

UNIT-I INTRODUCTION

1.1 CLOUD COMPUTING

1.1.1 Introduction to Cloud Computing

Computing as a service has seen a phenomenal growth in recent years. The primary motivation for this growth has been the promise of reduced capital and operating expenses, and the ease of dynamically scaling and deploying new services without maintaining a dedicated compute infrastructure. Hence, cloud computing has begun to rapidly transform the way organizations view their IT resources. From a scenario of a single system consisting of single operating system and single application, organizations have been moving into cloud computing, where resources are available in abundance and the user has a wide range to choose from. Cloud computing is a model for enabling convenient, on-demand network access to a shared pool of configurable computing resources that can be rapidly provisioned and released with service provider interaction or minimal management effort.

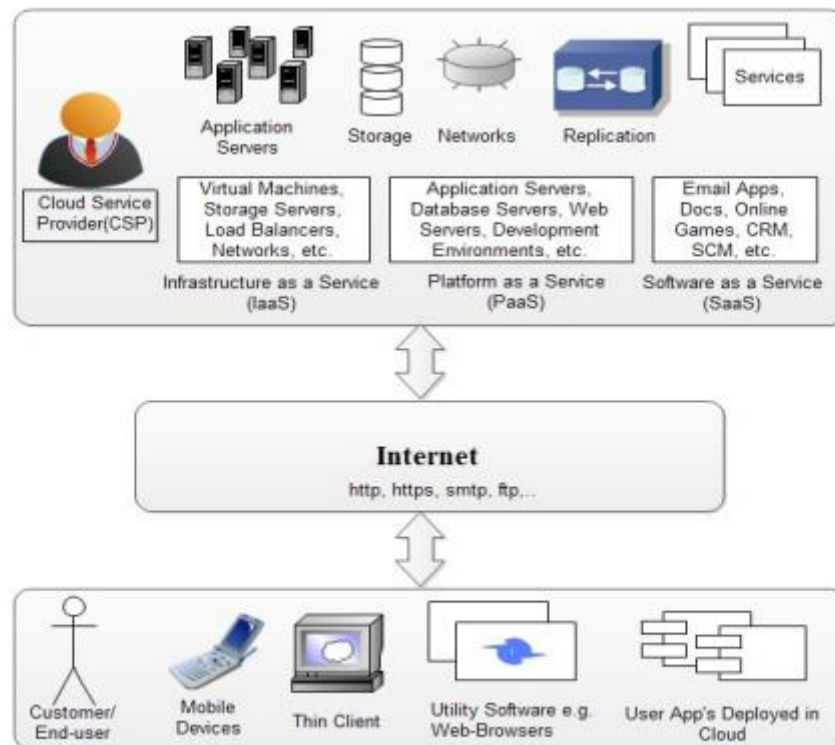


Fig. 1.1 Cloud Computing Architecture.

Cloud computing consists of three distinct types of computing services delivered remotely to clients via the internet. Clients typically pay a monthly or annual service fee to providers, to gain access to systems that deliver software as a service, platforms as a service and infrastructure as a service to subscribers. Clients who subscribe to cloud computing services can reap a variety of benefits, depending on their particular business needs at a given point in time. The days of large capital investments in software and IT infrastructure are now a thing of the past for any enterprise that chooses to adopt the cloud computing model for procurement of IT services.

1.1.2 Types of service in cloud computing

SAAS(Software as a Service)

SaaS (Software as a Service) provides clients with the ability to use software applications on a remote basis via an internet web browser. Software as a service is also referred to as “software on demand”. Clients can access SaaS applications from anywhere via the web because service providers host applications and their associated data at their location. The primary benefit of SaaS, is a lower cost of use, since subscriber fees require a much smaller investment than what is typically encountered under the traditional model of software delivery. Licensing fees, installation costs, maintenance fees and support fees that are routinely associated with the traditional model of software delivery can be virtually eliminated by subscribing to the SaaS model of software delivery. Examples of SaaS include: Google Applications and internet based email applications like Yahoo! Mail, Hotmail and Gmail.

PAAS(Platform as a Service)

PaaS (Platform as a Service) provides clients with the ability to develop and publish customized applications in a hosted environment via the web. It represents a new model for software development that is rapidly increasing in its popularity. An example of PaaS is Salesforce.com. PaaS provides a framework for agile software development, testing, deployment and maintenance in an integrated environment. Like SaaS, the primary benefit of PaaS, is a lower cost of use, since subscriber fees require a much smaller investment than what is typically encountered when implementing traditional tools for software development, testing and deployment. PaaS providers handle platform maintenance and system upgrades, resulting in a more efficient and cost effective solution for enterprise software development.

IAAS(Infrastructure as a Service)

IaaS (Infrastructure as a Service) allows clients to remotely use IT hardware and resources on a “pay-as-you-go” basis. It is also referred to as HaaS (hardware as a service). Major IaaS players include companies like IBM, Google and Amazon.com. IaaS employs virtualization, a method of creating and managing infrastructure resources in the “cloud”. IaaS provides small start up firms with a major advantage, since it allows them to gradually expand their IT infrastructure without the need for large capital investments in hardware and peripheral systems.

1.2 DEFINITION OF CLOUD

Cloud Computing is the use of hardware and software to deliver a service over a network (typically the Internet). With cloud computing, users can access files and use applications from any device that can access the Internet. An example of a Cloud Computing provider is Google's Gmail. Cloud Computing lets you store and access your applications or data over remote computers instead of your own computer.

Cloud Computing can be defined as delivering computing power (CPU, RAM, Network Speeds, Storage OS software) a service over a network (usually on the internet) rather than physically having the computing resources at the customer location.

Example: AWS, Azure, Google Cloud

1.2.1 Why Cloud Computing?

With increase in computer and Mobile user's, data storage has become a priority in all fields. Large and small scale businesses today thrive on their data & they spent a huge amount of money to maintain this data. It requires a strong IT support and a storage hub. Not all businesses can afford high cost of in-house IT infrastructure and back up support services. For them Cloud Computing is a cheaper solution. Perhaps its efficiency in storing data, computation and less maintenance cost has succeeded to attract even bigger businesses as well.

Cloud computing decreases the hardware and software demand from the user's side. The only thing that user must be able to run is the cloud computing systems interface software, which can be as simple as Web browser, and the Cloud network takes care of the rest. We all have experienced cloud computing at some instant of time, some of the popular cloud services we have used or we are still using are mail services like Gmail, Hotmail or Yahoo etc.

While accessing e-mail service our data is stored on cloud server and not on our computer. The technology and infrastructure behind the cloud is invisible. It is less important whether cloud services are based on HTTP, XML, Ruby, PHP or other specific technologies as far as it is user friendly and functional. An individual user can connect to cloud system from his/her own devices like desktop, laptop or mobile. Cloud computing harnesses small business effectively having limited resources, it gives small businesses access to the technologies that previously were out of their reach.

1.2.2 Benefits of Cloud Computing

First of all, 'cloud' is just a metaphor to describe the technology. Basically, cloud is nothing but a data center filled with hundreds of components like servers, routers, and storage units. Cloud data centers could be anywhere in the world; also you can access it from anywhere with an Internet-connected device. Why do people use it? Because of the following benefits it has:



Fig 1.2 Benefits of cloud computing

Pay-per-use Model: You only have to pay for the services you use, and nothing more!

24/7 Availability: It is always online! There is no such time that you cannot use your cloud service; you can use it whenever you want.

Easily Scalable: It is very easy to scale up and down or turn it off as per customers' needs. For instance, if your website's traffic increases only on Friday nights, you can opt for scaling up your servers that particular day of the week and then scaling down for the rest of the week.

Security: Cloud computing offers amazing data security. Especially if the data is mission-critical, then that data can be wiped off from local drives and kept on the cloud only for your access to stop it ending up in wrong hands.

Easily Manageable: You only have to pay subscription fees; all maintenance, up-gradation and delivery of services are completely maintained by the Cloud Provider. This is backed by the Service-level Agreement (SLA).

1.3 EVOLUTION OF CLOUD COMPUTING

A recent sensation in the realm of outsourcing is called Cloud Computing. Cloud is a huge collection of effortlessly approachable imaginary like utilities that can be used and accessed from anywhere, (for example s/w, h/w, advancement operating environments and applications). These operating environments and applications could be alterably re-designed to acclimate to a varying burden, permitting likewise for best environment utilization. These environments and facilities are ordinarily known to be a per utilization payment arrangement where in insurances are guaranteed by the service issuer by method of altered service level agreements.

While the happening to contemporary workstation systems administration happened inside mid1970s, not any talk of something remotely looking like a thought like "cloud computing" occurred until at long last in the ballpark of an a few years after the fact in 8f when John Back of Sun Microsystems begat the noteworthy motto, "The real system is The true machine." As prophetic similarly Sun was a while back, fittings (essential Figure and systems administration) had been none, of these influential none, of these commoditized enough to attain this vision around then. Cloud computing was still leastways a decade off. Meanwhile, Sun's Unix-delightful working framework and servers turned into the "new iron," swapping mainframes that been around for numerous eras.

Sun's machines utilized open systems administration models, for example TCP/IP. This empowered projects running on only one machine to identify with running systems on different models. Such requisitions ordinarily emulated the customer server building design model. Around this time period, Sir Tim Berners-Lee prescribed the thought connected with imparting learning spared on numerous hosting space to be made accessible to the planet through customer machines. Documents might be placed holding hypertext, course book with metadata made up of area data as a referrals for the thing portrayed by means of that content.

Despite the fact that the exact history of cloud computing is not the existing (the first business and customer cloud computing administrations sites – salesforce.com in addition to Google, were presented in 1999), the story is trussed straight to the improvement of the Internet and business engineering, since cloud computing is the method to the issue connected with how the Internet can help fortify business innovation. Business innovation has a drawn out and intriguing history, one that is almost as long on the grounds that business itself, yet the specific advancements that about straight impacted the of cloud computing begin with The true rise of workstations as suppliers of genuine business results.

The internet's "youth" Whereby it came to be clear that Arpanet was a decently major ordeal and some huge workstation organizations were made In the nineteen seventies, the thoughts and likewise components that had been prescribed in the 50s and 60s were being created decisively. Additionally, a large portion of the world's greatest machine organizations were begun, and the internet was conceived. Inside 1971, Intel, set up in the previous decade, acquainted the earth with the precise first microchip, and Apple organization architect Ray Tomlinson composed a system that permitted individuals to send correspondences starting with one machine then onto the next, from this point forward sending the essential message that a great deal of individuals might recognize similarly message.

The plant seeds were being seeded for the increment of the internet. The internet's global starting Where the Internet as a spot for both business and correspondence carried its own weight. The specific 1990s joined the planet in an unmatched way, beginning together with Cern's ease the World Wide-cut Web for general (that is, non-business) use in 1991. In Michael became bonkers, a browser reputed to be Mosaic permitted pictures to be demonstrated on Internet, and private organizations were permitted to work with Internet despite anything that might have happened before, as well. When firms were on the web, they started to assume the business potential

outcomes that accompanied having the ability to achieve the planet in an immediate, and a percentage of the greatest players online were established.

Marc Andreessen and John Clark established Netscape in 94' and none excessively early, since 1995 saving machine internet activity gave up to business endeavors like Netscape. In the meantime, stalwarts of the true internet Amazon.com and likewise eBay were started by Jeff Bezos and Pierre Omidyan, severally. The internet's "adulthood" notwithstanding cloud computing ascent Where the dab com house of cards blasts as a pimple notwithstanding cloud computing goes to the real fore The remainder of the nineties and starting with the 2000s were an incredible opportunity to find or put resources into an internet-based organization. Cloud computing had the right environment to lose, as multi-leaseholder architectures, profoundly overarching fast data transfer capacity and regular programming interoperability details were created in this specific time. Salesforce.net appeared in late 90s and was the essential webpage to convey business requisitions from the "typical" site – what on earth is presently called cloud computing.

Amazon.com presented Amazon Web Services in 2004. This gave clients the capacity with a specific end goal to store information and hang a huge amount of people to work with exceptionally little obligations, (for example Hardware Turk), around some different administrations. Face book had been established in '04, revolutionizing the methodology clients impart and the way they store their own particular information (their photos and film), inadvertently making the cloud an individual administration.

Most as of late, cloud computing organizations have been pondering how they might make their stock all the more join. In 2010 Salesforce.com started the cloud-based database at Database.com planned for engineers, denoting the advancement of could computing administrations that might be utilized on any unit, run on practically any stage and composed in diverse modifying dialect. Obviously, the long run of the internet and cloud computing have in prior times demonstrated hard to compute, however so farseeing as organizations strive to unite the globe and serve in which joined planet with new ways, there'll dependably be a need for both the internet and cloud computing.

1.4 UNDERLYING PRINCIPLES OF PARALLEL AND DISTRIBUTED COMPUTING

1.4.1 Parallel Computing

Cloud computing is intimately tied to parallel and distributed processing. Cloud applications are based on the client–server paradigm. Such applications run multiple instances of the service and require reliable and in-order delivery of messages. Parallel computing is a term usually used in the area of High Performance Computing (HPC). It specifically refers to performing calculations or simulations using multiple processors. Supercomputers are designed to perform parallel computation. These system do not necessarily have shared memory (as incorrectly claimed by other answers

What is Parallel Computing?

Parallel computing is a type of computation in which many calculations or execution of processes is carried out simultaneously. Whereas, a distributed system is a system whose components are located on different networked computers which communicate and coordinate their actions by passing messages to one another. Thus, this is the fundamental difference between parallel and distributed computing.

Parallel computing is also called parallel processing. There are multiple processors in parallel computing. Each of them performs the computations assigned to them. In other words, in parallel computing, multiple calculations are performed simultaneously. The systems that support parallel computing can have a shared memory or distributed memory. In shared memory systems, all the processors share the memory. In distributed memory systems, memory is divided among the processors.

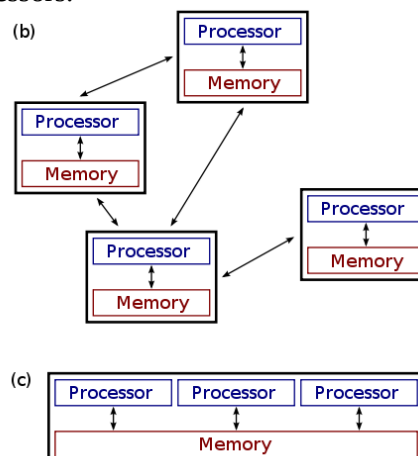
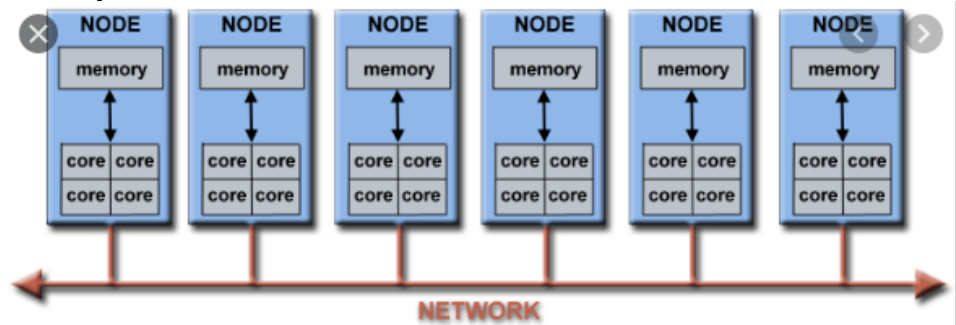


Figure 1.5 Parallel computing architecture

There are multiple advantages to parallel computing. As there are multiple processors working simultaneously, it increases the CPU utilization and improves the performance. Moreover, failure in one processor does not affect the functionality of other processors. Therefore, parallel computing provides reliability. On the other hand, increasing processors is costly. Furthermore, if one processor requires instructions of another, the processor might cause latency.



In the simplest sense, parallel computing is the simultaneous use of multiple compute resources to solve a computational problem:

- A problem is broken into discrete parts that can be solved concurrently
- Each part is further broken down to a series of instructions
- Instructions from each part execute simultaneously on different processors
- An overall control/coordination mechanism is employed

Parallel Computing

It is the use of multiple processing elements simultaneously for solving any problem. Problems are broken down into instructions and are solved concurrently as each resource which has been applied to work is working at the same time.

Advantages of Parallel Computing over Serial Computing are as follows:

- It saves time and money as many resources working together will reduce the time and cut potential costs.
- It can be impractical to solve larger problems on Serial Computing.
- It can take advantage of non-local resources when the local resources are finite.

Serial Computing 'wastes' the potential computing power, thus Parallel Computing makes better work of hardware.

Types of Parallelism:

Bit-level parallelism: It is the form of parallel computing which is based on the increasing processor's size. It reduces the number of instructions that the system must execute in order to perform a task on large-sized data.

Example: Consider a scenario where an 8-bit processor must compute the sum of two 16-bit integers. It must first sum up the 8 lower-order bits, then add the 8 higher-order bits, thus requiring two instructions to perform the operation. A 16-bit processor can perform the operation with just one instruction.

Instruction-level parallelism: A processor can only address less than one instruction for each clock cycle phase. These instructions can be re-ordered and grouped which are later on executed concurrently without affecting the result of the program. This is called instruction-level parallelism.

Task Parallelism: Task parallelism employs the decomposition of a task into subtasks and then allocating each of the subtasks for execution. The processors perform execution of sub tasks concurrently.

Why parallel computing?

- The whole real world runs in dynamic nature i.e. many things happen at a certain time but at different places concurrently. This data is extensively huge to manage.
- Real world data needs more dynamic simulation and modeling, and for achieving the same, parallel computing is the key.
- Parallel computing provides concurrency and saves time and money.
- Complex, large datasets, and their management can be organized only and only using parallel computing's approach.
- Ensures the effective utilization of the resources. The hardware is guaranteed to be used effectively whereas in serial computation only some part of hardware was used and the rest rendered idle.

Also, it is impractical to implement real-time systems using serial computing.

Applications of Parallel Computing:

- Data bases and Data mining.
- Real time simulation of systems.
- Science and Engineering.
- Advanced graphics, augmented reality and virtual reality.

Limitations of Parallel Computing:

- It addresses such as communication and synchronization between multiple sub-tasks and processes which is difficult to achieve.
- The algorithms must be managed in such a way that they can be handled in the parallel mechanism.
- The algorithms or program must have low coupling and high cohesion.
- But it's difficult to create such **programs**.
- More technically skilled and expert programmers can code a parallelism based program well.

Future of Parallel Computing: The computational graph has undergone a great transition from serial computing to parallel computing. Tech giant such as Intel has already taken a step towards parallel computing by employing multicore processors. Parallel computation will revolutionize the way computers work in the future, for the better good. With all the world connecting to each other even more than before, Parallel Computing does a better role in helping us stay that way. With faster networks, distributed systems, and multi-processor computers, it becomes even more necessary.

1.4.2 What is Distributed Computing?

Distributed computing divides a single task between multiple computers. Each computer can communicate with others via the network. All computers work together to achieve a common goal. Thus, they all work as a single entity. A computer in the distributed system is a node while a collection of nodes is a cluster.

A Distributed System is composed of a collection of independent physically (and geographically) separated computers that do not share physical memory or a clock. Each processor has its own local memory and the processors communicate using local and wide area networks. The nodes of a distributed system may be of heterogeneous architectures.

A Distributed Operating System attempts to make this architecture seamless and transparent to the user to facilitate the sharing of heterogeneous resources in an efficient, flexible and robust manner. Its aim is to shield the user from the complexities of the architecture and make it appear to behave like a timeshared centralized environment.

Communication is the central issue for distributed systems as all process interaction depends on it. Exchanging messages between different components of the system incurs delays due to data propagation, execution of communication protocols and scheduling. Communication delays can lead to inconsistencies arising between different parts of the system at a given instant in time making it difficult to gather global information for decision making and making it difficult to distinguish between what may be a delay and what may be a failure.

Fault tolerance is an important issue for distributed systems. Faults are more likely to occur in distributed systems than centralized ones because of the presence of communication links and a greater number of processing elements, any of which can fail. The system must be capable of reinitializing itself to a state where the integrity of data and state of ongoing computation is preserved with only some possible performance degradation.

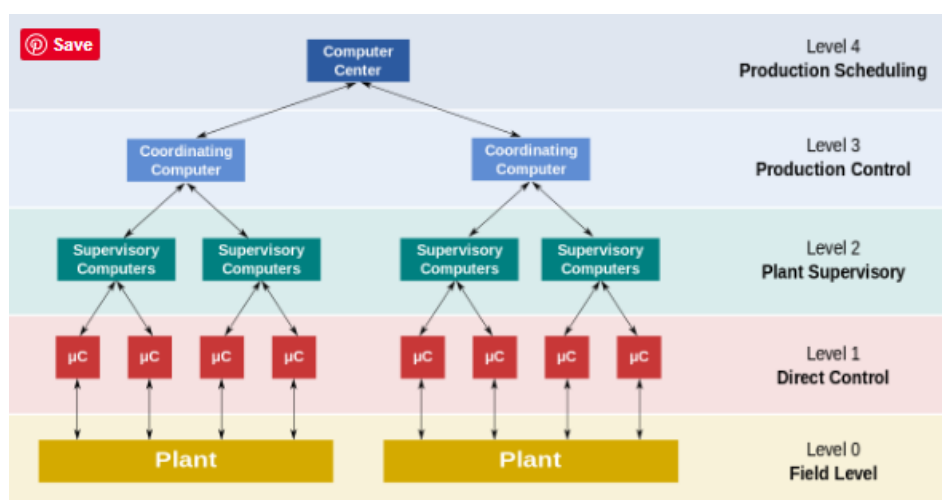
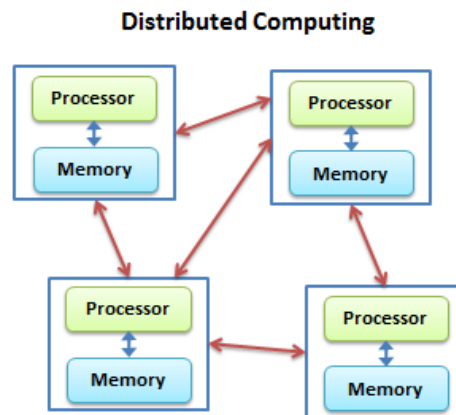


Figure 1.6 Distributed Computing architecture

There are multiple advantages of using distributed computing. It allows scalability and makes it easier to share resources easily. It also helps to perform computation tasks efficiently. On the other hand, it is difficult to develop distributed systems. Moreover, there can be network issues.

Distributed Computing

In daily life, an individual can use a computer to work with applications such as Microsoft Word, Microsoft PowerPoint. Complex problems may not be accomplished by using a single computer. Therefore, the single problem can be divided into multiple tasks and distributed to many computers. These computers can communicate with other computers through the network. They all perform similarly to a single entity. The process of dividing a single task among multiple computers is known as distributed computing. Each computer in a distributed system is known as a node. A set of nodes is a cluster.



In distributed computing systems, multiple system processors can communicate with each other using messages that are sent over the network. Such systems are increasingly available these days because of the availability at low price of computer processors and the high-bandwidth links to connect them.

The following reasons explain why a system should be built distributed, not just parallel:

Scalability: As distributed systems do not have the problems associated with shared memory, with the increased number of processors, they are obviously regarded as more scalable than parallel systems.

Reliability: The impact of the failure of any single subsystem or a computer on the network of computers defines the reliability of such a connected system. Definitely, distributed systems demonstrate a better aspect in this area compared to the parallel systems.

Data sharing: Data sharing provided by distributed systems is similar to the data sharing provided by distributed databases. Thus, multiple organizations can have distributed systems with the integrated applications for data exchange.

Resources sharing: If there exists an expensive and a special purpose resource or a processor, which cannot be dedicated to each processor in the system, such a resource can be easily shared across distributed systems.

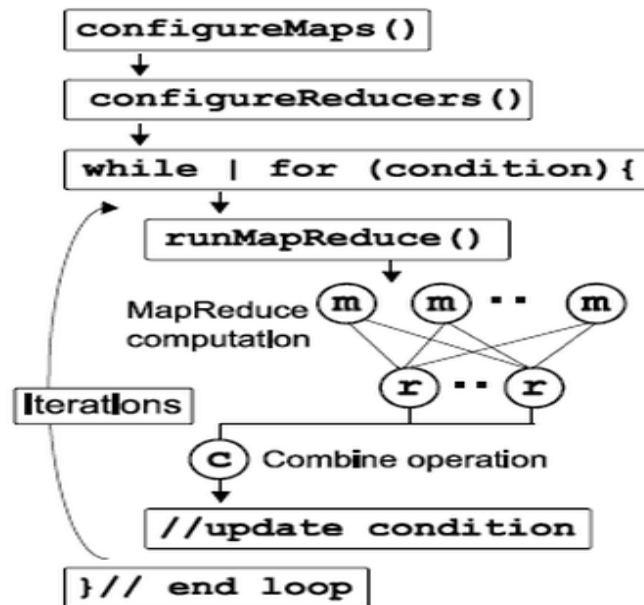
Heterogeneity and modularity: A system should be flexible enough to accept a new heterogeneous processor to be added into it and one of the processors to be replaced or removed from the system without affecting the overall system processing capability. Distributed systems are observed to be more flexible in this respect.

Geographic construction: The geographic placement of different subsystems of an application may be inherently placed as distributed. Local processing may be forced by the low communication bandwidth more specifically within a wireless network.

Economic: With the evolution of modern computers, high-bandwidth networks and workstations are available at low cost, which also favors distributed computing for economic reasons.

MapReduce

As we discussed that map reduce is really a robust framework manage large amount of data. The map reduce framework has to involve a lot of overhead when dealing with iterative map reduce. Twister is a great framework to perform iterative map reduce.



Additional functionality:

1.) Static and variable Data : Any iterative algorithm requires a static and variable data. Variable data are computed with static data (Usually the larger part of both) to generate another set of variable data. The process is repeated till a given condition and constrain is met. In a normal map-reduce function using Hadoop or DryadLINQ the static data are loaded uselessly every time the computation has to be performed. This is an extra overhead for the computation. Even though they remain fixed throughout the computation they have to be loaded again and again.

Twister introduces a “config” phase for both map and reduces to load any static data that is required. Loading static data for once is also helpful in running a long running Map/Reduce task

2.) Fat Map task : To save the access a lot of data the map is provided with an option of configurable map task, the map task can access large block of data or files. This makes it easy to add heavy computational weight on the map side.

3.) Combine operation: Unlike GFS where the output of reducer are stored in separate files, Twister comes with a new phase along with map reduce called combine that’s collectively adds up the output coming from all the reducer.

4.) Programming extensions: Some of the additional functions to support iterative functionality of Twister are:
i) `mapReduceBCast(Value value)` for sending a single to all map tasks. For example, the “Value” can be a set of parameters, a resource (file or executable) name, or even a block of data

ii) `configureMaps(Value[]values)` and `configureReduce(Value[]values)` to configure map and reduce with additional static data

TWISTER ARCHITECTURE

The Twister is designed to effectively support iterative MapReduce function. To reach this flexibility it reads data from the local disk of the worker nodes and handle the intermediate data data in the distributed memory of the workers mode.

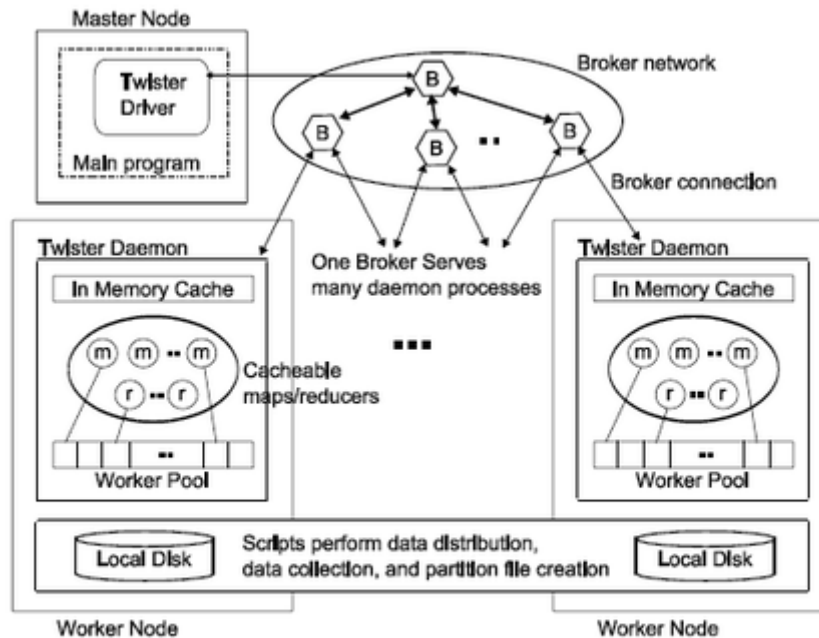


Figure 2. Architecture of Twister.

The messaging infrastructure in twister is called broker network and it is responsible to perform data transfer using publish/subscribe messaging.

Twister has three main entity:

1. Client Side Driver responsible to drive entire MapReduce computation
2. Twister Daemon running on every working node.
3. The broker Network.

Access Data

1. To access input data for map task it either reads data from the local disk of the worker nodes.
2. receive data directly via the broker network.

They keep all data read as file and having data as native file allows Twister to pass data directly to any executable. Additionally they allow tool to perform typical file operations like

- (i) create directories, (ii) delete directories, (iii) distribute input files across worker nodes, (iv) copy a set of resources/input files to all worker nodes, (v) collect output files from the worker nodes to a given location, and (vi) create partition-file for a given set of data that is distributed across the worker nodes.

Intermediate Data

The intermediate data are stored in the distributed memory of the worker node. Keeping the map output in distributed memory enhances the speed of the computation by sending the output of the map from these memory to reduces.

Messaging

The use of publish/subscribe messaging infrastructure improves the efficiency of Twister runtime. It use scalable NaradaBrokering messaging infrastructure to connect difference Broker network and reduce load on any one of them.

Fault Tolerance

There are three assumption for for providing fault tolerance for iterative mapreduce:

- (i) failure of master node is rare and no support is provided for that.
- (ii) Independent of twister runtime the communication network can be made fault tolerant.
- (iii) the data is replicated among the nodes of the computation infrastructure. Based on these assumptions we try to handle failures of map/reduce tasks, daemons, and worker nodes failures.

What is the Difference between Parallel and Distributed Computing?

S.NO	PARALLEL COMPUTING	DISTRIBUTED COMPUTING
1.	Many operations are performed simultaneously	System components are located at different locations
2.	Single computer is required	Uses multiple computers

3.	Multiple processors perform multiple operations	Multiple computers perform multiple operations
4.	It may have shared or distributed memory	It have only distributed memory
5.	Processors communicate with each other through bus	Computer communicate with each other through message passing.
6.	Improves the system performance	Improves system scalability, fault tolerance and resource sharing capabilities

1.5 CLOUD CHARACTERISTICS

- **On Demand Self-services**
- **Broad Network Access**
- **Resource Pooling**
- **Rapid Elasticity**
- **Measured Service**
- **Dynamic Computing Infrastructure**
- **IT Service-centric Approach**
- **Minimally or Self-managed Platform**
- **Consumption-based Billing**
- **Multi Tenancy**
- **Managed Metering**

On-Demand Self-Service

A consumer can unilaterally provision computing capabilities, such as server time and network storage, as needed automatically without requiring human interaction with each service provider.

Broad network access

Capabilities are available over the network and accessed through standard mechanisms that promote use by heterogeneous thin or thick client platforms (e.g., mobile phones, tablets, laptops, and workstations).

Resource pooling

The provider's computing resources are pooled to serve multiple consumers using a multi-tenant model, with different physical and virtual resources dynamically assigned and reassigned according to consumer demand. There is a sense of location independence in that the customer generally has no control or knowledge over the exact location of the provided resources but may be able to specify location at a higher level of abstraction (e.g., country, state, or datacenter). Examples of resources include storage, processing, memory, and network bandwidth.

Rapid elasticity

Capabilities can be elastically provisioned and released, in some cases automatically, to scale rapidly outward and inward commensurate with demand. To the consumer, the capabilities available for provisioning often appear to be unlimited and can be appropriated in any quantity at any time.

Measured service.

Cloud systems automatically control and optimize resource use by leveraging a metering capability at some level of abstraction appropriate to the type of service (e.g., storage, processing, bandwidth, and active user accounts). Resource usage can be monitored, controlled, and reported, providing transparency for both the provider and consumer of the utilized service.

Large Network Access

The user can access the data of the cloud or upload the data to the cloud from anywhere just with the help of a device and an internet connection. These capabilities are available all over the network and accessed with the help of internet.

Availability

The capabilities of the Cloud can be modified as per the use and can be extended a lot. It analyzes the storage usage and allows the user to buy extra Cloud storage if needed for a very small amount.

Automatic System

Cloud computing automatically analyzes the data needed and supports a metering capability at some level of services. We can monitor, control, and report the usage. It will provide transparency for the host as well as the customer.

Economical

It is the one-time investment as the company (host) has to buy the storage and a small part of it can be provided to the many companies which save the host from monthly or yearly costs. Only the amount which is spent is on the basic maintenance and a few more expenses which are very less.

Security

Cloud Security, is one of the best features of cloud computing. It creates a snapshot of the data stored so that the data may not get lost even if one of the servers gets damaged. The data is stored within the storage devices, which cannot be hacked and utilized by any other person. The storage service is quick and reliable.

1.6 ELASTICITY IN CLOUD

Elasticity is the ability to grow or shrink infrastructure resources dynamically as needed to adapt to workload changes in an autonomic manner, maximizing the use of resources.

Elasticity is the degree to which a system is able to adapt to workload changes by provisioning and de-provisioning resources in an autonomic manner, such that at each point in time the available resources match the current demand as closely as possible.

Elasticity in cloud infrastructure involves enabling the hypervisor to create virtual machines or containers with the resources to meet the real-time demand. Scalability often is discussed at the application layer, highlighting capability of a system, network or process to handle a growing amount of work, or its potential to be enlarged in order to accommodate that growth.

Dimensions and Core Aspects

Any given adaptation process is defined in the context of at least one or possibly multiple types of resources that can be scaled up or down as part of the adaptation. Each resource type can be seen as a separate dimension of the adaptation process with its own elasticity properties. If a resource type is a container of other resources types, like in the case of a virtual machine having assigned CPU cores and RAM, elasticity can be considered at multiple levels. Normally, resources of a given resource type can only be provisioned in discrete units like CPU cores, virtual machines (VMs), or physical nodes. For each dimension of the adaptation process with respect to a specific resource type, elasticity captures the following core aspects of the adaptation:

Speed

The speed of scaling up is defined as the time it takes to switch from an under-provisioned state to an optimal or over-provisioned state. The speed of scaling down is defined as the time it takes to switch from an over-provisioned state to an optimal or under-provisioned state. The speed of scaling up/down does not correspond directly to the technical resource provisioning/de-provisioning time.

Precision

The precision of scaling is defined as the absolute deviation of the current amount of allocated resources from the actual resource demand. As discussed above, elasticity is always considered with respect to one or more resource types. Thus, a direct comparison between two systems in terms of elasticity is only possible if the same resource types (measured in identical units) are scaled. To evaluate the actual observable elasticity in a given scenario, as a first step, one must define the criterion based on which the amount of provisioned resources is considered to match the actual current demand needed to satisfy the system's given performance requirements.

Over Provisioning and Under Provisioning

The main reason for cloud elasticity is to avoid either over provisioning or under provisioning of resources. Giving a cloud user either too much or too little data and resources will put that user at a disadvantage. If an enterprise has too many resources, they'll be paying for assets they aren't using. If they have too few resources, they can't run their processes correctly. Elastic systems can detect changes in workflows and processes in the cloud, automatically correcting resource provisioning to adjust for updated user projects.

- Modern business operations live on consistent performance and instant service availability.
- Cloud scalability and cloud elasticity handle these two business aspects in equal measure.
- Cloud scalability is an effective solution for businesses whose workload requirements are increasing slowly and predictably.
- Cloud elasticity is a cost-effective solution for the business with dynamic and unpredictable resource demands.

Notations and Preliminaries

For clarity and convenience, Notations describes the correlated variables which are used in the following sections. To elaborate the essence of cloud elasticity, we give the various states that are used in our discussion. Let n denote the number of VMs in service and let r be the number of requests in the system.

(1) Just-in-Need State. A cloud platform is in a just-in-need state if $\sum_{i=1}^n t_i$ is defined as the accumulated time in all just-in-need states.

(2) Over-provisioning State. A cloud platform is in an over-provisioning state if $0 < j < i$. T_o is defined as the accumulated time in all over-provisioning states.

(3) Under-provisioning State. A cloud platform is in an under-provisioning state if $j > 3i$. T_u is defined as the accumulated time in all under-provisioning states.

Notice that constants 1 and 3 in this paper are only for illustration purpose and can be any other values, depending on how an elastic cloud platform is managed. Different cloud users and/or applications may prefer different bounds of the hypothetical just-in-need states. The length of the interval between the upper (e.g.,) and lower (e.g.,) bounds controls the re-provisioning frequency. Narrowing down the interval leads to higher re-provision frequency for a fluctuating workload.

The just-in-need computing resource denotes a balanced state, in which the workload can be properly handled and quality of service (QoS) can be satisfactorily guaranteed. Computing resource over-provisioning, though QoS can be achieved, leads to extra but unnecessary cost to rent the cloud resources. Computing resource under-provisioning, on the other hand, delays the processing of workload and may be at the risk of breaking QoS commitment.

We present our elasticity definition for a realistic cloud platform and present mathematical foundation for elasticity evaluation. The definition of elasticity is given from a computational point of view and we develop a calculation formula for measuring elasticity value in virtualized clouds. Let T_m be the measuring time, which includes all the periods in the just-in-need, over-provisioning, and under-provisioning states; that is, $T_m = T_j + T_o + T_u$.

Definition : The elasticity E of a cloud perform is the percentage of time when the platform is in just-in-need states; that is, $E = T_j / T_m = 1 - T_o / T_m - T_u / T_m$.

Broadly defining, elasticity is the capability of delivering preconfigured and just-in-need virtual machines adaptively in a cloud platform upon the fluctuation of the computing resources required. Practically it is determined by the time needed from an under-provisioning or over-provisioning state to a balanced resource provisioning state. Definition 1 provides a mathematical definition which is easily and accurately measurable. Cloud platforms with high elasticity exhibit high adaptively, implying that they switch from an over-provisioning or an under-provisioning state to a balanced state almost in real time. Other cloud platforms take longer time to adjust and reconfigure computing resources. Although it is recognized that high elasticity can also be achieved via physical host standby, we argue that, with virtualization-enabled computing resource provisioning, elasticity can be delivered in a much easier way due to the flexibility of service migration and image template generation.

$$E = 1 - \frac{(T_o + T_u)}{T_m} = 1 - \frac{T_o}{T_m} - \frac{T_u}{T_m}, E = 1 - \frac{(T_o + T_u)}{T_m} = 1 - \frac{T_o}{T_m} - \frac{T_u}{T_m},$$

where T_m denotes the total measuring time, in which T_o is the over-provisioning time which accumulates each single period of time that the cloud platform needs to switch from an over-provisioning state to a balanced state and is the T_u under-provisioning time which accumulates each single period of time that the cloud platform needs to switch from an under-provisioning state to a corresponding balanced state.

Let P_j, P_o , and P_u be the accumulated probabilities of just-in-need states, over-provisioning states, and under-provisioning states, respectively. If T_m is sufficiently long, we have

$$P_j = T_j / T_m, P_o = T_o / T_m, \text{ and } P_u = T_u / T_m. \text{ Therefore, we get}$$

$$E = P_j = 1 - P_o - P_u.$$

Equation (E1) can be used when elasticity is measured by monitoring a real system. Equation (Pj) can be used when elasticity is calculated by using our CTMC model. If elasticity metrics are well defined, elasticity of cloud platforms could easily be captured, evaluated, and compared.

We would like to mention that the primary factors of elasticity, that is, the amount, frequency, and time of resource re-provisioning, are all summarized in T_o and T_u (i.e. P_o , and P_u). Elasticity can be increased by changing these factors. For example, one can maintain a list of standby or underutilized compute nodes. These nodes are prepared for the upcoming surge of workload, if there is any, to minimize the time needed to start these nodes. Such a hot standby strategy increases cloud elasticity by reducing T_u .

Elastic Cloud Platform Modeling

To model elastic cloud platforms, we make the following assumptions.

- (i) All VMs are homogeneous with the same service capability and are added/removed one at a time. λ
- (ii) The user request arrivals are modeled as a Poisson process with rate .
- (iii) The service time, the start-up time, and the shut-off time of each VM are governed by exponential distributions with rates α, μ , and , respectively .
- (iv) Let i denote the number of virtual machines that are currently in service, and let denote the number of requests that are receiving service or in waiting.

(v) Let State $v(i,j)$ denote the various states of a cloud platform when the virtual machine number is j and the request number is i . Let the hypothetical just-in-need state, over-provisioning state, and under-provisioning state be JIN, OP, and UP, respectively. We can set the equations of the relation between the virtual machine number and the request number as follows:

Let i denote the various states of a cloud platform when the virtual machine number is j and the request number is i . Let the hypothetical just-in-need state, over-provisioning state, and under-provisioning state be JIN, OP, and UP, respectively. We can set the equations of the relation between the virtual machine number and the request number as follows:

$$\text{state}v(i, j) = \begin{cases} \text{OP,} & \text{if } 0 \leq j \leq i; \\ \text{JIN,} & \text{if } i < j \leq 3i; \\ \text{UP,} & \text{if } j > 3i. \end{cases} \quad \text{state}v(i, j) = \begin{cases} \text{OP,} & \text{if } 0 \leq j \leq i; \\ \text{JIN,} & \text{if } i < j \leq 3i; \\ \text{UP,} & \text{if } j > 3i. \end{cases}$$

The hypothetical just-in-need state, over-provisioning state, and under-provisioning state are listed in Table

VM number	Overprovisioning state	Just-in-need state	Underprovisioning state
1	$0 \leq j \leq 1$	$1 < j \leq 3$	$j > 3$
2	$0 \leq j \leq 2$	$2 < j \leq 6$	$j > 6$
3	$0 \leq j \leq 3$	$3 < j \leq 9$	$j > 9$
4	$0 \leq j \leq 4$	$4 < j \leq 12$	$j > 12$

For example:

Streaming Services. Netflix is dropping a new season of Mind hunter. The notification triggers a significant number of users to get on the service and watch or upload the episodes. Resource-wise, it is an activity spike that requires swift resource allocation.

E-commerce. Amazon has a Prime Day event with many special offers, sells-offs, promotions, and discounts. It attracts an immense amount of customers on the service that is doing different activities. Actions include searching for products, bidding, buying stuff, writing reviews, rating products. This diverse activity requires a very flexible system that can allocate resources to one sector without dragging down others.

Elasticity in Cloud Computing

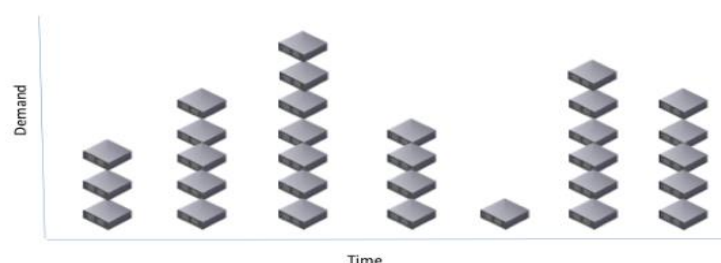


Figure 1.7 Elasticity in cloud

Cloud elasticity is a popular feature associated with scale-out solutions (horizontal scaling), which allows for resources to be dynamically added or removed when needed. Elasticity is generally associated with public cloud resources and is more commonly featured in pay-per-use or pay-as-you-grow services. This means IT managers are not paying for more resources than they are consuming and any given time. In virtualized environments cloud elasticity could include the ability to dynamically deploy new virtual machines or shutdown inactive virtual machines.

Elasticity as the scaling of system resources to increase or decrease capacity, whereby definitions and

Specifically state that the amount of provisioned re- in a broader sense, it could also contain manual steps. Without a defined adaptation process, a scalable system cannot behave in an elastic manner, as scalability on its own does not include temporal aspects. When evaluating elasticity, the following points need to be checked beforehand:

Autonomic Scaling: What adaptation process is used for autonomic scaling?

• **Elasticity Dimensions:** What is the set of resource types scaled as part of the adaptation process?

• **Resource Scaling Units:** For each resource type, in what unit is the amount of allocated resources varied?

• **Scalability Bounds:** For each resource type, what is the upper bound on the amount of resources that can be allocated?

What is Cloud Scalability?

Scalability is one of the preeminent features of cloud computing. In the past, a system's scalability relied on the company's hardware, and thus, was severely limited in resources. With the adoption of cloud computing, scalability has become much more available and more effective.

Automatic scaling opened up numerous possibilities for the implementation of big data machine learning models and data analytics to the fold.

Overall, Cloud Scalability covers expected and predictable workload demands and also handles rapid and unpredictable changes in the scale of operation. The pay-as-you-expand pricing model makes possible the preparation of the infrastructure and its spending budget in the long term without too much strain.

There are several types of cloud scalability:

Vertical, aka Scale-Up - the ability to handle an increasing workload by adding resources to the existing infrastructure. It is a short term solution to cover immediate needs.

Horizontal, aka Scale-Out - the expansion of the existing infrastructure with new elements to tackle more significant workload requirements. It is a long term solution aimed to cover present and future resource demands with room for expansion.

Diagonal scalability is a more flexible solution that combines adding and removal of resources according to the current workload requirements. It is the most cost-effective scalability solution by far.

1.8 ON-DEMAND PROVISIONING

On-demand computing is a delivery model in which computing resources are made available to the user as needed. The resources may be maintained within the user's enterprise, or made available by a cloud service provider. When the services are provided by a third-party, the term cloud computing is often used as a synonym for on-demand computing.

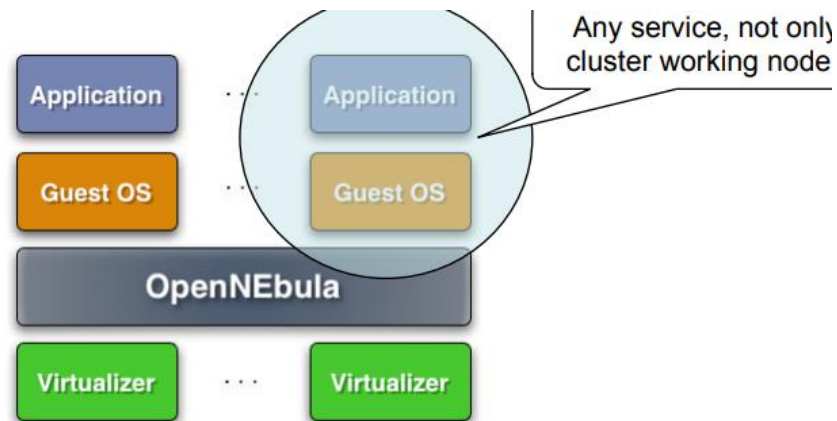
The on-demand model was developed to overcome the common challenge to an enterprise of being able to meet fluctuating demands efficiently. Because an enterprise's demand on computing resources can vary drastically from one time to another, maintaining sufficient resources to meet peak requirements can be costly. Conversely, if an enterprise tried to cut costs by only maintaining minimal computing resources, it is likely there will not be sufficient resources to meet peak requirements.

The on-demand model provides an enterprise with the ability to scale computing resources up or down with the click of a button, an API call or a business rule. The model is characterized by three attributes: scalability, pay-per-use and self-service. Whether the resource is an application program that helps team members collaborate or additional storage for archiving images, the computing resources are elastic, metered and easy to obtain.

Many on-demand computing services in the cloud are so user-friendly that non-technical end users can easily acquire computing resources without any help from the organization's information technology (IT) department. This has advantages because it can improve business agility, but it also has disadvantages because shadow IT can pose security risks. For this reason, many IT departments carry out periodic cloud audits to identify greyneet on-demand applications and other rogue IT.

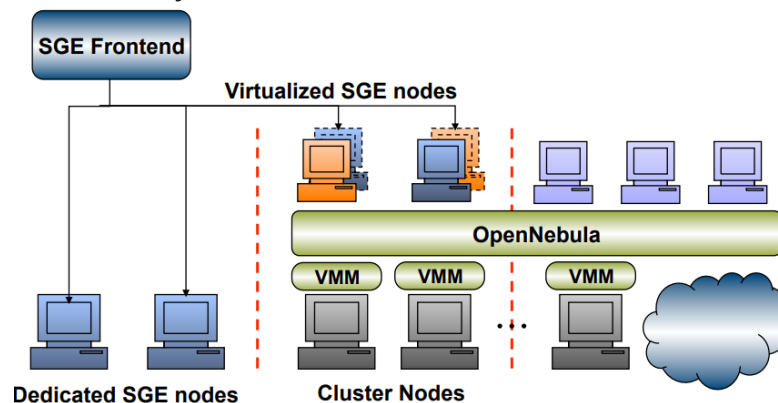
a) Local On-demand Resource Provisioning

- The Engine for the Virtual Infrastructure
- Virtualization of Cluster and HPC Systems Benefits
- Open Nebula creates a distributed virtualization layer
- Extend the benefits of VM Monitors from one to multiple resources
- Decouple the VM (service) from the physical location
- Transform a distributed physical infrastructure into a flexible and elastic virtual infrastructure, which adapts to the changing demands of the VM Service workload



b) Remote On-demand Resource Provisioning

- Access to Cloud Systems
- Federation of Cloud Systems
- The RESERVOIR Project



Benefit of Remote Provisioning

The virtualization of the local infrastructure supports a virtualized alternative to contribute resources to a Grid infrastructure.

- Simpler deployment and operation of new middleware distributions
- Lower operational costs
- Easy provision of resources to more than one infrastructure or VO
- Easy support for VO-specific worker nodes
- Performance partitioning between local and grid clusters

VMs to Provide pre-Created Software Environments for Jobs

- Extensions of job execution managers to create per-job basis VMs so as to provide a pre-defined environment for job execution
- Those approaches still manage jobs
- The VMs are bounded to a given PM and only exist during job execution
- Condor, SGE, MOAB, Globus GridWay...
- Job Execution Managers for the Management of VMs
- Job execution managers enhanced to allow submission of VMs
- Those approaches manage VMs as jobs
- Condor, "pilot" backend in Globus VWS

Differences between VMs and Jobs as basic Management Entities

- VM structure: Images with fixed and variable parts for migration...
- VM life-cycle: Fixed and transient states for contextualization, live migration.
- VM duration: Long time periods ("forever")
- VM groups (services): Deploy ordering, affinity, rollback management...
- VM elasticity: Changing of capacity requirements and number of VMs
- Different Metrics in the Allocation of Physical Resources

Capacity provisioning:

Probability of SLA violation for a given cost of provisioning including support for server consolidation, partitioning.

- HPC scheduling: Turnaround time, wait time, throughput

VMware DRS, Platform Orchestrator, IBM Director, Novell ZENworks, Enomalism, Xenoserver

- Advantages:

- Open-source (Apache license v2.0)
- Open and flexible architecture to integrate new virtualization technologies
- Support for the definition of any scheduling policy (consolidation, workload balance, affinity, SLA...)
- LRM-like CLI and API for the integration of third-party tools

Static Provisioning: For applications that have predictable and generally unchanging demands/workloads, it is possible to use “static provisioning” effectively. With advance provisioning, the customer contracts with the provider for services and the provider prepares the appropriate resources in advance of start of service. The customer is charged a flat fee or is billed on a monthly basis.

Dynamic Provisioning: In cases where demand by applications may change or vary, “dynamic provisioning” techniques have been suggested whereby VMs may be migrated on-the-fly to new compute nodes within the cloud. With dynamic provisioning, the provider allocates more resources as they are needed and removes them when they are not. The customer is billed on a pay-per-use basis. When dynamic provisioning is used to create a hybrid cloud, it is sometimes referred to as cloud bursting.

User Self-provisioning: With user self- provisioning (also known as cloud self- service), the customer purchases resources from the cloud provider through a web form, creating a customer account and paying for resources with a credit card. The provider's resources are available for customer use within hours, if not minutes. **Parameters for Resource Provisioning**

i) Response time: The resource provisioning algorithm designed must take minimal time to respond when executing the task.

ii) Minimize Cost: From the Cloud user point of view cost should be minimized.

iii) Revenue Maximization: This is to be achieved from the Cloud Service Provider’s view.

iv) Fault tolerant: The algorithm should continue to provide service in spite of failure of nodes.

v) Reduced SLA Violation: The algorithm designed must be able to reduce SLA violation.

vi) Reduced Power Consumption: VM placement & migration techniques must lower power consumption.

Resource Provisioning Strategies For efficiently making use of the Cloud Resources, resource provisioning techniques are to be used.

There are many resource provisioning techniques both static and dynamic provisioning each having its own pros and cons. The provisioning techniques are used to improve QoS parameters , minimize cost for cloud user and maximize revenue for the Cloud Service Provider improve response time , deliver services to cloud user even in presence of failures, improve performance reduces SLA violation, efficiently uses cloud resources reduces power consumption

c) Static Resource Provisioning Techniques

Aneka’s deadline driven provisioning technique is used for scientific application as scientific applications require large computing power. Aneka is a cloud application platform which is capable of provisioning resources which are obtained from various sources such as public and private clouds, clusters, grids and desktop grids. This technique efficiently allocates resources thereby reducing application execution time. Because resource failures are inevitable it is a good idea to efficiently couple private and public cloud using an architectural framework for realizing the full potential of hybrid clouds. Proposes a failure- aware resource provisioning algorithm that is capable of providing cloud users’ QoS requirements. This provides resource provisioning policies and proposes a scalable hybrid infrastructure to assure QoS of the users. This improves the deadline violation rate by 32% and 57% improvement in slowdown with a limited cost on a public cloud. Since resources held by single cloud are usually limited it is better to get resources from other participating clouds. But this is difficult to provide right resources from different cloud providers because management policies are different and description about various resources is different in each organization. Also interoperability is hard to achieve. To overcome this, Inter Cloud Resource Provisioning (ICRP) system is proposed in where resources and tasks are described semantically and stored using resource ontology and using a semantic scheduler and a set of inference rules resources are assigned. With the increasing functionality and complexity in Cloud computing, resource failure cannot be avoided. So the proposed strategy in addresses the question of provisioning resources to applications in the presence of failures in a hybrid cloud computing environment. It takes into account the workload model and the failure correlations to redirect requests to appropriate cloud providers. This is done using real failure traces and workload models, and it is found that the deadline violation rate of users’ request is reduced by 20% with a limited cost on Amazon Public Cloud.

d) Dynamic Resource provisioning Techniques

The algorithm proposed is suitable for web applications where response time is one of the important factors. For web applications guaranteeing average response time is difficult because traffic patterns are highly dynamic and difficult to predict accurately and also due to the complex nature of the multi-tier web applications it is difficult to identify bottlenecks and resolving them automatically. This provisioning technique proposes a working prototype system for automatic detection and resolution of bottlenecks in a multi-tier cloud hosted web applications. This

improves response time and also identifies over provisioned resources. VM based resource management is a heavy weight task. So this is less flexible and less resource efficient. To overcome this, a lightweight approach called Elastic Application Container [EAC] is used for provisioning the resources where EAC is a virtual resource unit for providing better resource efficiency and more scalable applications. This EAC-oriented platform and algorithm is to support multi tenant cloud use . Dynamic creation of the tenant is done by integrating cloud based services on the fly. But dynamic creation is by building the required components from the scratch. Even though multitenant systems save cost, but incur huge reconfiguration costs.

Sl. No.	On demand Provisioning Techniques	Merits	Challenges
1	Deadline-driven provisioning of resources for scientific applications in hybrid clouds with Aneka	Able to efficiently allocate resources from different sources in order to reduce application execution times.	Not suitable for HPC-data intensive applications.
2	Dynamic provisioning in multi-tenant service clouds	Matches tenant functionalities with client requirements.	Does not work for testing on real-life cloud-based system and across several domains.
3	Elastic Application Container: A Lightweight Approach for Cloud Resource Provisioning	Outperforms in terms of Flexibility and resource efficiency.	Not suitable for web applications and supports only one type of programming language, Java.
4	Hybrid Cloud Resource Provisioning Policy in the Presence of Resource Failures	Able to adopt user the workload model to provide flexibility in the choice of strategy based on the desired level of QoS, the needed performance, and the available budget.	Not suitable to run real experiments.
5	Provisioning of Requests for Virtual Machine Sets with Placement Constraints in IaaS Clouds	Runtime efficient & can provide an effective means of online VM-to-PM mapping and also Maximizes revenue.	Not practical for medium to large problems.
6	Failure-aware resource provisioning for hybrid Cloud infrastructure	Able to improve the users' QoS about 32% in terms of deadline violation rate and 57% in terms of slowdown with a limited cost on a public cloud.	Not able to run real experiments and also not able to move VMs between public and private clouds to deal with resource failures in the local infrastructures.
7	VM Provisioning Method to Improve the Profit and SLA Violation of Cloud Service Providers	Reduces SLA violations & Improves Profit.	Increases the problem of resource allocation and load balancing among the datacenters.
8	Risk Aware Provisioning and Resource Aggregation based Consolidation of Virtual Machines	Significant amount of reduction in the numbers required to host 1000 VMs and enables to turn off unnecessary servers.	Takes into account only CPU requirements of VMs.
9	Semantic based Resource Provisioning and Scheduling in Inter-cloud Environment	Enables the fulfillment of customer requirements to the maximum by providing additional resources to the cloud system participating in a federated cloud environment thereby solving the interoperability problem.	QoS parameters like response time and throughput has to be achieved for interactive applications.
10	Design and implementation of adaptive power-aware virtual machine provisioned (APA-VMP) using swarm intelligence	Efficient VM placement and significant reduction in power.	Not suitable for conserving power in modern data centers.
11	Adaptive resource provisioning for read intensive multi-tier applications in the cloud	Automatic Identification and resolution of bottlenecks in multitier web application hosted on a cloud.	Not suitable for n-tier clustered application hosted on a cloud.

12	Optimal Resource Provisioning for Cloud Computing Environment	Efficiently provisions Cloud Resources for SaaS users with a limited budget and Deadline thereby optimizing QoS.	Applicable only for SaaS users and SaaS providers.
----	---	--	--

CHAPTER-2

CLOUD ENABLING TECHNOLOGIES

2.1 SERVICE-ORIENTED ARCHITECTURE (SOA)

A service-oriented architecture (SOA) is essentially a collection of services. These services communicate with each other. The communication can involve either simple data passing or it could involve two or more services coordinating some activity. Some means of connecting services to each other is needed.

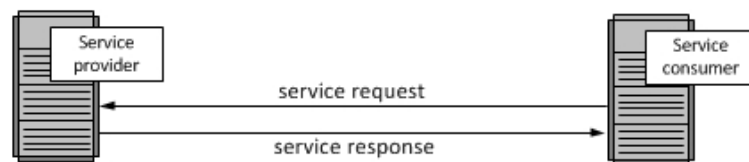
Service-Oriented Architecture (SOA) is a style of software design where services are provided to the other components by application components, through a communication protocol over a network. Its principles are independent of vendors and other technologies. In service oriented architecture, a number of services communicate with each other, in one of two ways: through passing data or through two or more services coordinating an activity.

Services

If a service-oriented architecture is to be effective, we need a clear understanding of the term service. A service is a function that is well-defined, self-contained, and does not depend on the context or state of other services. See Service.

Connections

The technology of Web Services is the most likely connection technology of service-oriented architectures. The following figure illustrates a basic service-oriented architecture. It shows a service consumer at the right sending a service request message to a service provider at the left. The service provider returns a response message to the service consumer. The request and subsequent response connections are defined in some way that is understandable to both the service consumer and service provider. How those connections are defined is explained in Web Services Explained. A service provider can also be a service consumer.



Web services which are built as per the SOA architecture tend to make web service more independent. The web services themselves can exchange data with each other and because of the underlying principles on which they are created, they don't need any sort of human interaction and also don't need any code modifications. It ensures that the web services on a network can interact with each other seamlessly.

Benefit of SOA

- **Language Neutral Integration:** Regardless of the developing language used, the system offers and invoke services through a common mechanism. Programming language neutralization is one of the key benefits of SOA's integration approach.
- **Component Reuse:** Once an organization built an application component, and offered it as a service, the rest of the organization can utilize that service.
- **Organizational Agility:** SOA defines building blocks of capabilities provided by software and it offers some service(s) that meet some organizational requirement; which can be recombined and integrated rapidly.
- **Leveraging Existing System:** This is one of the major use of SOA which is to classify elements or functions of existing applications and make them available to the organizations or enterprise.

2.1.1 SOA Architecture

SOA architecture is viewed as five horizontal layers. These are described below:

Consumer Interface Layer: These are GUI based apps for end users accessing the applications.

Business Process Layer: These are business-use cases in terms of application.

Services Layer: These are whole-enterprise, in service inventory.

Service Component Layer: are used to build the services, such as functional and technical libraries.

Operational Systems Layer: It contains the data model.

SOA Governance

It is a notable point to differentiate between IT governance and SOA governance. IT governance focuses on managing business services whereas SOA governance focuses on managing Business services. Furthermore in service oriented organization, everything should be characterized as a service in an organization. The cost that governance put forward becomes clear when we consider the amount of risk that it eliminates with the good understanding of service, organizational data and processes in order to choose approaches and processes for policies for monitoring and generate performance impact.

SOA Architecture Protocol

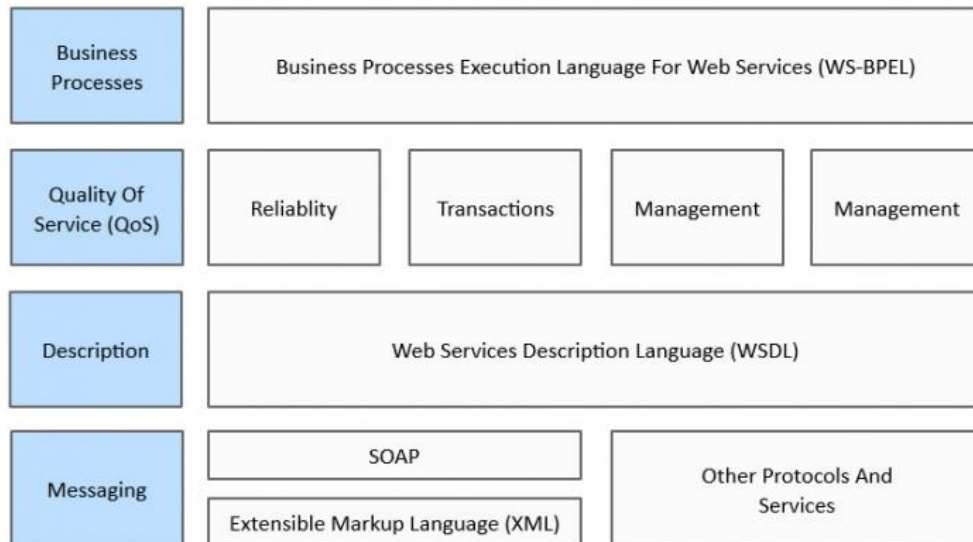


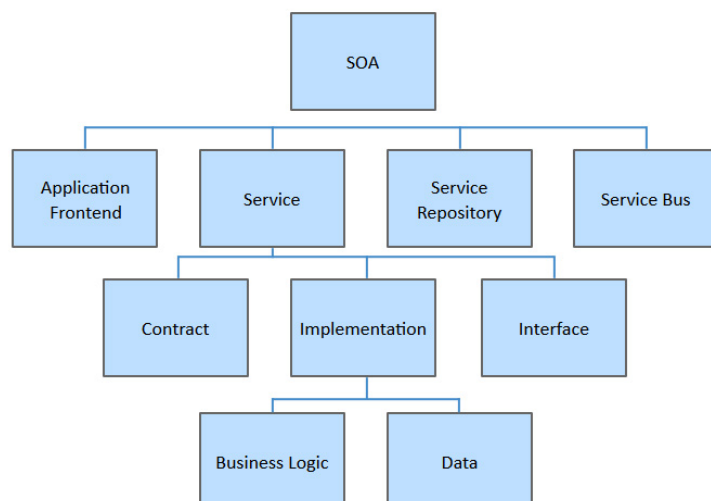
Figure 2.2 SOA Protocol Diagram

Here lies the protocol stack of SOA showing each protocol along with their relationship among each protocol. These components are often programmed to comply with SCA (Service Component Architecture), a language that has broader but not universal industry support. These components are written in BPEL (Business Process Execution Languages), Java, C#, XML etc and can apply to C++ or FORTRAN or other modern multi-purpose languages such as Python, PP or Ruby. With this, SOA has extended the life of many all-time famous applications.

SOA Security

- With the vast use of cloud technology and its on-demand applications, there is a need for well - defined security policies and access control.
- With the betterment of these issues, the success of SOA architecture will increase.
- Actions can be taken to ensure security and lessen the risks when dealing with SOE (Service Oriented Environment).
- We can make policies that will influence the patterns of development and the way services are used. Moreover, the system must be set-up in order to exploit the advantages of public cloud with resilience. Users must include safety practices and carefully evaluate the clauses in these respects.

Elements of SOA



2.1.2 SOA is based on some key principles which are mentioned below

1. **Standardized Service Contract** - Services adhere to a service description. A service must have some sort of description which describes what the service is about. This makes it easier for client applications to understand what the service does.
2. **Loose Coupling** – Less dependency on each other. This is one of the main characteristics of web services which just states that there should be as less dependency as possible between the web services and the client invoking the web service. So if the service functionality changes at any point in time, it should not break the client application or stop it from working.
3. **Service Abstraction** - Services hide the logic they encapsulate from the outside world. The service should not expose how it executes its functionality; it should just tell the client application on what it does and not on how it does it.
4. **Service Reusability** - Logic is divided into services with the intent of maximizing reuse. In any development company re-usability is a big topic because obviously one wouldn't want to spend time and effort building the same code again and again across multiple applications which require them. Hence, once the code for a web service is written it should have the ability work with various application types.
5. **Service Autonomy** - Services should have control over the logic they encapsulate. The service knows everything on what functionality it offers and hence should also have complete control over the code it contains.
6. **Service Statelessness** - Ideally, services should be stateless. This means that services should not withhold information from one state to the other. This would need to be done from either the client application. An example can be an order placed on a shopping site. Now you can have a web service which gives you the price of a particular item. But if the items are added to a shopping cart and the web page navigates to the page where you do the payment, the responsibility of the price of the item to be transferred to the payment page should not be done by the web service. Instead, it needs to be done by the web application.
7. **Service Discoverability** - Services can be discovered (usually in a service registry). We have already seen this in the concept of the UDDI, which performs a registry which can hold information about the web service.
8. **Service Composability** - Services break big problems into little problems. One should never embed all functionality of an application into one single service but instead, break the service down into modules each with a separate business functionality.
9. **Service Interoperability** - Services should use standards that allow diverse subscribers to use the service. In web services, standards as XML and communication over HTTP is used to ensure it conforms to this principle.

2.1.3 Implementing Service-Oriented Architecture

- When it comes to implementing service-oriented architecture (SOA), there is a wide range of technologies that can be used, depending on what your end goal is and what you're trying to accomplish.
- Typically, Service-Oriented Architecture is implemented with web services, which makes the “functional building blocks accessible over standard internet protocols.”

An example of a web service standard is SOAP, which stands for Simple Object Access Protocol. In a nutshell, SOAP “is a messaging protocol specification for exchanging structured information in the implementation of web services in computer networks.

The importance of Service-Oriented Architecture

There are a variety of ways that implementing an SOA structure can benefit a business, particularly, those that are based around web services. Here are some of the foremost:

Creates reusable code

The primary motivator for companies to switch to an SOA is the ability to reuse code for different applications. By reusing code that already exists within a service, enterprises can significantly reduce the time that is spent during the development process. Not only does the ability to reuse services decrease time constraints, but it also lowers costs that are often incurred during the development of applications. Since SOA allows varying languages to communicate through a central interface, this means that application engineers do not need to be concerned with the type of environment in which these services will be run. Instead, they only need to focus on the public interface that is being used.

Creates reusable code

The primary motivator for companies to switch to an SOA is the ability to reuse code for different applications. By reusing code that already exists within a service, enterprises can significantly reduce the time that is spent during the development process. Not only does the ability to reuse services decrease time

constraints, but it also lowers costs that are often incurred during the development of applications. Since SOA allows varying languages to communicate through a central interface, this means that application engineers do not need to be concerned with the type of environment in which these services will be run. Instead, they only need to focus on the public interface that is being used.

Promotes interaction

A major advantage in using SOA is the level of interoperability that can be achieved when properly implemented. With SOA, no longer will communication between platforms be hindered in operation by the languages on which they are built. Once a standardized communication protocol has been put in place, the platform systems and the varying languages can remain independent of each other, while still being able to transmit data between clients and services. Adding to this level of interoperability is the fact that SOA can negotiate firewalls, thus ensuring that companies can share services that are vital to operations.

Allows for scalability

When developing applications for web services, one issue that is of concern is the ability to increase the scale of the service to meet the needs of the client. All too often, the dependencies that are required for applications to communicate with different services inhibit the potential for scalability. However, with SOA this is not the case. By using an SOA where there is a standard communication protocol in place, enterprises can drastically reduce the level of interaction that is required between clients and services, and this reduction means that applications can be scaled without putting added pressure on the application, as would be the case in a tightly-coupled environment.

Reduced costs

In business, the ability to reduce costs while still maintaining a desired level of output is vital to success, and this concept holds true with customized service solutions as well. By switching to an SOA-based system, businesses can limit the level of analysis that is often required when developing customized solutions for specific applications. This cost reduction is facilitated by the fact that loosely coupled systems are easier to maintain and do not necessitate the need for costly development and analysis. Furthermore, the increasing popularity in SOA means reusable business functions are becoming commonplace for web services which drive costs lower.

2.2 REST AND SYSTEMS OF SYSTEMS

Representational State Transfer (REST) is an architecture principle in which the web services are viewed as resources and can be uniquely identified by their URLs. The key characteristic of a RESTful Web service is the explicit use of HTTP methods to denote the invocation of different operations.

Representational state transfer (REST) is a distributed system framework that uses Web protocols and technologies. The REST architecture involves client and server interactions built around the transfer of resources. The Web is the largest REST implementation.

REST may be used to capture website data through interpreting extensible markup language (XML) Web page files with the desired data. In addition, online publishers use REST when providing syndicated content to users by activating Web page content and XML statements. Users may access the Web page through the website's URL, read the XML file with a Web browser, and interpret and use data as needed.

Basic REST constraints include:

Client and Server: The client and server are separated from REST operations through a uniform interface, which improves client code portability.

Stateless: Each client request must contain all required data for request processing without storing client context on the server.

Cacheable: Responses (such as Web pages) can be cached on a client computer to speed up Web Browsing. Responses are defined as cacheable or not cacheable to prevent clients from reusing stale or inappropriate data when responding to further requests.

Layered System: Enables clients to connect to the end server through an intermediate layer for improved scalability.

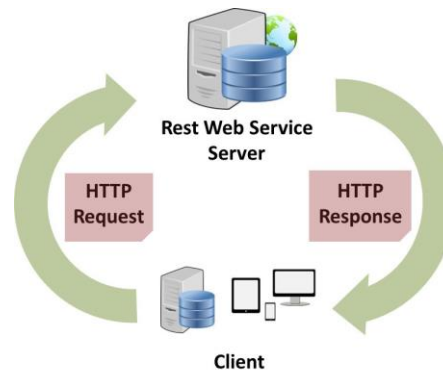


Figure 2.4 Representation state transfer architecture

2.2.1 The basic REST design principle uses the HTTP protocol methods for typical CRUD operations:

- POST - Create a resource
- GET - Retrieve a resource
- PUT – Update a resource
- DELETE - Delete a resource

The major advantages of REST-services are:

- They are highly reusable across platforms (Java, .NET, PHP, etc) since they rely on basic HTTP protocol
- They use basic XML instead of the complex SOAP XML and are easily consumable
- REST-based web services are increasingly being preferred for integration with backend enterprise services.
- In comparison to SOAP based web services, the programming model is simpler and the use of native XML instead of SOAP reduces the serialization and deserialization complexity as well as the need for additional third-party libraries for the same.

2.2.2 Overview of Architecture

In J2EE applications, the Java API or services are exposed as either Stateless Session Bean API (Session Façade pattern) or as SOAP web services. In case of integration of these services with client applications using non-Java technology like .NET or PHP etc, it becomes very cumbersome to work with SOAP Web Services and also involves considerable development effort.

The approach mentioned here is typically intended for service integrations within the organization where there are many services which can be reused but the inter-operability and development costs using SOAP create a barrier for quick integrations. Also, in scenarios where a service is not intended to be exposed on the enterprise ESB or EAI by the internal Governance organization, it becomes difficult to integrate 2 diverse-technology services in a point-to-point manner.

For example – In a telecom IT environment:

- Sending an SMS to the circle-specific SMSC's which is exposed as a SOAP web service or an EJB API; Or
- Creating a Service Request in a CRM application exposed as a Database stored procedure (e.g. Oracle CRM) exposed over ESB using MQ or JMS bindings; Or
- Creating a Sales Order request for a Distributor from a mobile SMS using the SMSGateway.
- If above services are to be used by a non-Java application, then the integration using SOAP web services will be cumbersome and involve extended development.

This new approach has been implemented in the form of a framework so that it can be reused in other areas where a Java Service can be exposed as a REST-like resource.

The architecture consists of a Front Controller which acts as the central point for receiving requests and providing response to the clients. The Front Controller delegates the request processing to the ctionController which contains the processing logic of this framework. The ActionController performs validation, maps the request to the appropriate Action and invokes the action to generate response. Various Helper Services are provided for request processing, logging and exception handling which can be used by the ActionController as well as the individual Actions.

Service Client

This is a client application which needs to invoke the service. This component can be either Java-based or any other client as long as it is able to support the HTTP methods

Common Components

These are the utility services required by the framework like logging, exception handling and any common functions or constants required for implementation. Apache Commons logging with Log4j implementation is used in the sample code.

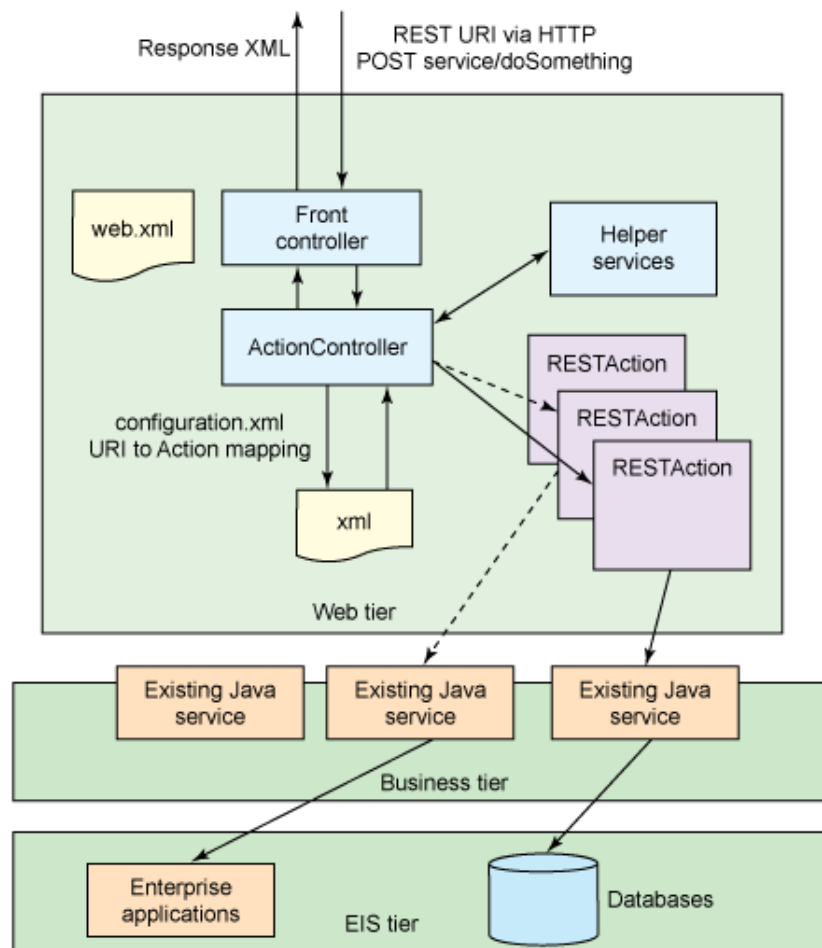


Figure 2.3 REST-Like enablement framework

RESTServiceServlet

The framework uses the Front Controller pattern for centralized request processing and uses this Java Servlet component for processing the input requests. It supports common HTTP methods like GET, PUT, POST and DELETE.

RESTActionController

This component is the core framework controller which manages the core functionality of loading the services and framework configuration, validation of requests and mapping the requests with configured REST actions and executing the actions.

RESTConfiguration

This component is responsible for loading and caching the framework configuration as well as the various REST services configuration at run-time. This component is used by the RESTActionController to identify the correct action to be called for a request as well as validate the input request.

RESTMapping

This component stores the REST action mappings specified in the configuration file. The mapping primarily consists of the URI called by client and the action class which does the processing.

ActionContext

This component encapsulates all the features required for execution of the REST action. It assists developers in providing request and response handling features so that the developer has to only code the actual business logic implementation. It hides the protocol specific request and response objects from the Action component and hence allows independent testing of the same like a POJO. It also provides a handle to the XML Binding Service so that Java business objects can be easily converted to XML and vice-versa based on the configured XML Binding API. The RESTActionController configures this component dynamically and provides it to the Action component.

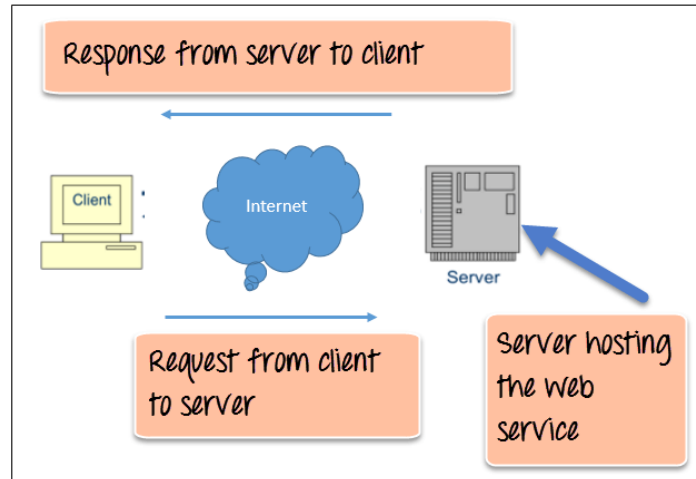
2.3 WEB SERVICES

- **What is Web Service?**
- **Type of Web Service**
- **Web Services Advantages**

- **Web Service Architecture**
- **Web Service Characteristics**

What is Web Service?

- Web service is a standardized medium to propagate communication between the client and server applications on the World Wide Web.
- A web service is a software module which is designed to perform a certain set of tasks.
- The web services can be searched for over the network and can also be invoked accordingly.
- When invoked the web service would be able to provide functionality to the client which invokes that web service.



Web Service Architecture Diagram

- The above diagram shows a very simplistic view of how a web service would actually work. The client would invoke a series of web service calls via requests to a server which would host the actual web service.
- These requests are made through what is known as remote procedure calls. Remote Procedure Calls(RPC) are calls made to methods which are hosted by the relevant web service.
- As an example, Amazon provides a web service that provides prices for products sold online via amazon.com. The front end or presentation layer can be in .Net or Java but either programming language would have the ability to communicate with the web service.
- The main component of a web service is the data which is transferred between the client and the server, and that is XML. XML (Extensible markup language) is a counterpart to HTML and easy to understand the intermediate language that is understood by many programming languages.
- So when applications talk to each other, they actually talk in XML. This provides a common platform for application developed in various programming languages to talk to each other.
- Web services use something known as SOAP (Simple Object Access Protocol) for sending the XML data between applications. The data is sent over normal HTTP.
- The data which is sent from the web service to the application is called a SOAP message. The SOAP message is nothing but an XML document. Since the document is written in XML, the client application calling the web service can be written in any programming language.

2.3.1 Type of Web Service

There are mainly two types of web services.

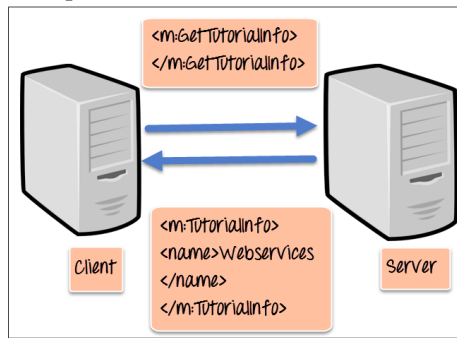
- SOAP web services.
- RESTful web services.

In order for a web service to be fully functional, there are certain components that need to be in place. These components need to be present irrespective of whatever development language is used for programming the web service.

SOAP (Simple Object Access Protocol)

- SOAP is known as a transport-independent messaging protocol. SOAP is based on transferring XML data as SOAP Messages. Each message has something which is known as an XML document.
- Only the structure of the XML document follows a specific pattern, but not the content. The best part of Web services and SOAP is that its all sent via HTTP, which is the standard web protocol.
- Each SOAP document needs to have a root element known as the <Envelope> element. The root element is the first element in an XML document.
- The "envelope" is in turn divided into 2 parts. The first is the header, and the next is the body.

- The header contains the routing data which is basically the information which tells the XML document to which client it needs to be sent to.
- The body will contain the actual message.
- The diagram below shows a simple example of the communication via SOAP.



WSDL (Web services description language)

- The client invoking the web service should know where the web service actually resides.
- Secondly, the client application needs to know what the web service actually does, so that it can invoke the right web service.
- This is done with the help of the WSDL, known as the Web services description language.
- The WSDL file is again an XML-based file which basically tells the client application what the web service does. By using the WSDL document, the client application would be able to understand where the web service is located and how it can be utilized.

Web Service Example

An example of a WSDL file is given below.

```
<definitions>
  <message name="TutorialRequest">
    <part name="TutorialID" type="xsd:string"/>
  </message>
  <message name="TutorialResponse">
    <part name="TutorialName" type="xsd:string"/>
  </message>
  <portType name="Tutorial_PortType">
    <operation name="Tutorial">
      <input message="tns:TutorialRequest"/>
      <output message="tns:TutorialResponse"/>
    </operation>
  </portType>
  <binding name="Tutorial_Binding" type="tns:Tutorial_PortType">
    <soap:binding style="rpc"
      transport="http://schemas.xmlsoap.org/soap/http"/>
    <operation name="Tutorial">
      <soap:operation soapAction="Tutorial"/>
      <input>
        <soap:body
          encodingStyle="http://schemas.xmlsoap.org/soap/encoding/"
          namespace="urn:examples:Tutorialservice"
          use="encoded"/>
      </input>
      <output>
        <soap:body
          encodingStyle="http://schemas.xmlsoap.org/soap/encoding/"
          namespace="urn:examples:Tutorialservice"
          use="encoded"/>
      </output>
    </operation>
  </binding>
</definitions>
```

The important aspects to note about the above WSDL declaration are as follows;

<message> - The message parameter in the WSDL definition is used to define the different data elements for each operation performed by the web service. So in the example above, we have 2 messages which can be exchanged between the web service and the client application, one is the "TutorialRequest", and the other is the "TutorialResponse" operation. The TutorialRequest contains an element called "TutorialID" which is of the type string. Similarly, the TutorialResponse operation contains an element called "TutorialName" which is also a type string.

<portType> - This actually describes the operation which can be performed by the web service, which in our case is called Tutorial. This operation can take 2 messages; one is an input message, and the other is the output message.

<binding> - This element contains the protocol which is used. So in our case, we are defining it to use http (<http://schemas.xmlsoap.org/soap/http>). We also specify other details for the body of the operation, like the namespace and whether the message should be encoded.

Universal Description, Discovery, and Integration (UDDI)

- UDDI is a standard for describing, publishing, and discovering the web services that are provided by a particular service provider. It provides a specification which helps in hosting the information on web services.
- Now we discussed in the previous topic about WSDL and how it contains information on what the Web service actually does.
- But how can a client application locate a WSDL file to understand the various operations offered by a web service? So UDDI is the answer to this and provides a repository on which WSDL files can be hosted.
- So the client application will have complete access to the UDDI, which acts as a database containing all the WSDL files.
- Just as a telephone directory has the name, address and telephone number of a particular person, the same way the UDDI registry will have the relevant information for the web service.

2.3.2 Web Services Advantages

We already understand why web services came about in the first place, which was to provide a platform which could allow different applications to talk to each other.

Exposing Business Functionality on the network - A web service is a unit of managed code that provides some sort of functionality to client applications or end users. This functionality can be invoked over the HTTP protocol which means that it can also be invoked over the internet. Nowadays all applications are on the internet which makes the purpose of Web services more useful. That means the web service can be anywhere on the internet and provide the necessary functionality as required.

Interoperability amongst applications - Web services allow various applications to talk to each other and share data and services among themselves. All types of applications can talk to each other. So instead of writing specific code which can only be understood by specific applications, you can now write generic code that can be understood by all applications

A Standardized Protocol which everybody understands - Web services use standardized industry protocol for the communication. All the four layers (Service Transport, XML Messaging, Service Description, and Service Discovery layers) uses well-defined protocols in the web services protocol stack.

Reduction in cost of communication - Web services use SOAP over HTTP protocol, so you can use your existing low-cost internet for implementing web services.

Web service Architecture

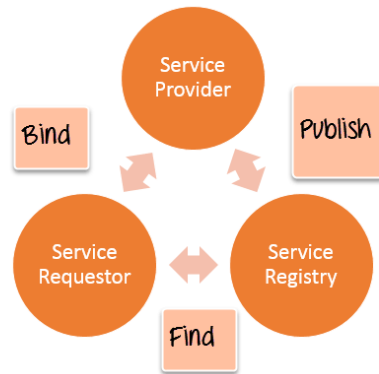
Every framework needs some sort of architecture to make sure the entire framework works as desired. Similarly, in web services, there is an architecture which consists of three distinct roles as given below

Provider - The provider creates the web service and makes it available to client application who want to use it.

Requestor - A requestor is nothing but the client application that needs to contact a web service. The client application can be a .Net, Java, or any other language based application which looks for some sort of functionality via a web service.

Broker - The broker is nothing but the application which provides access to the UDDI. The UDDI, as discussed in the earlier topic enables the client application to locate the web service.

The diagram below showcases how the Service provider, the Service requestor and Service registry interact with each other.



Publish - A provider informs the broker (service registry) about the existence of the web service by using the broker's publish interface to make the service accessible to clients

Find - The requestor consults the broker to locate a published web service

Bind - With the information it gained from the broker(service registry) about the web service, the requestor is able to bind, or invoke, the web service.

2.4 PUBLISH-SUBSCRIBE MODEL

Pub/Sub brings the flexibility and reliability of enterprise message-oriented middleware to the cloud. At the same time, Pub/Sub is a scalable, durable event ingestion and delivery system that serves as a foundation for modern stream analytics pipelines. By providing many-to-many, asynchronous messaging that decouples senders and receivers, it allows for secure and highly available communication among independently written applications. Pub/Sub delivers low-latency, durable messaging that helps developers quickly integrate systems hosted on the Google Cloud Platform and externally.

Publish-subscribe (pub/sub) is a messaging pattern where publishers push messages to subscribers. In software architecture, pub/sub messaging provides instant event notifications for distributed applications, especially those that are decoupled into smaller, independent building blocks. In laymen's terms, pub/sub describes how two different parts of a messaging pattern connect and communicate with each other.

How Pub/Sub Works

Publish/ Subscribe Pattern



Figure 2.3 Pub/Sub Pattern

These are three central components to understanding pub/sub messaging pattern:

Publisher: Publishes messages to the communication infrastructure

Subscriber: Subscribes to a category of messages

Communication infrastructure (channel, classes): Receives messages from publishers and maintains subscribers' subscriptions.

The publisher will categorize published messages into classes where subscribers will then receive the message. Figure 2.3 offers an illustration of this messaging pattern. Basically, a publisher has one input channel that splits into multiple output channels, one for each subscriber. Subscribers can express interest in one or more classes and only receive messages that are of interest.

The thing that makes pub/sub interesting is that the publisher and subscriber are unaware of each other. The publisher sends messages to subscribers, without knowing if there are any actually there. And the subscriber receives messages, without explicit knowledge of the publishers out there. If there are no subscribers around to receive the topic-based information, the message is dropped.

2.4.1 Core concepts

Topic: A named resource to which messages are sent by publishers.

Subscription: A named resource representing the stream of messages from a single, specific topic, to be delivered to the subscribing application. For more details about subscriptions and message delivery semantics, see the **Subscriber Guide**.

Message: The combination of data and (optional) attributes that a publisher sends to a topic and is eventually delivered to subscribers.

Message attribute: A key-value pair that a publisher can define for a message. For example, key `iana.org/language_tag` and value `en` could be added to messages to mark them as readable by an English-speaking subscriber.

Publisher-subscriber relationships

A publisher application creates and sends messages to a topic. Subscriber applications create a subscription to a topic to receive messages from it. Communication can be one-to-many (fan-out), many-to-one (fan-in), and many-to-many.

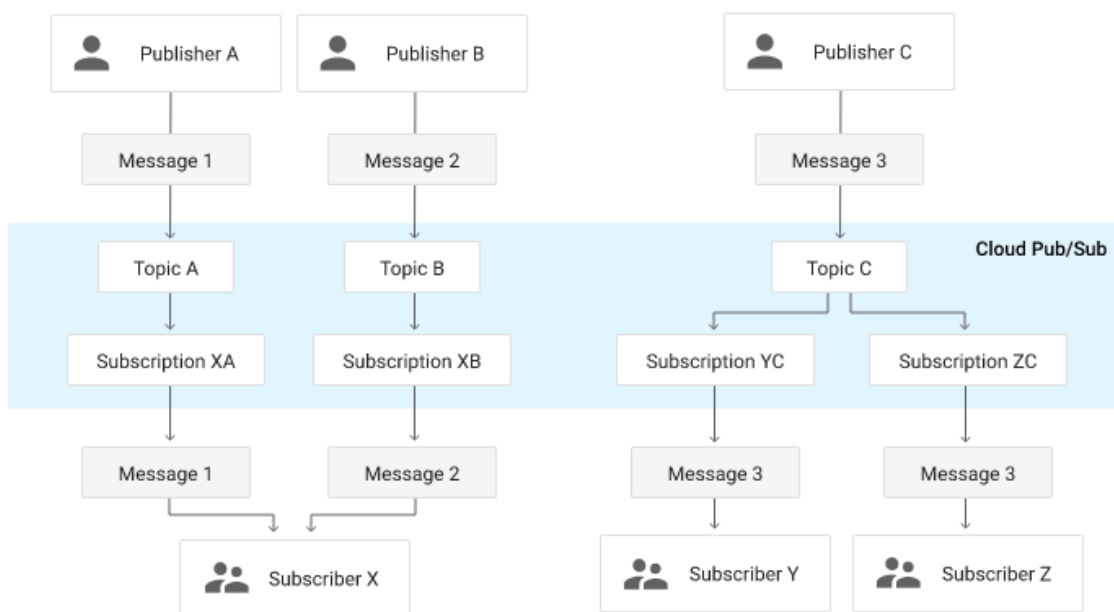


Figure 2.4 Pub/Sub relationship diagram

Common use cases

Balancing workloads in network clusters. For example, a large queue of tasks can be efficiently distributed among multiple workers, such as Google Compute Engine instances.

Implementing asynchronous workflows. For example, an order processing application can place an order on a topic, from which it can be processed by one or more workers.

Distributing event notifications. For example, a service that accepts user signups can send notifications whenever a new user registers, and downstream services can subscribe to receive notifications of the event.

Refreshing distributed caches. For example, an application can publish invalidation events to update the IDs of objects that have changed.

Logging to multiple systems. For example, a Google Compute Engine instance can write logs to the monitoring system, to a database for later querying, and so on.

Data streaming from various processes or devices. For example, a residential sensor can stream data to backend servers hosted in the cloud.

Reliability improvement. For example, a single-zone Compute Engine service can operate in additional zones by subscribing to a common topic, to recover from failures in a zone or region.

Content-Based Pub-Sub Models

In the publish-subscribe model, filtering is used to process the selection of messages for reception and processing, with the two most common being topic-based and content-based.

In a **topic-based system**, messages are published to named channels (topics). The publisher is the one who creates these channels. Subscribers subscribe to those topics and will receive messages from them whenever they appear.

In a **content-based system**, messages are only delivered if they match the constraints and criteria that are defined by the subscriber.

2.5 BASICS OF VIRTUALIZATION

The term 'Virtualization' can be used in many respect of computer. It is the process of creating a virtual environment of something which may include hardware platforms, storage devices, OS, network resources, etc. The cloud's virtualization mainly deals with the server virtualization.

Virtualization is the ability which allows sharing the physical instance of a single application or resource among multiple organizations or users. This technique is done by assigning a name logically to all those physical resources & provides a pointer to those physical resources based on demand.

Over an existing operating system & hardware, we generally create a virtual machine which and above it we run other operating systems or applications. This is called Hardware Virtualization. The virtual machine provides a separate environment that is logically distinct from its underlying hardware. Here, the system or the machine is the host & virtual machine is the guest machine.

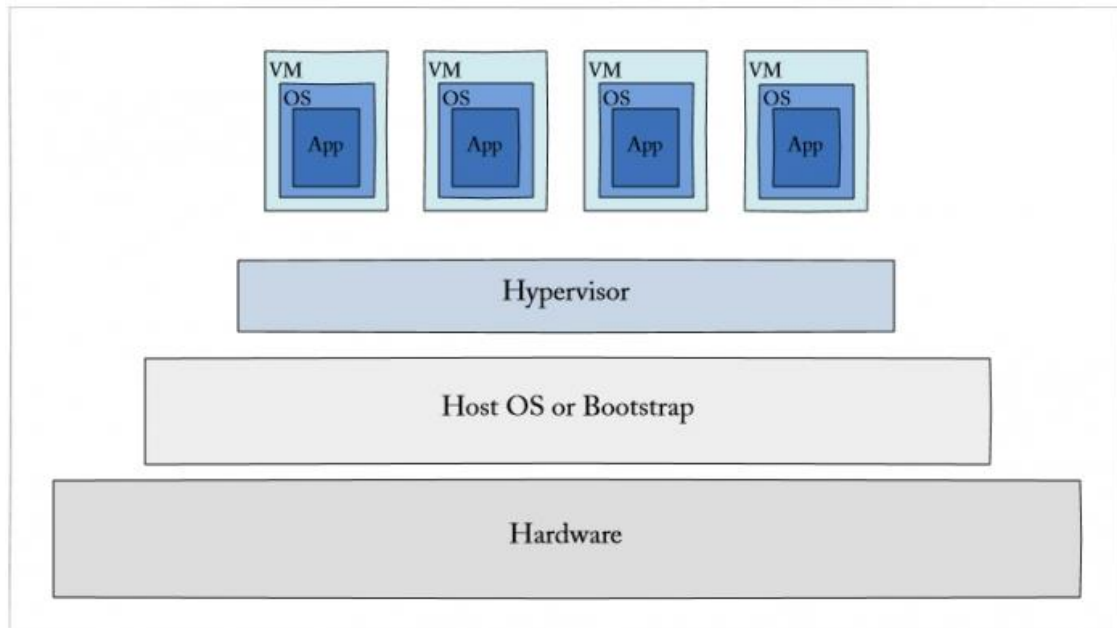


Figure - The Cloud's Virtualization

There are several approaches or ways to virtualizes cloud servers.

These are:

Grid Approach: where the processing workloads are distributed among different physical servers, and their results are then collected as one.

OS - Level Virtualization: Here, multiple instances of an application can run in an isolated form on a single OS

Hypervisor-based Virtualization: which is currently the most widely used technique With hypervisor's virtualization, there are various sub-approaches to fulfill the goal to run multiple applications & other loads on a single physical host. A technique is used to allow virtual machines to move from one host to another without any requirement of shutting down. This technique is termed as "Live Migration". Another technique is used to actively load balance among multiple hosts to efficiently utilize those resources available in a virtual machine, and the concept is termed as Distributed Resource Scheduling or Dynamic Resource Scheduling.

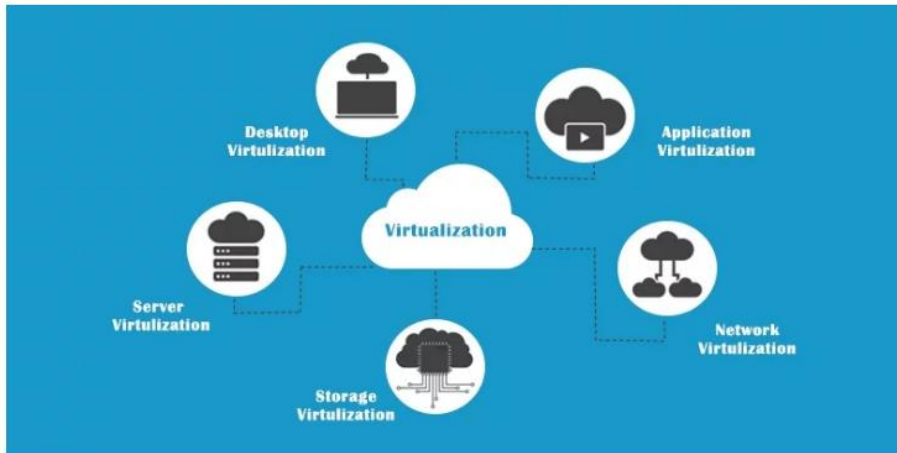
2.5.1 VIRTUALIZATION

Virtualization is the process of creating a virtual environment on an existing server to run your desired program, without interfering with any of the other services provided by the server or host platform to other users. The Virtual environment can be a single instance or a combination of many such as operating systems, Network or Application servers, computing environments, storage devices and other such environments.

Virtualization in Cloud Computing is making a virtual platform of server operating system and storage devices. This will help the user by providing multiple machines at the same time it also allows sharing a single physical instance of resource or an application to multiple users. Cloud Virtualizations also manage the workload by transforming traditional computing and make it more scalable, economical and efficient.

TYPES OF VIRTUALIZATION

- i. Operating System Virtualization
- ii. Hardware Virtualization
- iii. Server Virtualization
- iv. Storage Virtualization



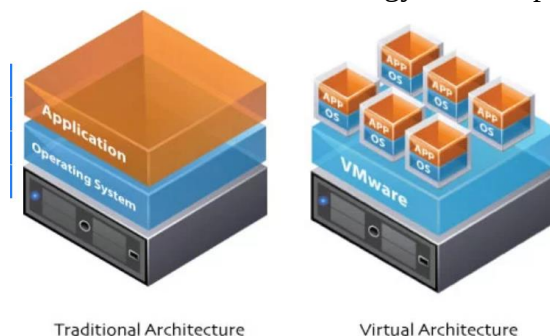
Virtualization Architecture

Benefits for Companies

- Removal of special hardware and utility requirements
- Effective management of resources
- Increased employee productivity as a result of better accessibility
- Reduced risk of data loss, as data is backed up across multiple storage locations

Benefits for Data Centers

- Maximization of server capabilities, thereby reducing maintenance and operation costs
- Smaller footprint as a result of lower hardware, energy and manpower requirements



Access to the virtual machine and the host machine or server is facilitated by a software known as Hypervisor. Hypervisor acts as a link between the hardware and the virtual environment and distributes the hardware resources such as CPU usage, memory allotment between the different virtual environments.

Hardware Virtualization

Hardware virtualization also known as hardware-assisted virtualization or server virtualization runs on the concept that an individual independent segment of hardware or a physical server, may be made up of multiple smaller hardware segments or servers, essentially consolidating multiple physical servers into virtual servers that run on a single primary physical server. Each small server can host a virtual machine, but the entire cluster of servers is treated as a single device by any process requesting the hardware. The hardware resource allotment is done by the hypervisor. The main advantages include increased processing power as a result of maximized hardware utilization and application uptime.

Subtypes:

Full Virtualization – Guest software does not require any modifications since the underlying hardware is fully simulated.

Emulation Virtualization – The virtual machine simulates the hardware and becomes independent of it. The guest operating system does not require any modifications.

Para virtualization – the hardware is not simulated and the guest software runs their own isolated domains.

Software Virtualization

Software Virtualization involves the creation of an operation of multiple virtual environments on the host machine. It creates a computer system complete with hardware that lets the guest operating system to run. For example, it lets you run Android OS on a host machine natively using a Microsoft Windows OS, utilizing the same hardware as the host machine does.

Subtypes:

Operating System Virtualization – hosting multiple OS on the native OS

In operating system virtualization in Cloud Computing, the virtual machine software installs in the operating system of the host rather than directly on the hardware system. The most important use of operating system virtualization is for testing the application on different platforms or operating system. Here, the software is present in the hardware, which allows different applications to run.

Application Virtualization – hosting individual applications in a virtual environment separate from the native OS.

Service Virtualization – hosting specific processes and services related to a particular application.

Server Virtualization

In server virtualization in Cloud Computing, the software directly installs on the server system and use for a single physical server can divide into many servers on the demand basis and balance the load. It can be also stated that the server virtualization is masking of the server resources which consists of number and identity. With the help of software, the server administrator divides one physical server into multiple servers.

Memory Virtualization

Physical memory across different servers is aggregated into a single virtualized memory pool. It provides the benefit of an enlarged contiguous working memory. You may already be familiar with this, as some OS such as Microsoft Windows OS allows a portion of your storage disk to serve as an extension of your RAM.

Subtypes:

Application-level control – Applications access the memory pool directly

Operating system level control – Access to the memory pool is provided through an operating system

Storage Virtualization

Multiple physical storage devices are grouped together, which then appear as a single storage device. This provides various advantages such as homogenization of storage across storage devices of multiple capacity and speeds, reduced downtime, load balancing and better optimization of performance and speed. Partitioning your hard drive into multiple partitions is an example of this virtualization.

Subtypes:

Block Virtualization – Multiple storage devices are consolidated into one

File Virtualization – Storage system grants access to files that are stored over multiple hosts

Data Virtualization

It lets you easily manipulate data, as the data is presented as an abstract layer completely independent of data structure and database systems. Decreases data input and formatting errors.

Network Virtualization

In network virtualization, multiple sub-networks can be created on the same physical network, which may or may not is authorized to communicate with each other. This enables restriction of file movement across networks and enhances security, and allows better monitoring and identification of data usage which lets the network administrator's scale up the network appropriately. It also increases reliability as a disruption in one network doesn't affect other networks, and the diagnosis is easier.

Hardware Virtualization

Hardware virtualization in Cloud Computing, used in server platform as it is flexible to use Virtual Machine rather than physical machines. In hardware virtualizations, virtual machine software installs in the hardware system and then it is known as hardware virtualization. It consists of a hypervisor which use to control and monitor the process, memory, and other hardware resources. After the completion of hardware virtualization process, the user can install the different operating system in it and with this platform different application can use.

Storage Virtualization

In storage virtualization in Cloud Computing, a grouping is done of physical storage which is from multiple network storage devices this is done so it looks like a single storage device. It can implement with the help of software applications and storage virtualization is done for the backup and recovery process. It is a sharing of the physical storage from multiple storage devices.

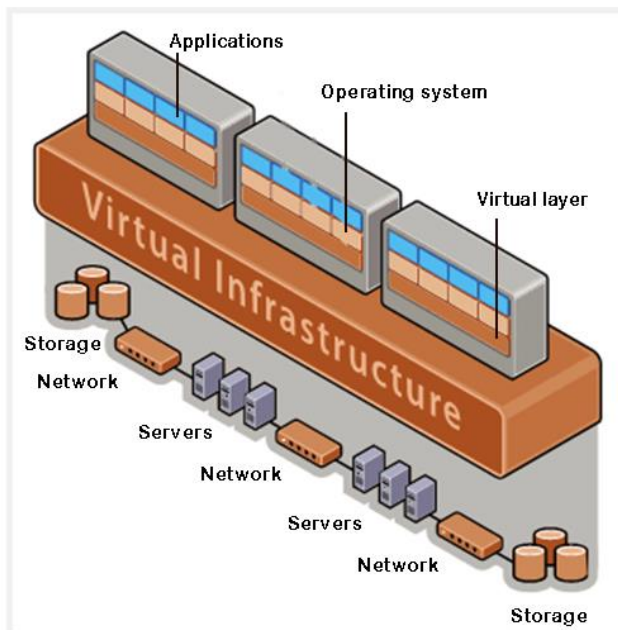
2.5.2 Subtypes:

Internal network: Enables a single system to function like a network

External network: Consolidation of multiple networks into a single one, or segregation of a single network into multiple ones.

Desktop Virtualization

This is perhaps the most common form of virtualization for any regular IT employee. The user's desktop is stored on a remote server, allowing the user to access his desktop from any device or location. Employees can work conveniently from the comfort of their home. Since the data transfer takes place over secure protocols, any risk of data theft is minimized.



Benefits of Virtualization

Virtualizations in Cloud Computing has numerous benefits, let's discuss them one by one:



i. Security

During the process of virtualization security is one of the important concerns. The security can be provided with the help of firewalls, which will help to prevent unauthorized access and will keep the data confidential. Moreover, with the help of firewall and security, the data can protect from harmful viruses malware and other cyber threats. Encryption process also takes place with protocols which will protect the data from other threads. So, the customer can virtualize all the data store and can create a backup on a server in which the data can store.

ii. Flexible operations

With the help of a virtual network, the work of it professional is becoming more efficient and agile. The network switch implement today is very easy to use, flexible and saves time. With the help of virtualization in Cloud Computing, technical problems can solve in physical systems. It eliminates the problem of recovering the data from crashed or corrupted devices and hence saves time.

iii. Economical

Virtualization in Cloud Computing, save the cost for a physical system such as hardware and servers. It stores all the data in the virtual server, which are quite economical. It reduces the wastage, decreases the electricity bills along with the maintenance cost. Due to this, the business can run multiple operating system and apps in a particular server.

iv. Eliminates the risk of system failure

While performing some task there are chances that the system might crash down at the wrong time. This failure can cause damage to the company but the virtualizations help you to perform the same task in multiple devices at the same time. The data can store in the cloud it can retrieve anytime and with the help of any device. Moreover, there is two working server side by side which makes the data accessible every time. Even if a server crashes with the help of the second server the customer can access the data.

v. Flexible transfer of data

The data can transfer to the virtual server and retrieve anytime. The customers or cloud provider don't have to waste time finding out hard drives to find data. With the help of virtualization, it will very easy to locate the required data and transfer them to the allotted authorities. This transfer of data has no limit and can transfer to a long distance with the minimum charge possible. Additional storage can also provide and the cost will be as low as possible.

Which Technology to use?

Virtualization is possible through a wide range of Technologies which are available to use and are also OpenSource. We prefer using XEN or KVM since they provide the best virtualization experience and performance.

- XEN
- KVM
- OpenVZ

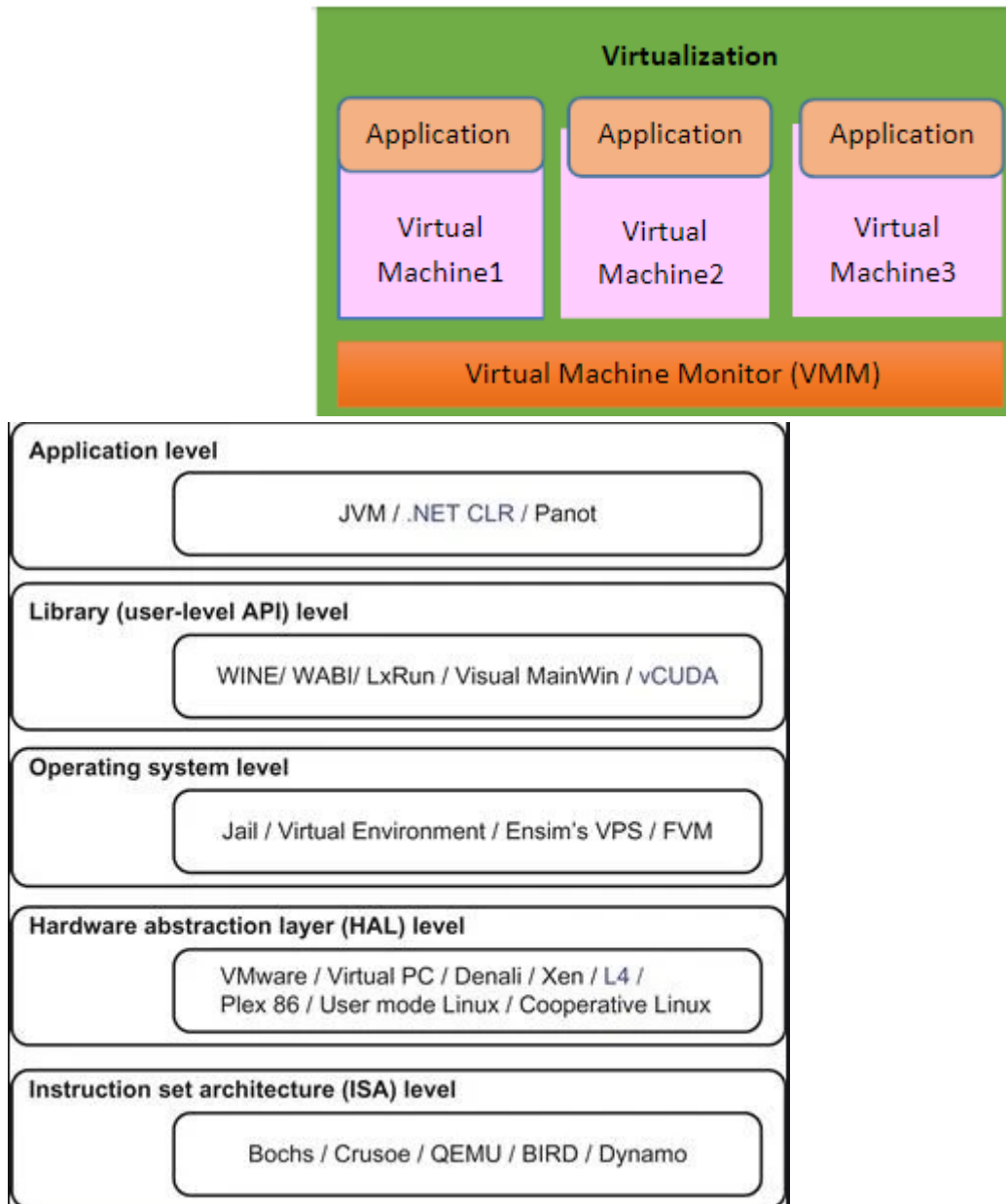
Conclusion

With the help of Virtualization in Cloud Computing, companies can implement cloud computing. This article proves that virtualization in Cloud computing is an important aspect in cloud computing and can maintain and secure the data. virtualization lets you easily outsource your hardware and eliminate any energy costs associated with its operation. Although it may not work for everyone, however the efficiency, security and cost advantages are considerable for you to consider employing it as part of your operations. But whatever type of virtualization you may need, always look for service providers that provide straightforward tools to manage your

RESOURCES AND MONITOR USAGE

2.6 LEVELS OF VIRTUALIZATION IMPLEMENTATION.

- a) Instruction Set Architecture Level.
- b) Hardware Abstraction Level.
- c) Operating System Level.
- d) Library Support Level.
- e) User-Application Level.



Virtualization at ISA (instruction set architecture) level

Virtualization is implemented at ISA (Instruction Set Architecture) level by transforming physical architecture of system's instruction set into software completely. The host machine is a physical platform containing various components, such as process, memory, Input/output (I/O) devices, and buses. The VMM installs the guest systems on this machine. The emulator gets the instructions from the guest systems to process and execute. The emulator transforms those instructions into native instruction set, which are run on host machine's hardware. The instructions include both the I/O-specific ones and the processor-oriented instructions. For an emulator to be efficacious, it has to imitate all tasks that a real computer could perform.

Advantages:

It is a simple and strong method of conversion into virtual architecture. On a single physical structure, this architecture makes simple to implement multiple systems on single physical structure. The instructions given by the guest system is translated into instructions of the host system. This architecture makes the host system to adjust to the changes in architecture of the guest system. The binding between the guest system and the host is not rigid, but making it very flexible. The infrastructure of this kind could be used for creating virtual machines on platform, for example: X86 on any platform such as Sparc, X86, Alpha, etc.

Disadvantage: The instructions should be interpreted before being executed. And therefore the system with ISA level of virtualization shows poor performance.

Virtualization at HAL (hardware abstraction layer) level

The virtualization at the HAL (Hardware Abstraction Layer) is the most common technique, which is used in computers on x86 platforms that increases the efficiency of virtual machine to handle various tasks. This architecture becomes economical and relatively for practical use. In case, if emulator communication is required to the critical processes, the simulator undertakes the tasks and it performs the appropriate multiplexing. The working of virtualization technique wants catching the execution of privileged instructions by virtual machine, which passes these instructions to VMM to be handled properly. This is necessary because of the possible existence of multiple virtual machines, each having its own OS that could issue separate privileged instructions. Execution of privileged instructions needed complete attention of CPU. If, this is not managed properly by VMM, and it will raise an exception, which will result in system crash. Trapping and forwarding the instructions to VMM, helps in managing the system suitably, and thereby avoiding different risks. We cannot fully virtualized all platforms, with the help of this technique. Even in X86 platform, it is detected that some privileged instructions fail without being trapped, because their execution is not privileged appropriately. Such occurrences need some workaround in virtualization technique, to pass the control of such execution of fault instructions to the VMM, which would handle them properly. Code Scanning and dynamic instruction rewriting are some of examples of the techniques to enable VMM to have control of execution of fault privileged instructions.

Virtualization at OS (operating system) level

To overcome redundancy and time consumption issues, virtualization at the operating system level is implemented. This kind of technique includes the sharing of both the OS and hardware. The physical machine is being separated from logical structure (virtual system) by separate virtualization layer, which could be compared with VMMs in functions. This layer is built on the top of the OS, which could enable the user to access to multiple machines, which is isolated from others and is running independently. The virtualization technique at the OS level, keeps the environment for running of applications properly. IT keeps the OS, the user-libraries, application-specific data structures separately. Thus, the application is not able to differentiate between the virtual environment (VE) and the real. The main idea behind OS level virtualization implementation is that the virtual environment rests indistinguishable from the real one. The virtualization layer imitates the operating environment, which is recognized on the physical machine, in order to provide a Virtual environment for application, thus by making partitions of each virtual system, whenever needed. An orderly managed partitioning and multiplexing permits to disseminate complete operating environments, which are separated from physical machine.

Virtualization at library level

Programming the applications in more systems needs a widespread list of Application Program Interfaces (APIs) to be disseminated by implementing several libraries at user level. These APIs are used to save users from miniature details involved with programming related to the OS and facilitate the programmers to write programs more easily. At the user level library operation, a different virtual environment is provided, in that kind of perception. This virtual environment is created above the OS layer, which could expose a different class of binary interfaces together. This type of virtualization is well-defined as an implementation of various set of ABIs (Application Binary Interfaces). The APIs are being implemented with the help of the base system and execute the function of ABI/API emulation.

Virtualization at application level

The user level programs and the operating systems are executed on applications, which behave like real machines. The memory mapped I/O processing technique or I/O mapped input/output processing is used to deal with hardware. Thus, an application might be taken simply as a block of instructions, which are being executed on a machine. The Java Virtual Machine (JVM) carried a new aspect to virtualization and it is known as application level virtualization. The main concept after this type of virtualization is to produce a virtual machine that works distinctly at the application level and functions in a way similar as a normal machine. We can run our applications on those virtual machines as if we are running our applications on physical machines. This type faces a little threat to the security of the system. However, these machines

should have an operating environment delivered to the applications in the form of a separate environment or in the form of a hosted OS of their own.

A comparison between implementation levels of virtualization

Various implementation levels of virtualization have their own set of merits and demerits. For example, ISA level virtualization gives high flexibility for the applications but the performance is very poor. Likewise, the other levels such as HAL-level, OS-level, Library and Application Level also have both negatives and positives. The OS-level and the HAL-level virtualizations are the best on performance, but the implementations are complex and the applications flexibility is also not very good. The Application level implementation provides larger application isolation feature, but low flexibility, poor performance and high implementation complexity makes it less desirable. Library level virtualizations have medium performance, medium complexity, but poor isolation feature and low flexibility.

Table 1: Comparison of Implementation Levels of Virtualization

Implementa- tion Level	Perfor- mance	Implementa- tion Com- plexity	Applica- tion flexi- bility	Applica- tion Isola- tion
ISA	Very poor	Medium	Excellent	Medium
HAL	Excellent	High	Medium	Very Good
OS Level	Excellent	Medium	Low	Very Poor
Application Level	Medium	Low	Low	Very Poor
Library Level	Poor	High	Low	Excellent

Requirements for virtualization design The design of virtual systems becomes indistinct with OSs, which have functionalities comparable to virtual systems. So we need to have definite dissimilarities in the design of virtualized systems. The virtualized design requirements are generally viewed as follows:

Equivalence requirement

A machine which is developed through virtualization should have a logical equivalence with real machines. The emulator should match the capabilities of physical system in terms of computational performance. The emulator system should be able to execute all applications and the programs are designed to execute on real machines with certain exception of timing.

Resource control requirement

A computer is a combination of resources such as memory, processors and I/O devices. These resources must be controlled and managed effectively by VMM. The VMM must enforce isolation between virtualized systems. The virtual machines should not face any interference.

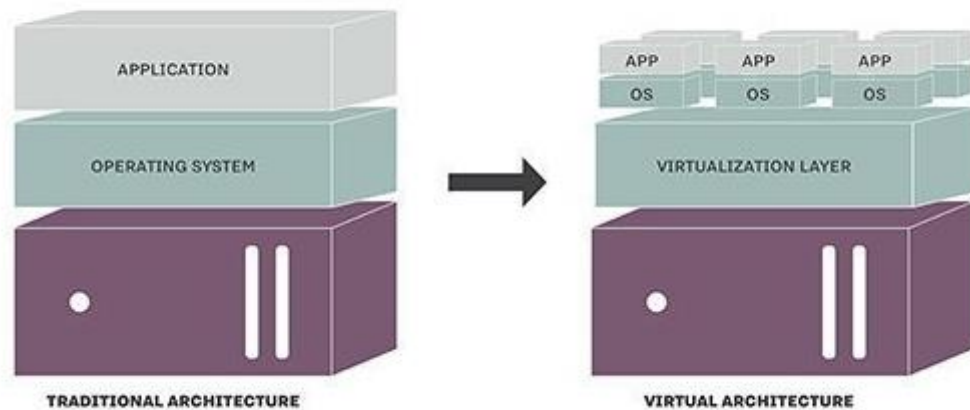
Efficiency requirement

The virtual machines must be efficient in performance as the real system. Virtualization is done with the purpose to get proficient software without physical hardware. Thus, the emulator should be capable of interpreting all the instructions that might be interpreted safely in a physical system.

2.7 VIRTUALIZATION STRUCTURES

A virtualization architecture is a conceptual model specifying the arrangement and interrelationships of the particular components involved in delivering a virtual -- rather than physical -- version of something, such as an operating system (OS), a server, a storage device or network resources.

TRADITIONAL AND VIRTUAL ARCHITECTURE



Virtualization is commonly hypervisor-based. The hypervisor isolates operating systems and applications from the underlying computer hardware so the host machine can run multiple virtual machines (VM) as guests that share the system's physical compute resources, such as processor cycles, memory space, network bandwidth and so on.

Type 1 hypervisors, sometimes called bare-metal hypervisors, run directly on top of the host system hardware. Bare-metal hypervisors offer high availability and resource management. Their direct access to system hardware enables better performance, scalability and stability. Examples of type 1 hypervisors include Microsoft Hyper-V, Citrix XenServer and VMware ESXi.

A type 2 hypervisor, also known as a hosted hypervisor, is installed on top of the host operating system, rather than sitting directly on top of the hardware as the type 1 hypervisor does. Each guest OS or VM runs above the hypervisor. The convenience of a known host OS can ease system configuration and management tasks. However, the addition of a host OS layer can potentially limit performance and expose possible OS security flaws. Examples of type 2 hypervisors include VMware Workstation, Virtual PC and Oracle VM VirtualBox.

The main alternative to hypervisor-based virtualization is containerization. Operating system virtualization, for example, is a container-based kernel virtualization method. OS virtualization is similar to partitioning. In this architecture, an operating system is adapted so it functions as multiple, discrete systems, making it possible to deploy and run distributed applications without launching an entire VM for each one. Instead, multiple isolated systems, called containers, are run on a single control host and all access a single kernel.

2.8 VIRTUALIZATION STRUCTURES/TOOLS AND MECHANISMS

In general, there are three typical classes of VM architecture.

virtualization, and host-based virtualization.

The hypervisor supports hardware-level virtualization like CPU, memory, disk and network interfaces.

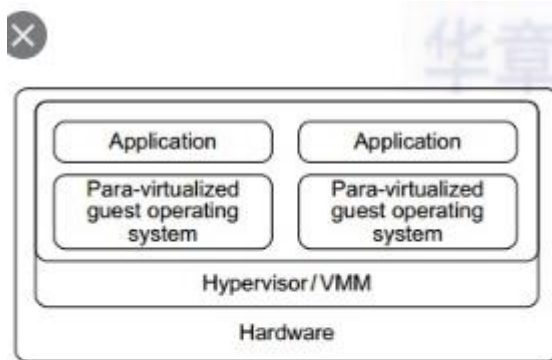


FIGURE 3.7

Para-virtualized VM architecture, which involves modifying the guest OS kernel to replace nonvirtualizable instructions with hypercalls for the hypervisor or the VMM to carry out the virtualization

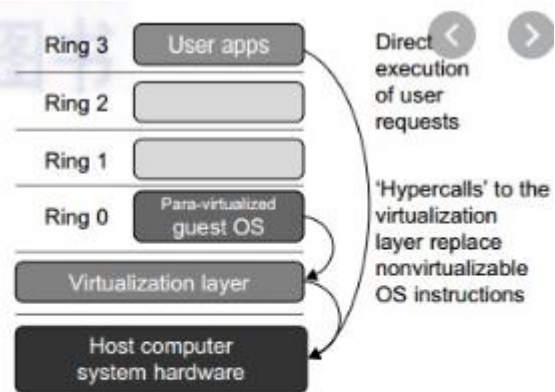
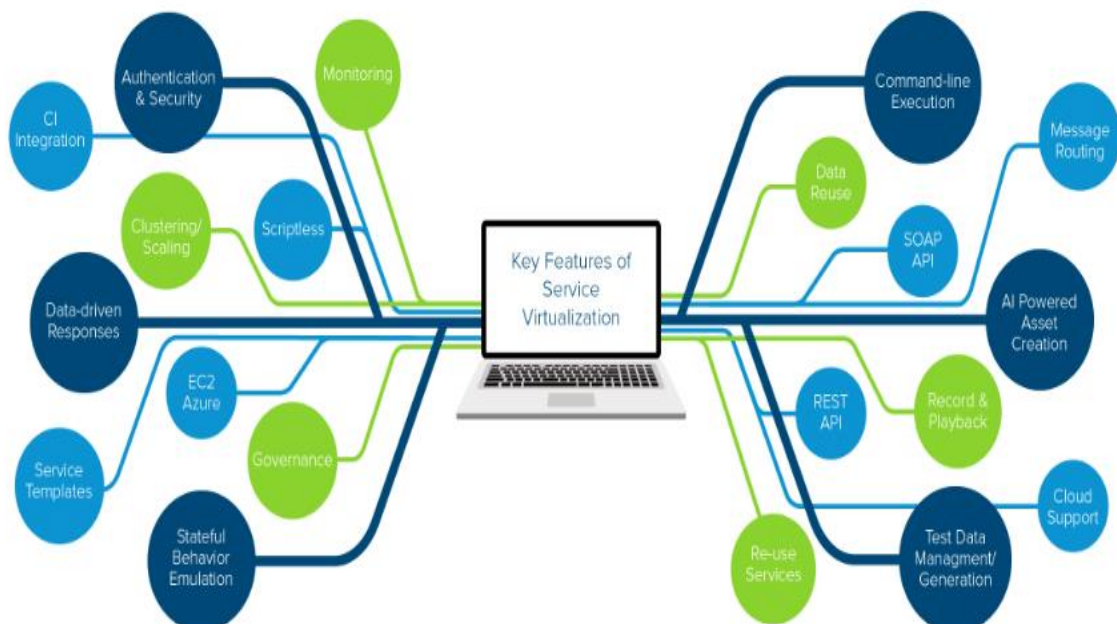


FIGURE 3.8

The use of a para-virtualized guest OS assisted by an intelligent compiler to replace nonvirtualizable OS instructions by hypercalls.

- Before virtualization, the operating system manages the hardware.
- After virtualization, a virtualization layer is inserted between the hardware and the operating system. In such a case, the virtualization layer is responsible for converting portions of the real hardware into virtual hardware.
- Therefore, different operating systems such as Linux and Windows can run on the same physical machine, simultaneously.
- Depending on the position of the virtualization layer, there are several classes of VM architectures, namely the hypervisor architecture, para-virtualization, and host-based virtualization.
- The hypervisor is also known as the VMM (Virtual Machine Monitor). They both perform the same virtualization operations.



A modern technology that helps teams simulate dependent services that are out of your control for testing, service virtualization is a key enabler to any test automation project.

By creating stable and predictable test environments with service virtualization, your test automation will be reliable and accurate, but there are several different approaches and tools available on the market.

What should you look for in a service virtualization solution to make sure that you're maximizing your return on investment?

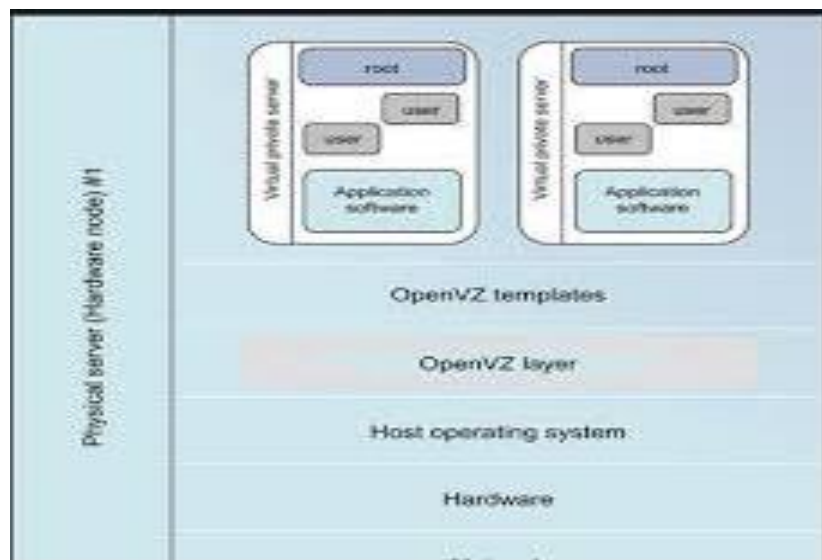
Lightweight Service Virtualization Tools

Free or open-source tools are great tools to start with because they help you get started in a very ad hoc way, so you can quickly learn the benefits of service virtualization. Some examples of lightweight tools include Traffic Parrot, Mockito, or the free version of Parasoft Virtualize. These solutions are usually sought out by individual development teams to "try out" service virtualization, brought in for a very specific project or reason.

While these tools are great for understanding what service virtualization is all about and helping individual users make the case for broader adoption across teams, the downside of these lightweight tools is that it's often challenging for those users to garner full organizational traction because the tools lack the breadth of capability and ease-of-use required for less technical users to be successful. Additionally, while these tools are free in the short term, they become more expensive as you start to look into maintenance and customization.

Enterprise Service Virtualization Tools

More heavyweight tooling is available through vendor-supported tools, designed to support power users that want daily access to create comprehensive virtual services.



You can read the most recent comparison of enterprise-scale service virtualization tools from industry analyst Theresa Lanowitz to look at all the players -- Theresa's summary chart is shown to the left.

These enterprise-grade solutions are designed to align better with deployment and team usage in mind. When an organization wants to implement service virtualization as a part of its continuous integration and DevOps pipeline, enterprise solutions integrate tightly through native plug-ins into their build pipelines. Additionally, these solutions can handle large volumes of traffic while still being performant. On the downside of these solutions, of course, is cost — enterprise solutions and the customer support that comes with them are far from free.

How to Choose the Best Service Virtualization Tool for You?

Most organizations won't self-identify into a specific tooling category such as lightweight or enterprise, but rather have specific needs that they need to make sure they get from their solution. Whether it's specific protocol support or a way to efficiently handle lots of application change, the best way to choose a service virtualization solution that's right for you is to look at the different features and capabilities that you may require and ensure that your tooling choice has those capabilities.

As opposed to trying to focus on generic pros and cons of different solutions, I always try and stress to clients the importance of identifying what you uniquely need for your team and your projects. It's also important to identify future areas of capabilities that you may not be ready for now, but will just be sitting

there in your service virtualization solution for when your test maturity and user adoption grows. So what are those key capabilities?

2.8.1 Key Capabilities of Service Virtualization

Ease-Of-Use and Core Capabilities:

- Ability to use the tool without writing scripts
- Ability to rapidly create virtual services before the real service is available
- Intelligent response correlation
- Data-driven responses
- Ability to re-use services
- A custom extensibility framework
- Support for authentication and security
- Configurable performance environments
- Support for clustering/scaling

Capabilities for optimized workflows:

- Record and playback
- AI-powered asset creation
- Test data management / generation
- Data re-use
- Service templates
- Message routing
- Fail-over to a live system
- Stateful behavior emulation

Automation Capabilities:

- CI integration
- Build system plugins
- Command-line execution
- Open APIs for DevOps integration
- Cloud support (EC2, Azure)

Management and Maintenance Support:

- Governance
- Environment management
- Monitoring
- A process for managing change
- On-premise and browser-based access

Supported Technologies:

- REST API virtualization
- SOAP API virtualization
- Asynchronous API messaging
- MQ/JMS virtualization
- IoT and microservice virtualization
- Database virtualization
- Webpage virtualization
- File transfer virtualization
- Mainframe and fixed-length
- EDI virtualization
- Fix, Swift, etc.

We see best Service Virtualization Tools. Some of the popular Service Virtualization Tools are as follows:

1. IBM Rational Test Virtualization Server
2. Micro Focus Data Simulation Software
3. Broadcom Service Virtualization
4. Smartbear ServiceVPro
5. Tricentis Tosca Test-Driven Service Virtualization

IBM RATIONAL TEST VIRTUALIZATION SERVER

IBM Rational Test Virtualization Server software enables early and frequent testing in the development lifecycle. It removes dependencies by virtualizing part or all of an application or database so software testing teams don't have to wait for the availability of those resources to begin. Combined with Integration Tester, you can achieve continuous software testing.

Features:

- Virtualize services, software and applications.
- Update, reuse and share virtualized environments
- Get support for middleware technologies
- Benefit from integration with other tools
- Flexible pricing and deployment

MICRO FOCUS DATA SIMULATION SOFTWARE

Application simulation software to keep you on schedule and focused on service quality—not service constraints.

Features:

- Easily create simulations of application behavior.
- Model the functional network and performance behavior of your virtual services by using step-by-step wizards.
- Modify data, network, and performance models easily.
- Manage from anywhere with support for user roles, profiles, and access control lists.
- Virtualize what matters: create simulations incorporating a wide array of message formats, transport types, and even ERP application protocols to test everything from the latest web service to a legacy system.
- Easily configure and use virtual services in your daily testing practices. Service Virtualization features fully integrate into LoadRunner, Performance Center, ALM, and Unified Functional Testing.

2.8.2 BROADCOM SERVICE VIRTUALIZATION (FORMERLY CA SERVICE VIRTUALIZATION)

Service Virtualization (formerly CA Service Virtualization) simulates unavailable systems across the software development lifecycle (SDLC), allowing developers, testers, integration, and performance teams to work in parallel for faster delivery and higher application quality and reliability. You'll be able to accelerate software release cycle times, increase quality and reduce software testing environment infrastructure costs.

Features:

- Accelerate time-to-market by enabling parallel software development, testing and validation.
- Test earlier in the SDLC where it is less expensive and disruptive to solve application defects.
- Reduce demand for development environments or pay-per-use service charges.

Smartbear ServiceVPro

Smartbear ServiceVPro is a Service API Mocking and Service Virtualization Tool. API virtualization in ServiceV Pro helps you deliver great APIs on time and under budget, and does so for a fraction of the cost typically associated with traditional enterprise service virtualization suites. Virtualize REST & SOAP APIs, TCP, JDBC, and more to accelerate development and testing cycles.

Features:

- Create virtual services from an API definition, record and use an existing service, or start from scratch to generate a virtual service.
- Create, configure, and deploy your mock on local machines, or deploy inside a public or private cloud to share. Analyze traffic & performance of each virtual service from a web UI.
- Generate dynamic mock data instantly

- Simulate Network Performance & Server-Side Behavior
- Real-time Service Recording & Switching

TRICENTIS TOSCA TEST-DRIVEN SERVICE VIRTUALIZATION

Tricentis Test-Driven Service Virtualization simulates the behavior of dependent systems that are difficult to access or configure so you can continuously test without delays.

Features:

- Reuse Tests as Service Virtualization Scenarios
- More Risk Coverage With Test-Driven Service Virtualization
- Effortless Message Verification and Analysis
- Create and Maintain Virtual Services with Ease

WireMock:

WireMock is a simulator for HTTP-based APIs. Some might consider it a service virtualization tool or a mock server. It enables you to stay productive when an API you depend on doesn't exist or isn't complete. It supports testing of edge cases and failure modes that the real API won't reliably produce. And because it's fast it can reduce your build time from hours down to minutes.

Features:

- Flexible Deployment: Run WireMock from within your Java application, JUnit test, Servlet container or as a standalone process.
- Powerful Request Matching: Match request URLs, methods, headers cookies and bodies using a wide variety of strategies. First class support for JSON and XML.
- Record and Playback: Get up and running quickly by capturing traffic to and from an existing API.

Conclusion:

- We have included most of the tools we have come across. If we missed any tool, please share in the comments and we will include in our list of Service Virtualization Tools. You may also want to check out our ultimate list of API Testing Tools that contains Popular API Testing Tools.

2.9 WHAT IS CPU VIRTUALIZATION

CPU virtualization involves a single CPU acting as if it were multiple separate CPUs. The most common reason for doing this is to run multiple different operating systems on one machine. CPU virtualization emphasizes performance and runs directly on the available CPUs whenever possible. The underlying physical resources are used whenever possible and the virtualization layer runs instructions only as needed to make virtual machines operate as if they were running directly on a physical machine. When many virtual machines are running on an ESXi host, those virtual machines might compete for CPU resources. When CPU contention occurs, the ESXi host time-slices the physical processors across all virtual machines so each virtual machine runs as if it has its specified number of virtual processors.

To support virtualization, processors such as the x86 employ a special running mode and instructions, known as hardware-assisted virtualization. In this way, the VMM and guest OS run in different modes and all sensitive instructions of the guest OS and its applications are trapped in the VMM. To save processor states, mode switching is completed by hardware. For the x86 architecture, Intel and AMD have proprietary technologies for hardware-assisted virtualization.

2.9.1 HARDWARE SUPPORT FOR VIRTUALIZATION

Modern operating systems and processors permit multiple processes to run simultaneously. If there is no protection mechanism in a processor, all instructions from different processes will access the hardware directly and cause a system crash. Therefore, all processors have at least two modes, user mode and supervisor mode, to ensure controlled access of critical hardware. Instructions running in supervisor mode

are called privileged instructions. Other instructions are unprivileged instructions. In a virtualized environment, it is more difficult to make OSes and applications run correctly because there are more layers in the machine stack. Example 3.4 discusses Intel's hardware support approach.

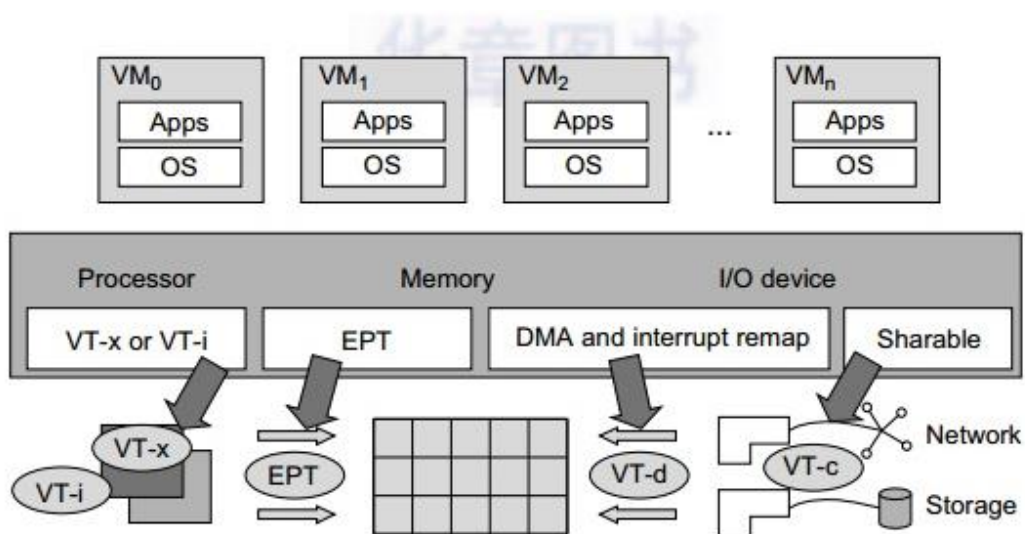
At the time of this writing, many hardware virtualization products were available. The VMware Workstation is a VM software suite for x86 and x86-64 computers. This software suite allows users to set up multiple x86 and x86-64 virtual computers and to use one or more of these VMs simultaneously with the host operating system. The VMware Workstation assumes the host-based virtualization. Xen is a hypervisor for use in IA-32, x86-64, Itanium, and PowerPC 970 hosts. Actually, Xen modifies Linux as the lowest and most privileged layer, or a hypervisor.

One or more guest OS can run on top of the hypervisor. KVM (Kernel-based Virtual Machine) is a Linux kernel virtualization infrastructure. KVM can support hardware-assisted virtualization and paravirtualization by using the Intel VT-x or AMD-v and VirtIO framework, respectively. The VirtIO framework includes a paravirtual Ethernet card, a disk I/O controller, a balloon device for adjusting guest memory usage, and a VGA graphics interface using VMware drivers.

Example 3.4 Hardware Support for Virtualization in the Intel x86 Processor

Since software-based virtualization techniques are complicated and incur performance overhead, Intel provides a hardware-assist technique to make virtualization easy and improve performance. Figure 3.10 provides an overview of Intel's full virtualization techniques. For processor virtualization, Intel offers the VT-x or VT-i technique. VT-x adds a privileged mode (VMX Root Mode) and some instructions to processors. This enhancement traps all sensitive instructions in the VMM automatically. For memory virtualization, Intel offers the EPT, which translates the virtual address to the machine's physical addresses to improve performance. For I/O virtualization, Intel implements VT-d and VT-c to support this.

Since software-based virtualization techniques are complicated and incur performance overhead, Intel provides a hardware-assist technique to make virtualization easy and improve performance. Figure 3.10 provides an overview of Intel's full virtualization techniques. For processor virtualization, Intel offers the VT-x or VT-i technique. VT-x adds a privileged mode (VMX Root Mode) and some instructions to processors. This enhancement traps all sensitive instructions in the VMM automatically. For memory virtualization, Intel offers the EPT, which translates the virtual address to the machine's physical addresses to improve performance. For I/O virtualization, Intel implements VT-d and VT-c to support this.



Intel hardware support for virtualization of processor, memory, and I/O devices.

2.9.2 CPU VIRTUALIZATION

A VM is a duplicate of an existing computer system in which a majority of the VM instructions are executed on the host processor in native mode. Thus, unprivileged instructions of VMs run directly on the host machine for higher efficiency. Other critical instructions should be handled carefully for correctness and stability. The critical instructions are divided into three categories: privileged instructions, control-sensitive instructions, and behavior-sensitive instructions. Privileged instructions execute in a privileged mode and will be trapped if executed outside this mode. Control-sensitive instructions attempt to change the configuration of resources used. Behavior-sensitive instructions have different behaviors depending on the configuration of resources, including the load and store operations over the virtual memory.

A CPU architecture is virtualizable if it supports the ability to run the VM's privileged and unprivileged instructions in the CPU's user mode while the VMM runs in supervisor mode. When the privileged instructions including control- and behavior-sensitive instructions of a VM are executed, they are trapped in the VMM. In this case, the VMM acts as a unified mediator for hardware access from different VMs to guarantee the correctness and stability of the whole system. However, not all CPU architectures are virtualizable. RISC CPU architectures can be naturally virtualized because all control- and behavior-sensitive instructions are privileged instructions. On the contrary, x86 CPU architectures are not primarily designed to support virtualization. This is because about 10 sensitive instructions, such as SGDT and SMSW, are not privileged instructions. When these instructions execute in virtualization, they cannot be trapped in the VMM.

On a native UNIX-like system, a system call triggers the 80h interrupt and passes control to the OS kernel. The interrupt handler in the kernel is then invoked to process the system call. On a paravirtualization system such as Xen, a system call in the guest OS first triggers the 80h interrupt normally. Almost at the same time, the 82h interrupt in the hypervisor is triggered. Incidentally, control is passed on to the hypervisor as well. When the hypervisor completes its task for the guest OS system call, it passes control back to the guest OS kernel. Certainly, the guest OS kernel may also invoke the hypercall while it's running. Although paravirtualization of a CPU lets unmodified applications run in the VM, it causes a Small Performance Penalty.

2.9.3 HARDWARE-ASSISTED CPU VIRTUALIZATION

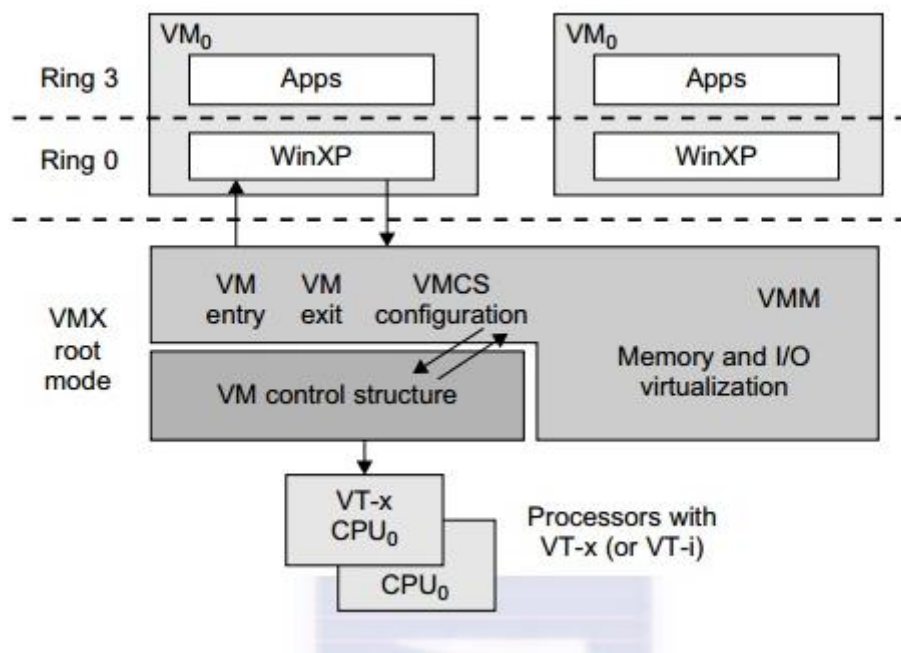
This technique attempts to simplify virtualization because full or paravirtualization is complicated. Intel and AMD add an additional mode called privilege mode level (some people call it Ring-1) to x86 processors. Therefore, operating systems can still run at Ring 0 and the hypervisor can run at Ring -1. All the privileged and sensitive instructions are trapped in the hypervisor automatically. This technique removes the difficulty of implementing binary translation of full virtualization. It also lets the operating system run in VMs without modification.

Example: Intel Hardware-Assisted CPU Virtualization

Although x86 processors are not virtualizable primarily, great effort is taken to virtualize them. They are used widely in comparing RISC processors that the bulk of x86-based legacy systems cannot discard easily. Virtualization of x86 processors is detailed in the following sections. Intel's VT-x technology is an example of hardware-assisted virtualization, as shown in Figure 3.11. Intel calls the privilege level of x86 processors the VMX Root Mode. In order to control the start and stop of a VM and allocate a memory page to maintain the CPU state for VMs, a set of additional instructions is added. At the time of this writing, Xen, VMware, and the Microsoft Virtual PC all implement their hypervisors by using the VT-x technology.

Generally, hardware-assisted virtualization should have high efficiency. However, since the transition from the hypervisor to the guest OS incurs high overhead switches between processor modes, it sometimes cannot outperform binary translation. Hence, virtualization systems such as VMware now use a hybrid approach, in which a few tasks are offloaded to the hardware but the rest is still done in software.

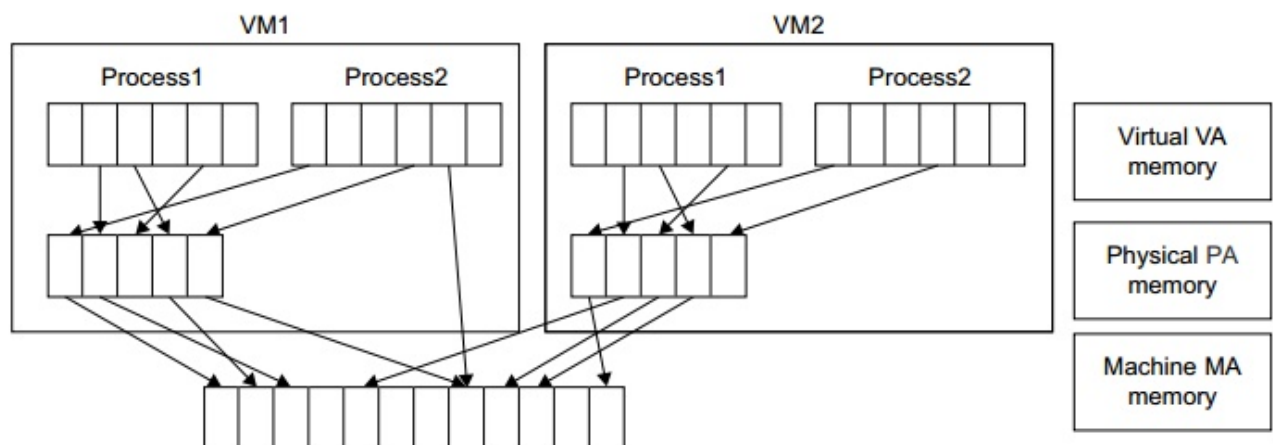
In addition, para-virtualization and hardware-assisted virtualization can be combined to improve the performance further.



Intel hardware-assisted CPU virtualization.

3.10 MEMORY VIRTUALIZATION

Virtual memory virtualization is similar to the virtual memory support provided by modern operating systems. In a traditional execution environment, the operating system maintains mappings of virtual memory to machine memory using page tables, which is a one-stage mapping from virtual memory to machine memory. All modern x86 CPUs include a memory management unit (MMU) and a translation lookaside buffer (TLB) to optimize virtual memory performance. However, in a virtual execution environment, virtual memory virtualization involves sharing the physical system memory in RAM and dynamically allocating it to the physical memory of the VMs.



Two-level memory mapping procedure.

That means a two-stage mapping process should be maintained by the guest OS and the VMM, respectively: virtual memory to physical memory and physical memory to machine memory. Furthermore, MMU virtualization should be supported, which is transparent to the guest OS. The guest

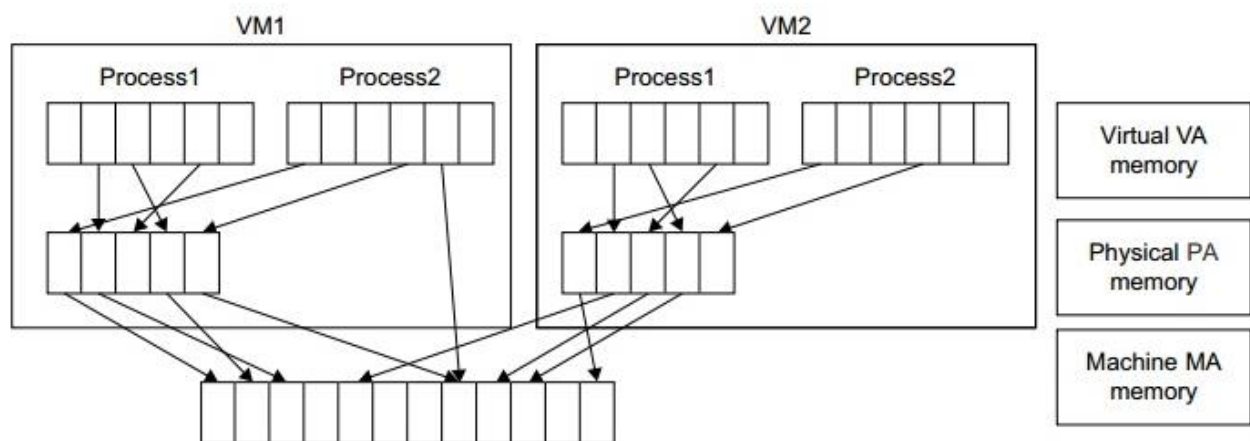
OS continues to control the mapping of virtual addresses to the physical memory addresses of VMs. But the guest OS cannot directly access the actual machine memory. The VMM is responsible for mapping the guest physical memory to the actual machine memory. Figure shows the two-level memory mapping procedure. Since each page table of the guest OSes has a separate page table in the VMM corresponding to it, the VMM page table is called the shadow page table. Nested page tables add another layer of indirection to virtual memory. The MMU already handles virtual-to-physical translations as defined by the OS. Then the physical memory addresses are translated to machine addresses using another set of page tables defined by the hypervisor. Since modern operating systems maintain a set of page tables for every process, the shadow page tables will get flooded. Consequently, the performance overhead and cost of memory will be very high.

VMware uses shadow page tables to perform virtual-memory-to-machine-memory address translation. Processors use TLB hardware to map the virtual memory directly to the machine memory to avoid the two levels of translation on every access. When the guest OS changes the virtual memory to a physical memory mapping, the VMM updates the shadow page tables to enable a direct lookup. The AMD Barcelona processor has featured hardware-assisted memory virtualization since 2007. It provides hardware assistance to the two-stage address translation in a virtual execution environment by using a technology called nested paging.

Example: Extended Page Table by Intel for Memory Virtualization

Since the efficiency of the software shadow page table technique was too low, Intel developed a hardware-based EPT technique to improve it, as illustrated in Figure 3.13. In addition, Intel offers a Virtual Processor ID (VPID) to improve use of the TLB. Therefore, the performance of memory virtualization is greatly improved. In Figure 3.13, the page tables of the guest OS and EPT are all four-level.

When a virtual address needs to be translated, the CPU will first look for the L4 page table pointed to by Guest CR3. Since the address in Guest CR3 is a physical address in the guest OS, the CPU needs to convert the Guest CR3 GPA to the host physical address (HPA) using EPT. In this procedure, the CPU will check the EPT TLB to see if the translation is there. If there is no required translation in the EPT TLB, the CPU will look for it in the EPT. If the CPU cannot find the translation in the EPT, an EPT violation exception will be raised. When the GPA of the L4 page table is obtained, the CPU will calculate the GPA of the L3 page table by using the GVA and the content of the L4 page table. If the entry corresponding to the GVA in the L4

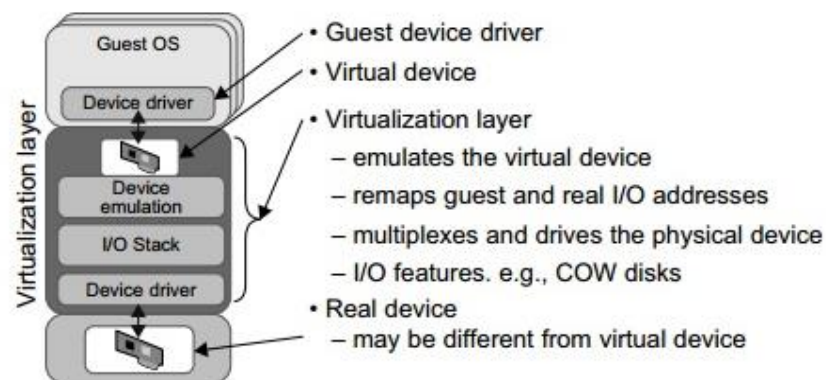


Two-level memory mapping procedure.

page table is a page fault, the CPU will generate a page fault interrupt and will let the guest OS kernel handle the interrupt. When the PGA of the L3 page table is obtained, the CPU will look for the EPT to get the HPA of the L3 page table, as described earlier. To get the HPA corresponding to a GVA, the CPU needs to look for the EPT five times, and each time, the memory needs to be accessed four times. Therefore, there are 20 memory accesses in the worst case, which is still very slow. To overcome this shortcoming, Intel increased the size of the EPT TLB to decrease the number of memory accesses.

4. I/O VIRTUALIZATION

I/O virtualization involves managing the routing of I/O requests between virtual devices and the shared physical hardware. At the time of this writing, there are three ways to implement I/O virtualization: full device emulation, para-virtualization, and direct I/O. Full device emulation is the first approach for I/O virtualization. Generally, this approach emulates well-known, real-world devices.



Device emulation for I/O virtualization implemented inside the middle layer that maps real I/O devices into the virtual devices for the guest device driver to use.

All the functions of a device or bus infrastructure, such as device enumeration, identification, interrupts, and DMA, are replicated in software. This software is located in the VMM and acts as a virtual device. The I/O access requests of the guest OS are trapped in the VMM which interacts with the I/O devices. The full device emulation approach is shown in Figure.

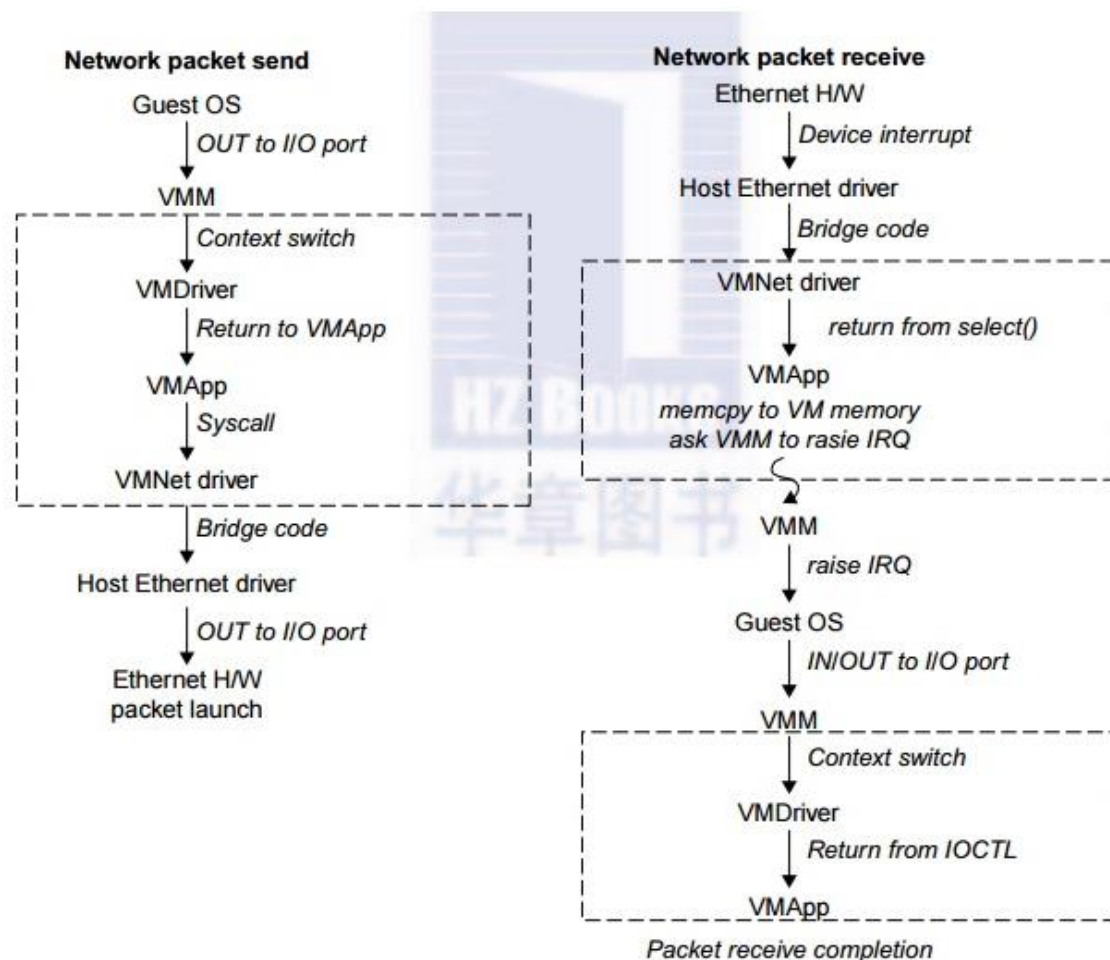
A single hardware device can be shared by multiple VMs that run concurrently. However, software emulation runs much slower than the hardware it emulates [10,15]. The para-virtualization method of I/O virtualization is typically used in Xen. It is also known as the split driver model consisting of a frontend driver and a backend driver. The frontend driver is running in Domain U and the backend driver is running in Domain 0. They interact with each other via a block of shared memory. The frontend driver manages the I/O requests of the guest OSes and the backend driver is responsible for managing the real I/O devices and multiplexing the I/O data of different VMs. Although para-I/O-virtualization achieves better device performance than full device emulation, it comes with a higher CPU overhead.

Direct I/O virtualization lets the VM access devices directly. It can achieve close-to-native performance without high CPU costs. However, current direct I/O virtualization implementations focus on networking for mainframes. There are a lot of challenges for commodity hardware devices. For example, when a physical device is reclaimed (required by workload migration) for later reassignment, it may have been set to an arbitrary state (e.g., DMA to some arbitrary memory locations) that can function incorrectly or even crash the whole system. Since software-based I/O virtualization requires a very high overhead of device emulation, hardware-assisted I/O virtualization is critical. Intel VT-d supports the remapping of I/O DMA transfers and device-generated interrupts. The architecture of VT-d provides the flexibility to support multiple usage models that may run unmodified, special-purpose, or “virtualization-aware” guest OSes.

Another way to help I/O virtualization is via self-virtualized I/O (SV-IO) [47]. The key idea of SV-IO is to harness the rich resources of a multicore processor. All tasks associated with virtualizing an I/O device are encapsulated in SV-IO. It provides virtual devices and an associated access API to VMs and a management API to the VMM. SV-IO defines one virtual interface (VIF) for every kind of virtualized I/O device, such as virtual network interfaces, virtual block devices (disk), virtual camera devices, and others. The guest OS interacts with the VIFs via VIF device drivers. Each VIF consists of two message queues. One is for outgoing messages to the devices and the other is for incoming messages from the devices. In addition, each VIF has a unique ID for identifying it in SV-IO.

The VMware Workstation runs as an application. It leverages the I/O device support in guest OSes, host OSes, and VMM to implement I/O virtualization. The application portion (VMAp) uses a driver loaded into the host operating system (VMDriver) to establish the privileged VMM, which runs directly on the hardware. A given physical processor is executed in either the host world or the VMM world, with the VMDriver facilitating the transfer of control between the two worlds. The VMware Workstation employs full device emulation to implement I/O virtualization. Figure 3.15 shows the functional blocks used in sending and receiving packets via the emulated virtual NIC.

Example VMware Workstation for I/O Virtualization



Functional blocks involved in sending and receiving network packets.

The virtual NIC models an AMD Lance Am79C970A controller. The device driver for a Lance controller in the guest OS initiates packet transmissions by reading and writing a sequence of virtual I/O ports; each read or write switches back to the VMAp to emulate the Lance port accesses. When the last OUT instruction of the sequence is encountered, the Lance emulator calls a normal write() to the VMNet driver. The VMNet driver then passes the packet onto the network via a host NIC and then the VMAp

switches back to the VMM. The switch raises a virtual interrupt to notify the guest device driver that the packet was sent. Packet receives occur in reverse.

4.2 VIRTUALIZATION IN MULTI-CORE PROCESSORS

Virtualizing a multi-core processor is relatively more complicated than virtualizing a uni-core processor. Though multicore processors are claimed to have higher performance by integrating multiple processor cores in a single chip, multi-core virtualization has raised some new challenges to computer architects, compiler constructors, system designers, and application programmers. There are mainly two difficulties: Application programs must be parallelized to use all cores fully, and software must explicitly assign tasks to the cores, which is a very complex problem.

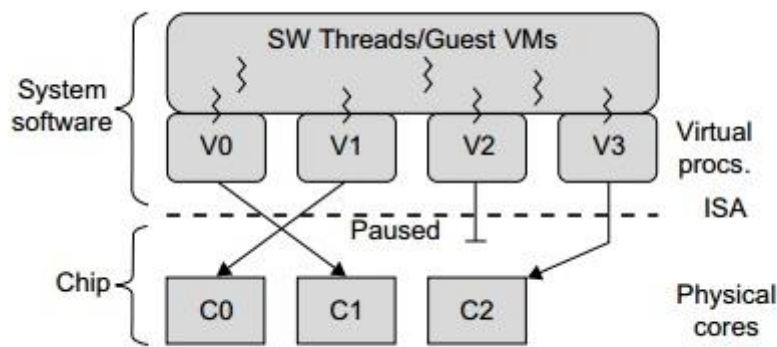
Concerning the first challenge, new programming models, languages, and libraries are needed to make parallel programming easier. The second challenge has spawned research involving scheduling algorithms and resource management policies. Yet these efforts cannot balance well among performance, complexity, and other issues. What is worse, as technology scales, a new challenge called dynamic heterogeneity is emerging to mix the fat CPU core and thin GPU cores on the same chip, which further complicates the multi-core or many-core resource management. The dynamic heterogeneity of hardware infrastructure mainly comes from less reliable transistors and increased complexity in using the transistors.

4.2.1 Physical versus Virtual Processor Cores

Wells, proposed a multicore virtualization method to allow hardware designers to get an abstraction of the low-level details of the processor cores. This technique alleviates the burden and inefficiency of managing hardware resources by software. It is located under the ISA and remains unmodified by the operating system or VMM (hypervisor). Figure 3.16 illustrates the technique of a software-visible VCPU moving from one core to another and temporarily suspending execution of a VCPU when there are no appropriate cores on which it can run.

4.3 VIRTUAL HIERARCHY

The emerging many-core chip multiprocessors (CMPs) provides a new computing landscape. Instead of supporting time-sharing jobs on one or a few cores, we can use the abundant cores in a space-sharing, where single-threaded or multithreaded jobs are simultaneously assigned to separate groups of cores for long time intervals. This idea was originally suggested by Marty and Hill [39]. To optimize for space-shared workloads, they propose using virtual hierarchies to overlay a coherence and caching hierarchy onto a physical processor. Unlike a fixed physical hierarchy, a virtual hierarchy can adapt to fit how the work is space shared for improved performance and performance isolation.

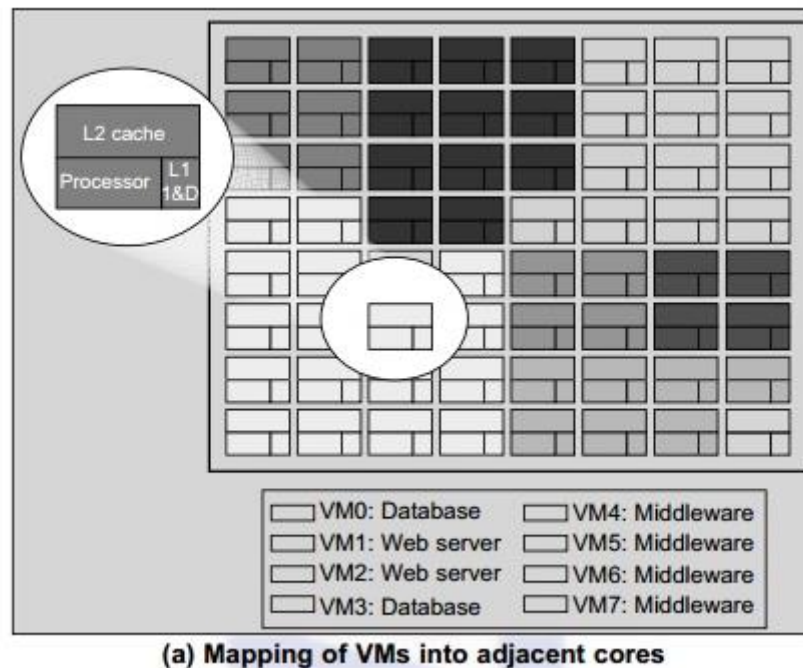


Multicore virtualization method that exposes four VCPUs to the software, when only three cores are actually present.

Today's many-core CMPs use a physical hierarchy of two or more cache levels that statically determine the cache allocation and mapping. A virtual hierarchy is a cache hierarchy that can adapt to fit the workload or mix of workloads. The hierarchy's first level locates data blocks close to the cores needing them for faster access, establishes a shared-cache domain, and establishes a point of coherence for faster communication. When a miss leaves a tile, it first attempts to locate the block (or sharers) within the first level. The first level can also provide isolation between independent workloads. A miss at the L1 cache can invoke the L2 access.

The idea is illustrated in Figure Space sharing is applied to assign three workloads to three clusters of virtual cores: namely VM0 and VM3 for database workload, VM1 and VM2 for web server workload, and VM4–VM7 for middleware workload. The basic assumption is that each workload runs in its own VM. However, space sharing applies equally within a single operating system. Statically distributing the directory among tiles can do much better, provided operating systems or hypervisors carefully map virtual pages to physical frames. Marty and Hill suggested a two-level virtual coherence and caching hierarchy that harmonizes with the assignment of tiles to the virtual clusters of VMs.

Figure illustrates a logical view of such a virtual cluster hierarchy in two levels. Each VM operates in an isolated fashion at the first level. This will minimize both miss access time and performance interference with other workloads or VMs. Moreover, the shared resources of cache capacity, inter-connect links, and miss handling are mostly isolated between VMs. The second level maintains a globally shared memory. This facilitates dynamically repartitioning resources without costly cache flushes. Furthermore, maintaining globally shared memory minimizes changes to existing system software and allows virtualization features such as content-based page sharing. A virtual hierarchy adapts to space-shared workloads like multiprogramming and server consolidation. Figure 3.17 shows a case study focused on consolidated server workloads in a tiled architecture. This many-core mapping scheme can also optimize for space-shared multiprogrammed workloads in a single-OS environment.



CMP server consolidation by space-sharing of VMs into many cores forming multiple virtual clusters to execute various workloads.

4.6 VIRTUALIZATION SUPPORT AND DISASTER RECOVERY.

Virtualization provides flexibility in disaster recovery. When servers are virtualized, they are containerized into VMs, independent from the underlying hardware. Therefore, an organization does not need the same physical servers at the primary site as at its secondary disaster recovery site.

Other benefits of virtual disaster recovery include ease, efficiency and speed. Virtualized platforms typically provide high availability in the event of a failure. Virtualization helps meet recovery time objectives (RTOs) and recovery point objectives (RPOs), as replication is done as frequently as needed, especially for critical systems. DR planning and failover testing is also simpler with virtualized workloads than with a physical setup, making disaster recovery a more attainable process for organizations that may not have the funds or resources for physical DR.

In addition, consolidating physical servers with virtualization saves money because the virtualized workloads require less power, floor space and maintenance. However, replication can get expensive, depending on how frequently it's done.

Adding VMs is an easy task, so organizations need to watch out for VM sprawl. VMs operating without the knowledge of DR staff may fall through the cracks when it comes time for recovery. Sprawl is particularly dangerous at larger companies where communication may not be as strong as at a smaller organization with fewer employees. All organizations should have strict protocols for deploying virtual machines.

Virtual disaster recovery planning and testing

Virtual infrastructures can be complex. In a recovery situation, that complexity can be an issue, so it's important to have a comprehensive [DR plan](#).

A virtual disaster recovery plan has many similarities to a traditional DR plan. An organization should:

- Decide which systems and data are the most critical for recovery, and document them.
- Get management support for the DR plan
- Complete a risk assessment and business impact analysis to outline possible risks and their potential impacts.
- Document steps needed for recovery.
- Define RTOs and RPOs.
- Test the plan.

As with a traditional DR setup, you should clearly define who is involved in planning and testing, and the role of each staff member. That extends to an actual recovery event, as staff should be ready for their tasks during an unplanned incident.

The organization should review and test its virtual disaster recovery plan on a regular basis, especially after any changes have been made to the production environment. Any physical systems should also be tested. While it may be complicated to test virtual and physical systems at the same time, it's important for the sake of **business continuity**.

Virtual disaster recovery vs. physical disaster recovery

Virtual disaster recovery, though simpler than traditional DR, should retain the same standard goals of meeting RTOs and RPOs, and ensuring a business can continue to function in the event of an unplanned incident.

The traditional disaster recovery process of duplicating a data center in another location is often expensive, complicated and time-consuming. While a physical disaster recovery process typically involves multiple steps, virtual disaster recovery can be as simple as a click of a button for failover.

Rebuilding systems in the virtual world is not necessary because they already exist in another location, thanks to replication. However, it's important to monitor backup systems. It's easy to "set it and forget it" in the virtual world, which is not advised and is not as much of a problem with physical systems.

Like with physical disaster recovery, the virtual disaster recovery plan should be tested. Virtual disaster recovery, however, provides testing capabilities not available in a physical setup. It is easier to do a DR test in the virtual world without affecting production systems, as virtualization enables an organization to bring up servers in an isolated network for testing. In addition, deleting and recreating DR servers is much simpler than in the physical world.

Virtual disaster recovery is possible with physical servers through physical-to-virtual backup. This process creates virtual backups of physical systems for recovery purposes.

For the most comprehensive data protection, experts advise having an offline copy of data. While virtual disaster recovery vendors provide capabilities to protect against cyberattacks such as ransomware, physical tape storage is the one true offline option that guarantees data is safe during an attack.

Trends and future directions

With ransomware now a constant threat to business, virtual disaster recovery vendors are including capabilities specific to recovering from an attack. Through **point-in-time copies**, an organization can roll back its data recovery to just before the attack hit.

The convergence of backup and DR is a major trend in data protection. One example is **instant recovery** also called recovery in place -- which allows a backup snapshot of a VM to run temporarily on secondary storage following a disaster. This process significantly reduces RTOs.

Hyper-convergence, which combines storage, compute and virtualization, is another major trend. As a result, hyper-converged backup and recovery has taken off, with newer vendors such as Cohesity and Rubrik leading the charge. Their cloud-based hyper-converged backup and recovery systems are accessible to smaller organizations, thanks to lower cost and complexity.

These newer vendors are pushing the more established players to do more with their storage and recovery capabilities.

Major vendors

There are several data protection vendors that offer comprehensive virtual backup and disaster recovery. Some key players include:

- Acronis Disaster Recovery Service protects virtual and physical systems.
- Nakivo Backup & Replication provides data protection for VMware, Microsoft Hyper-V and AWS Elastic Compute Cloud.
- SolarWinds Backup features recovery to VMware, Microsoft Hyper-V, Microsoft Azure and Amazon VMs.
- Veeam Software started out only protecting VMs but has since grown into one of the leading data protection vendors, offering backup and recovery for physical and cloud workloads as well.
- VMware, a pioneer in virtualization, provides DR through such products as Site Recovery Manager and vSphere Replication.

How Virtualization Benefit Disaster Recovery

Most of us are aware of the importance of backing up data, but there's a lot more to disaster recovery than backup alone. It's important to recognize the fact that disaster recovery and backup are not interchangeable. Rather, backup is a critical element of disaster recovery. However, when a system failure occurs, it's not just your files that you need to recover – you'll also need to restore a complete working environment.

Virtualization technology has come a long way in recent years to completely change the way organizations implement their disaster-recovery strategies. Consider, for a moment, how you would deal with a system failure in the old days: You'd have to get a new server or repair the existing one before manually reinstalling all your software, including the operating system and any applications you use for work. Unfortunately, disaster recovery didn't stop there. Without virtualization, you'd then need to manually restore all settings and access credentials to what they were before.

In the old days, a more efficient disaster-recovery strategy would involve redundant servers that would contain a full system backup that would be ready to go as soon as you needed it. However, that also meant increased hardware and maintenance costs from having to double up on everything.

How Does Virtualization Simplify Disaster Recovery?

When it comes to backup and disaster recovery, virtualization changes everything by consolidating the entire server environment, along with all the workstations and other systems into a single virtual machine. A virtual machine is effectively a single file that contains everything, including your operating systems, programs, settings, and files. At the same time, you'll be able to use your virtual machine the same way you use a local desktop.

Virtualization greatly simplifies disaster recovery, since it does not require rebuilding a physical server environment. Instead, you can move your virtual machines over to another system and access them as normal. Factor in cloud computing, and you have the complete flexibility of not having to depend on in-house hardware at all. Instead, all you'll need is a device with internet access and a remote desktop application to get straight back to work as though nothing happened.

What Is the Best Way to Approach Server Virtualization?

Almost any kind of computer system can be virtualized, including workstations, data storage, networks, and even applications. A virtual machine image defines the hardware and software parameters of the system, which means you can move it between physical machines that are powerful enough to run it, including those accessed through the internet.

Matters can get more complicated when you have many servers and other systems to virtualize. For example, you might have different virtual machines for running your apps and databases, yet they all depend on one another to function properly. By using a tightly integrated set of systems, you'll be able to simplify matters, though it's usually better to keep your total number of virtual machines to a minimum to simplify recovery processes.

How Can the Cloud Help?

Although virtualization is carried out on a CPU level by a powerful server system, it's cheaper and easier for smaller businesses to move their core operations to the cloud. That way, you don't need to worry about maintaining your own hardware and additional redundant server systems for backup and disaster recovery purposes.

Instead, everything will be hosted in a state-of-the-art remote data center complete with redundant systems, uninterruptible power supplies, and the physical, technical and administrative security measures needed to keep your data safe. That way, your team will be able to access everything they need to do their jobs by connecting to a remote, virtualized desktop from almost any device with an internet connection.

Recover to any hardware

By using a virtualized environment you don't have to worry about having completely redundant hardware. Instead you can use almost any x86 platform as a backup solution, this allows you to save money by repurposing existing hardware and also gives your company more agility when it comes to hardware failure as almost any virtual server can be restarted on different hardware.

Backup and restore full images

By having your system completely virtualized each of your server's files are encapsulated in a single image file. An image is basically a single file that contains all of server's files, including system files, programs, and data; all in one location. By having these images it makes managing your systems easy and backups become as simple as duplicating the image file and restores are simplified to simply mounting the image on a new server.

Run other workloads on standby hardware

A key benefit to virtualization is reducing the hardware needed by utilizing your existing hardware more efficiently. This frees up systems that can now be used to run other tasks or be used as a hardware redundancy. This mixed with features like VMware's High Availability, which restarts a virtual machine on a different server when the original hardware fails, or for a more robust disaster recovery plan you can use Fault Tolerance, which keeps both servers in sync with each other leading to zero downtime if a server should fail.

Easily copy system data to recovery site

Having an offsite backup is a huge advantage if something were to happen to your specific location, whether it be a natural disaster, a power outage, or a water pipe bursting, it is nice to have all your information at an offsite location. Virtualization makes this easy by easily copying each virtual machines image to the offsite location and with the easy customizable automation process, it doesn't add any more strain or man hours to the IT department.

Benefits of cloud-based disaster recovery

Online Tech cloud-based disaster recovery With the growing popularity of the cloud, more and more companies are turning to it for their production sites. But what about cloud-based disaster recovery? Does it offer the same kind of benefits? As disaster recovery can be complex, time-consuming and very expensive, it pays to plan ahead to figure out just what your business needs. Putting your disaster recovery plan in the cloud can help alleviate some of the fears that come with setting it up.

Here are four big benefits to cloud-based disaster recovery:

Faster recovery

The big difference between cloud-based disaster recovery and traditional recovery practices is the difference in RPO and RTO. With cloud-based DR, your site has the capability to recover from a warm site right away, drastically reducing your RPO and RTO times from days, or even weeks, to hours. Whereas traditional disaster recovery involved booting up from a cold site, cloud recovery is different. Thanks to virtualization, the entire server, including the operating system, applications, patches and data are encapsulated into a single software bundle or virtual server. This virtual server can be copied or backed up to an offsite data center and spun up on a virtual host in a matter of minutes in the event of a disaster. For organizations that can't afford to wait after a disaster, a cloud-based solution could mean the difference between staying in business or closing its doors.

Financial savings

Cloud storage is very cost effective, as you can pay for storing only what you need. Without capital expenses to worry about, you can use "pay-as-you-go" model systems that help keep your TCO low. You also don't have to store a ton of backup tapes that could take days or to access in an emergency. When it's already expensive to implement a DR plan, having your recovery site in the cloud can help make it more affordable.

Scalability

Putting your disaster recovery site in the cloud allows for a lot of flexibility, so increasing or decreasing your storage capacity as your business demands it is easier than with traditional backup. Rather than having to commit to a specific amount of storage for a certain time and worry whether you're meeting or exceeding those requirements, you can scale your storage as needed.

Security

Despite any myths to the contrary, having a cloud-based disaster recovery plan is quite secure with the right provider. Cloud service providers can argue they offer just as much, if not more, security features than traditional infrastructure. But when it comes to disaster recovery for your business, you can't afford to take chances. Make sure you shop around and ask the tough questions when it comes to backing up your production site.

Virtual desktops

In most offices, employees are still dependent on desktop computers. Their workstations grant them access to everything from customer relationship software to company databases and when these computers go down, there's no way to get work done. Virtualized desktops allow users to access their files and even computing power from across the internet.

Instead of logging on to an operating system stored on a hard drive just a few inches away from their keyboard, employees can take advantage of server hardware to store their files across a network.

With barebones computers, employees can log in to these virtual desktops either in the office or from home. Floods, fires and other disasters won't prevent your team from working because they can continue remotely.

Virtual applications

Devoting a portion of your server's hardware and software resources to virtual desktops requires a fair amount of computing power. If the majority of your employees' time is spent working with just one or two pieces of software, you can virtualized just those applications.

If a hurricane destroyed your office and the hardware inside it, virtualized applications can be restored in minutes. They don't need to be installed on the machines that use them, and as long as you have backups these applications can be streamed to employee computers just like a cloud-based application.

Virtual servers

If you use virtual desktops or applications, it makes perfect sense to use virtual servers as well. With a little help from a managed services provider, your servers can be configured to automatically create virtual backups. Beyond preventing data loss, these backups also make it possible to restore server functionality with offsite restorations.

Virtualized servers are incredibly useful when clients need access to a website or database that you maintain in the office. For example, if you provide background checks on tenants to rental property owners through your website, an unexpected power outage won't cause an interruption of service. Your virtualization solution will boot up a backup server away from the power outage and your customers will be none the wiser.

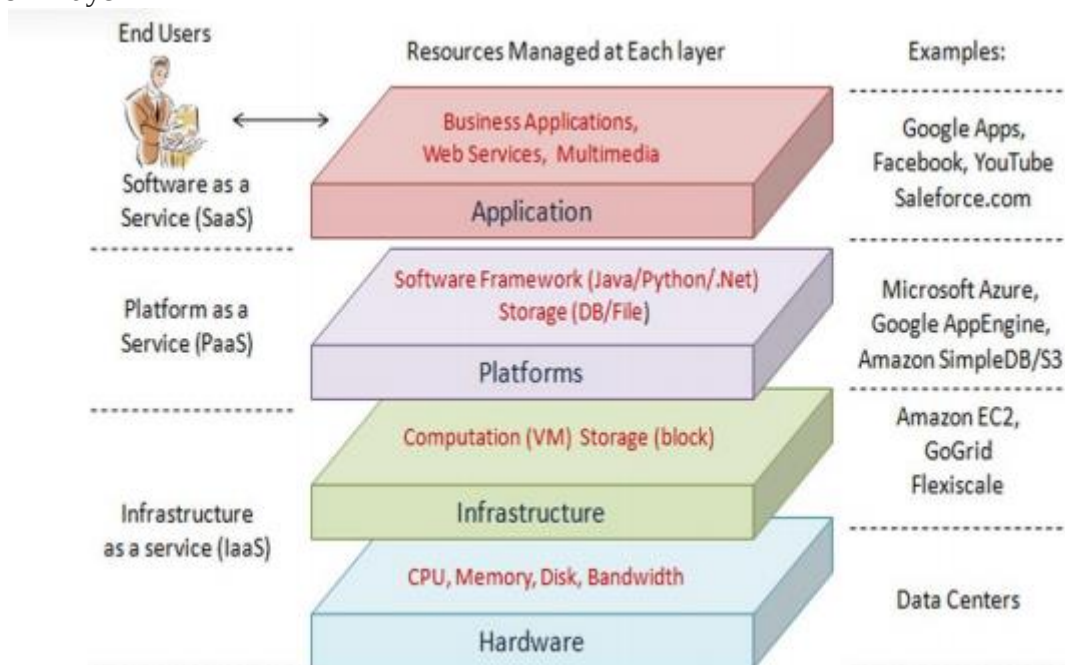
UNIT III CLOUD ARCHITECTURE, SERVICES AND STORAGE

3.1 Layered Cloud Architecture Design

Cloud Computing is a recent technology trend whose aim is to deliver computing utilities as Internet services. Many companies have already offered successful commercial Cloud services including SaaS, PaaS and IaaS. But those services are all computer-based and designed for Web browsers. Currently there is no Cloud architecture whose purpose is to provide special services for digital appliances in smart home.

The architecture of cloud computing categorized into four different layers:

- The physical layer
- Application layer
- Infrastructure layer
- platform layer



Cloud computing consist of four different layers, each layer having their own functionalities, moreover we can able to know the services provided by the cloud computing are also mentioned in the below figure. Let us have a look to all the four layers with the help of diagram.

Hardware Layer

Physical resources of the cloud are managed in this layer, including physical servers, routers, switches, power and cooling systems. In practice, the data centers are place where hardware layer are implemented. A data center usually contains thousands of servers that are organized in racks and interconnected through switches, routers or other fabrics. Typical issues at hardware layer include hardware configuration, fault tolerance, traffic management, power and cooling resource management.

Infrastructure Layer

The basic purpose of infrastructure is to delivering basic storage and compute capabilities as standardized services over the internet. It is also known as the virtualization layer. The infrastructure layer creates cluster of storage and computing resources by partitioning the physical resources using virtualization technologies such as Xen, KVM and VMware. This layer is an essential component of cloud computing, since many key features, such as dynamic resource assignment, are only made available through virtualization technologies .

The services provided by this layer to the consumer is to storage, networks, and other fundamental computing resources, where different arbitrary software can be run or deploy by the consumer, which can include operating systems and applications. The underlying cloud infrastructure are not manage by the consumer but has control over operating systems, storage, and deployed applications and possibly limited control of select networking components (e.g., host firewalls). IaaS refers to on-demand provisioning of infrastructural resources, usually in terms of VMs. The cloud owner who offers IaaS is called an IaaS provider. An example of IaaS provider includes Amazon EC2, GoGrid and Flexiscale.

Platform layer

It is built on top of the infrastructure layer. It consists of operating systems and application frameworks. The main purpose of the platform layer is to minimize the burden of deploying applications directly into VM containers. For example, Google App Engine operates at the platform layer to provide API support for implementing storage, database and business logic of typical web applications.

Application layer At the highest level of the hierarchy, the application layer consists of the actual cloud applications. Different from traditional applications, cloud applications can leverage the automatic-scaling feature to achieve better performance, availability and lower operating cost

The advantages are:

- We only need to understand the layers beneath the one we are working on;
- Each layer is replaceable by an equivalent implementation, with no impact on the other layers;
- Layers are optimal candidates for standardisation;
- A layer can be used by several different higher-level layers.

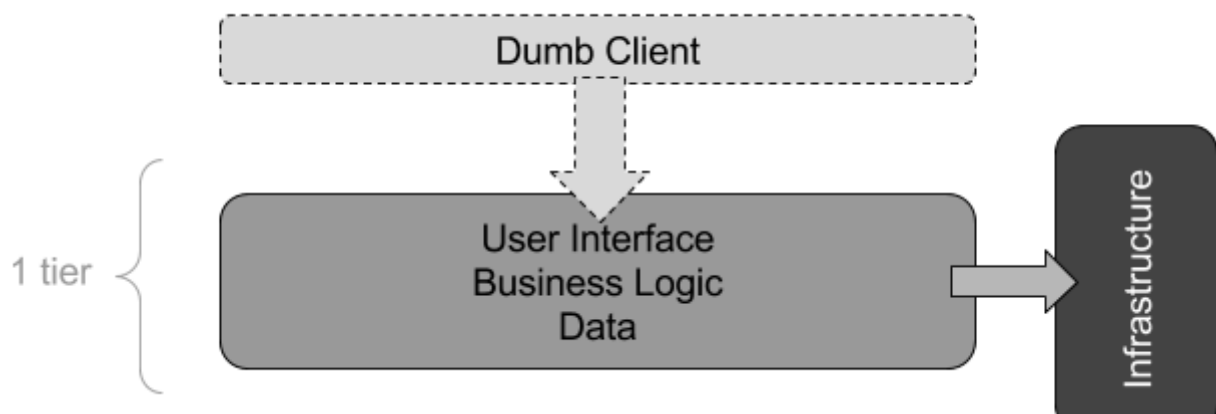
The disadvantages are:

- Layers can not encapsulate everything (a field that is added to the UI, most likely also needs to be added to the DB);
- Extra layers can harm performance, especially if in different tiers.

The 60s and 70s

Although software development started during the 50s, it was during the 60s and 70s that it was effectively born as we know it today, as the activity of building applications that can be delivered, deployed and used by others that are not the developers themselves.

At this point, however, applications were very different than today. There was no GUI (which only came into existence in the early 90s, maybe late 80s), all applications were usable only through a CLI, displayed in a dumb terminal who would just transmit whatever the user typed to the application which was, most likely, being used from the same computer.



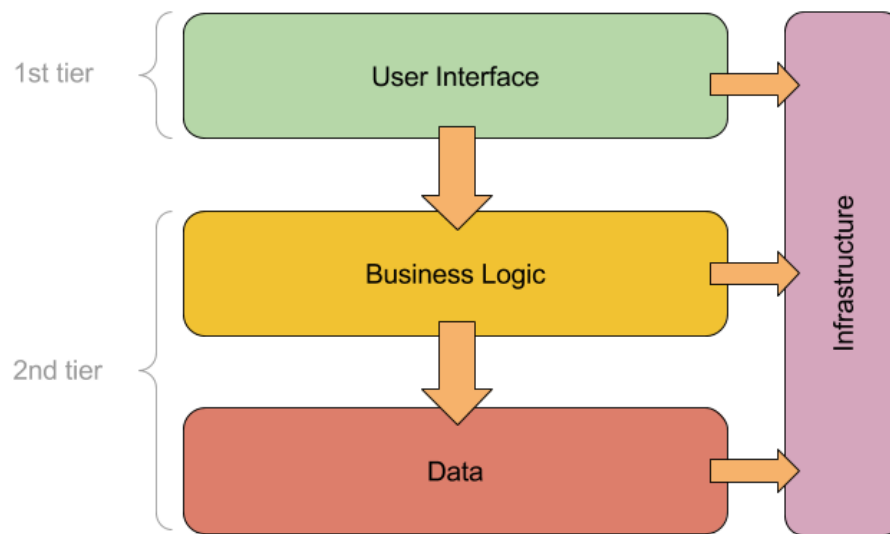
Applications were quite simple so weren't built with layering in mind and they were deployed and used on one computer making it effectively a one-tier application, although at some point the dumb client might even have been remote. While these applications were very simple, they were not scalable, for example, if we needed to update the software to a new version, we would have to do it on every computer that would have the application installed.

Layering during the 80s and 90s

During the 1980s, enterprise applications come to life and we start having several users in a company using desktop computers who access the application through the network.

At this time, there were mostly three layers:

- **User Interface (Presentation):** The user interface, be it a web page, a CLI or a native desktop application;
- **A native Windows application** as the client (rich client), which the common user would use on his desktop computer, that would communicate with the server in order to actually make things happen. The client would be in charge of the application flow and user input validation;
- **Business logic (Domain):** The logic that is the reason why the application exists;
- **An application server**, which would contain the business logic and would receive requests from the native client, act on them and persist the data to the data storage;
- **Data source:** The data persistence mechanism (DB), or communication with other applications.
- **A database server**, which would be used by the application server for the persistence of data.



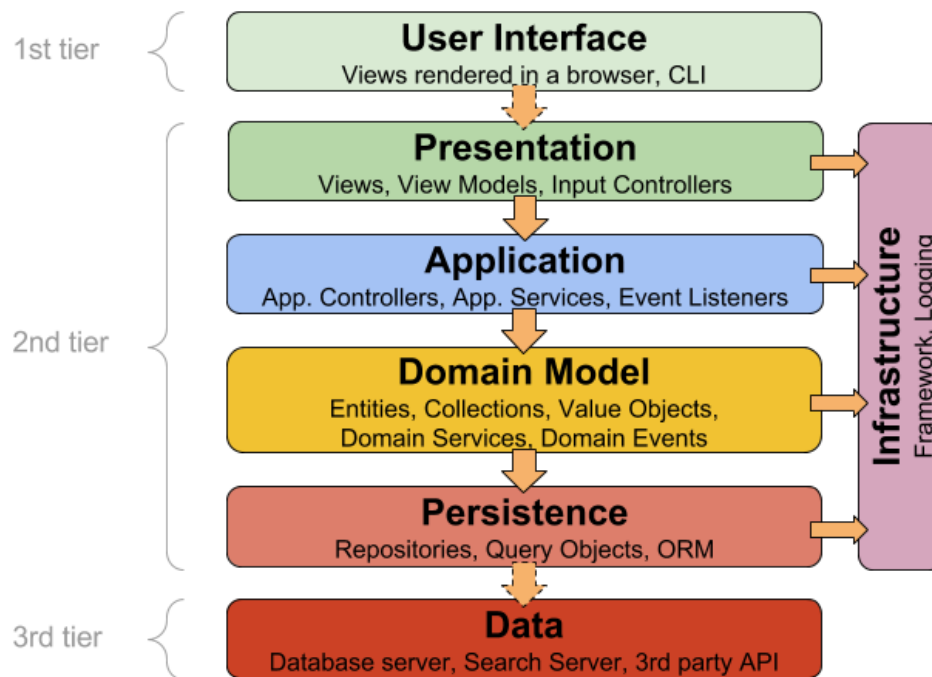
With this shift in usability context, layering started to be a practise, although it only started to be a common widespread practice during the 1990s (Fowler 2002) with the rise of **client/server** systems. This was effectively a **two-tier** application, where the client would be a rich client application used as the application interface, and the server would have the business logic and the data source.

This architecture pattern solves the **scalability** problem, as several users could use the application independently, we would just need another desktop computer, install the client application in it and that was it. However, if we would have a few hundred, or even just a few tenth of clients, and we would want to update the application it would be a **highly complex** operation as we would have to update the clients one by one.

Layering after the mid 90s

Roughly between 1995 and 2005, with the generalised shift to a cloud context, the increase in application users, application complexity and infrastructure complexity we end up seeing an evolution of the layering scheme, where a typical implementation of this layering could be:

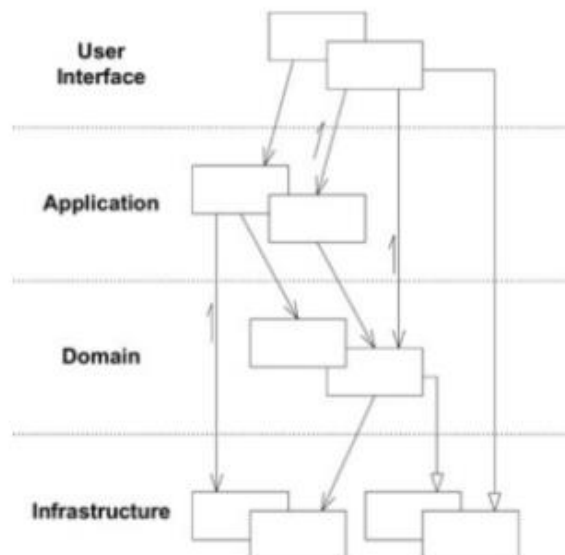
- A **native browser application**, rendering and running the user interface, sending requests to the server application;
- An **application server**, containing the presentation layer, the application layer, the domain layer, and the persistence layer;
- A **database server**, which would be used by the application server for the persistence of data.



This is a three-tier architecture pattern, also known as n-tier. It is a scalable solution and solves the problem of updating the clients as the user interface lives and is compiled on the server, although it is rendered and ran on the client browser.

Layering after the early 2000s

In 2003, Eric Evans published his emblematic book Domain-Driven Design: Tackling Complexity in the Heart of Software. Among the many key concepts published in that book, there was also a vision for the layering of a software system:



User Interface

Responsible for drawing the screens the users use to interact with the application and translating the user's inputs into application commands. It is important to note that the "users" can be human but can also be other applications, which corresponds entirely to the Boundary objects in the EBI Architecture by Ivar Jacobson (more on this in a later post);

Application Layer

Orchestrates Domain objects to perform tasks required by the users. It does not contain business logic. This relates to the Interactors in the EBI Architecture by Ivar Jacobson, except that Jacobson's interactors were any object that was not related to the UI nor an Entity;

Domain Layer

This is the layer that contains all business logic, the Entities, Events and any other object type that contains Business Logic. It obviously relates to the Entity object type of EBI. This is the heart of the system;

Infrastructure

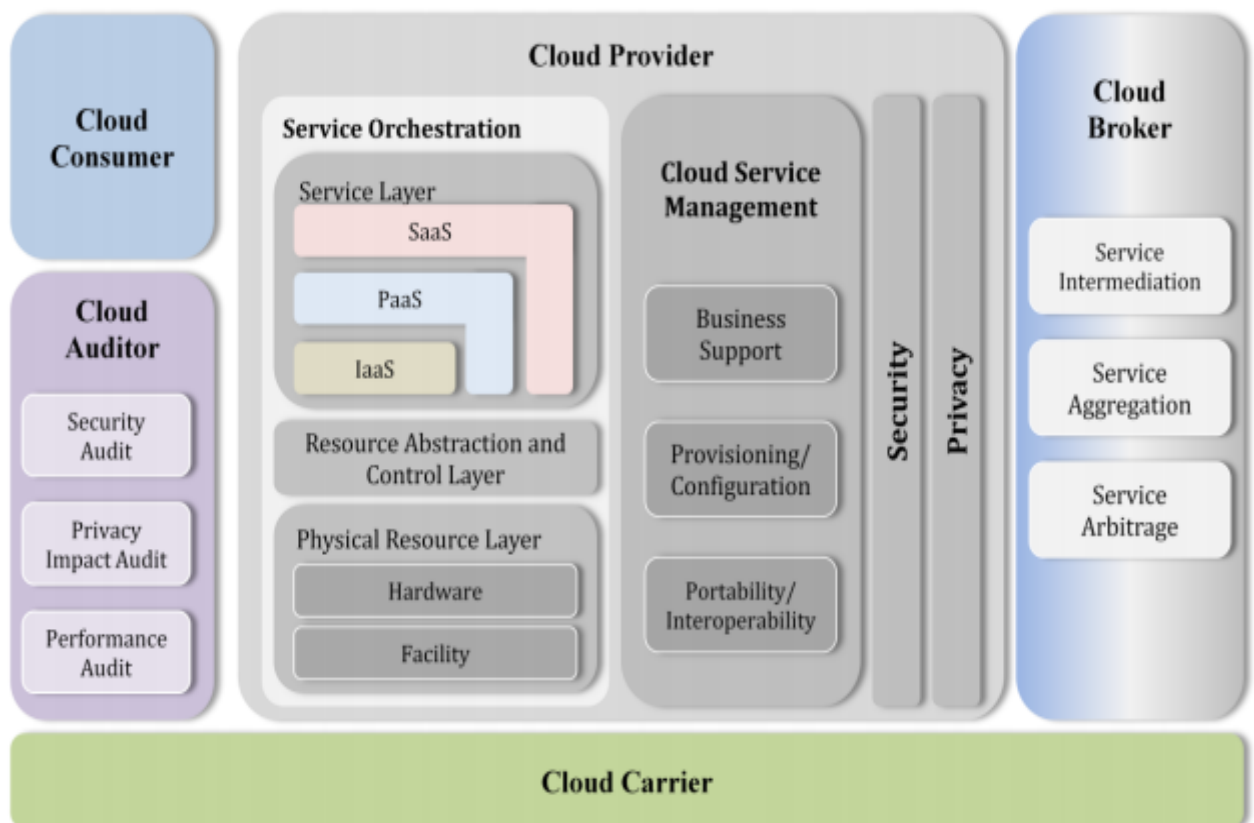
The technical capabilities that support the layers above, ie. persistence or messaging.

3. 2NIST Cloud Computing Reference Architecture

The National Institute of Standards and Technology (NIST) has been designated by Federal Chief Information Officer (CIO) Vivek Kundra with technical leadership for US government (USG) agency efforts related to the adoption and development of cloud computing standards. The goal is to accelerate the federal government's adoption of secure and effective cloud computing to reduce costs and improve services. The NIST strategy is to build a USG Cloud Computing Technology Roadmap which focuses on the highest priority USG cloud computing security, interoperability and portability requirements, and to lead efforts to develop standards and guidelines in close consultation and collaboration with standards bodies, the private sector, and other stakeholders.

The Conceptual Reference

An overview of the NIST cloud computing reference architecture, which identifies the major actors, their activities and functions in cloud computing. The diagram depicts a generic high-level architecture and is intended to facilitate the understanding of the requirements, uses, characteristics and standards of cloud computing.



The NIST cloud computing reference architecture defines five major actors: cloud consumer, cloud provider, cloud carrier, cloud auditor and cloud broker. Each actor is an entity (a person or an organization) that participates in a transaction or process and/or performs tasks in cloud computing. Table 1 briefly lists the actors defined in the NIST cloud computing reference architecture. The general activities of the actors are discussed in the remainder of this section, while the details of the architectural elements are discussed.

Figure 2 illustrates the interactions among the actors. A cloud consumer may request cloud services from a cloud provider directly or via a cloud broker. A cloud auditor conducts independent audits and may

contact the others to collect necessary information. The details will be discussed in the following sections and presented in increasing level of details in successive diagrams.

Actor	Definition
Cloud Consumer	A person or organization that maintains a business relationship with, and uses service from, <i>Cloud Providers</i> .
Cloud Provider	A person, organization, or entity responsible for making a service available to interested parties.
Cloud Auditor	A party that can conduct independent assessment of cloud services, information system operations, performance and security of the cloud implementation.
Cloud Broker	An entity that manages the use, performance and delivery of cloud services, and negotiates relationships between <i>Cloud Providers</i> and <i>Cloud Consumers</i> .
Cloud Carrier	An intermediary that provides connectivity and transport of cloud services from <i>Cloud Providers</i> to <i>Cloud Consumers</i> .

Fig. Actors in cloud computing

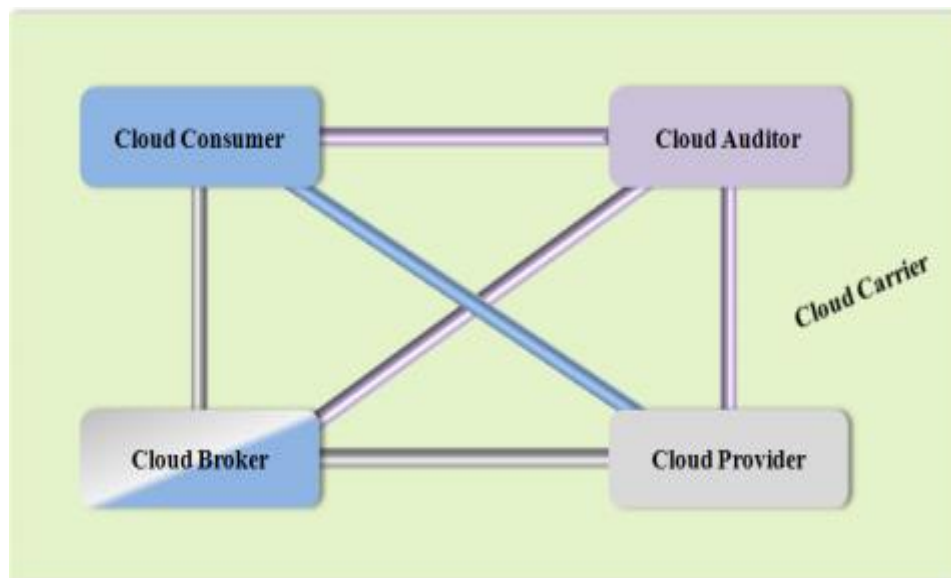


Figure 2: Interactions between the Actors in Cloud Computing

Example Usage Scenario 1: A cloud consumer may request service from a cloud broker instead of contacting a cloud provider directly. The cloud broker may create a new service by combining multiple services or by enhancing an existing service. In this example, the actual cloud providers are invisible to the cloud consumer and the cloud consumer interacts directly with the cloud

broker.

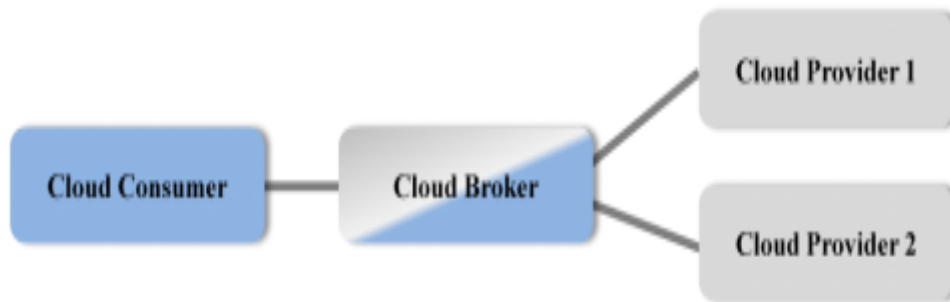
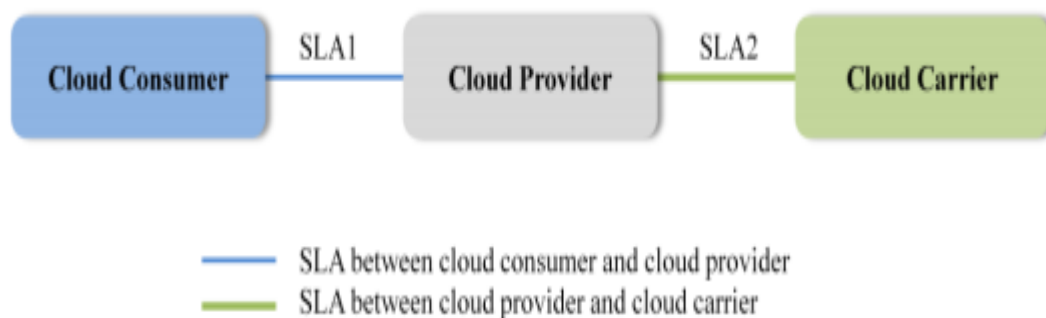


Figure 3: Usage Scenario for Cloud Brokers

Example Usage Scenario 2: Cloud carriers provide the connectivity and transport of cloud services from cloud providers to cloud consumers. As illustrated in Figure 4, a cloud provider participates in and arranges for two unique service level agreements (SLAs), one with a cloud carrier (e.g. SLA2) and one with a cloud consumer (e.g. SLA1). A cloud provider arranges service level agreements (SLAs) with a cloud carrier and may request dedicated and encrypted connections to ensure the cloud services are consumed at a consistent level according to the contractual obligations with the cloud consumers. In this case, the provider may specify its requirements on capability, flexibility and functionality in SLA2 in order to provide essential requirements in SLA1.



Example Usage Scenario 3: For a cloud service, a cloud auditor conducts independent assessments of the operation and security of the cloud service implementation. The audit may involve interactions with both the Cloud Consumer and the Cloud Provider.

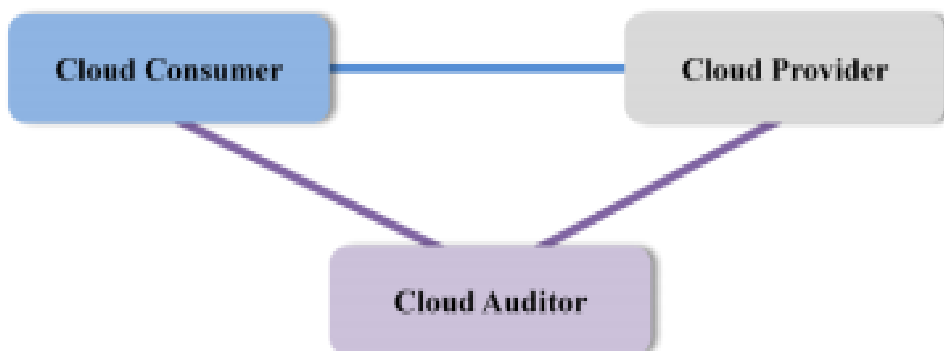


Figure 5: Usage Scenario for Cloud Auditors

Cloud Consumer

The cloud consumer is the principal stakeholder for the cloud computing service. A cloud consumer represents a person or organization that maintains a business relationship with, and uses the service from a cloud provider. A cloud consumer browses the service catalog from a cloud provider, requests the appropriate service, sets up service contracts with the cloud provider, and uses the service. The cloud

consumer may be billed for the service provisioned, and needs to arrange payments accordingly. Cloud consumers need SLAs to specify the technical performance requirements fulfilled by a cloud provider. SLAs can cover terms regarding the quality of service, security, remedies for performance failures. A cloud provider may also list in the SLAs a set of promises explicitly not made to consumers, i.e. limitations, and obligations that cloud consumers must accept. A cloud consumer can freely choose a cloud provider with better pricing and more favorable terms. Typically a cloud provider's pricing policy and SLAs are non-negotiable, unless the customer expects heavy usage and might be able to negotiate for better contracts. Depending on the services requested, the activities and usage scenarios can be different among cloud consumers.

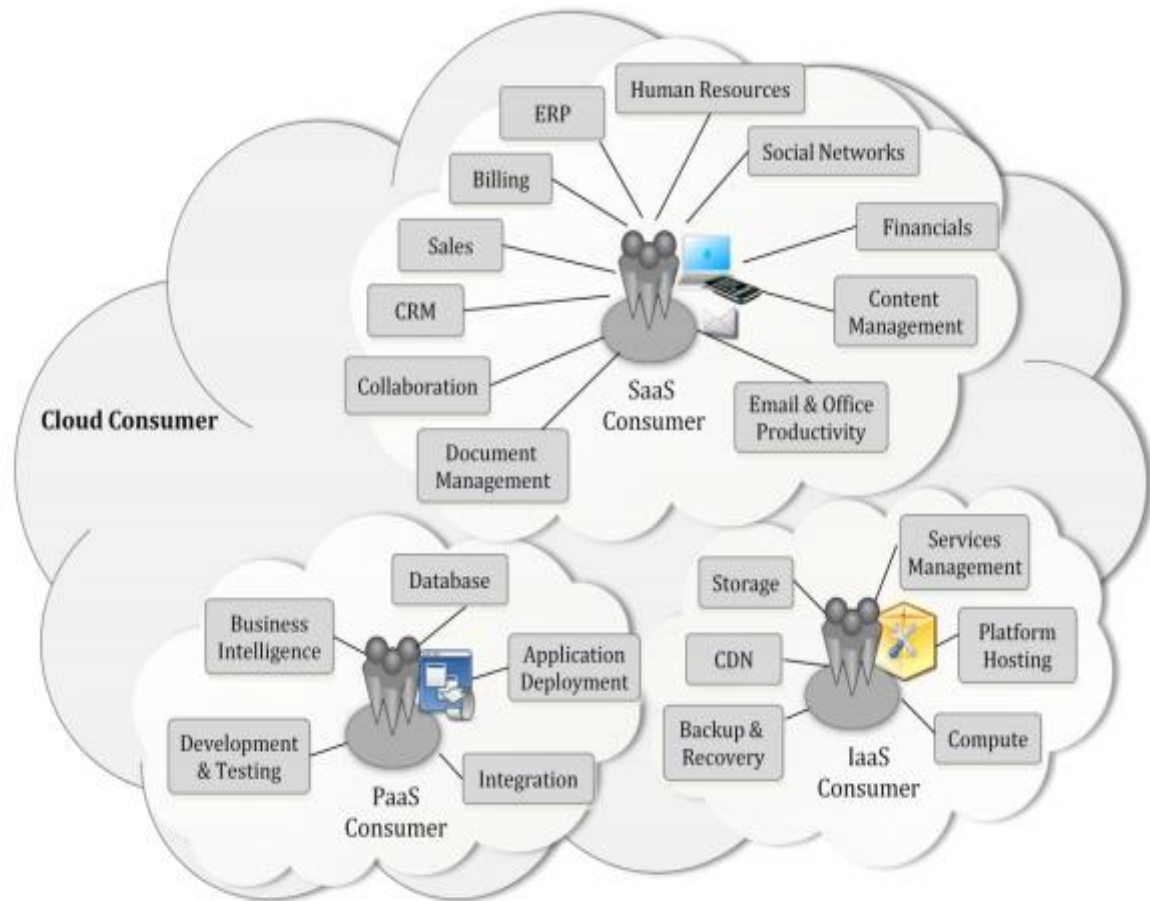


Figure 6: Example Services Available to a Cloud Consumer

SaaS applications in the cloud and made accessible via a network to the SaaS consumers. The consumers of SaaS can be organizations that provide their members with access to software applications, end users who directly use software applications, or software application administrators who configure applications for end users. SaaS consumers can be billed based on the number of end users, the time of use, the network bandwidth consumed, the amount of data stored or duration of stored data.

Cloud consumers of PaaS can employ the tools and execution resources provided by cloud providers to develop, test, deploy and manage the applications hosted in a cloud environment. PaaS consumers can be application developers who design and implement application software, application testers who run and test applications in cloud-based environments, application deployers who publish applications into the cloud, and application administrators who configure and monitor application performance on a platform.

PaaS consumers can be billed according to, processing, database storage and network resources consumed by the PaaS application, and the duration of the platform usage. Consumers of IaaS have access to virtual computers, network-accessible storage, network infrastructure components, and other fundamental computing resources on which they can deploy and run arbitrary software. The consumers of IaaS can be system developers, system administrators and IT managers who are interested in creating, installing, managing and monitoring services for IT infrastructure operations. IaaS consumers are

provisioned with the capabilities to access these computing resources, and are billed according to the amount or duration of the resources consumed, such as CPU hours used by virtual computers, volume and duration of data stored, network bandwidth consumed, number of IP addresses used for certain intervals.

Cloud Provider A cloud provider is a person, an organization; it is the entity responsible for making a service available to interested parties. A Cloud Provider acquires and manages the computing infrastructure required for providing the services, runs the cloud software that provides the services, and makes arrangement to deliver the cloud services to the Cloud Consumers through network access. For Software as a Service, the cloud provider deploys, configures, maintains and updates the operation of the software applications on a cloud infrastructure so that the services are provisioned at the expected service levels to cloud consumers. The provider of SaaS assumes most of the responsibilities in managing and controlling the applications and the infrastructure, while the cloud consumers have limited administrative control of the applications. For PaaS, the Cloud Provider manages the computing infrastructure for the platform and runs the cloud software that provides the components of the platform, such as runtime software execution stack, databases, and other middleware components. The PaaS Cloud Provider typically also supports the development, deployment and management process of the PaaS Cloud Consumer by providing tools such as integrated development environments (IDEs), development version of cloud software, software development kits (SDKs), deployment and management tools. The PaaS Cloud Consumer has control over the applications and possibly some the hosting environment settings, but has no or limited access to the infrastructure underlying the platform such as network, servers, operating systems (OS), or storage.

For IaaS, the Cloud Provider acquires the physical computing resources underlying the service, including the servers, networks, storage and hosting infrastructure. The Cloud Provider runs the cloud software necessary to makes computing resources available to the IaaS Cloud Consumer through a set of service interfaces and computing resource abstractions, such as virtual machines and virtual network interfaces. The IaaS Cloud Consumer in turn uses these computing resources, such as a virtual computer, for their fundamental computing needs Compared to SaaS and PaaS Cloud Consumers, an IaaS Cloud Consumer has access to more fundamental forms of computing resources and thus has more control over the more software components in an application stack, including the OS and network. The IaaS Cloud Provider, on the other hand, has control over the physical hardware and cloud software that makes the provisioning of these infrastructure services possible, for example, the physical servers, network equipments, storage devices, host OS and hypervisors for virtualization.

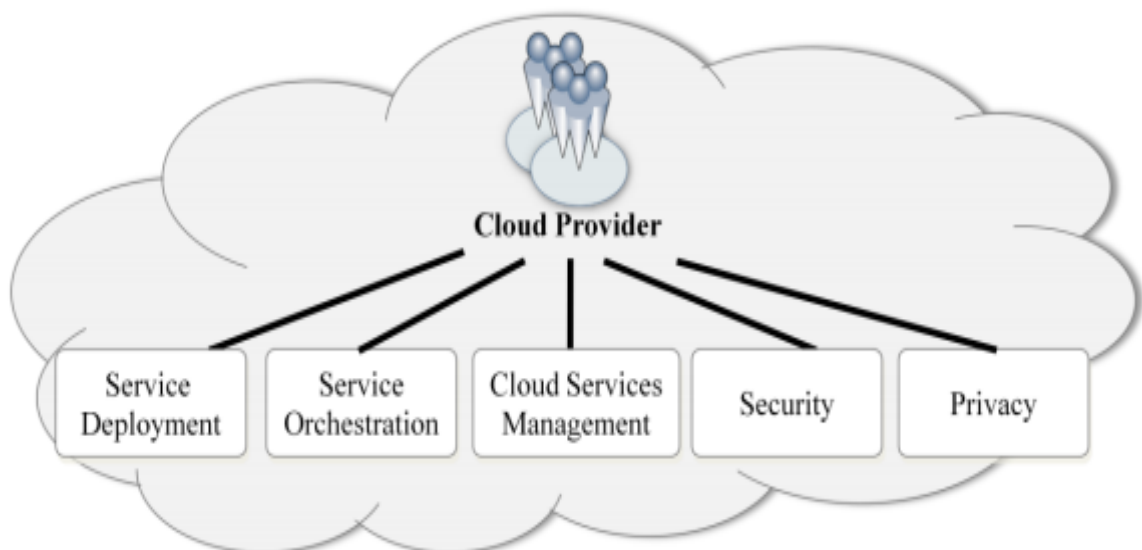


Figure 7: Cloud Provider - Major Activities

Cloud Auditor A cloud auditor is a party that can perform an independent examination of cloud service controls with the intent to express an opinion thereon. Audits are performed to verify conformance to standards through review of objective evidence. A cloud auditor can evaluate the services provided by a cloud provider in terms of security controls, privacy impact, performance.

Cloud Broker As cloud computing evolves, the integration of cloud services can be too complex for cloud consumers to manage. A cloud consumer may request cloud services from a cloud broker, instead of contacting a cloud provider directly. A cloud broker is an entity that manages the use, performance and delivery of cloud services and negotiates relationships between cloud providers and cloud consumers. In general, a cloud broker can provide services in three categories

Service Intermediation:

A cloud broker enhances a given service by improving some specific capability and providing value-added services to cloud consumers. The improvement can be managing access to cloud services, identity management, performance reporting, enhanced security, etc.

Service Aggregation:

A cloud broker combines and integrates multiple services into one or more new services. The broker provides data integration and ensures the secure data movement between the cloud consumer and multiple cloud providers.

Service Arbitrage:

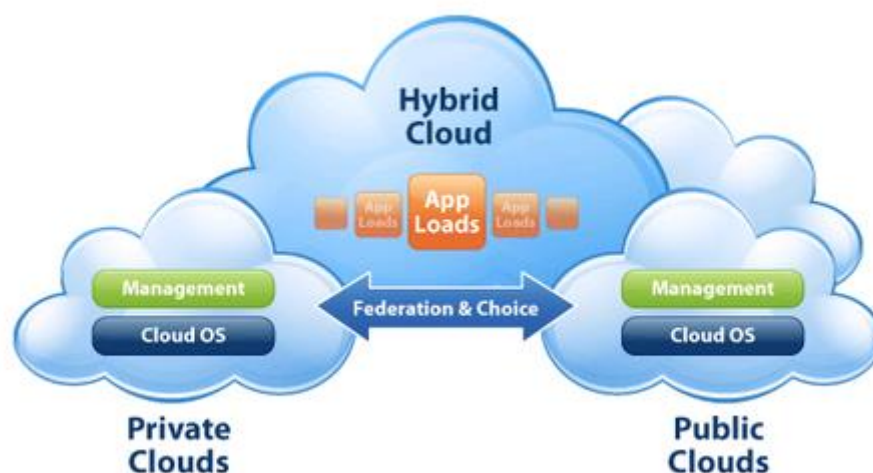
Service arbitrage is similar to service aggregation except that the services being aggregated are not fixed. Service arbitrage means a broker has the flexibility to choose services from multiple agencies. The cloud broker, for example, can use a credit-scoring service to measure and select an agency with the best score.

3.3 Public, Private and Hybrid Clouds

Cloud Deployment Models

Cloud computing spans a range of classifications, types and architecture models. The transformative networked computing model can be categorized into three major types:

- ✓ **Public cloud**
- ✓ **Private cloud**
- ✓ **Hybrid cloud**



Public Cloud: the cloud services are exposed to the public and can be used by anyone. Virtualization is typically used to build the cloud services that are offered to the public. An example of a public cloud is Amazon Web Services (AWS).

The **public cloud** refers to the cloud computing model with which the IT services are delivered across the Internet. The service may be free, freemium, or subscription-based, charged based on the computing resources consumed. The computing functionality may range from common services such as

email, apps and storage to the enterprise-grade OS platform or infrastructure environments used for software development and testing.

The cloud vendor is responsible for developing, managing and maintaining the pool of computing resources shared between multiple tenants from across the network. The defining features of a public cloud solution include high elasticity and scalability for IT-enabled services delivered at a low cost subscription-based pricing tier. As the most popular model of cloud computing services, the public cloud offers vast choices in terms of solutions and computing resources to address the growing needs of organizations of all sizes and verticals.

When to use the public cloud?

The public cloud is most suitable for situations with these needs:

- Predictable computing needs, such as communication services for a specific number of users.
- Apps and services necessary to perform IT and business operations.
- Additional resource requirements to address varying peak demands.
- Software development and test environments.

Advantages of public cloud

- No investments required to deploy and maintain the IT infrastructure.
- High scalability and flexibility to meet unpredictable workload demands.
- Reduced complexity and requirements on IT expertise as the cloud vendor is responsible to manage the infrastructure.
- Flexible pricing options based on different SLA offerings.
- The cost agility allows organizations to follow lean growth strategies and focus their investments on innovation projects.

Limitations of public cloud

- The total cost of ownership (TCO) can rise exponentially for large-scale usage, specifically for mid size to large enterprises.
- Not the most viable solution for security and availability sensitive mission-critical IT workloads
- Low visibility and control into the infrastructure, which may not suffice to meet regulatory Compliance.

Private Cloud: the cloud services used by a single organization, which are not exposed to the public. A private cloud resides inside the organization and must be behind a firewall, so only the organization has access to it and can manage it.

The **private cloud** refers to the cloud solution dedicated for use by a single organization. The data center resources may be located on-premise or operated by a third-party vendor off-site. The computing resources are isolated and delivered via a secure private network, and not shared with other customers.

Private cloud is customizable to meet the unique business and security needs of the organization. With greater visibility and control into the infrastructure, organizations can operate compliance-sensitive IT workloads without compromising on the security and performance previously only achieved with dedicated on-premise data centers.

When to use the private cloud?

The private cloud is often suitable for:

- Highly-regulated industries and government agencies.
- Technology companies that require strong control and security over their IT workloads and the underlying infrastructure.
- Large enterprises that require advanced data center technologies to operate efficiently and cost-Effectively.
- Organizations that can afford to invest in high performance and availability technologies.

Advantages of private cloud

- Dedicated and secure environments that cannot be accessed by other organizations.
- Compliance to stringent regulations as organizations can run protocols, configurations and measures to customize security based on unique workload requirements.
- High scalability and efficiency to meet unpredictable demands without compromising on security and performance.
- High SLA performance and efficiency. Flexibility to transform the infrastructure based on ever-changing business and IT needs of the Organization.

Limitations of private cloud

- Expensive solution with a relatively high total cost of ownership as compared to public cloud alternatives for short-term use cases.
- Mobile users may have limited access to the private cloud considering the high security measures in place
- The infrastructure may not offer high scalability to meet unpredictable demands if the cloud data center is limited to on-premise computing resources

Hybrid Cloud: the cloud services can be distributed among public and private clouds, where sensitive applications are kept inside the organization's network (by using a private cloud), whereas other services can be hosted outside the organization's network (by using a public cloud). Users can then interchangeably use private as well as public cloud services in every day operations.

The hybrid cloud

The **hybrid cloud** refers to the cloud infrastructure environment that is a mix of public and private cloud solutions. The resources are typically orchestrated as an integrated infrastructure environment. Apps and data workloads can share the resources between public and private cloud deployment based on organizational business and technical policies around security, performance, scalability, cost and efficiency, among other aspects.

For instance, organizations can use private cloud environments for their IT workloads and complement the infrastructure with public cloud resources to accommodate occasional spikes in network traffic. As a result, access to additional computing capacity does not require the high [CapEx](#) of a private cloud environment but is delivered as a short-term IT service via a public cloud solution. The environment itself is seamlessly integrated to ensure optimum performance and scalability to changing business needs.

When to use the hybrid cloud

Here's who the hybrid cloud might suit best:

- Organizations serving multiple verticals facing different IT security, regulatory and performance Requirements.
- Optimizing cloud investments without compromising on the value proposition of either public or private cloud technologies
- Improving security on existing cloud solutions such as SaaS offerings that must be delivered via secure private networks.
- Strategically approaching cloud investments to continuously switch and tradeoff between the best cloud service delivery model available in the market Advantages of hybrid cloud.
- Flexible policy-driven deployment to distribute workloads across public and private infrastructure environments based on security, performance and cost requirements.
- Scalability of public cloud environments is achieved without exposing sensitive IT workloads to the inherent security risks.
- High reliability as the services are distributed across multiple data centers across public and private data centers.
- Improved security posture as sensitive IT workloads run on dedicated resources in private clouds
- while regular workloads are spread across inexpensive public cloud infrastructure to tradeoff for cost investments

Limitations of hybrid cloud

- It can get expensive.
- Strong compatibility and integration is required between cloud infrastructure spanning different locations and categories. This is a limitation with public cloud deployments, for which organizations lack direct control over the infrastructure.
- Additional infrastructure complexity is introduced as organizations operate and manage an evolving mix of private and public cloud architecture.

Difference	Private	Public	Hybrid
Tenancy	Single tenancy: there's only the data of a single organization stored in the	Multi-tenancy: the data of multiple organizations is stored in a shared	The data stored in the public cloud is usually multi-tenant, which means the data from multiple organizations is

	cloud.	environment.	stored in a shared environment. The data stored in private cloud is kept private by the organization.
Exposed to the Public	No: only the organization itself can use the private cloud services.	Yes: anyone can use the public cloud services.	The services running on a private cloud can be accessed only the organization's users, while the services running on public cloud can be accessed by anyone.
Data Center Location	Inside the organization's network.	Anywhere on the Internet where the cloud service provider's services are located.	Inside the organization's network for private cloud services as well as anywhere on the Internet for public cloud services.
Cloud Service Management	The organization must have their own administrators managing their private cloud services.	The cloud service provider manages the services, where the organization merely uses them.	The organization itself must manage the private cloud, while the public cloud is managed by the CSP.
Hardware Components	Must be provided by the organization itself, which has to buy physical servers to build the private cloud on.	The CSP provides all the hardware and ensures it's working at all times.	The organization must provide hardware for the private cloud, while the hardware of CSP is used for public cloud services.
Expenses	Can be quite expensive, since the hardware, applications and network have to be provided and managed by the organization itself.	The CSP has to provide the hardware, set-up the application and provide the network accessibility according to the SLA.	The private cloud services must be provided by the organization, including the hardware, applications and network, while the CSP manages the public cloud services.

3.4 IaaS – PaaS – SaaS

Cloud computing architecture comes in many different flavors, three of which are popular among enterprises attempting to launch and manage websites, microsites and apps including, IaaS, PaaS and SaaS.

The acronyms stand for:

- **IaaS - Infrastructure as a Service**
- **PaaS - Platform as a Service**
- **SaaS - Software as a Service**

Infrastructure as a service (IaaS)

A vendor provides clients pay-as-you-go access to storage, networking, servers and other computing resources in the cloud.

Platform as a service (PaaS)

A service provider offers access to a cloud-based environment in which users can build and deliver applications. The provider supplies underlying infrastructure.

Software as a service (SaaS)

A service provider delivers software and applications through the internet. Users subscribe to the software and access it via the web or vendor APIs.

What is Infrastructure as a Service? (IaaS)

With IaaS, a brand essentially buys or rents server space from a vendor. They can then take advantage of the scaling potential guaranteed by the vendor while managing every detail of their applications — from operating system to middleware to runtime — without any assistance from the IaaS vendor.

IaaS or “infrastructure-as-a-service” is often used to describe “cloud services” or “managed infrastructure services”. IaaS involves providers offering dedicated or cloud based server and network infrastructure where the provider generally manages the hardware (swapping failed hard drives and that sort of thing) and sometimes the operating system of the infrastructure itself,” said David Vogelpohl, VP of Web Strategy at Austin, TX.-based WP Engine. “Many IaaS providers provide additional offerings like easy deployment options, stand-alone products which can be used on the IaaS provider’s platform, and some application layer services and support. IaaS is generally a good fit for organizations looking for high levels of customization for their infrastructure.

IaaS Examples

- AWS (Amazon Web Services)
- Google Compute
- Microsoft Azure

What is Platform as a Service (PaaS)

With platform-as-a-service or PaaS, the vendor gives its clients or customers the same server space and flexibility, but with some additional tools to help build or customize applications more rapidly. Furthermore, a PaaS vendor handles things like runtime, middleware, operating system, virtualization and storage — although the client or customer manages their own applications and data.

PaaS describes [an offering made up of] both the infrastructure and software for building digital applications. PaaS providers generally specialize in creating certain types of applications, like eCommerce applications for example, Vogelpohl told CMSWire. He went on to explain how some PaaS providers offer dedicated or virtualized hardware, and some hide the infrastructure layer from the customer for ease of use. “PaaS is generally a good fit for organizations building a particular type of application which would benefit from the additional features and management offered by the PaaS for that type of application. PaaS can require a high degree of technical proficiency; however, PaaS providers often include products and features that make it easier for non-technical customers to create digital applications.

PaaS Examples

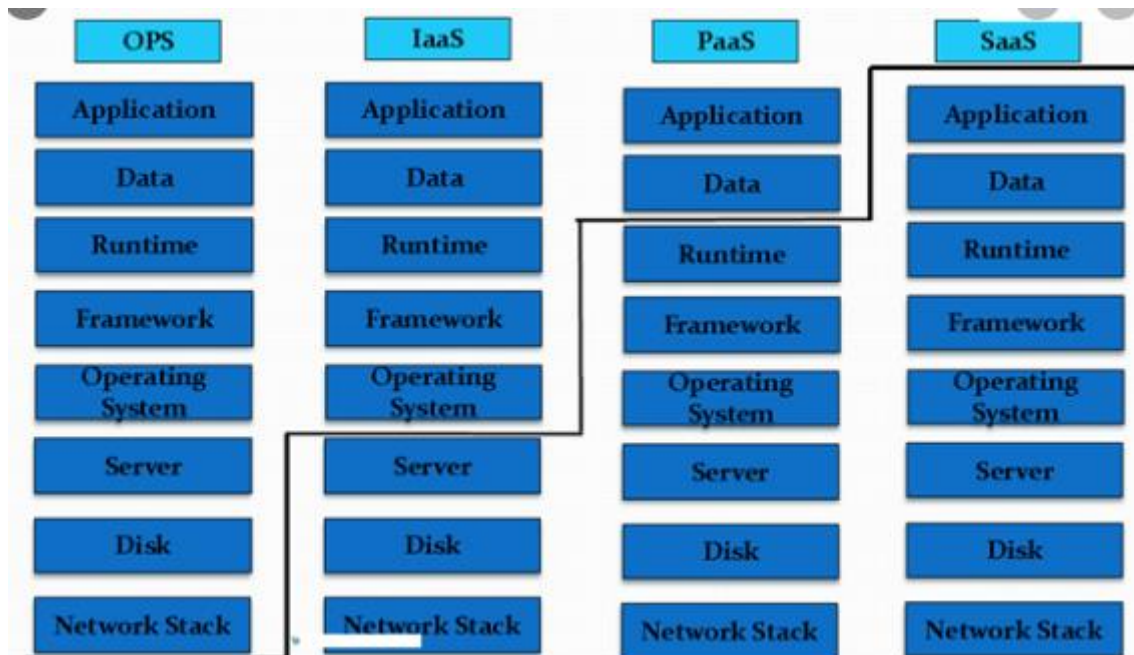
- Google App Engine
- Heroku
- OutSystems

What is Software as a Service (SaaS)

Software-as-a-service basically handles all the technical stuff while at the same time providing an application (or a suite of applications) that the client or customer can use to launch projects immediately or at least, faster than they would do with an IaaS or PaaS solution, both of which require more technical input from the client or customer. Coincidentally, most, if not all, SaaS vendors use IaaS or PaaS Solutions to support their suite of applications, handling the technical elements so their customers don’t have to. Whiteside told CMSWire that SaaS is the least hands-on of the three cloud computing solutions and is good if you don’t have developer resources but need to provide capabilities to end users. “You won’t have visibility or control of your infrastructure and are restricted by the capabilities and configuration of the software tools. This can be restrictive when you want to integrate with other systems you may own and run, but does allow you to get up and running quickly.

SaaS Examples

- Google G Suite
- Microsoft Office 365
- Mailchimp



3.5 Cloud Storage

Cloud storage is a cloud computing model that stores data on the Internet through a cloud computing provider who manages and operates data storage as a service. It's delivered on demand with just-in-time capacity and costs, and eliminates buying and managing your own data storage infrastructure.

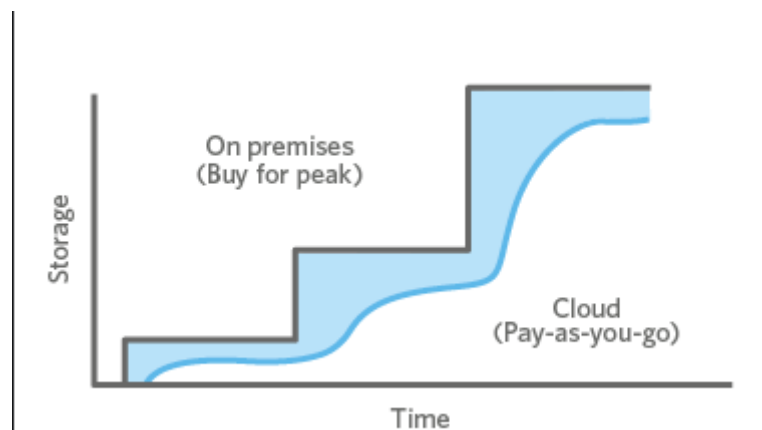


Fig Cloud Storage

Cloud storage is a service model in which data is transmitted and stored on remote storage systems, where it is maintained, managed, backed up and made available to users over a network (typically the internet). Users generally pay for their cloud data storage on a per-consumption, monthly rate. Although the per-gigabyte cost has been radically driven down, cloud storage providers have added operating expenses that can make the technology considerably more expensive to use. The security of cloud storage services continues to be a concern among users. Service providers have tried to allay those fears by enhancing their security capabilities by incorporating data encryption, multi-factor authentication and improved physical security into their services.

Types of cloud storage

There are three main cloud-based storage access models: public, private and hybrid.

Public cloud storage services provide a multi-tenant storage environment that is most suited for unstructured data on a subscription basis. Data is stored in the service providers' data centers with storage data spread across multiple regions or continents. Customers generally pay on a per-use basis similar to the utility payment model; in many cases, there are also transaction charges based on frequency and the volume of data being accessed. This market sector is dominated by Amazon Simple Storage

Service (S3), Amazon Glacier for cold or deep archival storage, Google Cloud Storage, Google Cloud Storage Nearline for cold data and Microsoft Azure.

Private cloud storage service is provided by in-house storage resources deployed as a dedicated environment protected behind an organization's firewall. Internally hosted private cloud storage implementations emulate some of the features of commercially available public cloud services, providing easy access and allocation of storage resources for business users, as well as object storage protocols. Private clouds are appropriate for users who need customization and more control over their data, or who have stringent data security or regulatory requirements.

Hybrid cloud storage is a mix of private cloud storage and third-party public cloud storage services with a layer of orchestration management to integrate operationally the two platforms. The model offers businesses flexibility and more data deployment options. An organization might, for example, store actively used and structured data in an on-premises cloud, and unstructured and archival data in a public cloud. A hybrid environment also makes it easier to handle seasonal or unanticipated spikes in data creation or access by "cloud bursting" to the external storage service and avoiding having to add in-house storage resources. In recent years, there has been increased adoption of the hybrid cloud model. Despite its benefits, a hybrid cloud presents technical, business and management challenges. For example, private workloads must access and interact with public cloud storage providers, so compatibility and reliable, ample network connectivity are very important factors. An enterprise-level cloud storage system should be scalable to suit current needs, accessible from anywhere and application-agnostic.

Cloud storage characteristics

Cloud storage is based on a virtualized infrastructure with accessible interfaces, near-instant elasticity and scalability, multi-tenancy and metered resources. Cloud-based data is stored in logical pools across disparate, commodity servers located on premises or in a data center managed by a third-party cloud provider. Using the RESTful API, an object storage protocol stores a file and its associated metadata as a single object and assigns it an ID number. When content needs to be retrieved, the user presents the ID to the system and the content is assembled with all its metadata, authentication and security.

In recent years, object storage vendors have added file system functions and capabilities to their object storage software and hardware largely because object storage was not being adopted fast enough. For example, a cloud storage gateway can provide a file system emulation front end to their object storage; that arrangement often allows applications to access the data without actually supporting an object storage protocol. All backup applications use the object storage protocol, which is one of the reasons why online backup to a cloud service was the initial successful application for cloud storage.

Most commercial cloud storage services use vast numbers of hard drive storage systems mounted in servers that are linked by a mesh-like network architecture. Service providers have also added high-performance layers to their virtual storage offerings, typically comprising some type of solid state drives (SSDs). High-performance clouds storage is generally most effective if the servers and applications accessing the storage are also resident in the cloud environment. Companies that use public cloud storage need to have the appropriate network access to the hosting service.

Benefits of cloud storage

Cloud storage provides many benefits that result in cost-savings and greater convenience for its users. These benefits include:

Pay for what is used. With a cloud storage service, customers only pay for the storage they actually use so there's no need for big capital expenses. While cloud storage costs are recurring rather than a one-time purchase, they are so low that even as an ongoing expense they may still be less than the cost of maintaining an in-house system.

Utility billing. Since customers only pay for the capacity they're using, cloud storage costs can decrease as usage drops. This is in stark contrast to using an in-house storage system, which will likely be overconfigured to handle anticipated growth; so, a company will pay for more than it needs initially, and the cost of the storage will never decrease.

Global availability. Cloud storage is typically available from any system anywhere at any time; one does not have to worry about operating system capability or complex allocation processes.

Ease of use. Cloud storage is easier to access and use, so developers, software testers and business users can get up and running quickly without have to wait for IT to allocate and configure storage resources.

Offsite security. By its very nature, public cloud storage offers a way to move copies of data to a remote site for backup and security purposes. Again, this represents a significant cost-savings when compared to a company maintaining its own remote facility.

An in-house cloud storage system can offer some of the above ease-of-use features of a public cloud service, but it will lack much of the storage capacity flexibility of a public service. Some hardware vendors are trying to address this issue by allowing their customers to turn on and off capacity that has already been installed in their arrays.

Drawbacks of cloud storage

There are some shortcomings to cloud storage -- particularly the public services -- that may deter companies from using these services or limit how they use them.

Security is the single most cited factor that may make a company reluctant -- or at least cautious -- about using public cloud storage. The concern is that once data leaves a company's premises, the company no longer has control over how it's handled and stored. There are also concerns about storing data that is regulated by specific compliance laws. Cloud providers address these concerns by making public the steps they take to protect their customers' data, such as encryption for data in flight and at rest, physical security and storing data at multiple locations.

Access to data stored in the cloud may also be an issue and could significantly increase the cost of using cloud storage. A company may need to upgrade its connection to the cloud storage service to handle the volume of data it expects to transmit; the monthly cost of an optical link can run into the thousands of dollars.

A company may run into performance issues if its in-house applications need to access the data it has stored in the cloud. In those cases, it will likely require either moving the servers and applications into the same cloud or bringing the necessary data back in-house.

If a company requires a lot of cloud storage capacity and frequently moves its data back and forth, the monthly costs can be quite high. Compared to deploying the storage in-house, the ongoing costs could eventually surpass the cost of implementing and maintaining the on-premises system.

Cloud storage pros/cons

Advantages of private cloud storage include high reliability and security. But this approach to cloud storage provides limited scalability and requires on-site resources and maintenance. Public cloud storage offers high scalability and a pay-as-you-go model with no need for an on-premises storage infrastructure. However, performance and security measures can vary by service provider. In addition, reliability depends on service provider availability and internet connectivity.

Cloud storage and data migration

Migrating data from one cloud storage service to another is an often-overlooked area. Cloud migrations have become more common due to market consolidation and price competition. Businesses tend to switch cloud storage providers either because of price -- which must be substantially cheaper to justify the cost and work of switching -- or when a cloud provider goes out of business or stops providing storage services. With public cloud providers, it is usually just as easy to copy data out of the cloud as it was to upload data to it. Available bandwidth can become a major issue, however. In addition, many providers charge extra to download data.

To mitigate concerns about a provider going out of business, you could copy data to more than one cloud storage service. While this increases cloud storage costs, it is often still cheaper than maintaining data locally.

Should that not be the case, or if bandwidth becomes a major sticking point, find out if the original and the new cloud storage service have a direct-connect relationship. This approach also removes the need of cloud storage customers to use their data centers as a bridge or go-between such as using an on-premises cache to facilitate the transfer of data between the two cloud storage providers.

3.6 Storage-as-a-Service

Storage as a service (SaaS) is a cloud business model in which a company leases or rents its storage infrastructure to another company or individuals to store data. Small companies and individuals often find this to be a convenient methodology for managing backups, and providing cost savings in personnel, hardware and physical space. As an alternative to storing magnetic tapes offsite in a vault, IT administrators are meeting their storage and backup needs by service level agreements (SLAs) with an SaaS provider, usually on a cost-per-gigabyte-stored and cost-per-data-transferred basis. The client transfers the data meant for storage to the service provider on a set schedule over the SaaS provider's wide area network or over the Internet.

The storage provider provides the client with the software required to access their stored data. Clients use the software to perform standard tasks associated with storage, including data transfers and data backups. Corrupted or lost company data can easily be restored. Storage as a service is prevalent among small to mid-sized businesses, as no initial budget is required to set up hard drives, servers and IT staff. SaaS is also marketed as an excellent technique to mitigate risks in disaster recovery by providing long-term data storage and enhancing business stability. Storage as a service is fast becoming the method of choice to all small and medium scale businesses. This is because storing files remotely rather than locally boasts an array of advantages for professional users.

Who uses storage as a service and why?

Storage as a Service is usually used by small or mid-sized companies that lack the budget to implement and maintain their own storage infrastructure.

Organizations use storage as a service to mitigate risks in disaster recovery, provide long-term retention and enhance business continuity and availability.

How storage as a service works?

The company would sign a service level agreement (SLA) whereby the STaaS provider agreed to rent storage space on a cost-per-gigabyte-stored and cost-per-data-transfer basis and the company's data would be automatically transferred at the specified time over the storage provider's proprietary WAN or the Internet. If the company ever loses its data, the network administrator could contact the STaaS provider and request a copy of the data.

Advantage of Storage as Services

Cost– factually speaking, backing up data isn't always cheap, especially when take the cost of equipment into account. Additionally, there is the cost of the time it takes to manually complete routine backups. Storage as a service reduces much of the cost associated with traditional backup methods, providing ample storage space in the cloud for a low monthly fee.

Invisibility – Storage as a service is invisible, as no physical presence of it is seen in its deployment and so it doesn't take up valuable office space.

Security – In this service type, data is encrypted both during transmission and while at rest, ensuring no unauthorized user access to files.

Automation – Storage as a service makes the tedious process of backing up easy to accomplish through automation. Users can simply select what and when they want to backup, and the service does all the rest.

Accessibility – By going for storage as a service, users can access data from smart phones, netbooks to desktops and so on.

Syncing – Syncing ensures your files are automatically updated across all of your devices. This way, the latest version of a file a user saved on their desktop is available on your smart phone.

Sharing – Online storage services allow the users to easily share data with just a few clicks

Collaboration – Cloud storage services are also ideal for collaboration purposes. They allow multiple people to edit and collaborate on a single file or document. Thus, with this feature users need not worry about tracking the latest version or who has made what changes.

Data Protection – By storing data on cloud storage services, data is well protected by all kind of catastrophes such as floods, earthquakes and human errors.

Disaster Recovery – as said earlier, data stored in cloud is not only protected from catastrophes by having the same copy at several places, but can also favor disaster recovery to ensure business continuity.

3.7 Advantages of Cloud Storage

Cloud Storage is a service where data is remotely maintained, managed, and backed up. The service allows the users to store files online, so that they can access them from any location via the

Internet. According to a recent survey conducted with more than 800 business decision makers and users worldwide, the number of organizations gaining competitive advantage through high cloud adoption has almost doubled in the last few years and by 2017, the public cloud services market is predicted to exceed \$244 billion. Now, let's look into some of the advantages and disadvantages of Cloud Storage.

Advantages of Cloud Storage

Usability: All cloud storage services reviewed in this topic have desktop folders for Mac's and PC's. This allows users to drag and drop files between the cloud storage and their local storage.

Bandwidth: You can avoid emailing files to individuals and instead send a web link to recipients through your email.

Accessibility: Stored files can be accessed from anywhere via Internet connection.

Disaster Recovery: It is highly recommended that businesses have an emergency backup plan ready in the case of an emergency. Cloud storage can be used as a back-up plan by businesses by providing a second copy of important files. These files are stored at a remote location and can be accessed through an internet connection.

Cost Savings: Businesses and organizations can often reduce annual operating costs by using cloud storage; cloud storage costs about 3 cents per gigabyte to store data internally. Users can see additional cost savings because it does not require internal power to store information remotely.

Disadvantages of Cloud Storage

Usability: Be careful when using drag/drop to move a document into the cloud storage folder. This will permanently move your document from its original folder to the cloud storage location. Do a copy and paste instead of drag/drop if you want to retain the document's original location in addition to moving a copy onto the cloud storage folder.

Bandwidth: Several cloud storage services have a specific bandwidth allowance. If an organization surpasses the given allowance, the additional charges could be significant. However, some providers allow unlimited bandwidth. This is a factor that companies should consider when looking at a cloud storage provider.

Accessibility: If you have no internet connection, you have no access to your data.

Data Security: There are concerns with the safety and privacy of important data stored remotely. The possibility of private data commingling with other organizations makes some businesses uneasy. If you want to know more about those issues that govern data security and privacy, here is an interesting article on the recent privacy debates.

Software: If you want to be able to manipulate your files locally through multiple devices, you'll need to download the service on all devices.

3.8 Cloud Storage Providers

Cloud storage lets users store and sync their data to an online server.

Because they are stored in the cloud rather than on a local drive, files are available on various devices. This allows a person to access files from multiple computers, as well as mobile devices to view, edit, and comment on files. It replaces workarounds like emailing yourself documents. Cloud storage can also act as a backup system for your hard drive.

Cloud storage solutions support a variety of file types. Supported files typically include:

- **Text documents**

- **Pictures**
- **Videos**
- **Audio files**

The most user-friendly cloud storage solutions integrate with other applications for easy edits, playback, and sharing. Cloud storage is used by individuals to manage personal files, as well as by businesses for file sharing and backup. Some feature sets are very important to businesses but may not be relevant to individuals. Admin and security features, for example, are designed for corporate enterprises where data security and availability are concerns for files stored in the cloud.

Cloud Storage Features & Capabilities

File Management

These features are core to every cloud storage platform. Typically, file management capabilities include:
A search function to easily find files and search within files

- **Device syncing to update files connected to the cloud across devices**
- **A web interface, with no install required**
- **Support for multiple file types**

Collaboration

Most cloud storage providers also feature collaboration functionality. Not all tools will have the same level of tracking and control. Collaboration features may include:

Notifications when files are changed by others

- **File sharing, with the ability to set editing and view-only permissions**
- **Simultaneous editing**
- **Change tracking and versioning**

Security & Administration

- Security and administration features are important considerations for enterprises. Security is particularly important for storing sensitive, private data in the cloud.
- Cloud storage providers offer different levels of security to address concerns. For example, Google Drive lets users set up two-step verification. CertainSafe is HIPAA compliant. Code42 lets users backup files in the cloud and on another machine. This is so that they are safe and available should something happen to the user's local drive, even without internet access.
- Products like Box and SpiderOak support password protected files. Consumer-friendly solutions like Dropbox and Google Drive do not.
- In addition to protection from outside access, users should consider the provider's security policy. Microsoft reserves the right to scan OneDrive user files. Google states it will not access user files on Google Drive unless prompted by law enforcement.

Possible security and administration features include:

- Single sign-on with Active Directory/SAML based identity providers
- Two-step verification for added security
- End-user encryption (for integrations)
- User and role management
- Control over file access, sharing and editing permissions
- Storage limits for individual users or groups
- Choosing where files will be stored, individual users' storage management
- Device management, restricting access to certain devices

File Sharing

File sharing is one of the most common uses for cloud storage. Most cloud storage providers offer a mechanism to let users share files. The level of access, versioning, and change tracking varies by product. Some providers put a cap on file upload size. This is important for anyone looking to upload and share files larger than 2GB.

File sharing is executed in a few different ways:

- **Making users co-owners of files**
- **Sending files to users**

- **Emailing users a link to the file in the cloud**

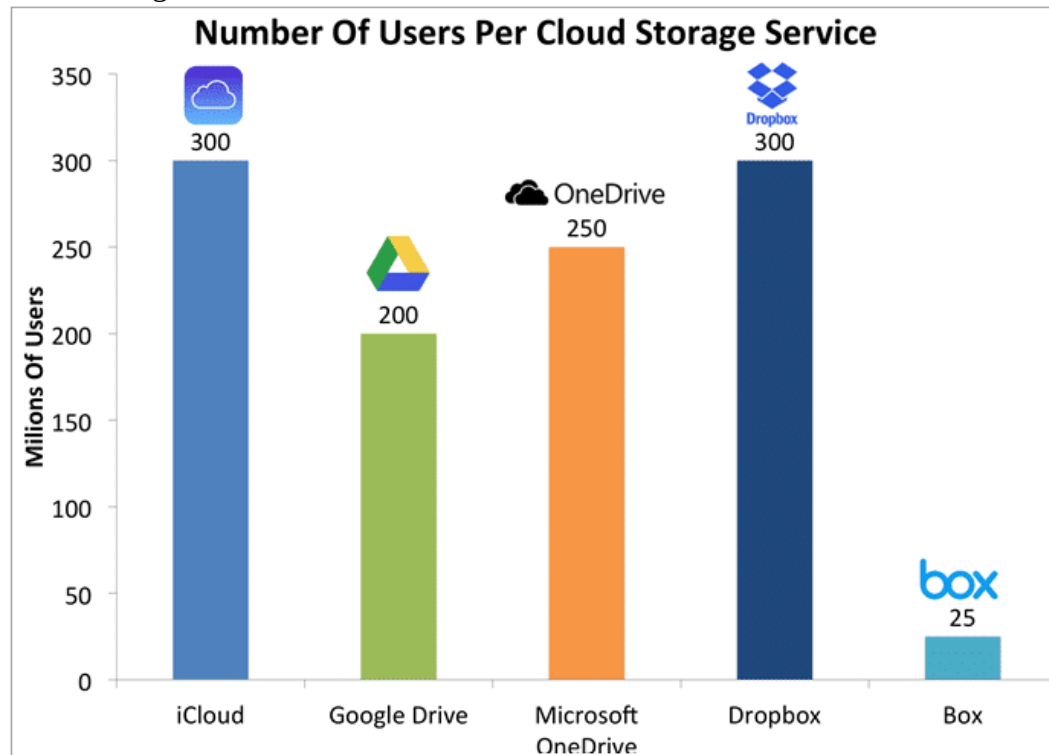
- Cloud storage solutions that prioritize availability and make file sharing easy often aren't as strong on security. The reverse is true as well.
- Cloud Storage Platform Considerations
- The platform's performance, reliability, and integrations are all important considerations for any business use case. Some enterprise cloud storage platforms monitor user activity and storage and offer reporting capabilities for platform administrators. International businesses can find multilingual and multi-currency capabilities on some platforms.

Free Cloud Storage

- Many cloud storage providers offer some amount of storage space for free. For example, DropBox offers 2GB of free storage, and Google Drive offers 15GB. Sometimes providers have a hard limit on free storage. Other providers, like Microsoft OneDrive, incentivize more storage with referral programs.
- Free accounts do not usually include all the features available to paid customers. Cloud storage vendors with advanced security features do not usually have free accounts.

Paid Cloud Storage

For users who need to move beyond free options, pricing for cloud storage is typically per user, per month. Plans usually have a fixed storage capacity, with prices increasing for more storage and/or added features. Users can find paid cloud storage options with monthly costs as low as \$10 for 1TB of storage.



- Many cloud storage providers offer a free plan for those who require the minimum out of their service. Cloud storage providers offer much data security for business users.
- If we are to compare the cloud storage providers, then all will look similar at first glance. Hence, most of them compare the providers based on the prices and decide which one to select. The features that you should look in the cloud storage providers include collaboration features, usability, and security provided by the company.
- Also, the support provided by these providers must also be considered. While selecting the cloud storage provider, you must consider your platform for use like Windows, Mac, iPhones, Androids, BlackBerry phones or mix. Big tech players have their own platforms for cloud storage as Windows have OneDrive and Mac has iCloud.

Cloud Storage Providers	Best For	Suitable for business size	Storage space plans	Platform	File Upload Limit	Price:
pCloud 	Storing large files.	Personal, Family, and Small businesses.	10GB 2TB	Windows, Mac, Linux, iOS, Android	2TB	Free storage of 10GB. Annual Plans: \$3.99 per month for 500 GB and \$7.99 per month for 2TB. Lifetime Plans: One-time fee of \$175 for 500GB and \$359 for 2TB.
IDrive 	It is mainly for Backup.	Freelancers, solo workers, teams, & businesses of any size.	5GB, 2TB, 5TB, 250GB , 500 GB, & 1.25 TB.	Windows, Mac, iOS, Android.	2GB	Free: 5GB IDrive Personal 2TB: \$104.25. IDrive Business: \$149.25.
Dropbox 	Light data users.	Freelancers, solo workers, teams, & businesses of any size.	2GB, 1TB, 2TB, 3TB, Till Unlimited.	Windows, Mac OS, Linux, Android, iOS, Windows phone.	Unlimited	Plans for Individuals starts at \$8.25/ month. Plans for teams start at \$12.50/user/month
Google Drive 	Teams & Collaboration	Individuals & Teams.	15GB, 100GB , 200GB .. Till Unlimited.	Windows, Mac OS, Android, iOS.	5TB	Free for 15GB. 200GB: \$2.99 per month. 2TB: \$9.99/month. 30TB: \$299.99/month.
OneDrive 	Windows users	--	5GB, 50GB, 1TB, 6TB, & Unlimited.	Windows, Android, iOS.	15GB	Free: 5GB. The paid plan starts at \$1.99 per month.
Box 	Enterprise solutions	Small teams and Enterprises.	10GB.	It is accessible from any device.	5GB	Free for 10GB. The paid plan starts at \$10/month.

3.8 S3-Simple Storage Service

Amazon S3 or Amazon Simple Storage Service is a service offered by Amazon Web Services that provides object storage through a web service interface. Amazon S3 uses the same scalable storage infrastructure that Amazon.com uses to run its global e-commerce network.

Amazon Simple Storage Service (Amazon S3) is an object storage service that offers industry-leading scalability, data availability, security, and performance. This means customers of all sizes and industries can use it to store and protect any amount of data for a range of use cases, such as websites, mobile applications, backup and restore, archive, enterprise applications, IoT devices, and big data analytics. Amazon S3 provides easy-to-use management features so you can organize your data and configure finely-tuned access controls to meet your specific business, organizational, and compliance requirements.

Fundamentally there are two types of storage:-

1. Object-Based Storage
2. Block Based Storage

Simple Storage Service (S3) - Object-Based Storage

Provides developers and IT Teams with secure durable, highly scalable object storage. Easy to use simple web interface to store and retire any amount of data from anywhere on the web.

It is a place to store your files on the AWS cloud Dropbox was born. By simplifying the User interface of S3. Think about Dropbox as a layer built on top of S3.

Data is spread across multiple devices and facilities

Think about S3 to store your photos or files.

1. Object Base Storage
2. Unlimited Storage
3. Files are Stored in Buckets/Folders
4. Names must be unique globally
5. Every time you have a successful upload you get a http 200 code back

S3 Is primarily used for:-

1. Store and Backup
2. Application File Hosting
3. Media Hosting
4. Software Delivery
5. Storing AMI's and Snapshots

Data consistency Model – S3

Read after write consistency for PUTS of new objects Eventual consistency for overwrite PUTS and deletes (i.e. takes time to propagate)

Objects Consists of the following

1. Key – this is simply the file name of the object
2. Value – the data and it is made up of a sequence of bytes.
3. Versioning – which version of the object is this
4. Meta Data: Data about data, the data about the data file you are storing.

Think, if you are storing a music track/song. This would have metadata like the information of the singer, the year it was released, the name of the album etc.

Sub resources

1. Access Control list – this determines who can access the file on S3. This can be done at the file level or at the Bucket level.
2. Torrent – supports the Bit torrent protocol
3. Built for 99.99% availability of the S3 Amazon SLA 99.9% platform
4. Durability guarantee – 99.9... (11.9s) (more)?
5. Tiered storage Availability
6. Lifecycle management
7. Versioning
8. Encryption
9. Secure the data using Access control lists and Bucket policies

S3 – IA (Infrequently Accessed) For data that is accessed less frequently but requires rapid access when needed. This here costs lesser than S3 but you are charged for the retrieval of the data.

S3 – RRS (Reduced Redundancy Storage) Basically less durability with the same level of availability.

e.g. This about data you could potentially regenerate like a tax calculation or a payslip. This is cheaper than the original SS. Suppose you create thumbnails for all your pictures. If you lose a thumbnail you could always regenerate it.

Image comparing the various types of S3

	Standard	Standard - Infrequent Access	Reduced Redundancy Storage
Durability	99.999999999%	99.999999999%	99.99%
Availability	99.99%	99.9%	99.99%
Concurrent facility fault tolerance	2	2	1
SSL Support	Yes	Yes	Yes
First byte latency	Milliseconds	Milliseconds	Milliseconds
Lifecycle Management Policies	Yes	Yes	Yes

Charging Model

1. Storage
2. Number of requests
3. Storage Management Pricing
4. Add metadata to see usage metrics.

Transfer Acceleration - Enables fast, easy and secure transfers of your files over long distances between your end users and an S3 bucket.

Transfer acceleration takes advantages of Amazon cloud front's globally distributed edge locations. As the data arrives at an edge location, the data is routed to Amazon S3 over an optimized network path.

Think of transfer acceleration as a combination of S3 + CDN natively supported by this Service. Basically, every user ends up going through the closest possible edge location which in turn talks to the actual S3 bucket.

Recap - S3

S3 Storage Classes

1. S3 (Durable, immediately available and frequently accessed)
2. S3 – IA (durable, immediately available, infrequently accessed.
3. S3 Reduced Redundancy Storage (Used for data that is easily reproducible, such thumbnails)

Core fundamentals of S3 objects

1. Key: Name of the object These are store in alphabetic order
2. Value: The data itself Version ID: The version of the object
3. Meta Data: The various attributes of the data

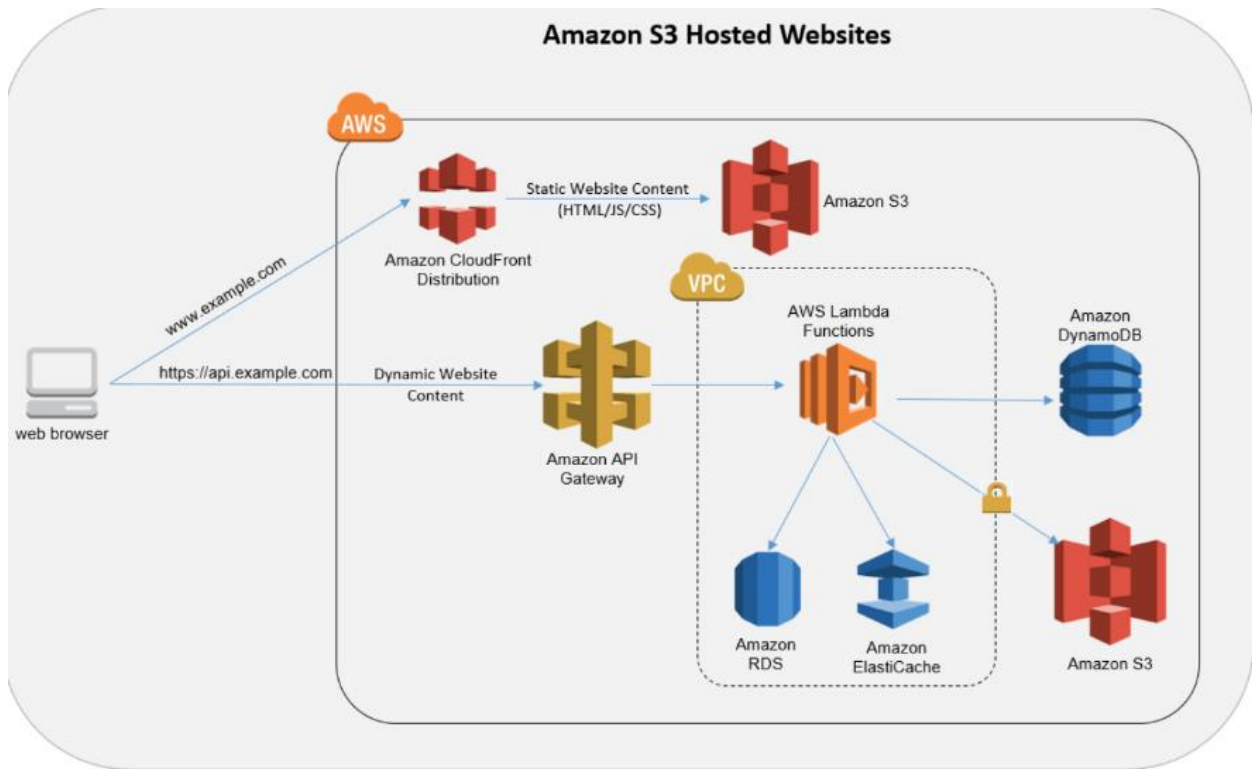
Sub resources

1. ACL: Access control lists
2. Torrent: bit Torrent protocol

Cross region Replication

This basically means that if you have this turned on then for a bucket AWS will automatically make a bucket available across 2 or more regions.

Example of an Amazon S3 Hosted Website Architecture



Securing your S3 Buckets

1. By default, all buckets are private
2. You can set up access control to your using
3. Bucket Policies
4. Access control lists (ACL)
5. S3 buckets can be configured to create access logs

Encryption for S3

1. In-transit
2. SSL/TLS (using HTTPS)
3. At Rest
4. Server Side Encryption
5. S3 Managed keys – SSE-S3
6. Server side Encryption
7. Key Management Service – Managed Key
8. SST – KMS
9. Client Side Encryption

Advantages of AWS S3 Service



Scalability on Demand

- If you want your application's scalability varying according to the change in traffic, then AWS S3 is a very good option.
- Scaling up or down is just mouse-clicks away when you use other attractive features of AWS.

Content Storage and Distribution

- S3 in Amazon could be used as the foundation for a Content Delivery Network. Because Amazon S3 is developed for content storage and distribution.

Big Data and Analytics on Amazon S3

- Amazon QuickSight UI can be connected with Amazon S3, and then large amounts of data can be analyzed with it.

Backup and Archive

- Whether you need timely backups of your website, or store static files for once, or store versions of files you are currently working on. S3 in Amazon has got you covered.

Disaster Recovery

- Storing data in multiple availability zones in a region gives the user the flexibility to recover files, which are lost, as soon as possible. Also, the cross-region replication technology can be used to store in any number of Amazon's worldwide data centers.

UNIT IV RESOURCE MANAGEMENT AND SECURITY IN CLOUD

4.5 Inter-cloud Resource Management

This section characterizes the various cloud service models and their extensions. The cloud service trends are outlined. Cloud resource management and intercloud resource exchange schemes are reviewed. We will discuss the defense of cloud resources against network threats

4.5.1 Extended Cloud Computing Services

There are six layers of cloud services, ranging from hardware, network, and collocation to infrastructure, platform, and software applications. We already introduced the top three service layers as SaaS, PaaS, and IaaS, respectively. The cloud platform provides PaaS, which sits on top of the IaaS infrastructure. The top layer offers SaaS. These must be implemented on the cloud platforms provided. Although the three basic models are dissimilar in usage, they are built one on top of another. The implication is that one cannot launch SaaS applications with a cloud platform. The cloud platform cannot be built if compute and storage infrastructures are not there.

Cloud application (SaaS)			Concur, RightNOW, Teleo, Kenexa, Webex, Blackbaud, salesforce.com, Netsuite, Kenexa, etc.
Cloud software environment (PaaS)			Force.com, App Engine, Facebook, MS Azure, NetSuite, IBM BlueCloud, SGI Cyclone, eBay
Cloud software infrastructure			Amazon AWS, OpSource Cloud, IBM Ensembles, Rackspace cloud, Windows Azure, HP, Banknorth
Computational resources (IaaS)	Storage (DaaS)	Communications (CaaS)	
Collocation cloud services (LaaS)			Savvis, Internap, NTTCommunications, Digital Realty Trust, 365 Main
Network cloud services (NaaS)			Owest, AT&T, AboveNet
Hardware/Virtualization cloud services (HaaS)			VMware, Intel, IBM, XenEnterprise

Fig A stack of six layers of cloud services

The bottom three layers are more related to physical requirements. The bottommost layer provides Hardware as a Service (HaaS). The next layer is for interconnecting all the hardware components, and is simply called Network as a Service (NaaS). Virtual LANs fall within the scope of NaaS. The next layer up offers Location as a Service (LaaS), which provides a collocation service to house, power, and secure all the physical hardware and network resources. Some authors say this layer provides Security as a Service (“SaaS”). The cloud infrastructure layer can be further subdivided as Data as a Service (DaaS) and Communication as a Service (CaaS) in addition to compute and storage in IaaS. We will examine commercial trends in cloud services in subsequent sections. Here we will mainly cover the top three layers with some success stories of cloud computing. cloud players are divided into three classes: (1) cloud service providers and IT administrators, (2) software developers or vendors, and (3) end users or business users. These cloud players vary in their roles under the IaaS, PaaS, and SaaS models. The table entries distinguish the three cloud models as viewed by different players. From the software vendors’ perspective, application performance on a given cloud platform is most important. From the providers’ perspective, cloud infrastructure performance is the primary concern. From the end users’ perspective, the quality of services, including security, is the most important.

Table Cloud Differences in Perspectives of Providers, Vendors, and Users

Cloud Players	IaaS	PaaS	SaaS
IT administrators/cloud providers	Monitor SLAs	Monitor SLAs and enable service platforms	Monitor SLAs and deploy software
Software developers (vendors)	To deploy and store data	Enabling platforms via configurators and APIs	Develop and deploy software
End users or business users	To deploy and store data	To develop and test web software	Use business software

4.5.1.1 Cloud Service Tasks and Trends

Cloud services are introduced in five layers. The top layer is for SaaS applications, as further subdivided into the five application areas, mostly for business applications. For example, CRM is heavily practiced in business promotion, direct sales, and marketing services. CRM offered the first SaaS on the cloud successfully. The approach is to widen market coverage by investigating customer behaviors and revealing opportunities by statistical analysis. SaaS tools also apply to distributed collaboration, and financial and human resources management. These cloud services have been growing rapidly in recent years.

PaaS is provided by Google, Salesforce.com, and Facebook, among others. IaaS is provided by Amazon, Windows Azure, and RackRack, among others. Collocation services require multiple cloud providers to work together to support supply chains in manufacturing. Network cloud services provide communications such as those by AT&T, Qwest, and AboveNet. Details can be found in Clou’s introductory book on business clouds . The vertical cloud services in Figure 4.25 refer to a sequence of

cloud services that are mutually supportive. Often, cloud mashup is practiced in vertical cloud applications.

4.5.1.2 Software Stack for Cloud Computing

Despite the various types of nodes in the cloud computing cluster, the overall software stacks are built from scratch to meet rigorous goals. Developers have to consider how to design the system to meet critical requirements such as high throughput, HA, and fault tolerance. Even the operating system might be modified to meet the special requirement of cloud data processing. Based on the observations of some typical cloud computing instances, such as Google, Microsoft, and Yahoo!, the overall software stack structure of cloud computing software can be viewed as layers. Each layer has its own purpose and provides the interface for the upper layers just as the traditional software stack does. However, the lower layers are not completely transparent to the upper layers.

The platform for running cloud computing services can be either physical servers or virtual servers. By using VMs, the platform can be flexible, that is, the running services are not bound to specific hardware platforms. This brings flexibility to cloud computing platforms. The software layer on top of the platform is the layer for storing massive amounts of data. This layer acts like the file system in a traditional single machine. Other layers running on top of the file system are the layers for executing cloud computing applications. They include the database storage system, programming for large-scale clusters, and data query language support. The next layers are the components in the software stack.

4.5.1.3 Runtime Support Services

As in a cluster environment, there are also some runtime supporting services in the cloud computing environment. Cluster monitoring is used to collect the runtime status of the entire cluster. One of the most important facilities is the cluster job management system introduced. The scheduler queues the tasks submitted to the whole cluster and assigns the tasks to the processing nodes according to node availability. The distributed scheduler for the cloud application has special characteristics that can support cloud applications, such as scheduling the programs written in MapReduce style. The runtime support system keeps the cloud cluster working properly with high efficiency. Runtime support is software needed in browser-initiated applications applied by thousands of cloud customers.

The SaaS model provides the software applications as a service, rather than letting users purchase the software. As a result, on the customer side, there is no upfront investment in servers or software licensing. On the provider side, costs are rather low, compared with conventional hosting of user applications. The customer data is stored in the cloud that is either vendor proprietary or a publicly hosted cloud supporting PaaS and IaaS.

4.5.2 Resource Provisioning and Platform Deployment

The emergence of computing clouds suggests fundamental changes in software and hardware architecture. Cloud architecture puts more emphasis on the number of processor cores or VM instances. Parallelism is exploited at the cluster node level. In this section, we will discuss techniques to provision computer resources or VMs. Then we will talk about storage allocation schemes to interconnect distributed computing infrastructures by harnessing the VMs dynamically.

4.5.2.1 Provisioning of Compute Resources (VMs)

Providers supply cloud services by signing SLAs with end users. The SLAs must commit sufficient resources such as CPU, memory, and bandwidth that the user can use for a preset period. Underprovisioning of resources will lead to broken SLAs and penalties. Overprovisioning of resources will lead to resource underutilization, and consequently, a decrease in revenue for the provider. Deploying an autonomous system to efficiently provision resources to users is a challenging problem. The difficulty comes from the unpredictability of consumer demand, software and hardware failures, heterogeneity of services, power management, and conflicts in signed SLAs between consumers and service providers. Efficient VM provisioning depends on the cloud architecture and management of cloud infrastructures. Resource provisioning schemes also demand fast discovery of services and data in cloud computing infrastructures. In a virtualized cluster of servers, this demands efficient installation of VMs, live VM migration, and fast recovery from failures. To deploy VMs, users treat them as physical hosts with customized operating systems for specific applications. For example, Amazon's EC2 uses Xen as the virtual machine monitor (VMM). The same VMM is used in IBM's Blue Cloud. In the EC2 platform, some predefined VM templates are also provided. Users can choose different kinds of VMs from the templates. IBM's Blue Cloud does not provide any VM templates. In general, any type of VM can run on top of Xen. Microsoft also applies virtualization in its Azure cloud platform. The provider should offer

resource-economic services. Power-efficient schemes for caching, query processing, and thermal management are mandatory due to increasing energy waste by heat dissipation from data centers. Public or private clouds promise to streamline the on-demand provisioning of software, hardware, and data as a service, achieving economies of scale in IT deployment and operation.

4.5.2.2 Resource Provisioning Methods

Three cases of static cloud resource provisioning policies.

- In case (a), overprovisioning with the peak load causes heavy resource waste (shaded area).
- In case (b), underprovisioning (along the capacity line) of resources results in losses by both user and provider in that paid demand by the users (the shaded area above the capacity) is not served and wasted resources still exist for those demanded areas below the provisioned capacity.
- In case (c), the constant provisioning of resources with fixed capacity to a declining user demand could result in even worse resource waste. The user may give up the service by canceling the demand, resulting in reduced revenue for the provider. Both the user and provider may be losers in resource provisioning without elasticity

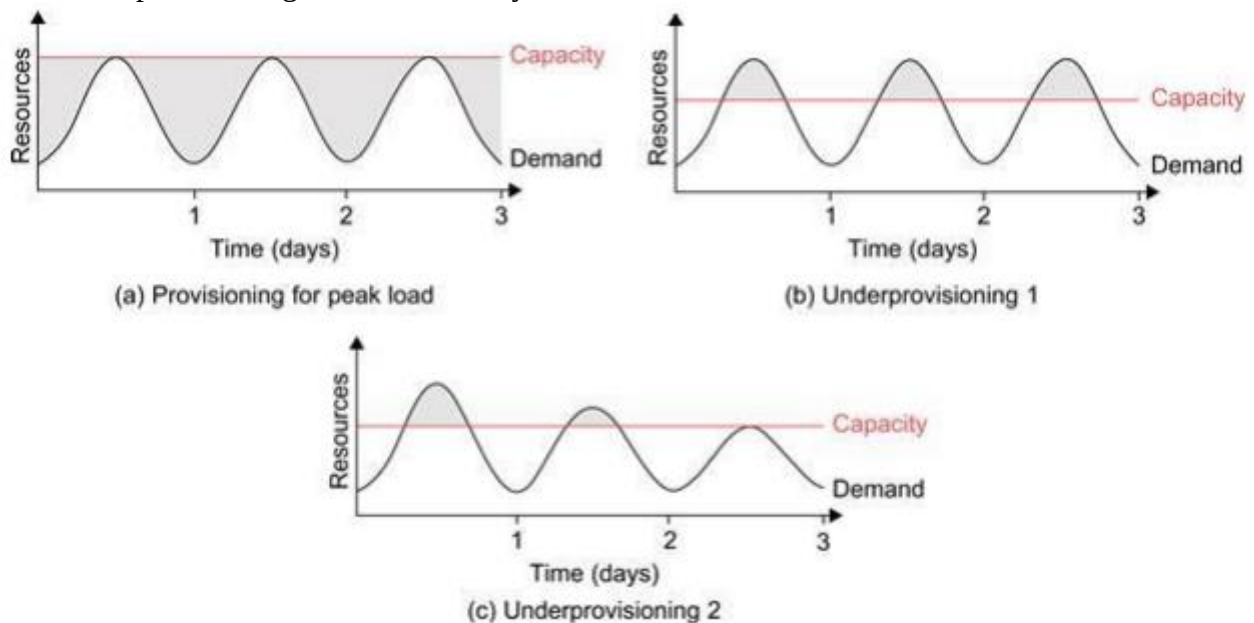


Fig Three cases of cloud resource provisioning without elasticity: (a) heavy waste due to overprovisioning, (b) underprovisioning and (c) under- and then overprovisioning C

Three resource-provisioning methods are presented in the following sections. The demand-driven method provides static resources and has been used in grid computing for many years. The event-driven method is based on predicted workload by time. The popularity-driven method is based on Internet traffic monitored. We characterize these resource provisioning methods as follows.

4.5.2.3 Demand-Driven Resource Provisioning

This method adds or removes computing instances based on the current utilization level of the allocated resources. The demand-driven method automatically allocates two Xeon processors for the user application, when the user was using one Xeon processor more than 60 percent of the time for an extended period. In general, when a resource has surpassed a threshold for a certain amount of time, the scheme increases that resource based on demand. When a resource is below a threshold for a certain amount of time, that resource could be decreased accordingly. Amazon implements such an auto-scale feature in its EC2 platform. This method is easy to implement. The scheme does not work out right if the workload changes abruptly. The x-axis is the time scale in milliseconds. In the beginning, heavy fluctuations of CPU load are encountered. All three methods have demanded a few VM instances initially. Gradually, the utilization rate becomes more stabilized with a maximum of 20 VMs (100 percent utilization) provided for demand-driven provisioning in Figure 4.25(a). However, the event-driven method reaches a stable peak of 17 VMs toward the end of the event and drops quickly in Figure 4.25(b). The popularity provisioning shown in Figure 4.25(c) leads to a similar fluctuation with peak VM utilization in the middle of the plot.

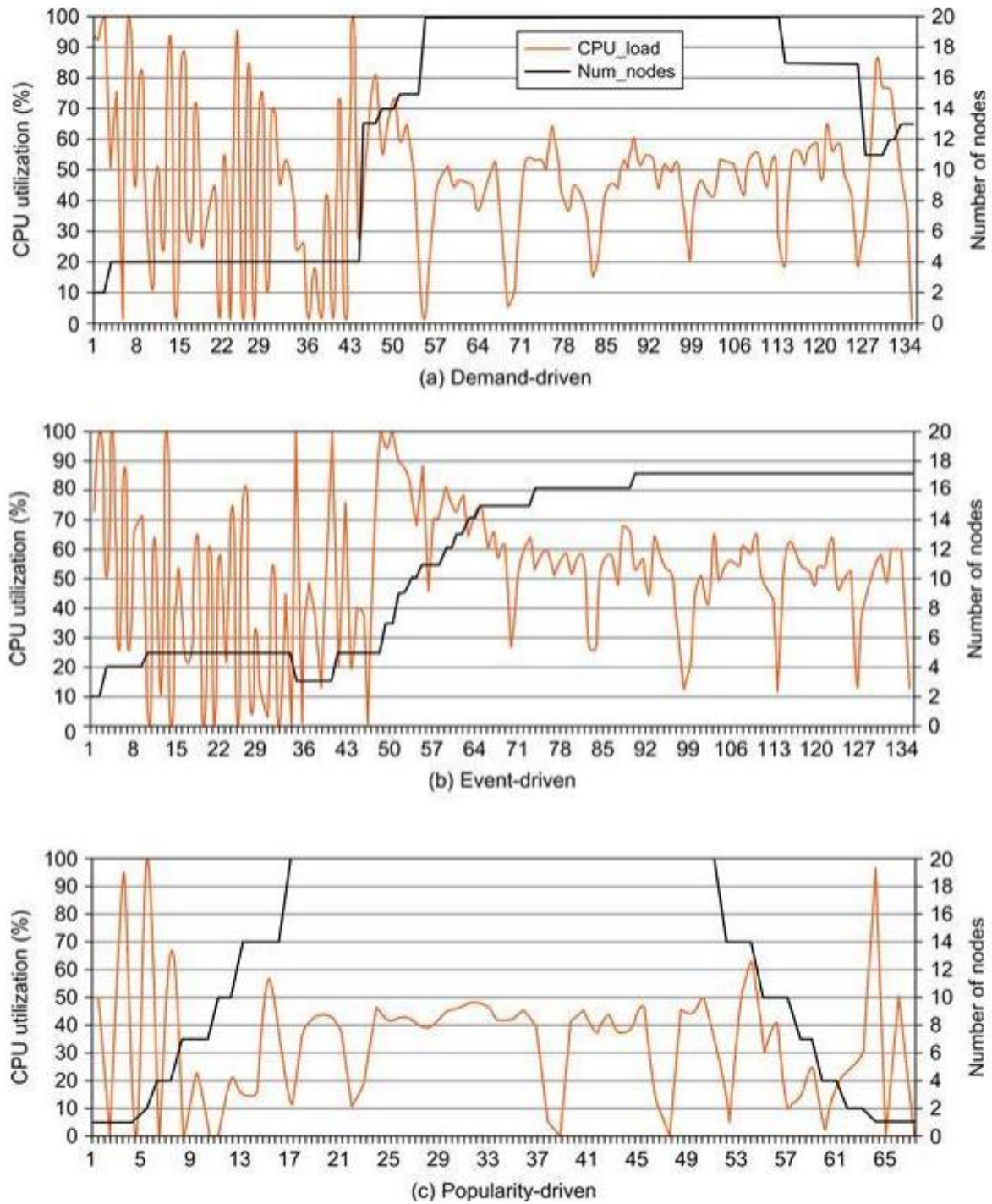


Fig. EC2 performance results on the AWS EC2 platform

4.5.2.4 Event-Driven Resource Provisioning

This scheme adds or removes machine instances based on a specific time event. The scheme works better for seasonal or predicted events such as Christmastime in the West and the Lunar New Year in the East. During these events, the number of users grows before the event period and then decreases during the event period. This scheme anticipates peak traffic before it happens. The method results in a minimal loss of QoS, if the event is predicted correctly. Otherwise, wasted resources are even greater due to events that do not follow a fixed pattern.

4.5.2.5 Popularity-Driven Resource Provisioning

In this method, the Internet searches for popularity of certain applications and creates the instances by popularity demand. The scheme anticipates increased traffic with popularity. Again, the scheme has a minimal loss of QoS, if the predicted popularity is correct. Resources may be wasted if traffic does not occur as expected. In Figure 4.25(c), EC2 performance by CPU utilization rate (the dark curve with the percentage scale shown on the left) is plotted against the number of VMs provisioned (the light curves with scale shown on the right, with a maximum of 20 VMs provisioned).

4.5.2.6 Dynamic Resource Deployment

The cloud uses VMs as building blocks to create an execution environment across multiple resource sites. The InterGrid-managed infrastructure was developed by a Melbourne University group. Dynamic resource deployment can be implemented to achieve scalability in performance. The InterGrid is a Java-implemented software system that lets users create execution cloud environments on top of all participating grid resources. Peering arrangements established between gateways enable the allocation of resources from multiple grids to establish the execution environment. In Figure 4.26, a scenario is illustrated by which an intergrid gateway (IGG) allocates resources from a local cluster to deploy applications in three steps: (1) requesting the VMs, (2) enacting the leases, and (3) deploying the VMs as requested. Under peak demand, this IGG interacts with another IGG that can allocate resources from a cloud computing provider.

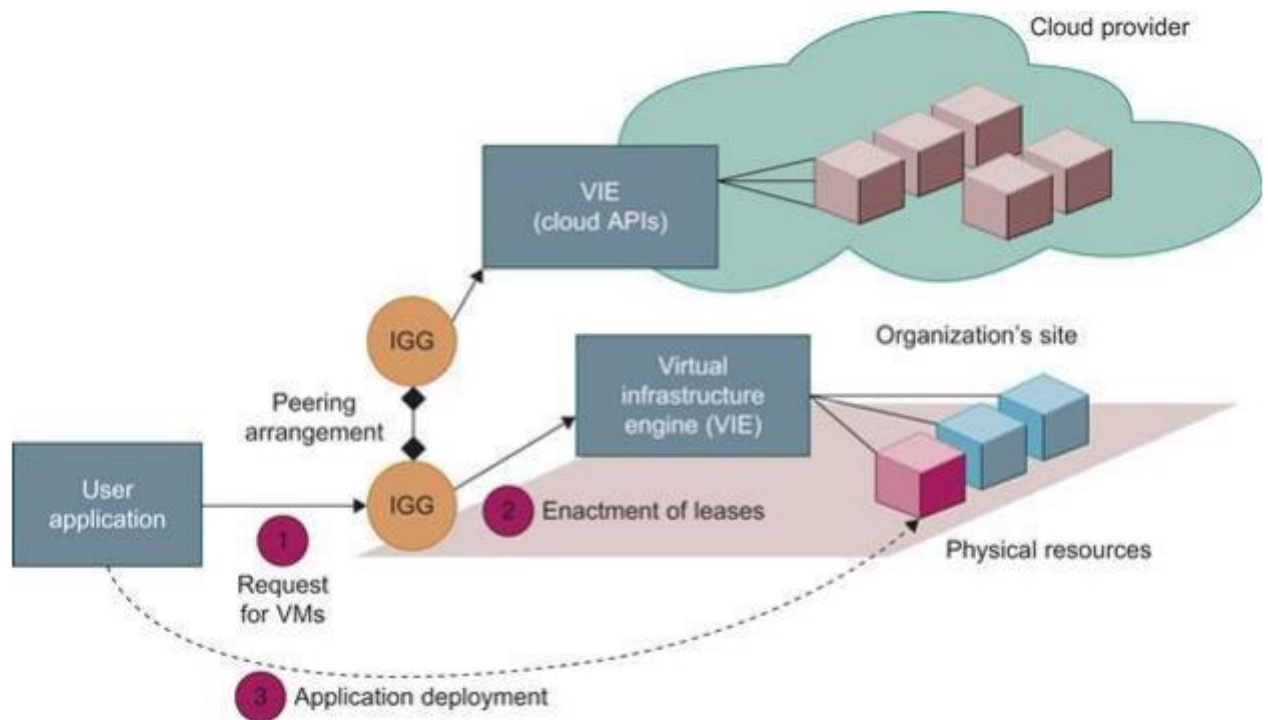


Fig Cloud resource deployment using an IGG (intergrid gateway) to allocate the VMs from a Local cluster to interact with the IGG of a public cloud provider

A grid has predefined peering arrangements with other grids, which the IGG manages. Through multiple IGGs, the system coordinates the use of InterGrid resources. An IGG is aware of the peering terms with other grids, selects suitable grids that can provide the required resources, and replies to requests from other IGGs. Request redirection policies determine which peering grid InterGrid selects to process a request and a price for which that grid will perform the task. An IGG can also allocate resources from a cloud provider. The cloud system creates a virtual environment to help users deploy their applications. These applications use the distributed grid resources.

The InterGrid allocates and provides a distributed virtual environment (DVE). This is a virtual cluster of VMs that runs isolated from other virtual clusters. A component called the DVE manager performs resource allocation and management on behalf of specific user applications. The core component of the IGG is a scheduler for implementing provisioning policies and peering with other gateways. The communication component provides an asynchronous message-passing mechanism. Received messages are handled in parallel by a thread pool.

4.5.2.7 Provisioning of Storage Resources

The data storage layer is built on top of the physical or virtual servers. As the cloud computing applications often provide service to users, it is unavoidable that the data is stored in the clusters of the cloud provider. The service can be accessed anywhere in the world. One example is e-mail systems. A typical large e-mail system might have millions of users and each user can have thousands of e-mails and consume multiple gigabytes of disk space. Another example is a web searching application. In storage technologies, hard disk drives may be augmented with solid-state drives in the future. This will provide reliable and high-performance data storage. The biggest barriers to adopting flash memory in data centers have been price, capacity, and, to some extent, a lack of sophisticated query-processing techniques.

However, this is about to change as the I/O bandwidth of solid-state drives becomes too impressive to ignore. A distributed file system is very important for storing large-scale data. However, other forms of data storage also exist. Some data does not need the namespace of a tree structure file system, and instead, databases are built with stored data files. In cloud computing, another form of data storage is (Key, Value) pairs. Amazon S3 service uses SOAP to access the objects stored in the cloud. Table 4.8 outlines three cloud storage services provided by Google, Hadoop, and Amazon.

Table 4.8 Storage Services in Three Cloud Computing Systems

Storage System		Features
GFS: Google System	File	Very large sustainable reading and writing bandwidth, mostly continuous accessing instead of random accessing. The programming interface is similar to that of the POSIX file system accessing interface.
HDFS: Distributed System	Hadoop File	The open source clone of GFS. Written in Java. The programming interfaces are similar to POSIX but not identical.
Amazon S3 and EBS		S3 is used for retrieving and storing data from/to remote servers. EBS is built on top of S3 for using virtual disks in running EC2 instances.

Many cloud computing companies have developed large-scale data storage systems to keep huge amount of data collected every day. For example, Google's GFS stores web data and some other data, such as geographic data for Google Earth. A similar system from the open source community is the Hadoop Distributed File System (HDFS) for Apache. Hadoop is the open source implementation of Google's cloud computing infrastructure. Similar systems include Microsoft's Cosmos file system for the cloud. Despite the fact that the storage service or distributed file system can be accessed directly, similar to traditional databases, cloud computing does provide some forms of structure or semistructure database processing capability. For example, applications might want to process the information contained in a web page. Web pages are an example of semistructural data in HTML format. If some forms of database capability can be used, application developers will construct their application logic more easily. Another reason to build a databaselike service in cloud computing is that it will be quite convenient for traditional application developers to code for the cloud platform.

Databases are quite common as the underlying storage device for many applications. Thus, such developers can think in the same way they do for traditional software development. Hence, in cloud computing, it is necessary to build databases like large-scale systems based on data storage or distributed file systems. The scale of such a database might be quite large for processing huge amounts of data. The main purpose is to store the data in structural or semi-structural ways so that application developers can use it easily and build their applications rapidly. Traditional databases will meet the performance bottleneck while the system is expanded to a larger scale. However, some real applications do not need such strong consistency. The scale of such databases can be quite large. Typical cloud databases include BigTable from Google, SimpleDB from Amazon, and the SQL service from Microsoft Azure.

4.5.3 Virtual Machine Creation and Management

In this section, we will consider several issues for cloud infrastructure management. First, we will consider the resource management of independent service jobs. Then we will consider how to execute third-party cloud applications. Cloud-loading experiments are used by a Melbourne research group on the French Grid'5000 system. This experimental setting illustrates VM creation and management. This case study example reveals major VM management issues and suggests some plausible solutions for workload-balanced execution. Figure 4.27 shows the interactions among VM managers for cloud creation and management. The managers provide a public API for users to submit and control the VMs.

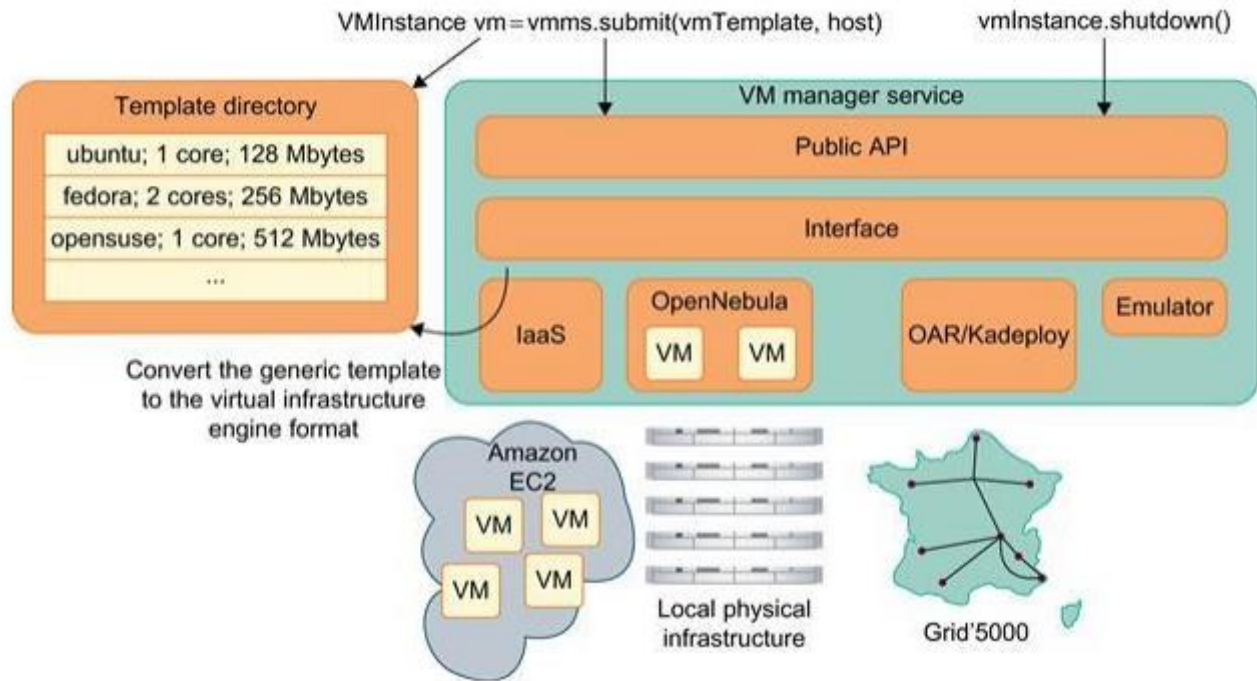


FIGURE 4.27 Interactions among VM managers for cloud creation and management; the manager provides a public API for users to submit and control the VMs

4.5.3.1 Independent Service Management

Independent services request facilities to execute many unrelated tasks. Commonly, the APIs provided are some web services that the developer can use conveniently. In Amazon cloud computing infrastructure, SQS is constructed for providing a reliable communication service between different providers. Even the endpoint does not run while another entity has posted a message in SQS. By using independent service providers, the cloud applications can run different services at the same time. Some other services are used for providing data other than the compute or storage services.

4.5.3.2 Running Third-Party

Applications Cloud platforms have to provide support for building applications that are constructed by third-party application providers or programmers. As current web applications are often provided by using Web 2.0 forms (interactive applications with Ajax), the programming interfaces are different from the traditional programming interfaces such as functions in runtime libraries. The APIs are often in the form of services. Web service application engines are often used by programmers for building applications. The web browsers are the user interface for end users. In addition to gateway applications, the cloud computing platform provides the extra capabilities of accessing backend services or underlying data. As examples, GAE and Microsoft Azure apply their own cloud APIs to get special cloud services. The WebSphere application engine is deployed by IBM for Blue Cloud. It can be used to develop any kind of web application written in Java. In EC2, users can use any kind of application engine that can run in VM instances.

4.5.3.3 Virtual Machine Manager

The VM manager is the link between the gateway and resources. The gateway doesn't share physical resources directly, but relies on virtualization technology for abstracting them. Hence, the actual resources it uses are VMs. The manager manage VMs deployed on a set of physical resources. The VM manager implementation is generic so that it can connect with different VIEs. Typically, VIEs can create and stop VMs on a physical cluster. The Melbourne group has developed managers for OpenNebula, Amazon EC2, and French Grid'5000. The manager using the OpenNebula OS (www.opennebula.org) to deploy VMs on local clusters. OpenNebula runs as a daemon service on a master node, so the VMM works as a remote user. Users submit VMs on physical machines using different kinds of hypervisors, such as Xen (www.xen.org), which enables the running of several operating systems on the same host concurrently.

The VMM also manages VM deployment on grids and IaaS providers. The InterGrid supports Amazon EC2. The connector is a wrapper for the command-line tool Amazon provides. The VM manager for

Grid'5000 is also a wrapper for its command-line tools. To deploy a VM, the manager needs to use its template.

4.5.3.4 Virtual Machine Templates

A VM template is analogous to a computer's configuration and contains a description for a VM with the following static information:

- The number of cores or processors to be assigned to the VM
- The amount of memory the VM requires
- The kernel used to boot the VM's operating system
- The disk image containing the VM's file system
- The price per hour of using a VM

The gateway administrator provides the VM template information when the infrastructure is set up. The administrator can update, add, and delete templates at any time. In addition, each gateway in the InterGrid network must agree on the templates to provide the same configuration on each site. To deploy an instance of a given VM, the VMM generates a descriptor from the template. This descriptor contains the same fields as the template and additional information related to a specific VM instance. Typically the additional information includes:

- The disk image that contains the VM's file system
- The address of the physical machine hosting the VM
- The VM's network configuration
- The required information for deployment on an IaaS provider

Before starting an instance, the scheduler gives the network configuration and the host's address; it then allocates MAC and IP addresses for that instance. The template specifies the disk image field. To deploy several instances of the same VM template in parallel, each instance uses a temporary copy of the disk image. Hence, the descriptor contains the path to the copied disk image. The descriptor's fields are different for deploying a VM on an IaaS provider. Network information is not needed, because Amazon EC2 automatically assigns a public IP to the instances. The IGG works with a repository of VM templates, called the VM template directory.

4.5.3.5 Distributed VM Management

The interactions between InterGrid's components. A distributed VM manager makes requests for VMs and queries their status. This manager requests VMs from the gateway on behalf of the user application. The manager obtains the list of requested VMs from the gateway. This list contains a tuple of public IP/private IP addresses for each VM with Secure Shell (SSH) tunnels. Users must specify which VM template they want to use and the number of VM instances needed, the deadline, the wall time, and the address for an alternative gateway. The local gateway tries to obtain resources from the underlying VIEs. When this is impossible, the local gateway starts a negotiation with any remote gateways to fulfill the request. When a gateway schedules the VMs, it sends the VM access information to the requester gateway. Finally, the manager configures the VM, sets up SSH tunnels, and executes the tasks on the VM. Under the peering policy, each gateway's scheduler uses conservative backfilling to schedule requests. When the scheduler can't start a request immediately using local resources, a redirection algorithm will be initiated.

Example

4.6. Experiments on an Inter Grid TestBed over the Grid'5000

The Melbourne group conducted two experiments to evaluate the InterGrid architecture. The first one evaluates the performance of allocation decisions by measuring how the IGG manages load via peering arrangements. The second considers its effectiveness in deploying a bag-of-tasks application. The experiment was conducted on the French experimental grid platform Grid'5000. Grid'5000 comprises 4,792 processor cores on nine grid sites across France. Each gateway represents one Grid'5000 site, as shown in Figure 4.28.

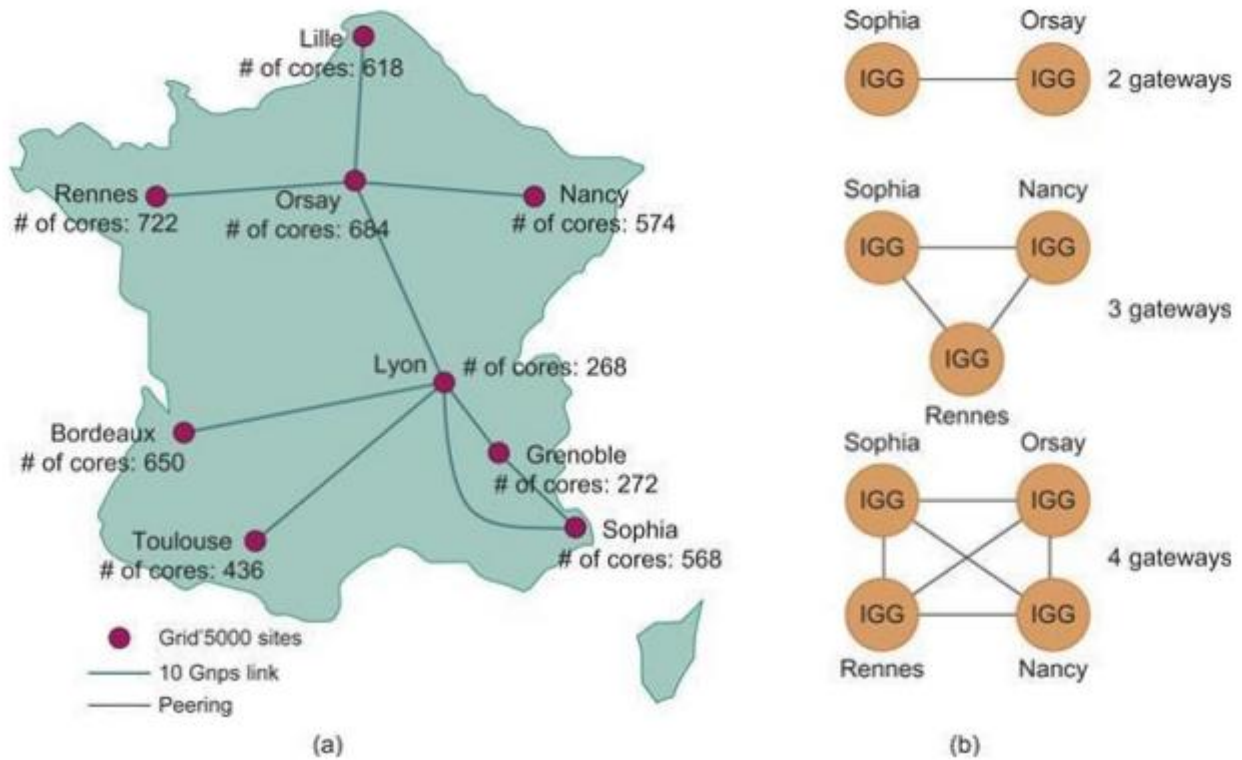


FIGURE 4.28 The InterGrid test bed over the French Grid'5000 located in nine cities across

To prevent the gateways from interfering with real Grid'5000 users, emulated VM managers were implemented to instantiate fictitious VMs. The number of emulated hosts is limited by the core number at each site. A balanced workload was configured among the sites. The maximum number of VMs requested does not exceed the number of cores in any site. The load characteristics are shown in Figure 4.29 under a four-gateway scenario. The teal bars indicate each grid site's load. The magenta bars show the load when gateways redirect requests to one another. The green bars correspond to the amount of load each gateway accepts from other gateways. The brown bars represent the amount of load that is redirected. The results show that the loading policy can balance the load across the nine sites. Rennes, a site with a heavy load, benefits from peering with other gateways as the gateway redirects a great share of its load to other sites.

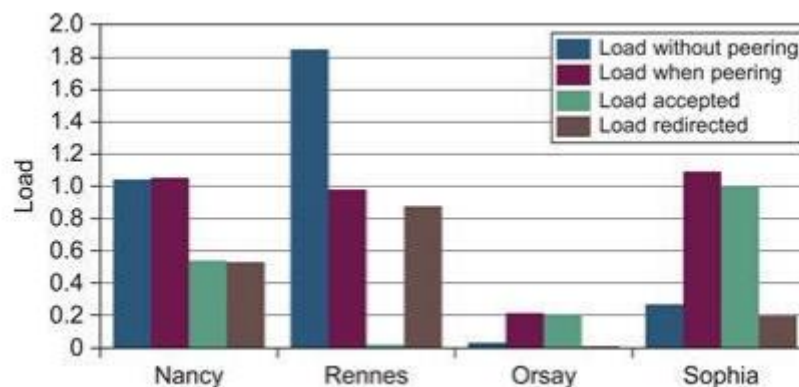


FIGURE 4.29 Cloud loading results at four gateways at resource sites in the Grid'5000 system

4.1 Inter Cloud Resource Management

Cloud computing paradigm provides management of resources and helps create extended portfolio of services. Through cloud computing, not only are services managed more efficiently, but also service discovery is made possible. To handle rapid increase in the content, media cloud plays a very vital role. But it is not possible for standalone clouds to handle everything with the increasing user demands. For scalability and better service provisioning, at times, clouds have to communicate with other clouds and

share their resources. This scenario is called Intercloud computing or cloud federation. The study on Intercloud computing is still in its start. Resource management is one of the key concerns to be addressed in Intercloud computing. Already done studies discuss this issue only in a trivial and simplistic way. In this study, we present a resource management model, keeping in view different types of services, different customer types, customer characteristic, pricing, and refunding. The presented framework was implemented using Java and NetBeans 8.0 and evaluated using CloudSim 3.0.3 toolkit. Presented results and their discussion validate our model and its efficiency.

Cloud computing is a handy solution for processing content in distributed environments. Cloud computing provides ubiquitous access to the content [6], without the hassle of keeping large storage and computing devices. Sharing large amount of media content is another feature that cloud computing provides [7]. Other than social media, traditional cloud computing provides additional features of collaboration and editing of content. Also, if content is to be shared, downloading individual files one by one is not easy. Cloud computing caters to this issue, since all the content can be accessed at once by other parties, with whom the content is being shared.

The increasing demands in cloud computing arena has resulted in more heterogeneous infrastructure, making interoperability an area of concern. Due to this, it becomes a challenge for cloud customers to select appropriate cloud service provider (CSP) and hence it ties them to a particular CSP. This is where intercloud computing comes into play. Although intercloud computing is still in its infancy, its purpose is to allow smooth interoperability between clouds, regardless of their underlying infrastructure. This allows users to migrate their workloads across clouds easily. Cloud brokerage is a promising aspect of intercloud computing .

Most of the data-intensive applications are now deployed on the clouds. These applications, storage, and data resource are so diversely located that they have to reach even cross-continental networks. Due to this, performance degradation in networks affects the performance of cloud systems and user requests. To ensure service quality, especially for bulk-data transfer, resource reservation and utilization become a critical issue .

Previous works mainly focus on integrated and collaborative uses of resources to meet application requirements. They do not focus on bulk-data transfer consistency and efficiency. They assume that all resources are connected by high-speed stable networks. Continuously growing cloud market faces new challenges now. Even though users have well collaborated end systems and resources are allocated according to their needs, still, bulk-data transfer for cross-continental users in remote places might create performance bottleneck. For instance, multimedia services like IP Television (IPTV) rely on availability of sufficient network resources and hence they have to be operated within the limitation of time constraints

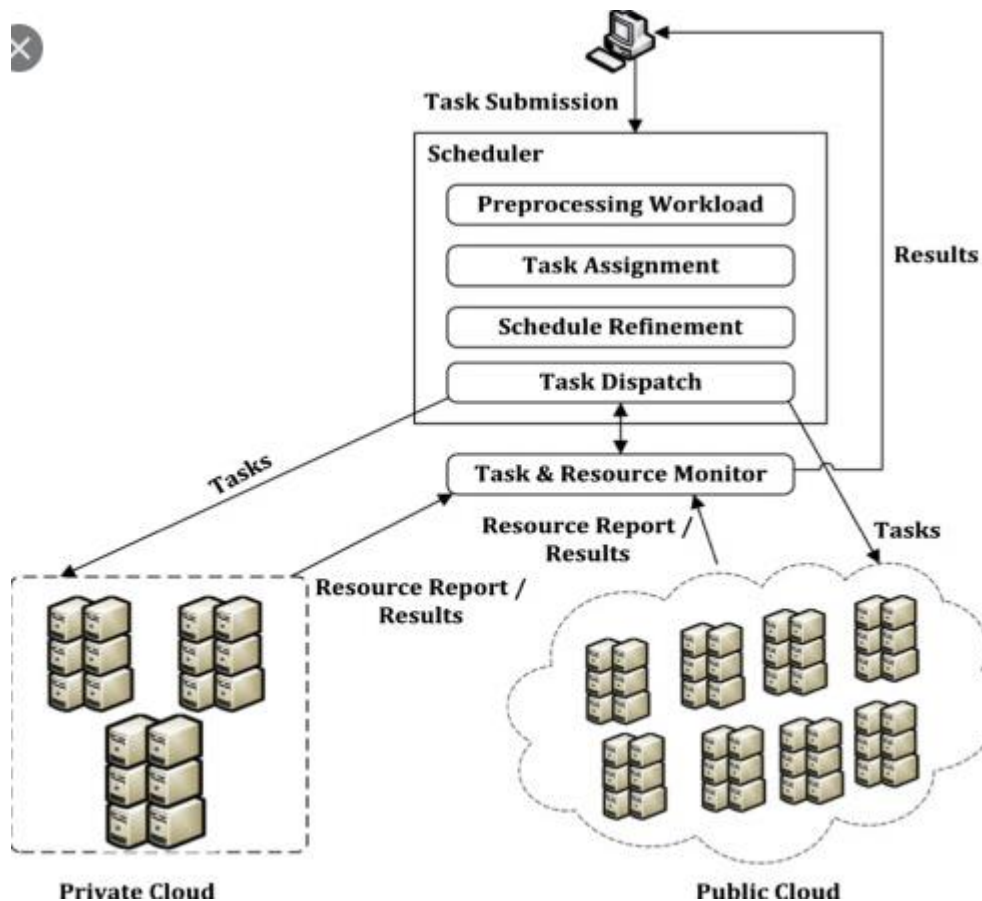
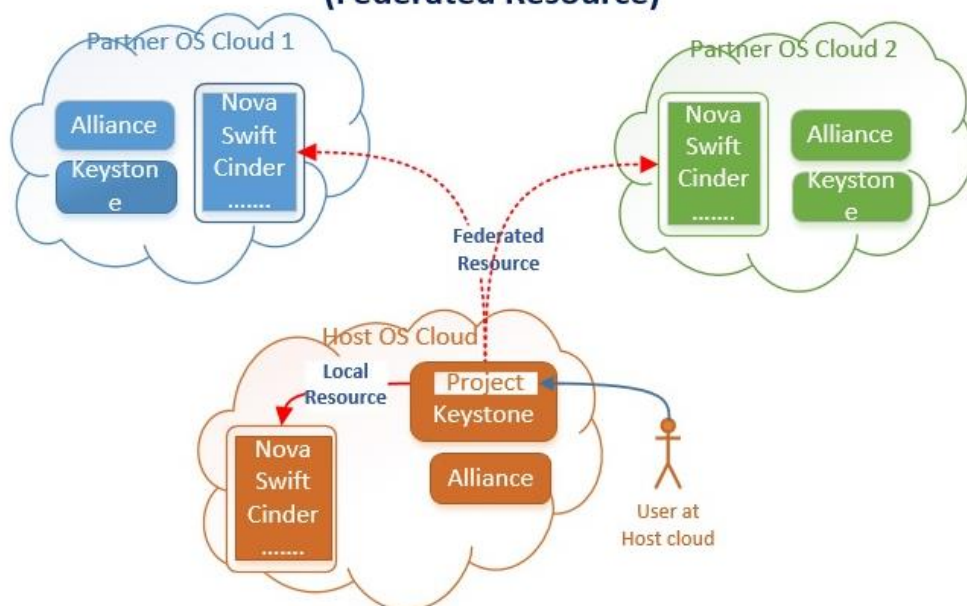


Fig. Resource management

Resource provisioning is required to allocate limited resource efficiently on the (partner cloud) resource provider cloud. Provisioning is also needed to provide sense of ownership to the end user, for cloud provider (partner/host) provisioning is required to address metering aspect.

Alliance service is responsible for provisioning remote resources. The resource provisioning will be done on local Keystone project at hosting cloud. It also maintains provisioning information in local database. The provisioning info will be used for the purpose of token generation and validation.

Resource provisioning (Federated Resource)



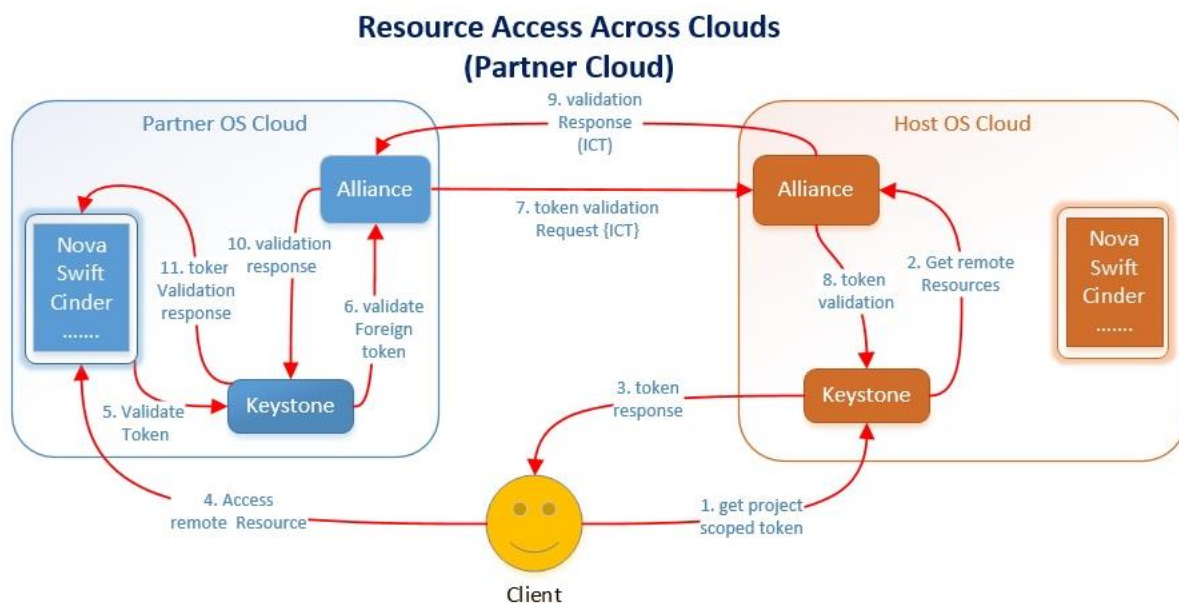
In the above picture user maintains his identity at one place (host cloud) and owns resources from remote cloud(s) on a local project. This is another benefit of resource federation, where user can use a single project in host cloud to scope all the remote resources across the cloud(s).

Resource Access Across Clouds

Resource access process start by getting an “X-Auth-Token” scoped to local Keystone project of "host" cloud. Keystone service at HC will talk to local Alliance to get information about remote resources associated with project

As part of token response client gets a service catalog containing endpoints to the remote (federated) resources. Client uses the remote resource endpoint to access the resource it provides (X-Host-Cloud-Id) host cloud identifier in request header and the X-Auth-Token{hc} got from host cloud.

Auth middle-ware protecting the resource at partner cloud intercepts the request and makes a call to Keystone for token validation. Keystone delegates such the token validation request to Alliance service which is not issued by it (foreign token) and have X-Host-Cloud-Id header associated. Alliance uses the cloud identifier (X-Host-Cloud-Id) from the header to lookup the paired host cloud and it's peer Alliance endpoint. Using the X-Auth-Token{hc}, it forms an InterCloud Federation Ticket and uses paired Alliance endpoint to validate user token Alliance at HC will coordinate with local Keystone to validate the token After successful inter-cloud token validation Alliance service provide the validate response to Keystone service running at PC. Keystone will caches the token in locale system and respond to middle-ware. Keystone will use the cached token for future token validations.



X-Auth-Token processing

Clients won't like to deal with multiple X-Auth-Tokens to access their resources across clouds (regions). Following are the options to solve this issue.

PKI tokens

PKI tokens can be used by clients to access resources across clouds. There won't be inter-cloud token validation required to validate the PKI tokens. PKI token are proven to be heavy, Federated Token can be a better solution.

Federated Tokens

Instead of generating new X-Auth-Token{pc} by partner cloud, partner cloud may choose to use the same X-Auth-Token{hc} issued by host cloud.

After successful inter-cloud token validation (as explained above) Alliance will cache the token locally and utilize the same X-Auth-Token for future communication. This option can be set as part of cloud pairing depending of level of trust between two cloud providers.

Note: Inter-cloud token validation is one time process or can be done multiple times over the period of communication by clients.

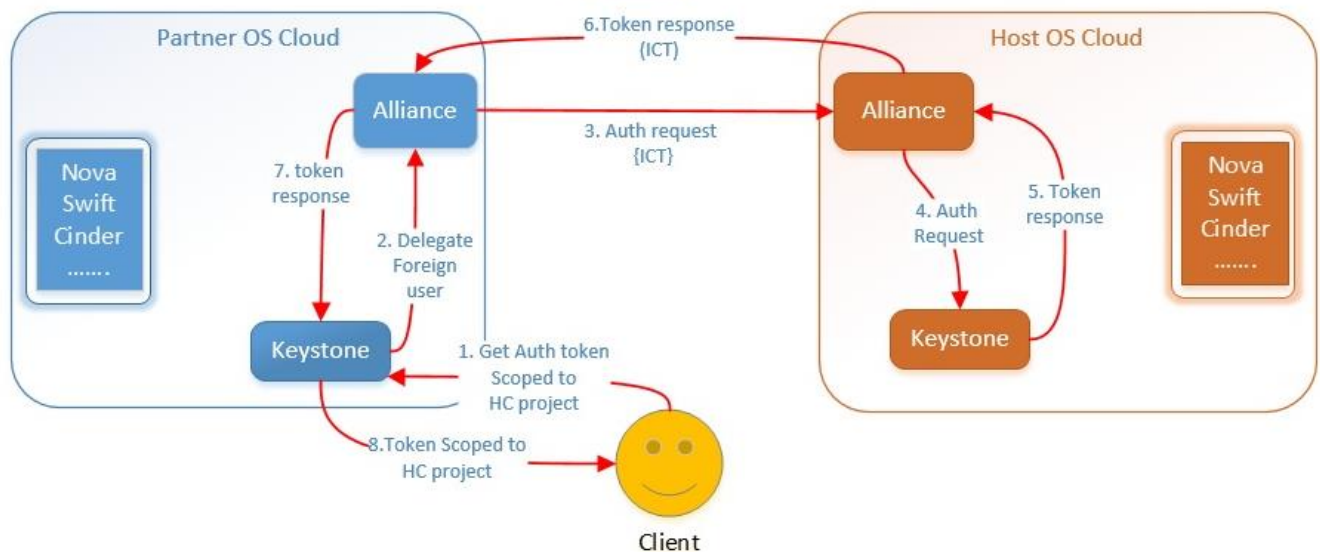
Federated Tokens by Eager Propagation

To support federated tokens partner cloud has to do inter-cloud token validation and cache the validate token response to make the future token validation more efficient. Another approach to solve the performance of inter-cloud token validation is to propagate the tokens to partner cloud in push mode. Host cloud will propagate token to the relevant partner using notification route.

SSO Across Cloud

In this mode, clients chooses to use PC's identity (Keystone) endpoint to make auth token request. Client provide credentials, project_id and cloud_id to the PC's identity service. Keystone will coordinate with Alliance service to get the token from remote cloud.

Token Request from Resource Provider



SSOut Across Cloud (or InterCloud Token Revocation)

Token revocation is an important aspect to maintain the security and system integrity. In case of resource federation use case, tokens revocation become more important as a stale token can cause bigger harm specially to the resource provider clouds.

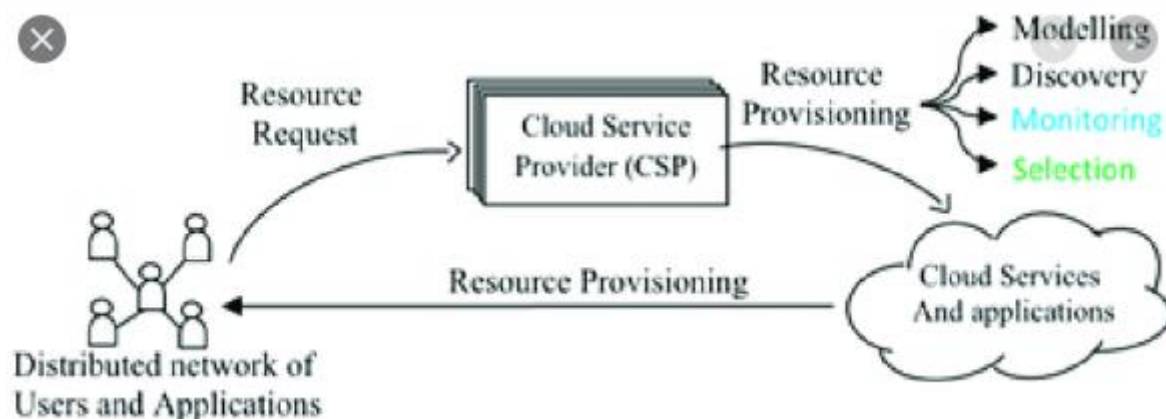
Inter-cloud token revocation will allow token revocation across cloud, e.g. Host cloud can initiate the token revocation for a token issued by itself or partner clouds can request/initiate the token revocation of a federated token.

Alliance service will be the interface between clouds to make the token revocation happen.

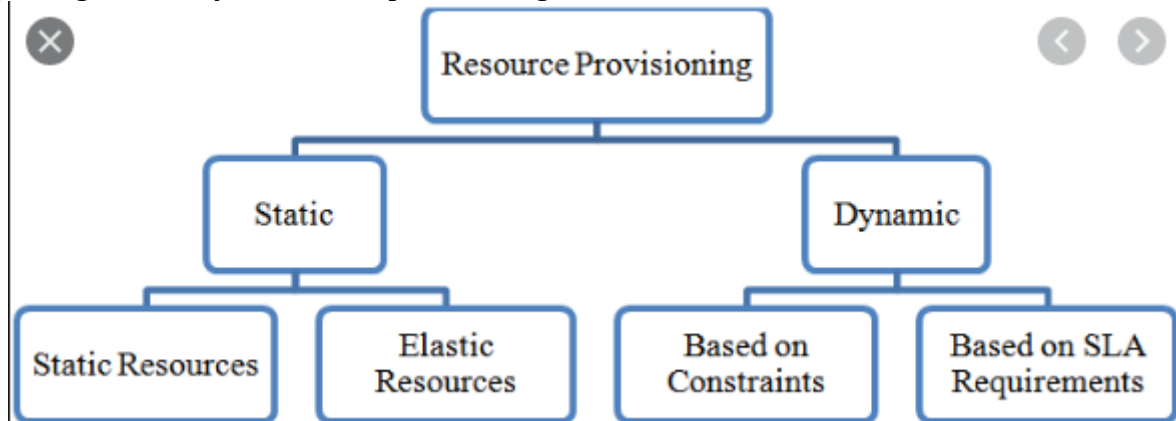
4.2 Resource Provisioning and Resource Provisioning Methods

NEED OF RESOURCE PROVISIONING

To increase the gratification and the chances or possibility of the users reaching the cloud, there is needed to increase the large number of the requests or feedbacks that gratified from the cloud. Therefore, because of these perspectives the profit becomes so higher to the cloud in the consequence. There is possibility to appeal the customers of the cloud computing application to the cloud is to merge or short time for the responding. To make the attraction of the customers or users with cloud, the cloud is to need for accepting the resource provisioning technique which creates or generates the highest rate of the business deal. For the developing of the higher qualities of the business deal, there is not needed to be settled with the lack of period of the time. By giving the preference to the last, the trade-offs is to be sorted among the transaction success and U-turn time. Hence the shortage of the time can be created as far as it is conceivable for having the main goal to keep the high rate of the dealing success .



The applications can be used properly by applying the purpose of resource provisioning which implies that to discover the reasonable resources for the appropriate workloads in time. The best consequences can get by using the more effective resources. The reasonable and appropriate workload discovery is one of the main goals that maintain the program of different workloads. For making the quality of services more effective there is needed to satisfy the parts or units like utility, availability, reliability, time, security, price and CPU etc. So the resource provisioning reflects the performance of the time for the various workloads. All the presentations depend upon the kind or type of workload. There are entirely two generic way of resource provisioning



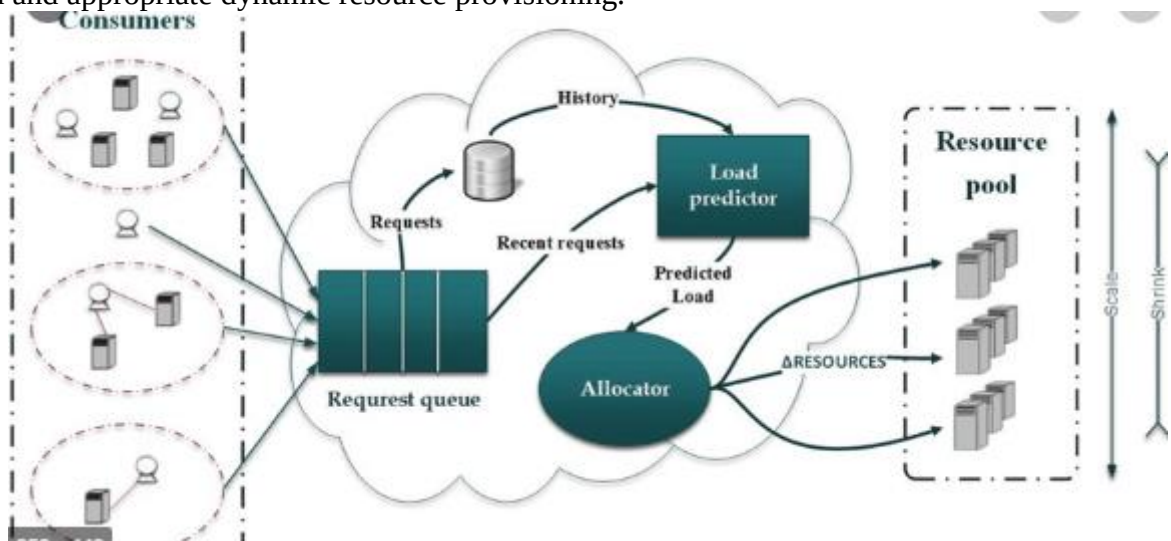
- **Static Resource Provisioning**
- **Dynamic Resource Provisioning**
- **User Self-provisioning**

Static Resource Provisioning

For an application all types of desired resources are required in the peak time normally. Mostly this type of cloud provisioning the misuse of resources and wastage of resources because of workload is not considered in the peak time. Despite of this the resource provider offer the maximum desired resource for the purpose of avoids the service level application violation .

Dynamic Resource Provisioning

The customer demand, requirements and workloads are changed rapidly so that the cloud computing contain the elasticity element to the level of advanced automation adaption in the way of resource provisioning. This aim can be achieved through making the automatically scaling up and down of the resources that are assigned to a particular customer. This method is used to match the existing resources with the consumer current needs and demands with more good and reasonable way. In this way the element of elasticity is helpful to overcome the problem of under and over provisioning and also helpful in good and appropriate dynamic resource provisioning.



User Self-provisioning: With user self-

provisioning (also known as cloud self-service), the customer purchases resources from the cloud provider through a web form, creating a customer account and paying for resources with a credit card. The provider's resources are available for customer use within hours, if not minutes

User Self-provisioning:

With user self-provisioning (also known as cloud self-service), the customer purchases resources from the cloud provider through a web form, creating a customer account and paying for resources with a credit card. The provider's resources are available for customer use within hours, if not minutes

User Self-provisioning: With user self-provisioning (also known as cloud self-service), the customer purchases resources from the cloud provider through a web form, creating a customer account and paying for resources with a credit card. The provider's resources are available for customer use within hours, if not minutes.

User Self-provisioning: With user self-provisioning (also known as cloud self-service), the customer purchases resources from the cloud provider through a web form, creating a customer account and paying for resources with a credit card. The provider's

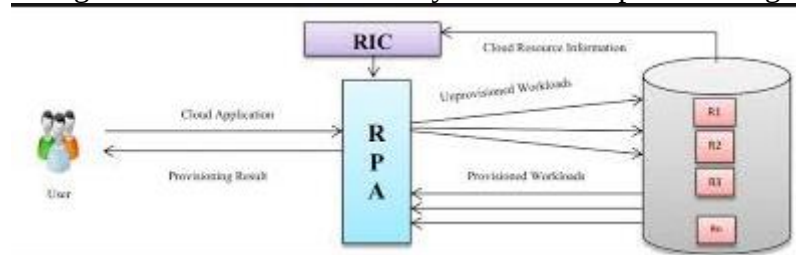
resources are available for customer use within hours, if not minutes.

Parameters of Resource Provisioning

- 1) **Response time:** The algorithm of resource provisioning is designed to give response in minimum time after completing any task
- 2) **Minimize Cost:** The cloud services cost should be less for the cloud consumer.
- 3) **Revenue Maximization:** The cloud services provider should be earned maximum revenue.
- 4) **Fault tolerant:** The algorithm provide services continuously in spite of collapse of nodes.
- 5) **Reduced SLA Violation:** The design of algorithm should be capable to decrease SLA violation.
- 6) **Reduced Power Consumption:** The placement & migration methods of virtual machine should be consume low power.

The basic model of resource provisioning in cloud

Cloud user sends their workload like cloud application to the resource provisioning agents and establish good interaction with them. Resource provisioning agent (RPA) does resource provisioning and provide most suitable resource according to the customer requirements. When resource provisioning agent received the workload from user, his connection and access with the resource information centre (RIC) that have all the desired information about all type of resources with a resource pool. After that output can be achieved depend on the workload requirements as précised by consumer. Through resource discovery we know about the available resources and desired resources list can be generated. On the other hand the selection of resources is a procedure of choosing the most appropriate workload resource competition and match depended on the quality of services need expressed by the cloud user in tenure of services level application from the catalog and list which is created by the resource provisioning.



Sl.

No.

Resource Provisioning Techniques

Merits

Challenges

1

Deadline-driven provisioning of resources for scientific applications in hybrid clouds with Aneka [5]

Able to efficiently allocate resources from different sources

in order to reduce application execution times.

Not suitable for HPC-data intensive applications.

2

Dynamic provisioning in multi-tenant service clouds [15]

Matches tenant functionalities with client requirements.

Does not work for testing on real-life cloud-based system and across several domains.

3

Elastic Application Container: A Lightweight Approach for Cloud Resource Provisioning [19]

Outperforms in terms of flexibility and resource efficiency.

Not suitable for web applications and supports only one type of programming language, Java.

International Journal of Research
in Computer and

Communication Technology, Vol 3, Issue 3, March- 2014

ISSN (Online) 2278- 5841

ISSN (Print) 2320- 5156

www.ijrcct.org Page 398

4

Hybrid Cloud Resource Provisioning Policy
in the

Presence of Resource Failures [31]

Able to adopt user the
workload model to provide
flexibility in the choice of
strategy based on the desired
level of QoS, the needed
performance, and the available
budget.

Not suitable to run real experiments.

5

Provisioning of Requests for Virtual
Machine Sets with Placement Constraints in
IaaS Clouds [38]

Runtime efficient & can provide
an effective means of online
VM-to-PM mapping and also

Maximizes revenue.

Not practical for medium to large problems.

6

Failure-aware resource provisioning for hybrid Cloud infrastructure [11]

Able to improve the users' QoS about 32% in terms of deadline violation rate and 57% in terms of slowdown with a limited cost on a public cloud.

Not able to run real experiments and also not able to move VMs between public and private clouds to deal with resource failures in the local infrastructures.

7

VM Provisioning Method to Improve the Profit and SLA Violation of Cloud Service Providers [27]

Reduces SLA violations & Improves Profit.

Increases the problem of resource allocation and load balancing among the datacenters.

8

Risk Aware Provisioning and Resource Aggregation based Consolidation of Virtual Machines [21]

Significant amount of reduction in the numbers required to host 1000 VMs and enables to turn off unnecessary servers.

Takes into account only CPU requirements of VMs.

9

Semantic based Resource Provisioning and Scheduling in Inter-cloud Environment [20]

Enables the fulfillment of customer requirements to the maximum by providing additional resources to the cloud system participating in a federated cloud environment thereby solving the interoperability problem.

QoS parameters like response time and throughput has to be achieved for interactive applications.

10

Design and implementation of adaptive power-aware virtual machine provisioner

(APA-VMP) using swarm intelligence [7]

Efficient VM placement and
significant reduction in power.

Not suitable for conserving power in
modern data centers.

11

Adaptive resource provisioning for read
intensive multi-tier applications in the cloud
[2]

Automatic Identification and
resolution of bottlenecks in
multitier web application hosted
on a cloud.

Not suitable for n-tier clustered
application hosted on a cloud.

12

Optimal Resource Provisioning for Cloud
Computing Environment [13]

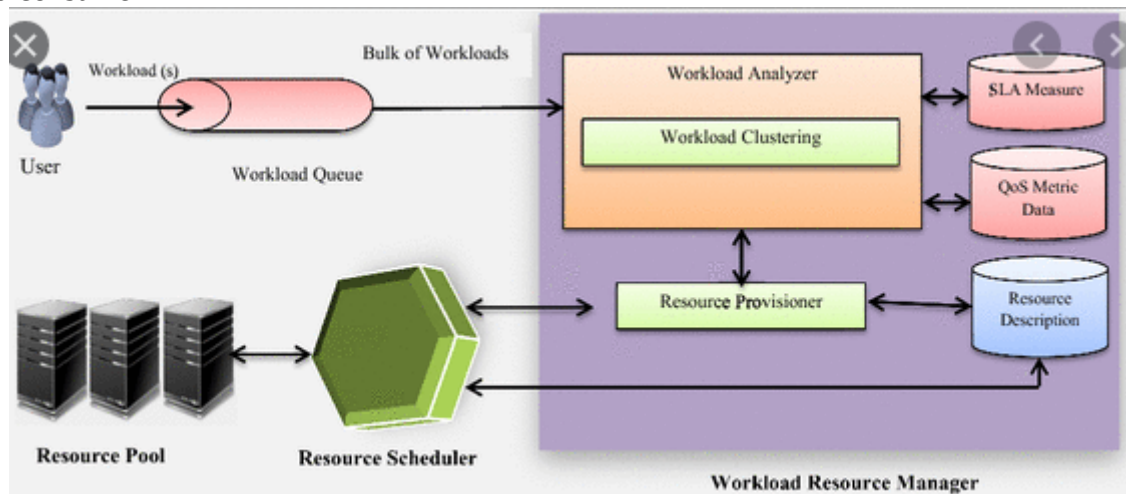
Efficiently provisions Cloud
Resources for SaaS users with a
limited budget and Deadline
thereby optimizing QoS.

Applicable only for SaaS users and
SaaS providers.

The procedure of cloud resource provisioning. Cloud consumer can interact with the help of cloud portal and also presents the quality of services (QoS) requirements of workload after the complete authentication procedure. Resource information centre (RIC) delivered the information that is based on customer requirements and available resources are checked by the resource provisioning agent (RPA). It

is also helpful in provision of desired resources to the cloud application workload for running or execution in the environment of cloud but after fulfill the condition that is demanded resources are present in the resource pool.

Resource provisioning agent (RPA) applications for again submit the workload with another quality of services requirement like a service level application article or document in the condition of if the desired resource is unavailable as the requirements of quality of services. Resource scheduler submitted the workloads after the appropriate provisioning of available resources. Resource scheduler requests to submit and present the whole workloads for all the available provisioned resources. Then resource provisioning agent again received the results and send these provisioning outputs and results to the cloud consumer



Resource Provisioning Mechanisms

1 QoS Based RPM

The major objective of such work is to provide provision on different resources before managing in an appropriate manner or way and then execute this application for getting optimal results to the end user.

2 Cost Based RPM

Minimize the total amount of resource provisioning cost like over provisioning cost and under provisioning cost. Cost reduction can assure the double capacity of application

3 SLA Based RPM SLA provisioning method depend on the admission control that can be maximizes the revenue and also the utilization of resources resource utilization and also pay attention on multiple type needs of SLA that are consumer described.

4 Time Based RPM Minimum execution time can double the application capacity as well as minimize the overhead cost of switching servers.

5. Energy Based RPM Enhance the resource utilization and must be reduce the consumption of power.

6 Dynamic Based RPM Decision related to different and changing environment such as electricity bills and user requirements. Fully and partially distributing the clouds computing services facilities with other consumers.

7 Adaptive Based RPM Methodologies which can be based on the virtualization for the purpose of resource provisioning depend on the need of application dynamically as well as minimize the consumption of power and energy by maximizing the server's usage.

8 Optimization Based RPM The running cost of consumer application can be reduced through advancing the energy resources and also meet the required deadline on time make sure that SLA objectives cannot be violated.

4.3 Global Exchange of Cloud Resources

4.5.4 Global Exchange of Cloud Resources

In order to support a large number of application service consumers from around the world, cloud infrastructure providers (i.e., IaaS providers) have established data centers in multiple geographical locations to provide redundancy and ensure reliability in case of site failures. For example, Amazon has data centers in the United States (e.g., one on the East Coast and another on the West Coast) and Europe. However, currently Amazon expects its cloud customers (i.e., SaaS providers) to express a preference regarding where they want their application services to be hosted. Amazon does not provide seamless/automatic mechanisms for scaling its hosted services across multiple geographically distributed data centers.

This approach has many shortcomings. First, it is difficult for cloud customers to determine in advance the best location for hosting their services as they may not know the origin of consumers of their services. Second, SaaS providers may not be able to meet the QoS expectations of their service consumers originating from multiple geographical locations. This necessitates building mechanisms for seamless federation of data centers of a cloud provider or providers supporting dynamic scaling of applications across multiple domains in order to meet QoS targets of cloud customers. Figure 4.30 shows the high-level components of the Melbourne group's proposed InterCloud architecture.

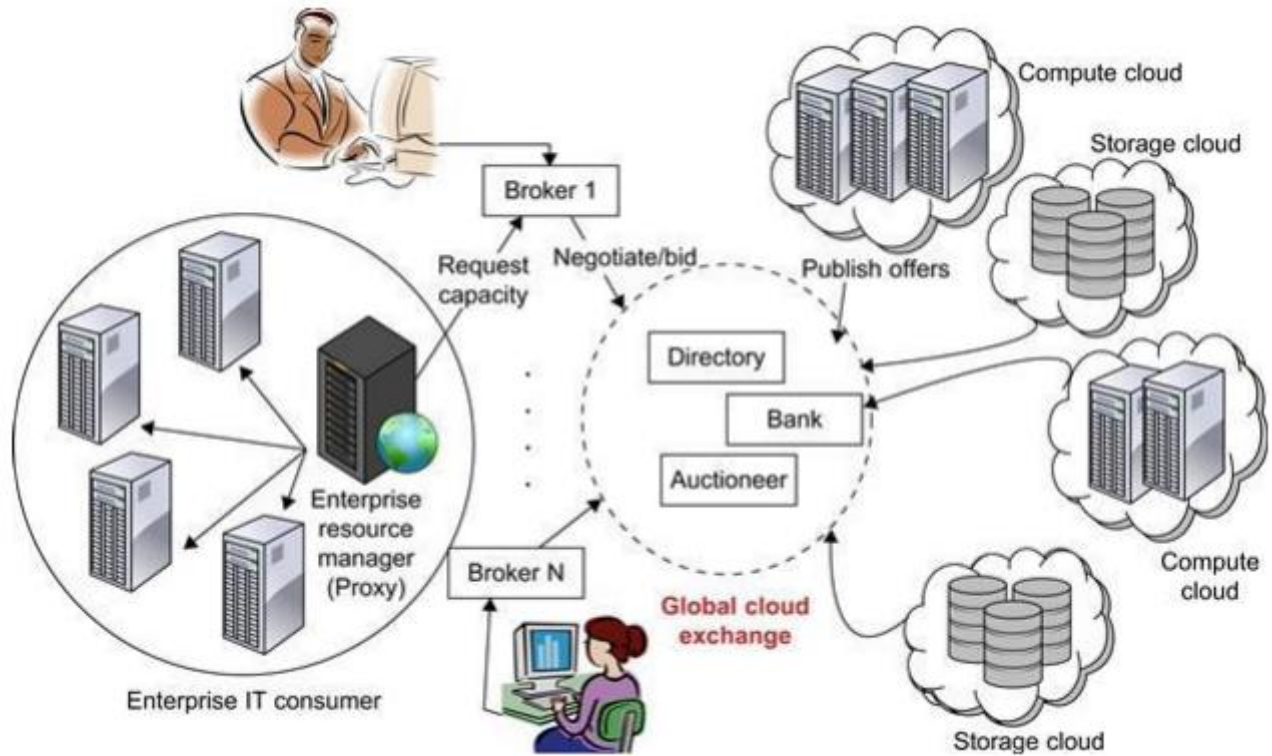


FIGURE 4.30 Inter-cloud exchange of cloud resources

In addition, no single cloud infrastructure provider will be able to establish its data centers at all possible locations throughout the world. As a result, cloud application service (SaaS) providers will have difficulty in meeting QoS expectations for all their consumers. Hence, they would like to make use of services of multiple cloud infrastructure service providers who can provide better support for their specific consumer needs. This kind of requirement often arises in enterprises with global operations and applications such as Internet services, media hosting, and Web 2.0 applications. This necessitates federation of cloud infrastructure service providers for seamless provisioning of services across different cloud providers. To realize this, the Cloudbus Project at the University of Melbourne has proposed InterCloud architecture supporting brokering and exchange of cloud resources for scaling applications across multiple clouds.

By realizing InterCloud architectural principles in mechanisms in their offering, cloud providers will be able to dynamically expand or resize their provisioning capability based on sudden spikes in workload demands by leasing available computational and storage capabilities from other cloud service providers; operate as part of a market-driven resource leasing federation, where application service providers such as Salesforce.com host their services based on negotiated SLA contracts driven by competitive market prices; and deliver on-demand, reliable, cost-effective, and QoS-aware services based on virtualization technologies while ensuring high QoS standards and minimizing service costs. They need to be able to utilize market-based utility models as the basis for provisioning of virtualized software services and federated hardware infrastructure among users with heterogeneous applications.

They consist of client brokering and coordinator services that support utility-driven federation of clouds: application scheduling, resource allocation, and migration of workloads. The architecture cohesively couples the administratively and topologically distributed storage and compute capabilities of clouds as part of a single resource leasing abstraction. The system will ease the cross-domain capability integration for on-demand, flexible, energy-efficient, and reliable access to the infrastructure based on virtualization technology. The Cloud Exchange (CEX) acts as a market maker for bringing together service producers and consumers. It aggregates the infrastructure demands from application brokers and

evaluates them against the available supply currently published by the cloud coordinators. It supports trading of cloud services based on competitive economic models such as commodity markets and auctions. CEx allows participants to locate providers and consumers with fitting offers. Such markets enable services to be commoditized, and thus will pave the way for creation of dynamic market infrastructure for trading based on SLAs. An SLA specifies the details of the service to be provided in terms of metrics agreed upon by all parties, and incentives and penalties for meeting and violating the expectations, respectively. The availability of a banking system within the market ensures that financial transactions pertaining to SLAs between participants are carried out in a secure and dependable environment.

4.4 Security Overview

Cloud computing security or, more simply, cloud security refers to a broad set of policies, technologies, applications, and controls utilized to protect virtualized IP, data, applications, services, and the associated infrastructure of cloud computing.

Ensure Local Backup

It is the essential precaution that one can take towards cloud data security. Misuse of data is one thing, but losing possible data from your end may result in dire consequences. Especially in the IT world, where information is everything organizations depend upon; losing data files could not only lead to a significant financial loss but may also attract legal action.

Avoid Storing Sensitive Information

Many companies refrain from storing personal data on their servers, and there is sensibility behind the decision — saving sensitive becomes a responsibility of the organization. Compromise with such data can lead to gruesome troubles for the firm. Giants such as Facebook have been dragged to court under such issues in the past. Additionally, uploading sensitive data is faulty from the customer's perspective too. Merely avoid storing such sensitive data on the cloud.

Use Encryption

Encrypting data before uploading it to the cloud is an excellent precaution against threats from unwanted hackers. Use local encryption as an additional layer of security. Known as zero-knowledge proof in cryptography, this method will even protect your data against service providers and administrators themselves. Therefore, choose a service provider who provides a prerequisite data encryption. Also if you're already opting for an encrypted cloud service, having a preliminary round of encryption for your files will give you a little extra security.

Apply Reliable Passwords

Utilize discretion and don't make your passwords predictable. Additionally, introduce a two-step verification process to enhance the security level of your data. Even if there is a breach in one security step, the other protects the data. Use updated patch levels so that hackers cannot break-in easily. There are numerous tips on the Internet to make a good password. Use your creativity to strengthen the password further and keep changing it at regular intervals.

Additional Security Measures

Although passwords are good for keeping data encrypted, applying additional measures are also important. Encryption stops unauthorized access of data, but it doesn't secure its existence. There are chances that your data might get corrupted over the time or that many people will have access to your data and password security seems unreliable. Your cloud must be secured with antivirus programs, admin controls, and other features that help protect data. A secure cloud system and its dedicated servers must use the right security tools and must function according to privilege controls to move data.

4.4.1 SECURITY BEST PRACTICES

Strategy & Policy

A holistic cloud security program should account for ownership and accountability (internal/external) of cloud security risks, gaps in protection/compliance, and identify controls needed to mature security and reach the desired end state.

Network Segmentation

In multi-tenant environments, assess what segmentation is in place between your resources and those of other customers, as well as between your own instances. Leverage a zone approach to isolate instances, containers, applications, and full systems from each other when possible.

Identity and Access Management and Privileged Access Management

Leverage robust identity management and authentication processes to ensure only authorized users to have access to the cloud environment, applications, and data. Enforce least privilege to restrict privileged access and to harden cloud resources (for instance, only expose resources to the Internet as is necessary, and de-activate unneeded capabilities/features/access). Ensure privileges are role-based, and that privileged access is audited and recorded via session monitoring.

Discover and Onboard Cloud Instances and Assets

Once cloud instances, services, and assets are discovered and grouped, bring them under management (i.e. managing and cycling passwords, etc.). Discovery and onboarding should be automated as much as possible to eliminate shadow IT.

Password Control (Privileged and Non-Privileged Passwords)

Never allow the use of shared passwords. Combine passwords with other authentication systems for sensitive areas. Ensure password management best practices.

Vulnerability Management

Regularly perform vulnerability scans and security audits, and patch known vulnerabilities.

Encryption

Ensure your cloud data is encrypted, at rest, and in transit.

Disaster Recovery

Be aware of the data backup, retention, and recovery policies and processes for your cloud vendor(s). Do they meet your internal standards? Do you have break-glass strategies and solutions in place?

Monitoring, Alerting, and Reporting

Implement continual security and user activity monitoring across all environments and instances. Try to integrate and centralize data from your cloud provider (if available) with data from in-house and other vendor solutions, so you have a holistic picture of what is happening in your environment.

4.5 Cloud Security Challenges

Here are the major security challenges that companies using cloud infrastructure have to prepare for.

Data breaches

A data breach might be the primary objective of a targeted attack or simply the result of human error, application vulnerabilities, or poor security practices. It might involve any kind of information that was not intended for public release, including personal health information, financial information, personally identifiable information, trade secrets, and intellectual property. An organization's cloud-based data may have value to different parties for different reasons.

Access management

Since cloud enables access to company's data from anywhere, companies need to make sure that not everyone has access to that data. This is done through various policies and guardrails that ensure only legitimate users have access to vital information, and bad actors are left out.

Data encryption

Implementing a cloud computing strategy means placing critical data in the hands of a third party, so ensuring the data remains secure both at rest (data residing on storage media) as well as when in transit is of paramount importance. Data needs to be encrypted at all times, with clearly defined roles when it comes to who will be managing the encryption keys. In most cases, the only way to truly ensure confidentiality of encrypted data that resides on a cloud provider's storage servers is for the client to own and manage the data encryption keys.

Denial of service (DoS/DDoS attacks)

Distributed denial-of-service attack (DDoS), like any denial-of-service attack (DoS), has as its final goal to stop the functioning of the targeted site so that no one can access it. The services of the targeted host connected to the internet are then stopped temporarily, or even indefinitely.

Advanced persistent threats (APTs)

APTs are a parasitical form of cyber attack that infiltrates systems to establish a foothold in the IT infrastructure of target companies, from which they steal data. APTs pursue their goals stealthily over extended periods of time, often adapting to the security measures intended to defend against them.

Conclusion

The cloud is not completely immune to security issues - no system will ever be. Bad actors are always developing new ways of exploiting security vulnerabilities and attacking systems, and often it's the human error that causes data breaches, loss of information and other unwanted consequences.

Still, cloud infrastructure is safer than traditional IT on-premises infrastructure, and it also brings with it other benefits like reduced costs, scalability and flexibility. Security threats directed toward cloud are on the rise - but so are security solutions developed to protect sensitive data and websites from malicious attacks.

Secure Application Programming Interfaces [APIs]:

Cloud APIs are the programming interfaces embedded into the cloud system. It automates the several tasks and makes the job easier. The APIs generally embedded are Representational State Transfer [REST]; Simple Object Access Protocol [SOAP]; XML-RPC or the JSON-RPC.

When an API is incorporated, the issues like identity, authentication, authorization, sessions, username, certificates, OAuth, Custom Authentic scheme, API key and, etc., must be addressed.

- While choosing the cloud service provider, the documentation of their API must be checked.
- You must hire a penetration tester to test the API provided and same measures must be taken while developing own APIs to ward-off security bugs if any.

Vulnerabilities in the system:

- The shortfalls in the virtual machines could be exploited for vulnerabilities.
- The virtual machines vulnerability includes hypervisors, VM hopping, virtual machine-based rootkits, denial of service attacks, data leakage, and more.
- The well-known existing vulnerabilities in the virtual machines include buffer overflows, denial of service, execution of malicious codes, and gain privileges.
- Another known vulnerability in the VMware products includes the path traversal vulnerability. If it gets exploited, the attacker will be able to control the guest VM image, break the access, disrupt the flow if the VM host is not disabled.

Loss of data:

Apart from the malicious attacks, the data could be lost permanently owing to accidental deletions, a physical catastrophe like the fire or the earthquake. It is recommended to follow the best practices for preventing hamper in business continuity and disaster recovery.

A few of the methods recommended are:

- Protect the data either at disk level or through scale-out storage
- Periodic backup of the data at cost-effective lower tier medium
- Journalled file system or checkpoint replication will enable to recover data

Loss of Revenue:

- Whenever a news hits the headlines telling about the company ABC's data breach, invariably it affects the revenue where we can expect about 50% drop in the first quarter. This loss is really huge for a company to recover.
- It is recommended that the company has to reduce the unmanaged cloud usage and thereto its associated risks. The IT teams must understand the uploaded data, shared data, and enforce adequate security and governing policies to protect the data.
- The companies must be aware of the associated risks related to the implementation of the cloud services and mitigate them, take proactive approaches in securing the data, and thus availing the clear benefits of the cloud.

Other Potential Threats

Alongside the potential security vulnerabilities relating directly to the cloud service, there are also a number of external threats which could cause an issue. Some of these are:

Man in the Middle attacks – where a third party manages to become a relay of data between a source and a destination. If this is achieved, the data being transmitted can be altered.

Distributed Denial of Service – a DDoS attack attempts to knock a resource offline by flooding it with too much traffic.

Account or Service Traffic Hijacking – a successful attack of this kind could provide an intruder with passwords or other access keys which allow them access to secure data.

4.6 Software-as-a-Service Security

Cloud access security brokerage

Cloud access security brokerages (CASBs) are the “integrated suites” of the SECaaS world. CASB vendors typically provide a range of services designed to help your company protect cloud infrastructure and data in whatever form it takes. According to McAfee, CASBs “are on-premises or cloud-hosted software that sit between cloud service consumers and cloud service providers to enforce security, compliance, and governance policies for cloud applications.” These tools monitor and act as security for all of a company's cloud applications.

Single sign-on

Single sign-on (SSO) services give users the ability to access all of their enterprise cloud apps with a single set of login credentials. SSO also gives IT and network administrators a better ability to monitor access and accounts. Some of the larger SaaS vendors already provide SSO capabilities for products within their suite, but chances are, you don't just use applications from one vendor, which is where a third-party SSO provider would come in handy.

Email security

It may not be the first application that comes to mind when you think about outsourcing security, but a massive amount of data travels in and out of your business through cloud-based email servers. SECaaS providers that focus on email security can protect you from the menagerie of threats and risks that are an intrinsic part of email like malvertising, targeted attacks, phishing, and data breaches. Some email security tools are part of a larger platform, while other vendors offer it as a standalone solution.

Website and app Security

Beyond protecting your data and infrastructure when using cloud-based applications, you also need to protect the apps and digital properties that you own and manage—like your website. This is another area where traditional endpoint and firewall protection will still leave you vulnerable to attacks, hacks, and breaches. Tools and services in this category are usually designed to expose and seal vulnerabilities in your external-facing websites, web applications, or internal portals and intranets.

Network security

Cloud-based network security applications help your business monitor traffic moving in and out of your servers and stop threats before they materialize. You may already use a hardware-based firewall, but with a limitless variety of threats spread across the internet today, it's a good idea to have multiple layers of security. Network security as a service, of course, means the vendor would deliver threat detection and intrusion prevention through the cloud.

4.7 Security Governance

Cloud security governance refers to the management model that facilitates effective and efficient security management and operations in the cloud environment so that an enterprise's business targets are achieved.

An organisation's board is responsible (and accountable to shareholders, regulators and customers) for the framework of standards, processes and activities that, together, make sure the organisation benefits securely from Cloud computing. We are the leading provider of information, books, products and services that help boards develop, implement and maintain a Cloud governance framework.

Trust boundaries in the Cloud

Organisations are responsible for their own information. The nature of Cloud computing means that at some point the organisation will rely on a third party for some element of the security of its data. The point at which the responsibility passes from your organisation to your supplier is called the 'trust boundary' and it occurs at a different point for Infrastructure as a Service (IaaS), Platform as a Service (PaaS) and Software as a Service (SaaS). Organisations need to satisfy themselves of the security and resilience of their Cloud service providers.

Cloud Controls Matrix

The Cloud Security Alliance (CSA) developed and maintains the Cloud Controls Matrix, a set of additional information security controls designed specifically for Cloud services providers (CSPs), and against which customers can carry out a security audit. BSI and the CSA have collaborated to offer a certification scheme (designed as an extension to ISO 27001) against which CSPs can achieve independent certification.

Cloud security certification

The CSA offers an open Cloud security certification process: STAR (Security, Trust and Assurance Registry). This scheme starts with self-assessment and progresses through process maturity to an externally certified maturity scheme, supported by an open registry of information about certified organisations.

Continuity and resilience in the Cloud

Cloud service providers are as likely to suffer operational outages as any other organisation. Physical infrastructure can also be negatively affected. Buyers of Cloud services should satisfy themselves that their CSPs are adequately resilient against operational risks. ISO22301 is an appropriate business continuity standard.

Data protection in the Cloud

- UK organisations that store personal data in the Cloud or that use a CSP must currently comply with the DPA.
- However, since the GDPR came into effect on 25 May 2018, data processors and data controllers are now accountable for the security of the personal data they process.
- CSPs and organisations that use them will need to implement appropriate technical and organisational measures to make sure that processing meets the GDPR's requirements and protects the rights of data subjects.

Enforcing cloud security governance policies

As policies are developed, they need to be enforced. The enforcement of cloud security policies needs a combination of people, processes, and technology working together—the people being stakeholders and the executive level, the processes being the procedures for amending policies when necessary, and the technology being the mechanisms that monitor compliance with the policies.

Each one of these factors is equally important, yet some businesses still experience difficulties in enforcing their frameworks due to a lack of support from stakeholders and the executive level, failure to plan ahead for amending policies when necessary or implementing inadequate technologies for monitoring compliance with cloud security governance policies.

Whereas the first two issues are out of our control, CloudHealth is the ideal solution for not only monitoring compliance with cloud security governance policies but also for preventing users from operating outside policy parameters. CloudHealth uses a process called policy-driven automation to effectively monitor and enforce cloud security governance policies, in which our cloud management platform is configured with business-specific policies, triggering certain actions to take if a policy is violated.

G-Cloud framework

The UK government's G-Cloud framework makes it faster and cheaper for the public sector to buy Cloud services. Suppliers are approved by the Crown Commercial Service (CCS) via the G-Cloud application process, which eliminates the need for them to go through a full tender process for each

buyer. Suppliers can sell Cloud services via an online catalogue called the Digital Marketplace under three categories **Cloud hosting** – Cloud platform or infrastructure services.

Cloud software – applications that are accessed over the Internet and hosted in the Cloud.

Cloud support – services to help buyers set up and maintain their Cloud services.

4.8 Virtual Machine Security

There are challenges introduced by the dynamism of virtualization in cloud:

Dynamic relocation of virtual machines (VMs): Hypervisors today move workloads based on the service level agreement (SLA), energy policy, resiliency policy, and a host of other reasons. IT administrators of today can no longer be sure where the workload resides in the data center.

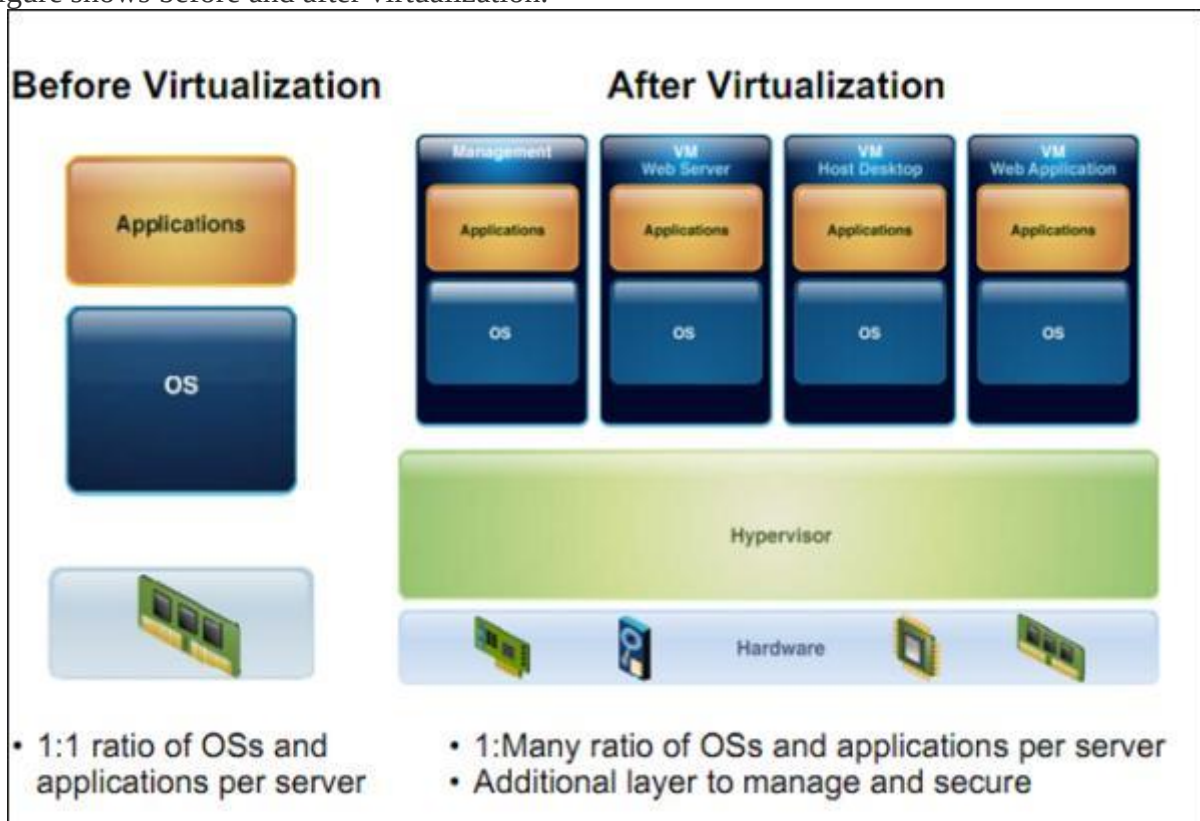
Increased infrastructure layers to manage and protect: Depending on the type of cloud model in use, there are a large number of additional infrastructure layers such as gateways, firewalls, access routers, and others, that need to be managed and protected, at the same time allowing access to the authorized users to perform their tasks.

Multiple operating systems and applications per server: On virtualized commodity hardware, multiple workloads on a physical server run concurrently, with multiple operating systems and even with same operating systems but at different patch levels.

Elimination of physical boundaries between systems: As virtualization adoption increases, workloads are co-located sharing the same physical infrastructure.

Tracking software and configuration of VMs: As IT infrastructure becomes virtualized, it is increasingly complex to manage software configuration including patch levels, security patches, security audits, and others, not just for guest operating systems but also for other virtualized infrastructure such as virtual distributed switches.

The figure shows before and after virtualization.

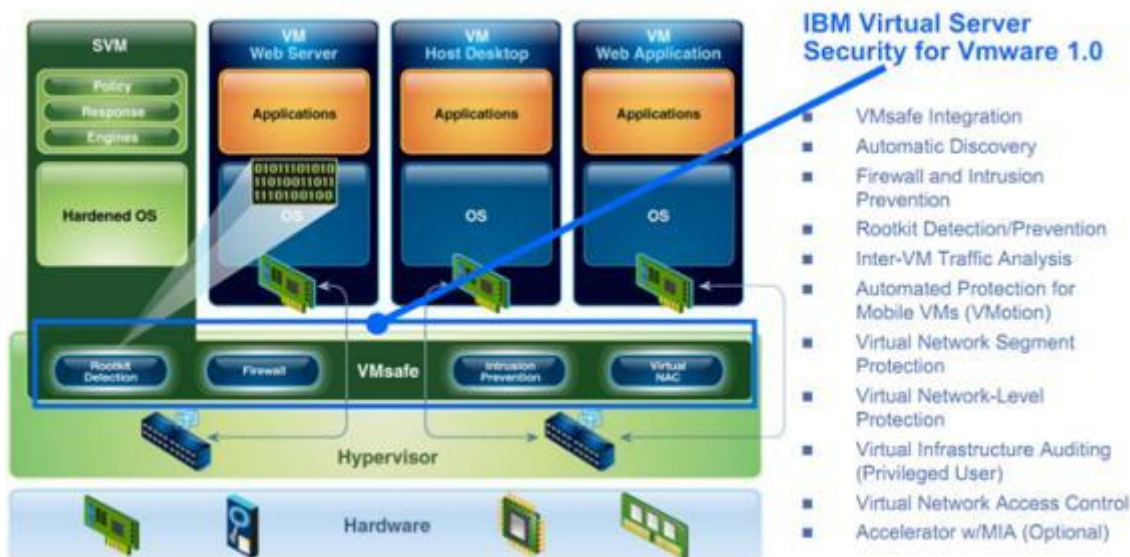


Traditional security products encounter new challenges in the virtualized world:

- Intrusiveness of existing solutions

- Reconfiguration of virtual network: Some existing solutions might require reconfiguration of the virtual network to allow for packet sniffing and protocol examination.
- Presence in the guest OS: For monitoring purposes, agents are required to be installed on the guest OS.
- Visibility and control gaps
- Virtual servers not connected to the physical network are invisible and unprotected.
- Lacks automation and transparency.
- Static security controls are too rigid. As VMs are moved around by the hypervisor, static controls need to be reapplied.
- No ability to deal with workload mobility exists.
- Resource overhead.
- Network traffic analysis in each guest OS is redundant, consuming more CPU cycles.

Virtual Server Protection for VMware is shown in the next figure.



Virtual Server Protection for VMware provides the following benefits:

- Dynamic protection of every layer of infrastructure, mitigating the risks introduced by virtualization.
- Meets regulatory and compliance requirements.
- Increases ROI of virtual infrastructure because it is easy to deploy and maintain security.

Integrated security benefits of Virtual Server Protection for VMware are as follows:

Transparency

- No reconfiguration of virtual network required
- No presence in guest OS

Security consolidation

- Only one Security Virtual Machine (SVM) required per physical server
- 1:many protection ratio

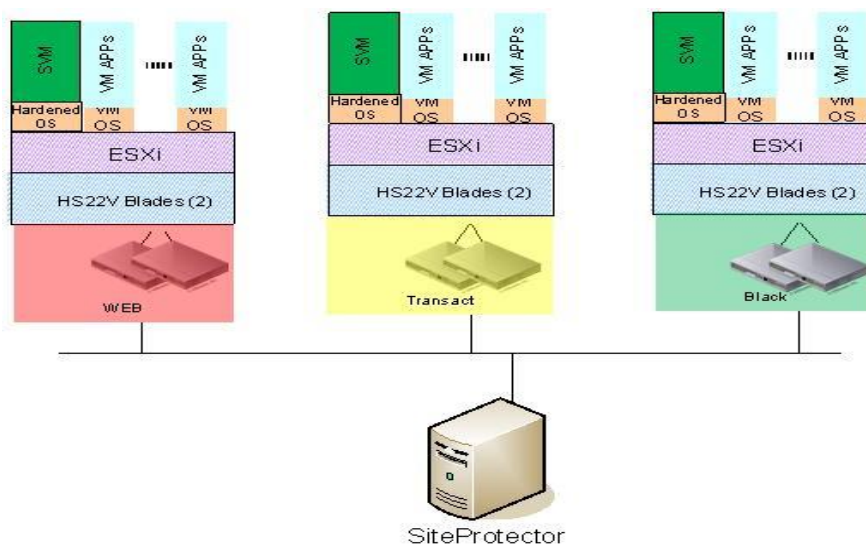
Automation

- Privileged presence gives SVM holistic view of the virtual network
- Protection applied automatically as each new VM comes online

Efficiency

- Eliminates redundant processing tasks
- Protection for any guest OS

An example of Virtual Server Protection for VMware architecture is shown in the next figure:



In the figure example, three ESX clusters are in three separate network zones (WEB, Transact, and Black), separated physically. Virtual Server Protection for VMware is deployed on each ESX host in the cluster (HS22V blades in this example), which monitors all VMs as they are brought online. All policy events data is forwarded by the SVM to the SiteProtector appliance.

4.9 IAM

Identity and access management (IAM) in enterprise IT is about defining and managing the roles and access privileges of individual network users and the circumstances in which users are granted (or denied) those privileges. Those users might be customers (customer identity management) or employees (employee identity management). The core objective of IAM systems is one digital identity per individual. Once that digital identity has been established, it must be maintained, modified and monitored throughout each user's "access lifecycle."

Need IAM?

Identity and access management is a critical part of any enterprise security plan, as it is inextricably linked to the security and productivity of organizations in today's digitally enabled economy.

Compromised user credentials often serve as an entry point into an organization's network and its information assets. Enterprises use identity management to safeguard their information assets against the rising threats of ransomware, criminal hacking, phishing and other malware attacks.

Access management: Access management refers to the processes and technologies used to control and monitor network access. Access management features, such as authentication, authorization, trust and security auditing, are part and parcel of the top ID management systems for both on-premises and cloud-based systems.

Active Directory (AD): Microsoft developed AD as a user-identity directory service for Windows domain networks. Though proprietary, AD is included in the Windows Server operating system and is thus widely deployed.

Biometric authentication: A security process for authenticating users that relies upon the user's unique characteristics. Biometric authentication technologies include fingerprint sensors, iris and retina scanning, and facial recognition.

Context-aware network access control: Context-aware network access control is a policy-based method of granting access to network resources according to the current context of the user seeking access. For example, a user attempting to authenticate from an IP address that hasn't been whitelisted would be blocked.

Credential: An identifier employed by the user to gain access to a network such as the user's password, public key infrastructure (PKI) certificate, or biometric information (fingerprint, iris scan).

De-provisioning: The process of removing an identity from an ID repository and terminating access privileges.

Digital identity: The ID itself, including the description of the user and his/her/its access privileges. ("Its" because an endpoint, such as a laptop or smartphone, can have its own digital identity.)

Entitlement: The set of attributes that specify the access rights and privileges of an authenticated security principal.

Identity as a Service (IDaaS): Cloud-based IDaaS offers identity and access management functionality to an organization's systems that reside on-premises and/or in the cloud.

Identity lifecycle management: Similar to access lifecycle management, the term refers to the entire set of processes and technologies for maintaining and updating digital identities. Identity lifecycle management includes identity synchronization, provisioning, de-provisioning, and the ongoing management of user attributes, credentials and entitlements.

Identity synchronization: The process of ensuring that multiple identity stores—say, the result of an acquisition—contain consistent data for a given digital ID.

Lightweight Directory Access Protocol (LDAP): LDAP is open standards-based protocol for managing and accessing a distributed directory service, such as Microsoft's AD

Multi-factor authentication (MFA): MFA is when more than just a single factor, such as a user name and password, is required for authentication to a network or system. At least one additional step is also required, such as receiving a code sent via SMS to a smartphone, inserting a smart card or USB stick, or satisfying a biometric authentication requirement, such as a fingerprint scan.

Password reset: In this context, it's a feature of an ID management system that allows users to re-establish their own passwords, relieving the administrators of the job and cutting support calls. The reset application is often accessed by the user through a browser. The application asks for a secret word or a set of questions to verify the user's identity.

Privileged account management: This term refers to managing and auditing accounts and data access based on the privileges of the user. In general terms, because of his or her job or function, a privileged user has been granted administrative access to systems. A privileged user, for example, would be able set up and delete user accounts and roles. Provisioning: The process of creating identities, defining their access privileges and adding them to an ID repository.

Risk-based authentication (RBA): Risk-based authentication dynamically adjusts authentication requirements based on the user's situation at the moment authentication is attempted. For example, when users attempt to authenticate from a geographic location or IP address not previously associated with them, those users may face additional authentication requirements.

Security principal: A digital identity with one or more credentials that can be authenticated and authorized to interact with the network.

Single sign-on (SSO): A type of access control for multiple related but separate systems. With a single username and password, a user can access a system or systems without using different credentials.

User behavior analytics (UBA): UBA technologies examine patterns of user behavior and automatically apply algorithms and analysis to detect important anomalies that may indicate potential security threats. UBA differs from other security technologies, which focus on tracking devices or security events. UBA is also sometimes grouped with entity behavior analytics and known as UEBA.

IAM vendors

The identity and access management vendor landscape is a crowded one, consisting of both pureplay providers such as Okta and OneLogin and large vendors such as IBM, Microsoft and Oracle. Below is a list of leading players based on Gartner's Magic Quadrant for Access Management, Worldwide, which was published in June 2017.

- Atos (Evidan)
- CA Technologies
- Centrify
- Covisint
- ForgeRock
- IBM Security Identity and Access Assurance
- I-Spring Innovations
- Micro Focus
- Microsoft Azure Active Directory
- Okta
- OneLogin
- Optimal idM
- Oracle Identity Cloud Service
- Ping

- SecureAuth

4.10 Security Standards.

In this document we focus on the following standards. This list is based on input from the ETSI working group on standards and the list of cloud standards published by NIST. We grouped closely related standards together for the sake of brevity.

- **HTML/XML**
- **WSDL/SOAP**
- **SAML/XACML**
- **OAuth/OpenID**
- **OData**
- **OVF**
- **OpenStack**
- **CAMP**
- **CIMI**
- **ODCA – SuoM**
- **SCAP**
- **ISO 27001**
- **ITIL**
- **SOC**
- **Tier Certification**
- **CSA CCM**

Note that it is a short list which is not exhaustive. There may well be other important standards or proposals interesting for discussion. The list of standards, with, per standard, a brief description of the standard.

4.10.1 Characteristics of standards

For each standard we will look at some key characteristics. These characteristics are not intended as means of qualification. Below for example, we may say that a standard is used only by a limited number of organizations, or that a standard is not publicly available, but this does not mean that the standard is inferior nor that it is better, than other standards. Similar, we may say that a security standard is not specific for IaaS, but that does not mean that it is not a relevant security standard for an IaaS provider or customer.

4.10.2 Application domain

We indicate the type of assets addressed by the standard, based on the types of assets introduced in Section 2.

- **Infrastructure as a Service**
- **Platform as a Service**
- **Software as a Service**
- **Facilities**
- **Organisation**

For example, we denote that the application domain of a standard is IaaS, if the standard contains requirements for IaaS assets, such as virtual machines or hypervisors. Similarly, we denote that a standard applies to Facilities if the standard contains requirements for setting up or maintaining facilities. Note that in the latter case the standard may be very relevant for cloud computing services, without being specific to one type of cloud service or the other.

4.10.3 Usage/Adoption

We indicate the estimate size of the user base, in terms of end-users or services. We use three levels:

- Globally (xxx) – thousands of organizations worldwide
- Widely (xx) - hundreds of organizations, regional or worldwide
- Limited (x) – tens of organizations or less, for example in pilots

4.10.4 Certification/auditing

We indicate whether or not there is a certification framework, to certify compliance with the standard,

or, alternatively, whether or not it is common to have third-party audits to certify compliance. We use three levels:

- Common (xxx): Audits are common and certification frameworks exist.
- Sometimes (xx): Audits of compliance to the standard are sometimes carried out.
- Hardly (x): De-facto standard. There is no audit or certification asserting compliance.

4.10.5 Availability/Openness

We indicate whether or not the standard is public and open, in terms of access and in terms of the review process. We distinguish three levels:

- Fully open (xxx) - Open consultation for drafts (like W3C, IETF, OASIS, etc.), and open access to final versions (or for a small fee, less than 100 euro).
- Partially open (xx) - Consultation is closed/membership, but there is open access to the standard.
- Closed (x) – Consultations are not open to the public, and the standard is not public either (or there is a substantial fee, more than 100 euro).

4.10.6 Existing standards

Cloud computing services are much more standard than traditional IT deployments and most cloud computing services have highly interoperable (and standard) interfaces. We mention some key standards, which allow customers to move data and processes more easily to other providers or to fall back on back-up services:

Like for other products and services, contracts and/or SLAs for cloud services are hardly standardized. The so-called ‘fine-print’ in contracts often hides important conditions and exceptions, and the terminology used in contracts or SLAs is often different from one provider to another. This means that customers have to read each contract and SLA in detail and sometimes consult a legal expert to understand clauses. Even if the customer has access to legal advice, it is often unpredictable how certain wordings in agreements will be interpreted in court. The standardization of IT services in cloud computing might enable further standardization of contracts and SLAs also. We mention one standard that defines specific standard service levels:

- HTML / XML allow users to integrate different cloud services and to (more easily) migrate data from one provider to another.
- WSDL/SOAP is an interface standard (which uses XML) which enables interoperability between products and services, facilitating integration and migration. An emerging standard with similar purpose.
- OAuth/OpenID and SAML/XACML allows customers to integrate a cloud service with other (existing) IDM solutions, allowing easire integration of an identity provider with other (existing) websites (SaaS for example). OAuth/OpenID and SAML/XACML also facilitates portability between cloud implementations that support the framework.
- SAML/XACML provides users with an interface to manage the provision of identification and user authentication between user and provider.
- OData is a standard for accessing and managing data, based on JSON. OData allows customers to integrate a (IaaS or SaaS) cloud service with other (existing) services with cloud ones, making the integration of this kind of service easier.
- OVF is a standard format for virtual machines. OVF allows customers to use existing virtual machines, and move virtual machine images more easily from one provider to another.
- SUoM is a standard developed by the Open Data Center Alliance, which describes a set of standard service parameters for IaaS, in 4 different levels (Bronze, Silver, Gold, Platinum), and covers among other things, security, availability, and elasticity.
- OpenStack is a standard software stack for IaaS. OpenStack dashboard could be used also to monitor the usage of cloud resources and it provides a standard API for managing cloud resources.
- CAMP provides users with artifacts and APIs to manage the provision of resources of her PaaS provider. During the life of the service, CAMP supports the modification of PaaS resources, according to user needs.

- CIMI provides users with an interface to manage the provision of resources of her IaaS provider. During the life of the service, CIMI supports the modification of IaaS resources, according to user needs.
- SCAP is a standard for specifying vulnerabilities. SCAP allows customers to keep track of security flaws and evaluate the state of infrastructure in terms of vulnerabilities and patching.

Mapping standards to use cases Table 1 summarizes this section and shows which standards are relevant for customers in the different use cases.

	Application domain					Other characteristics		
	IaaS	Paas	SaaS	Facilities	Organization	Usage/ Adoption	Certification/ Auditing	Openness/ Availability
HTML/XML	x		x			xxx	x	xxx
WSDL/SOAP	x		x			xxx	x	xxx
OAuth/OpenID			x			xxx	x	xxx
SAML			x			xxx	x	xxx
OData	x		x			x	x	xxx
OVF	x					xxx	x	xxx
OpenStack	x			x		xx	x	xx
CAMP		x				x	x	xx
CIMI	x					x	x	xxx
ODCA SUoM	x					x	x	xx
SCAP	x	x	x	x		xxx	x	xx
ISO 27001				x	x	xxx	xxx	xx
ITIL					x	xx	xxx	xx
SOC				x	x	xx	xxx	xx
Tier Certification				x		xx	xxx	x
CSA CCM				x	x	x	xxx	xxx

A.1 HTML/XML

Full title	HyperText Markup Language (HTML) / eXtensible Markup Language (XML)
Description	HTML is the markup language for web pages – it is used for displaying text, links, images for human readers. HTML requires no further introduction. XML is a mark-up language and structure for encoding data in a format that is machine-readable. It is used for exchanging data between systems, for example in web services, but it is also used in application programming interfaces (APIs), or to store configuration files or other internal system data. Hundreds of XML-based languages have been developed, including RSS, Atom, SOAP, and XHTML. XML-based formats have become the default for many office-productivity tools. XML has also been employed as the base language for communication protocols, such as WSDL, and XMPP.

Link	W3C http://www.w3.org/html/ - http://www.w3.org/XML/ - HTML5 specs are developed by the WHAT WG http://www.whatwg.org/
Organisation	World Wide Web Consortium – the HTML WG and the XML Core WG. The latest HTML specifications have been developed by the WHATWG .
Application domain	IaaS and SaaS XML allows exchange of data between applications (SaaS) and it provides a standard format for representing data which is used in plain storage service (IaaS).
Openness	Open - xxx - Development – Drafts are open to feedback and public comments. - Availability – Standards are freely available
Certification and compliance	No - x Compliance is often not formally certified. There are various schemes/tools that help in determining compliance (for example the W3C compliance validation checker, or XML editors that can validate XML documents).
Adoption	Globally – xxx Millions of companies use these standards, for external interfaces and internally to facilitate integration between products.

WSDL/SOAP

Full title	Web Services Description Language (WSDL)
Description	WSDL is an XML-based interface description language that is used for describing a web service. A WSDL description provides a machine-readable description of how the service can be called, what parameters it expects, and what data structures it returns. SOAP, an underlying standard, is used as a wrapper for transporting WSDL messages (for example over HTTP).
Link	http://www.w3.org/2000/xp/Group/
Organisation	World Wide Web Consortium : - XML Protocol Working Group; and - Web Services Description Working Group
Application domain	IaaS and SaaS WSDL allows integration of SaaS services – and WSDL is also used as standard for accessing data storage services (a type of IaaS).
Openness	Open - xxx: - Development – Discussion between W3C members (and potentially, non-members experts invited by the Group Chair). - Availability – Documents are freely available to download from XML Protocol /Web Service Description Working Groups
Certification and compliance	None - x Companies often implement WSDL on a voluntary basis, without any formal process to check compliance (such as certification). There are tools to validate interoperability and vendors sometimes participate in multi-vendor interoperability workshops.
Adoption	Globally – xxx Thousands of companies use WSDL.

SAML/XACML

Full title	Security Assertion Markup Language (SAML), Extensible Access Control Markup Language (XACML)
-------------------	--

Description	SAML/XACML are XML-based languages and protocols for authentication and authorisation (on the web and inside local networks) of users for accessing websites. SAML/XACML supports the integration of websites and intranet servers with authentication/authorisation services and products, providing SSO for users (aka federation). SAML/XACML is used widely in enterprise software and e-government, for example. OAuth/OpenID are alternatives more widely used in social media.
Link	http://saml.xml.org/wiki/saml-wiki-knowledgebase
Organisation	Organization for the Advancement of Structured Information Standards (OASIS)
Application domain	SaaS As a framework that allows to access to an HTTP service, it works on the API/GUI component of the cloud service model.
Openness	Open - xxx - Development – Standard is discussed by OASIS Security Services Technical Committee experts. - Availability – Document is freely available to download from OASIS website.
Certification and compliance	None - x Compliance to SAML and XACML is usually not formally audited or certified – there are multi-vendor interoperability workshops.
Adoption	Globally – xxx Thousands of applications use or support SAML, but it is estimated than only less of 10% of the available applications (in fact, it is being replaced by Oauth as standard de-factor for identity management)

OData

Full title	Open Data Protocol
Link	http://www.odata.org/
Organisation	Microsoft developed the standard. It has been proposed for adoption by Organization for the Advancement of Structured Information Standards (OASIS)
Description	OData is a web protocol for querying and updating data. OData applies and builds upon Web technologies such as HTTP, Atom Publishing Protocol and JSON to provide access to information from a variety of applications, services, and stores. OData can be used to expose and access information from a variety of sources including, but not limited to, relational databases, file systems, content management systems and traditional Web sites.
Application domain	IaaS and SaaS - OData provides a (REST-full) API for managing data.
Openness/availability	xxx - Open: - Development – Standard is open for discussion/feedback via the OASIS OData Technical Committee. - Availability – Document is freely available to download from OData website.
Certification/auditing	None - x No formal audits or certification scheme for compliance.
Adoption/usage	Limited – x Tens of applications and tens of live services implement OData Protocol at the moment of issuing this report (according to OData website).

ISO 27001

Full title	Data Center Site Infrastructure Tier Standard
-------------------	---

Link	http://uptimeinstitute.com/publications
Organisation	The Uptime Institute
Description	The standard is an objective basis for comparing the functionality, capacities, and relative cost of a particular site infrastructure design topology against others, or to compare group of sites.
Application domain	Facilities. The standard applies to the elements included in data centers: Hardware, housing and power/cooling.
Openness/availability	Not open: • Development – Elaborated and discussed by the Owners Advisory Committee (those organizations that have successfully achieved Tier Certification). • Availability – It is not available for download from Uptime Institute website; neither it is available for purchase.
Certification/audits	The Uptime Institute has retained the exclusive legal right to review, assess, and Certify data centers to the Institute's Tier Classification System. There are three steps: • Design Certification • Constructed Facility Certification • Operational Sustainability Rating
Adoption/usage	Widely adopted – There are 269 data centers certified from Tier II to Tier IV (according to Uptime Institute website).

UNIT V CLOUD TECHNOLOGIES AND ADVANCEMENTS

5.1 Hadoop

Apache Hadoop is an open source software framework used to develop data processing applications which are executed in a distributed computing environment. Applications built using HADOOP are run on large data sets distributed across clusters of commodity computers. Commodity computers are cheap and widely available. These are mainly useful for achieving greater computational power at low cost.

Similar to data residing in a local file system of a personal computer system, in Hadoop, data resides in a distributed file system which is called as a Hadoop Distributed File system. The processing model is based on 'Data Locality' concept wherein computational logic is sent to cluster nodes (server)

containing data. This computational logic is nothing, but a compiled version of a program written in a high-level language such as Java. Such a program, processes data stored in Hadoop HDFS.

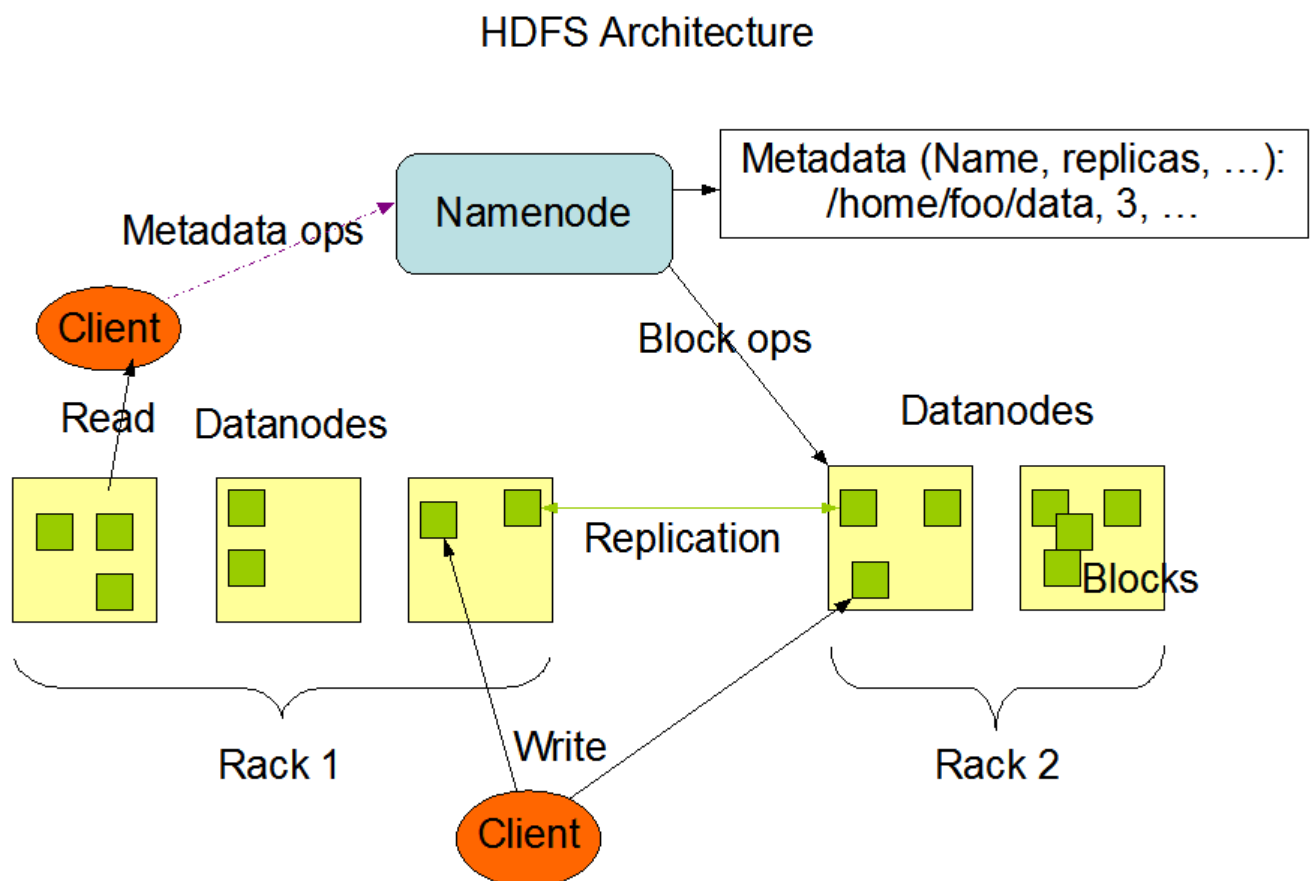
- Hadoop EcoSystem and Components
- Hadoop Architecture
- Features Of 'Hadoop'
- Network Topology In Hadoop

Apache Hadoop consists of two sub-projects –

Hadoop MapReduce: MapReduce is a computational model and software framework for writing applications which are run on Hadoop. These MapReduce programs are capable of processing enormous data in parallel on large clusters of computation nodes.

HDFS (Hadoop Distributed File System): HDFS takes care of the storage part of Hadoop applications. MapReduce applications consume data from HDFS. HDFS creates multiple replicas of data blocks and distributes them on compute nodes in a cluster. This distribution enables reliable and extremely rapid computations.

Although Hadoop is best known for MapReduce and its distributed file system- HDFS, the term is also used for a family of related projects that fall under the umbrella of distributed computing and large-scale data processing.



NameNode and DataNodes

HDFS has a master/slave architecture. An HDFS cluster consists of a single NameNode, a master server that manages the file system namespace and regulates access to files by clients. In addition, there are a number of DataNodes, usually one per node in the cluster, which manage storage attached to the nodes that they run on. HDFS exposes a file system namespace and allows user data to be stored in files. Internally, a file is split into one or more blocks and these blocks are stored in a set of DataNodes. The NameNode executes file system namespace operations like opening, closing, and renaming files and directories. It also determines the mapping of blocks to DataNodes. The DataNodes are responsible for serving read and write requests from the file system's clients. The DataNodes also perform block creation, deletion, and replication upon instruction from the NameNode.

The NameNode and DataNode are pieces of software designed to run on commodity machines. These machines typically run a GNU/Linux operating system (OS). HDFS is built using the Java

language; any machine that supports Java can run the NameNode or the DataNode software. Usage of the highly portable Java language means that HDFS can be deployed on a wide range of machines. A typical deployment has a dedicated machine that runs only the NameNode software. Each of the other machines in the cluster runs one instance of the DataNode software. The architecture does not preclude running multiple DataNodes on the same machine but in a real deployment that is rarely the case.

The existence of a single NameNode in a cluster greatly simplifies the architecture of the system. The NameNode is the arbitrator and repository for all HDFS metadata. The system is designed in such a way that user data never flows through the NameNode.

The File System Namespace

HDFS supports a traditional hierarchical file organization. A user or an application can create directories and store files inside these directories. The file system namespace hierarchy is similar to most other existing file systems; one can create and remove files, move a file from one directory to another, or rename a file. HDFS supports user quotas and access permissions. HDFS does not support hard links or soft links. However, the HDFS architecture does not preclude implementing these features.

While HDFS follows naming convention of the FileSystem, some paths and names (e.g. /.reserved and .snapshot) are reserved. Features such as transparent encryption and snapshot use reserved paths. The NameNode maintains the file system namespace. Any change to the file system namespace or its properties is recorded by the NameNode. An application can specify the number of replicas of a file that should be maintained by HDFS. The number of copies of a file is called the replication factor of that file. This information is stored by the NameNode.

Data Replication

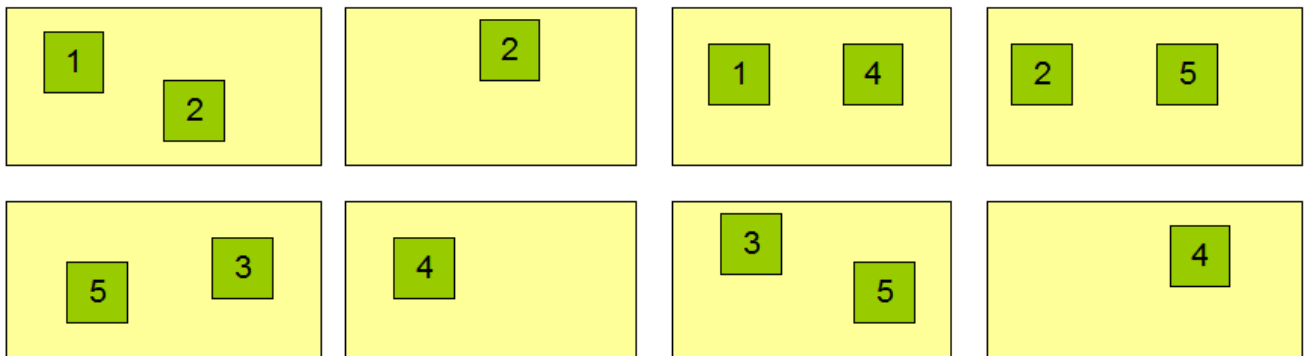
HDFS is designed to reliably store very large files across machines in a large cluster. It stores each file as a sequence of blocks. The blocks of a file are replicated for fault tolerance. The block size and replication factor are configurable per file. All blocks in a file except the last block are the same size, while users can start a new block without filling out the last block to the configured block size after the support for variable length block was added to append and hsync. An application can specify the number of replicas of a file.

The replication factor can be specified at file creation time and can be changed later. Files in HDFS are write-once (except for appends and truncates) and have strictly one writer at any time. The NameNode makes all decisions regarding replication of blocks. It periodically receives a Heartbeat and a Blockreport from each of the DataNodes in the cluster. Receipt of a Heartbeat implies that the DataNode is functioning properly. A Blockreport contains a list of all blocks on a DataNode.

Block Replication

```
Namenode (Filename, numReplicas, block-ids, ...)  
/users/sameerp/data/part-0, r:2, {1,3}, ...  
/users/sameerp/data/part-1, r:3, {2,4,5}, ...
```

Datanodes



Replication

The placement of replicas is critical to HDFS reliability and performance. Optimizing replica placement distinguishes HDFS from most other distributed file systems. This is a feature that needs lots of tuning and experience. The purpose of a rack-aware replica placement policy is to improve data reliability, availability, and network bandwidth utilization. The current implementation for the replica placement policy is a first effort in this direction. The short-term goals of implementing this policy are to validate it on production systems, learn more about its behavior, and build a foundation to test and research more sophisticated policies.

Large HDFS instances run on a cluster of computers that commonly spread across many racks. Communication between two nodes in different racks has to go through switches. In most cases, network bandwidth between machines in the same rack is greater than network bandwidth between machines in different racks. The NameNode determines the rack id each DataNode belongs to via the process outlined in Hadoop Rack Awareness. A simple but non-optimal policy is to place replicas on unique racks. This prevents losing data when an entire rack fails and allows use of bandwidth from multiple racks when reading data. This policy evenly distributes replicas in the cluster which makes it easy to balance load on component failure. However, this policy increases the cost of writes because a write needs to transfer blocks to multiple racks.

For the common case, when the replication factor is three, HDFS's placement policy is to put one replica on the local machine if the writer is on a datanode, otherwise on a random datanode in the same rack as that of the writer, another replica on a node in a different (remote) rack, and the last on a different node in the same remote rack. This policy cuts the inter-rack write traffic which generally improves write performance. The chance of rack failure is far less than that of node failure; this policy does not impact data reliability and availability guarantees. However, it does reduce the aggregate network bandwidth used when reading data since a block is placed in only two unique racks rather than three. With this policy, the replicas of a file do not evenly distribute across the racks. One third of replicas are on one node, two thirds of replicas are on one rack, and the other third are evenly distributed across the remaining racks. This policy improves write performance without compromising data reliability or read performance.

If the replication factor is greater than 3, the placement of the 4th and following replicas are determined randomly while keeping the number of replicas per rack below the upper limit (which is basically $(\text{replicas} - 1) / \text{racks} + 2$). Because the NameNode does not allow DataNodes to have multiple replicas of the same block, maximum number of replicas created is the total number of DataNodes at that time.

After the support for Storage Types and Storage Policies was added to HDFS, the NameNode takes the policy into account for replica placement in addition to the rack awareness described above. The NameNode chooses nodes based on rack awareness at first, then checks that the candidate node have storage required by the policy associated with the file. If the candidate node does not have the storage type, the NameNode looks for another node. If enough nodes to place replicas cannot be found in the first path, the NameNode looks for nodes having fallback storage types in the second path.

Replica Selection

To minimize global bandwidth consumption and read latency, HDFS tries to satisfy a read request from a replica that is closest to the reader. If there exists a replica on the same rack as the reader node, then that replica is preferred to satisfy the read request. If HDFS cluster spans multiple data centers, then a replica that is resident in the local data center is preferred over any remote replica.

5.2 MapReduce

What is MapReduce?

Hadoop MapReduce (Hadoop Map/Reduce) is a software framework for distributed processing of large data sets on computing clusters. It is a sub-project of the Apache Hadoop project. Apache Hadoop is an open-source framework that allows to store and process big data in a distributed environment across clusters of computers using simple programming models. MapReduce is the core component for data processing in Hadoop framework. In layman's term Mapreduce helps to split the input data set into a number of parts and run a program on all data parts parallel at once. The term MapReduce refers to two separate and distinct tasks. The first is the map operation, takes a set of data and converts it into another set of data, where individual elements are broken down into tuples (key/value pairs). The reduce operation combines those data tuples based on the key and accordingly modifies the value of the key.

Map Task

The Map task run in the following phases:-

a. RecordReader

The recordreader transforms the input split into records. It parses the data into records but does not parse records itself. It provides the data to the mapper function in key-value pairs. Usually, the key is the positional information and value is the data that comprises the record.

b. Map

In this phase, the mapper which is the user-defined function processes the key-value pair from the recordreader. It produces zero or multiple intermediate key-value pairs. The decision of what will be the key-value pair lies on the mapper function. The key is usually the data on which the reducer function does the grouping operation. And value is the data which gets aggregated to get the final result in the reducer function.

c. Combiner

The combiner is actually a localized reducer which groups the data in the map phase. It is optional. Combiner takes the intermediate data from the mapper and aggregates them. It does so within the small scope of one mapper. In many situations, this decreases the amount of data needed to move over the network. For example, moving (Hello World, 1) three times consumes more network bandwidth than moving (Hello World, 3). Combiner provides extreme performance gain with no drawbacks. The combiner is not guaranteed to execute. Hence it is not of overall algorithm.

d. Partitioner

Partitioner pulls the intermediate key-value pairs from the mapper. It splits them into shards, one shard per reducer. By default, partitioner fetches the hashCode of the key. The partitioner performs modulus operation by a number of reducers: $\text{key.hashCode()} \% (\text{number of reducers})$. This distributes the keyspace evenly over the reducers. It also ensures that key with the same value but from different mappers end up

into the same reducer. The partitioned data gets written on the local file system from each map task. It waits there so that reducer can pull it.

b. Reduce Task

The various phases in reduce task are as follows:

i. Shuffle and Sort

The reducer starts with shuffle and sort step. This step downloads the data written by partitioner to the machine where reducer is running. This step sorts the individual data pieces into a large data list. The purpose of this sort is to collect the equivalent keys together. The framework does this so that we could iterate over it easily in the reduce task. This phase is not customizable. The framework handles everything automatically. However, the developer has control over how the keys get sorted and grouped through a comparator object.

ii. Reduce

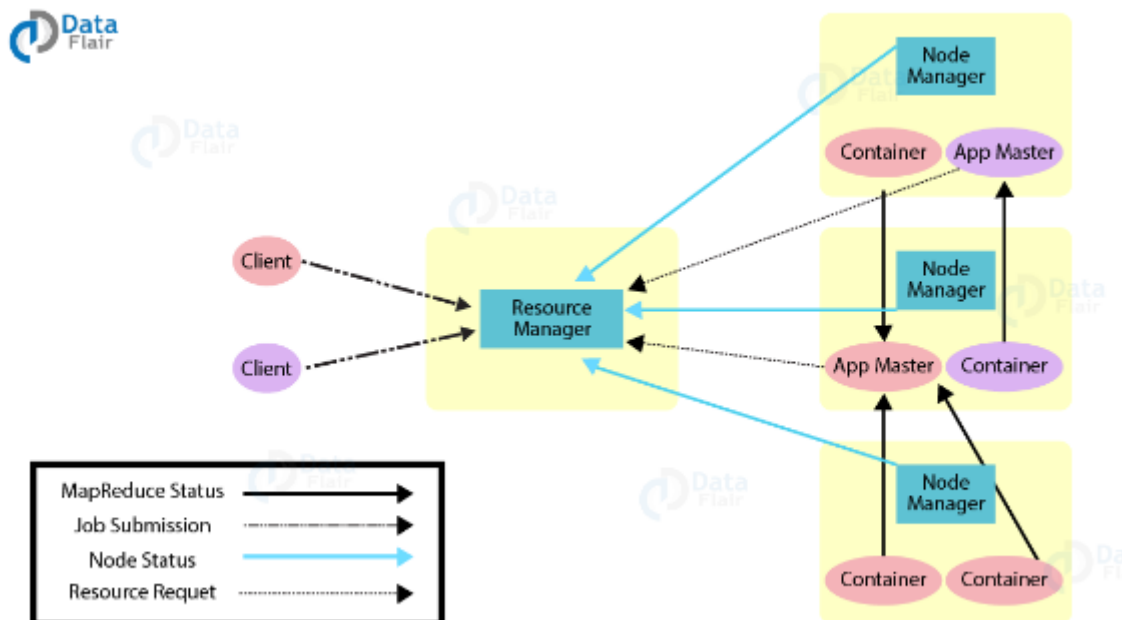
The reducer performs the reduce function once per key grouping. The framework passes the function key and an iterator object containing all the values pertaining to the key. We can write reducer to filter, aggregate and combine data in a number of different ways. Once the reduce function gets finished it gives zero or more key-value pairs to the output format. Like map function, reduce function changes from job to job. As it is the core logic of the solution.

iii. Output Format

This is the final step. It takes the key-value pair from the reducer and writes it to the file by record writer. By default, it separates the key and value by a tab and each record by a newline character. We can customize it to provide richer output format. But none the less final data gets written to HDFS.

YARN

YARN or Yet Another Resource Negotiator is the resource management layer of Hadoop. The basic principle behind YARN is to separate resource management and job scheduling/monitoring function into separate daemons. In YARN there is one global Resource Manager and per-application Application Master. An Application can be a single job or a DAG of jobs. Inside the YARN framework, we have two daemons Resource Manager and Node Manager. The Resource Manager arbitrates resources among all the competing applications in the system. The job of Node Manager is to monitor the resource usage by the container and report the same to Resource Manager. The resources are like CPU, memory, disk, network and so on.



i. Scheduler

Scheduler is responsible for allocating resources to various applications. This is a pure scheduler as it does not perform tracking of status for the application. It also does not reschedule the tasks which fail due to software or hardware errors. The scheduler allocates the resources based on the requirements of the applications.

ii. Application Manager

Following are the functions of ApplicationManager

- Accepts job submission.
- Negotiates the first container for executing ApplicationMaster. A container incorporates elements such as CPU, memory, disk, and network.
- Restarts the ApplicationMaster container on failure.

Functions of ApplicationMaster:-

- Negotiates resource container from Scheduler.
- Tracks the resource container status.
- Monitors progress of the application.

We can scale the YARN beyond a few thousand nodes through YARN Federation feature. This feature enables us to tie multiple YARN clusters into a single massive cluster. This allows for using independent clusters, clubbed together for a very large job.

iii. Features of Yarn

YARN has the following features:-

a. Multi-tenancy

YARN allows a variety of access engines (open-source or propriety) on the same Hadoop data set. These access engines can be of batch processing, real-time processing, iterative processing and so on.

b. Cluster Utilization

With the dynamic allocation of resources, YARN allows for good use of the cluster. As compared to static map-reduce rules in previous versions of Hadoop which provides lesser utilization of the cluster.

c. Scalability

Any data center processing power keeps on expanding. YARN's ResourceManager focuses on scheduling and copes with the ever-expanding cluster, processing petabytes of data.

d. Compatibility

MapReduce program developed for Hadoop 1.x can still run on this YARN. And this is without any disruption to processes that already work.

5.3 Virtual Box

VirtualBox is opensource software for virtualizing the X86 computing architecture. It acts as a hypervisor, creating a VM (Virtual Machine) in which the user can run another OS (operating system).

The operating system in which VirtualBox runs is called the "host" OS. The operating system running in the VM is called the "guest" OS. VirtualBox supports Windows, Linux, or macOS as its host OS. When configuring a virtual machine, the user can specify how many CPU cores, and how much RAM and disk space should be devoted to the VM. When the VM is running, it can be "paused." System execution is frozen at that moment in time, and the user can resume using it later.

Why Is VirtualBox Useful?

One:

VirtualBox allows you to run more than one operating system at a time. This way, you can run software written for one operating system on another (for example, Windows software on Linux or a Mac) without having to reboot to use it (as would be needed if you used partitioning and dual-booting). You can also configure what kinds of "virtual" hardware should be presented to each such operating system, and you can install an old operating system such as DOS or OS/2 even if your real computer's hardware is no longer supported by that operating system.

Two:

Sometimes, you may want to try out some new software, but would rather not chance it mucking up the pretty decent system you've got right now. Once installed, a virtual machine and its virtual hard disks can be considered a "container" that can be arbitrarily frozen, woken up, copied, backed up, and transported between hosts.

By using a VirtualBox feature called "snapshots", you can save a particular state of a virtual machine and revert back to that state, if necessary. This way, you can freely experiment with a computing environment. If something goes wrong (e.g. after installing misbehaving software or infecting the guest with a virus), you can easily switch back to a previous snapshot and avoid the need of frequent backups and restores.

Three:

Software vendors can use virtual machines to ship entire software configurations. For example, installing a complete mail server solution on a real machine can be a tedious task (think of rocket science!). With VirtualBox, such a complex setup (then often called an “appliance”) can be packed into a virtual machine. Installing and running a mail server becomes as easy as importing such an appliance into VirtualBox.

Along these same lines, I find the “clone” feature of virtual box just awesome! By cloning virtual machines, I’m able to move them from one machine to another along with all saved snapshots. If you try to imagine what it would involve to do something similar with physical machines, you will immediately see the power of this feature. Do have a look at my tutorial on moving virtual machines with snapshots.

Four:

On an enterprise level, virtualization can significantly reduce hardware and electricity costs. Most of the time, computers today only use a fraction of their potential power and run with low average system loads. A lot of hardware resources as well as electricity is thereby wasted. So, instead of running many such physical computers that are only partially used, one can pack many virtual machines onto a few powerful hosts and balance the loads between them.

VirtualBox Terminology

- When dealing with virtualization, it helps to acquaint oneself with a bit of crucial terminology, especially the following terms:

Host Operating System (Host OS):

- The operating system of the physical computer on which VirtualBox was installed. There are versions of VirtualBox for Windows, Mac OS X, Linux and Solaris hosts.

Guest Operating System (Guest OS):

- The operating system that is running inside the virtual machine.

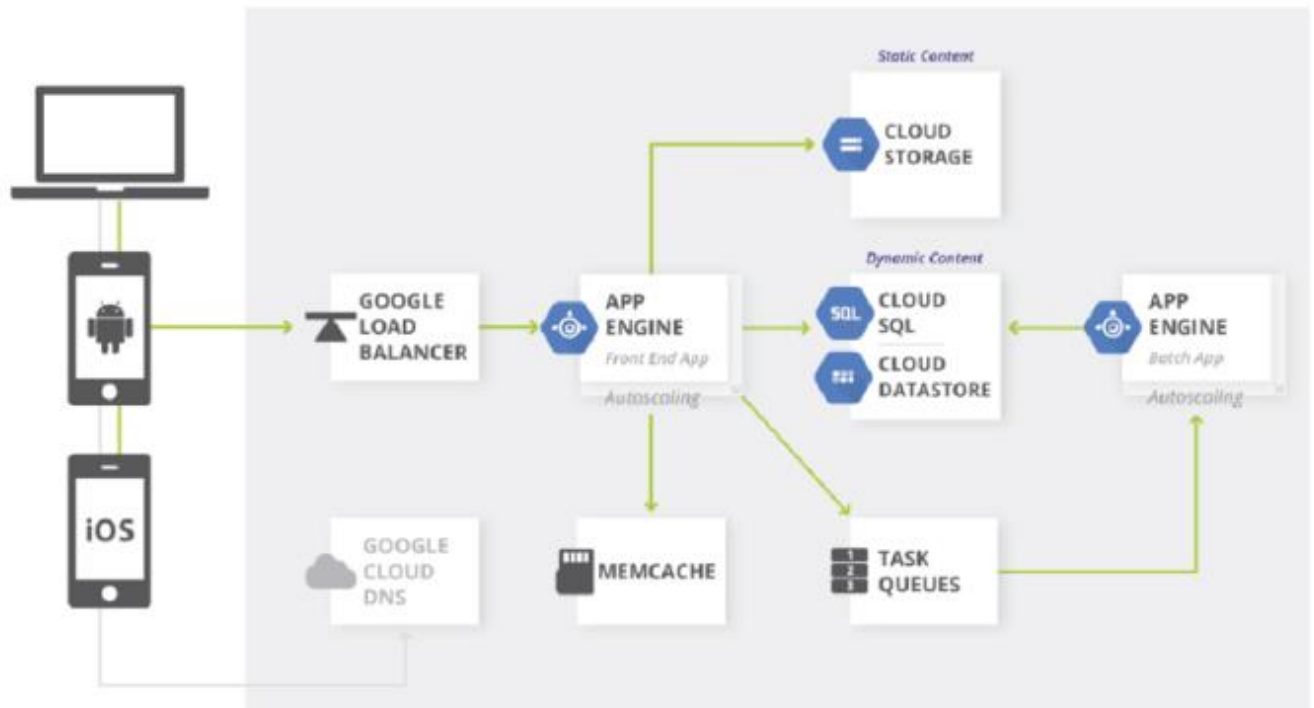
Virtual Machine (VM):

We’ve used this term often already. It is the special environment that VirtualBox creates for your guest operating system while it is running. In other words, you run your guest operating system “in” a VM. Normally, a VM will be shown as a window on your computers desktop, but depending on which of the various frontends of VirtualBox you use, it can be displayed in full screen mode or remotely on another computer.

5.4 Google App Engine

App Engine is a fully managed, serverless platform for developing and hosting web applications at scale. You can choose from several popular languages, libraries, and frameworks to develop your apps, then let App Engine take care of provisioning servers and scaling your app instances based on demand

- The App Engine requires that apps be written in Java or Python, store data in Google BigTable and use the Google query language. Non-compliant applications require modification to use App Engine.
- Google App Engine provides more infrastructure than other scalable hosting services such as Amazon Elastic Compute Cloud (EC2). The App Engine also eliminates some system administration and developmental tasks to make it easier to write scalable applications.
- Google App Engine is free up to a certain amount of resource usage. Users exceeding the per-day or per-minute usage rates for CPU resources, storage, number of API calls or requests and concurrent requests can pay for more of these resources.



Modern web applications

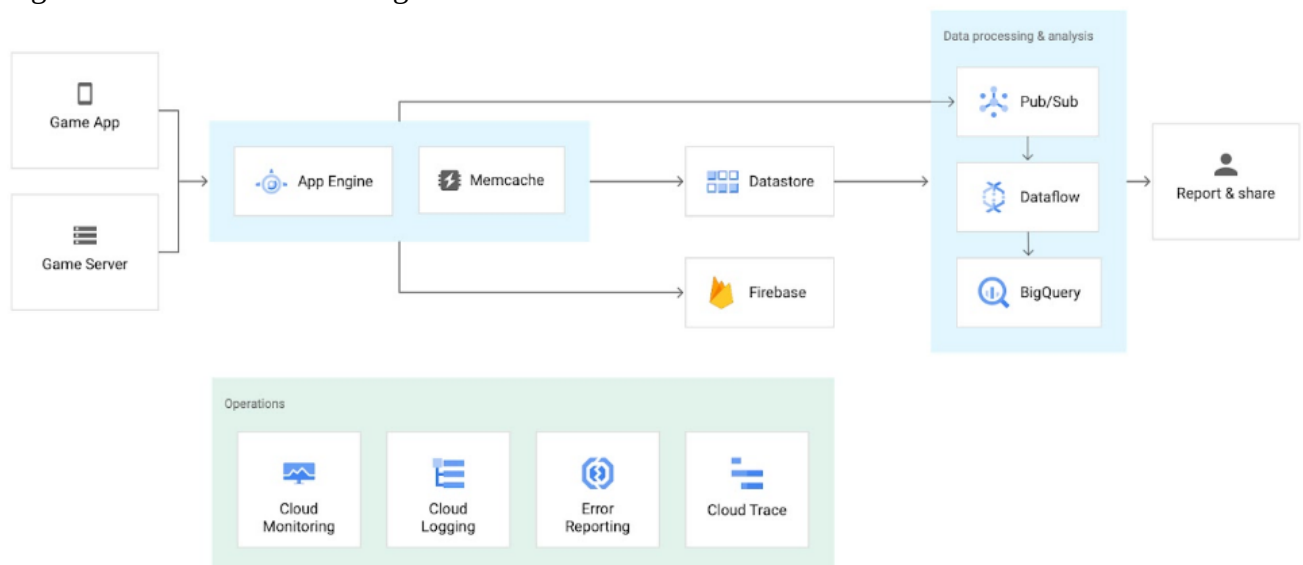
Quickly reach customers and end users by deploying web apps on App Engine. With zero-config deployments and zero server management, App Engine allows you to focus on writing code. Plus, App Engine automatically scales to support sudden traffic spikes without provisioning, patching, or monitoring.

Below is a sample reference architecture for building a simple web app using App Engine and Google Cloud.

Scalable mobile back ends

Whether you're building your first mobile app or looking to reach existing users via a mobile experience, App Engine automatically scales the hosting environment for you. Plus, seamless integration with Firebase provides an easy-to-use frontend mobile platform along with the scalable and reliable backend.

Below is sample reference architecture for a typical mobile app built using both Firebase and App Engine along with other services in Google Cloud.



Features

- Popular languages
- Build your application in Node.js, Java, Ruby, C#, Go, Python, or PHP—or bring your own language runtime.

Open and flexible

- Custom runtimes allow you to bring any library and framework to App Engine by supplying a Docker container.

Fully managed

- A fully managed environment lets you focus on code while App Engine manages infrastructure concerns.

Powerful application diagnostics

- Use Cloud Monitoring and Cloud Logging to monitor the health and performance of your app and Cloud Debugger and Error Reporting to diagnose and fix bugs quickly.

Application versioning

- Easily host different versions of your app, easily create development, test, staging, and production environments.

Traffic splitting

- Route incoming requests to different app versions, A/B test, and do incremental feature rollouts.

Application security

- Help safeguard your application by defining access rules with App Engine firewall and leverage managed SSL/TLS certificates* by default on your custom domain at no additional cost.

Services ecosystem

- Tap a growing ecosystem of Google Cloud services from your app including an excellent suite of cloud developer tools.

Advantages of Google App Engine

There are many advantages to the Google App Engine that helps to take your app ideas to the next level. This includes:

Infrastructure for Security

- Around the world, the Internet infrastructure that Google has is probably the most secure. There is rarely any type of unauthorized access to date as the application data and code are stored in highly secure servers.
- You can be sure that your app will be available to users worldwide at all times since Google has several hundred servers globally. Google's security and privacy policies are applicable to the apps developed using Google's infrastructure

Quick to Start

With no product or hardware to purchase and maintain, you can prototype and deploy the app to your users without taking much time.

Easy to Use

Google App Engine (GAE) incorporates the tools that you need to develop, test, launch, and update the applications.

Scalability

- For any app's success, this is among the deciding factors. Google creates its own apps using GFS, Big Table and other such technologies, which are available to you when you utilize the Google app engine to create apps.
- You only have to write the code for the app and Google looks after the testing on account of the automatic scaling feature that the app engine has. Regardless of the amount of data or number of users that your app stores, the app engine can meet your needs by scaling up or down as required.
- The good thing about Google App Engine as a manageable platform is that it has made it feasible for our engineers to effortlessly scale up their applications with no-operations skill. It, additionally, sets us up with the best practices as far as logging, security and releasing management is concerned.

Performance and Reliability

Google is among the leaders worldwide among global brands. So, when you discuss performance and reliability you have to keep that in mind. In the past 15 years, the company has created new benchmarks based on its services' and products' performance. The app engine provides the same reliability and performance as any other Google product.

Cost Savings

You don't have to hire engineers to manage your servers or to do that yourself. You can invest the money saved into other parts of your business.

Platform Independence

You can move all your data to another environment without any difficulty as there are not many dependencies on the app engine platform.

5.5 Programming Environment for Google

Creating a Google Cloud Platform project

To use Google's tools for your own site or app, you need to create a new project on Google Cloud Platform. This requires having a Google account.

- Go to the App Engine dashboard on the Google Cloud Platform Console and press the Create button.
- If you've not created a project before, you'll need to select whether you want to receive email updates or not, agree to the Terms of Service, and then you should be able to continue.
- Enter a name for the project, edit your project ID and note it down. For this tutorial, the following values are used:

Project Name: GAE Sample Site

Project ID: gaesamplesite

- **Click the Create button to create your project.**

Creating an application

- Each Cloud Platform project can contain one App Engine application. Let's prepare an app for our project.
 - We'll need a sample application to publish. If you've not got one to use, download and unzip this sample app.
 - Have a look at the sample application's structure — the website folder contains your website content and app.yaml is your application configuration file.
- 1) Your website content must go inside the website folder, and its landing page must be called index.html, but apart from that it can take whatever form you like.
 - 2) The app.yaml file is a configuration file that tells App Engine how to map URLs to your static files. You don't need to edit it.

Publishing your application

Now that we've got our project made and sample app files collected together, let's publish our app.

1. Open Google Cloud Shell.
2. Drag and drop the sample-app folder into the left pane of the code editor.
3. Run the following in the command line to select your project:
 - i. **gcloud config set project gaesamplesite**
4. Then run the following command to go to your app's directory:
 - i. **cd sample-app**
5. You are now ready to deploy your application, i.e. upload your app to App Engine:
 - i. **gcloud app deploy**
6. Enter a number to choose the region where you want your application located.
 - i. **Enter Y to confirm.**
7. Now navigate your browser to your-project-id.appspot.com to see your website online. For example, for the project ID gaesamplesite, go to gaesamplesite.appspot.com.

5.6 App Engine

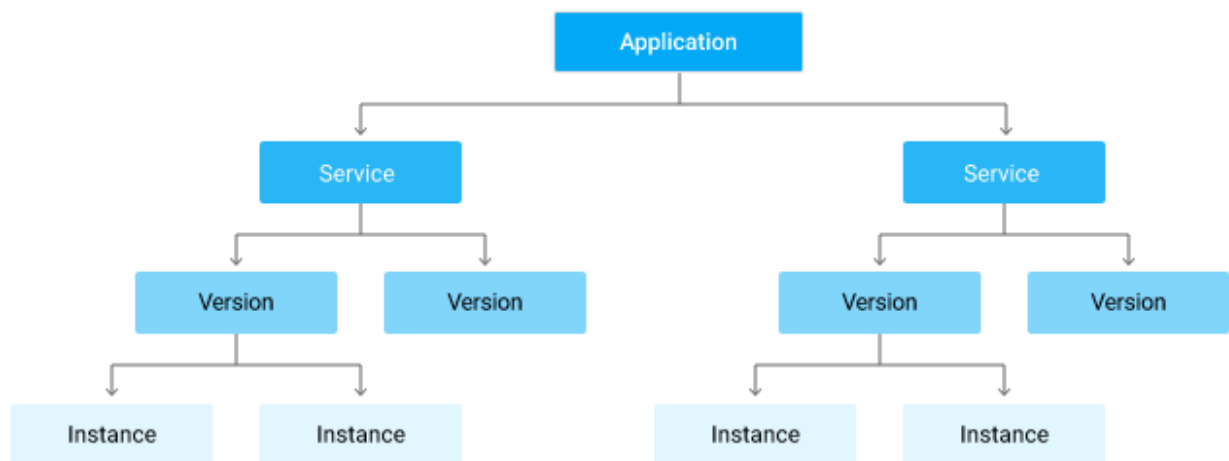
An App Engine app is made up of a single application resource that consists of one or more services. Each service can be configured to use different runtimes and to operate with different performance settings. Within each service, you deploy versions of that service. Each version then runs within one or more instances, depending on how much traffic you configured it to handle.

Components of an application

Your App Engine app is created under your Google Cloud project when you create an application resource. The App Engine application is a top-level container that includes the service, version, and instance resources that make up your app. When you create your App Engine app, all your resources are created in the region that you choose, including your app code along with a collection of settings, credentials, and your app's metadata.

Each App Engine application includes at least one service, the default service, which can hold as many versions of that service as you like.

The following diagram illustrates the hierarchy of an App Engine app running with multiple services. In this diagram, the app has two services that contain multiple versions, and two of those versions are actively running on multiple instances:



Services

Use services in App Engine to factor your large apps into logical components that can securely share App Engine features and communicate with one another. Generally, your App Engine services behave like microservices. Therefore, you can run your whole app in a single service or you can design and deploy multiple services to run as a set of microservices.

For example, an app that handles your customer requests might include separate services that each handle different tasks, such as:

- API requests from mobile devices
- Internal, administration-type requests
- Backend processing such as billing pipelines and data analysis

Each service in App Engine consists of the source code from your app and the corresponding App Engine configuration files. The set of files that you deploy to a service represent a single version of that service and each time that you deploy to that service, you are creating additional versions within that same service.

Versions

Having multiple versions of your app within each service allows you to quickly switch between different versions of that app for rollbacks, testing, or other temporary events. You can route traffic to one or more specific versions of your app by migrating or splitting traffic.

Instances

The versions within your services run on one or more instances. By default, App Engine scales your app to match the load. Your apps will scale up the number of instances that are running to provide consistent performance, or scale down to minimize idle instances and reduces costs. For more information about instances, see [How Instances are Managed](#).

Application requests

Each of your app's services and each of the versions within those services must have a unique name. You can then use those unique names to target and route traffic to specific resources using URLs, for example:

`https://VERSION_ID-dot-SERVICE_ID-dot-PROJECT_ID.REGION_ID.r.appspot.com`

Incoming user requests are routed to the services or versions that are configured to handle traffic. You can also target and route requests to specific services and versions. For more information, see [Handling Requests](#).

Logging application requests

When your application handles a request, it can also write its own logging messages to stdout and stderr. For details about your app's logs, see [Writing Application Logs](#).

Limits

The maximum number of services and versions that you can deploy depends on your app's pricing:

Limit	Free app	Paid app
• Maximum services per app	5	105
• Maximum versions per app	15	210

5.7 Open Stack

OpenStack is a free open standard cloud computing platform, mostly deployed as infrastructure-as-a-service in both public and private clouds where virtual servers and other resources are made available to users.

OpenStack is a set of software tools for building and managing cloud computing platforms for public and private clouds. Backed by some of the biggest companies in software development and hosting, as well as thousands of individual community members, many think that OpenStack is the future of cloud computing. OpenStack is managed by the OpenStack Foundation, a non-profit that oversees both development and community-building around the project.

Introduction to OpenStack

OpenStack lets users deploy virtual machines and other instances that handle different tasks for managing a cloud environment on the fly. It makes horizontal scaling easy, which means that tasks that benefit from running concurrently can easily serve more or fewer users on the fly by just spinning up more instances. For example, a mobile application that needs to communicate with a remote server might be able to divide the work of communicating with each user across many different instances, all communicating with one another but scaling quickly and easily as the application gains more users.

And most importantly, OpenStack is open source software, which means that anyone who chooses to can access the source code, make any changes or modifications they need, and freely share these changes back out to the community at large. It also means that OpenStack has the benefit of thousands of developers all over the world working in tandem to develop the strongest, most robust, and most secure product that they can.

How is OpenStack used in a cloud environment?

The cloud is all about providing computing for end users in a remote environment, where the actual software runs as a service on reliable and scalable servers rather than on each end-user's computer. Cloud computing can refer to a lot of different things, but typically the industry talks about running different items "as a service"—software, platforms, and infrastructure. OpenStack falls into the latter category and is considered Infrastructure as a Service (IaaS). Providing infrastructure means that OpenStack makes it easy for users to quickly add new instance, upon which other cloud components can

run. Typically, the infrastructure then runs a "platform" upon which a developer can create software applications that are delivered to the end users.

What are the components of OpenStack?

OpenStack is made up of many different moving parts. Because of its open nature, anyone can add additional components to OpenStack to help it to meet their needs. But the OpenStack community has collaboratively identified nine key components that are a part of the "core" of OpenStack, which are distributed as a part of any OpenStack system and officially maintained by the OpenStack community.

Nova is the primary computing engine behind OpenStack. It is used for deploying and managing large numbers of virtual machines and other instances to handle computing tasks.

Swift is a storage system for objects and files. Rather than the traditional idea of referring to files by their location on a disk drive, developers can instead refer to a unique identifier referring to the file or piece of information and let OpenStack decide where to store this information. This makes scaling easy, as developers don't have the worry about the capacity on a single system behind the software. It also allows the system, rather than the developer, to worry about how best to make sure that data is backed up in case of the failure of a machine or network connection.

Cinder is a block storage component, which is more analogous to the traditional notion of a computer being able to access specific locations on a disk drive. This more traditional way of accessing files might be important in scenarios in which data access speed is the most important consideration.

Neutron provides the networking capability for OpenStack. It helps to ensure that each of the components of an OpenStack deployment can communicate with one another quickly and efficiently.

Horizon is the dashboard behind OpenStack. It is the only graphical interface to OpenStack, so for users wanting to give OpenStack a try, this may be the first component they actually "see." Developers can access all of the components of OpenStack individually through an application programming interface (API), but the dashboard provides system administrators a look at what is going on in the cloud, and to manage it as needed.

Keystone provides identity services for OpenStack. It is essentially a central list of all of the users of the OpenStack cloud, mapped against all of the services provided by the cloud, which they have permission to use. It provides multiple means of access, meaning developers can easily map their existing user access methods against Keystone.

Glance provides image services to OpenStack. In this case, "images" refers to images (or virtual copies) of hard disks. Glance allows these images to be used as templates when deploying new virtual machine instances.

Ceilometer provides telemetry services, which allow the cloud to provide billing services to individual users of the cloud. It also keeps a verifiable count of each user's system usage of each of the various components of an OpenStack cloud. Think metering and usage reporting.

Heat is the orchestration component of OpenStack, which allows developers to store the requirements of a cloud application in a file that defines what resources are necessary for that application. In this way, it helps to manage the infrastructure needed for a cloud service to run.

Prerequisite for minimum production deployment

There are some basic requirements you'll have to meet to deploy OpenStack. Here are the prerequisites, drawn from the OpenStack manual.

Hardware: For OpenStack controller node, 12 GB RAM are needed as well as a disk space of 30 GB to run OpenStack services. Two SATA disks of 2 TB will be necessary to store volumes used by instances. Communication with compute nodes requires a network interface card (NIC) of 1 Gbps. For compute

nodes, 2 GB RAM will be sufficient to run three tiny instances on a single compute node. Two NIC 1 Gbps will allow communication with both the controller and other compute nodes.

Operating system (OS): OpenStack supports the following operating systems: CentOS, Debian, Fedora, Red Hat Enterprise Linux (RHEL), openSUSE, SLES Linux Enterprise Server and Ubuntu. Other system support is provided by different editors or can be developed by porting nova modules on the target platform.

5.8 Federation in the Cloud

“Cloud federation manages consistency and access controls when two or more independent geographically distinct Clouds share either authentication, files, computing resources, command and control or access to storage resources.”

Cloud federation introduces additional issues that have to be addressed in order to provide a secure environment in which to move applications and services among a collection of federated providers. Baseline security needs to be guaranteed across all cloud vendors that are part of the federation.

An interesting aspect is represented by the management of the digital identity across diverse organizations, security domains, and application platforms. In particular, the term federated identity management refers to standards-based approaches for handling authentication, single sign-on (SSO), role-based access control, and session management in a federated environment . This enables users to utilize services more effectively in a federated context by providing their authentication details only once to log into a network composed of several entities involved in a transaction. This capability is realized by either relying on open industry standards or openly published specifications (Liberty Alliance Identity Federation, OASIS Security Assertion Markup Language, and WS-Federation) such that interoperation can be achieved. No matter the specific protocol and framework, two main approaches can be considered:

Centralized federation model

This is the approach taken by several identity federation standards. It distinguishes two operational roles in an SSO transaction: the identity provider and the service provider.

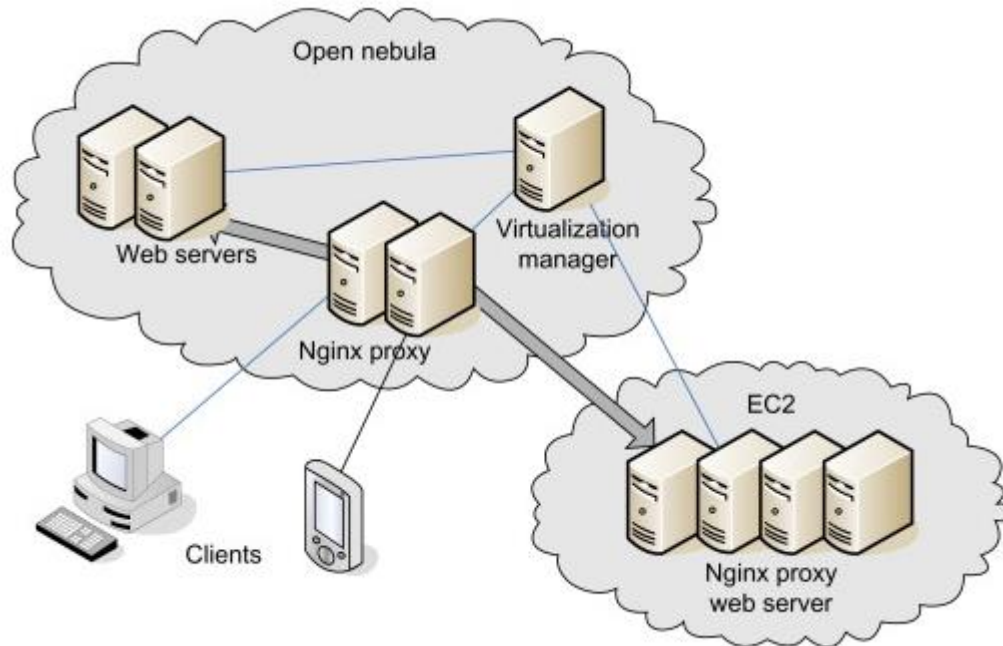
Claim-based model

This approach addresses the problem of user authentication from a different perspective and requires users to provide claims answering who they are and what they can do in order to access content or complete a transaction.

The first model is currently used today; the second constitutes a future vision for identity management in the cloud.

Digital identity management constitutes a fundamental aspect of security management in a cloud federation. To transparently perform operations across different administrative domains, it is of mandatory importance to have a robust framework for authentication and authorization, and federated identity management addresses this issue. Our previous considerations of security contribute to design and implement a secure system comprising the cloud vendor stack and the user application; federated identity management allows us to tie together the computing stacks of different vendors and present them as a single environment to users from a security point of view.

OpenNebula can be used in conjunction with a reverse proxy to form a cloud bursting hybrid cloud architecture with load balancing and virtualization support provided by OpenNebula . The OpenNebula VM controls server allocation in both the EC2 cloud as well as the OpenNebula cloud, while the Nginx proxy to which the clients are connected distributes load over the web servers both in EC2 as well as the OpenNebula cloud. In addition to web servers, the EC2 cloud also has its own Nginx load balancer.



Much research work has been developed around OpenNebula. For example, the University of Chicago has come up with an advance reservation system called Haizea Lease Manager. IBM Haifa has developed a policy-driven probabilistic admission control and dynamic placement optimization for site level management policies called the RESERVOIR Policy Engine, Nephele is an SLA-driven automatic service management tool developed by Telefonica and Virtual Cluster Tool for atomic cluster management with versioning with multiple transport protocols from CRS4 Distributed Computing Group.

Cloud Federations and Server Coalitions

In large-scale systems, coalition formation supports more effective use of resources, as well as convenient means to access these resources. It is therefore not surprising that coalition formation for computational grids has been investigated in the past. There is also little surprise that the interest in coalition formation migrated in recent years from computational grids to CRM. The interest in grid computing is fading away, while cloud computing is widely accepted today and its adoption by more and more institutions and individuals seems to be guaranteed at least for the foreseeable future.

Two classes of applications of cloud coalitions are reported in the literature:

1. Coalitions among CSPs for the formation of cloud federations. A cloud federation is an infrastructure allowing a group of CSPs to share resources; the goal is to balance the load and improve system reliability.
2. Coalitions among the servers of a data center. The goal is to assemble a pool of resources larger than the ones available from a single server.

In recent years the number of CSPs has increased significantly. The question if they should cooperate to share their resources led to the idea of cloud federations, groups of CSPs who have agreed on a set of common standards and are able to share their resources. The infrastructure of individual CSPs consists of a hierarchy of networks and millions of servers thus, a cloud federation would indeed be a very complex system.

The vast majority of ongoing research in this area is focused on game-theoretic aspects of coalition formation for cloud federations, while coalitions among the servers of a single cloud has received little attention in the past. This is likely to change due to the emerging interest in Big Data cloud applications which require more resources than a single server can provide. To address this problem, sets of identically configured servers able to communicate effectively among themselves form coalitions with sufficient resources for data- and computationally intensive problems.

Cloud coalition formation raises a number of technical, as well as nontechnical problems. Cloud federations require a set of standards. The cloud computing landscape is still evolving and an early

standardization may slowdown and negatively affects the adoption of new ideas and technologies. At the same time, CSPs want to maintain their competitive advantages by closely guarding the details of their internal algorithms and protocols.

Reaching agreements on a set of standards is particularly difficult when the infrastructure of the members of the group is designed to support different cloud delivery models. For example, it is hard to see how the IaaS could be supported by either SaaS or PaaS clouds. Thus, in spite of the efforts coordinated by the National Institute of Standards (NIST), the adoption of inter-operability standards supporting cloud federations seems a rather distant possibility, that resource management in one cloud is extremely challenging therefore, dynamic resource sharing among multiple cloud infrastructures seems infeasible at this time. Communication between the members of a cloud federation would also require dedicated networks with low latency and high bandwidth.

5.9 Four Levels of Federation

Creating a cloud federation involves research and development at different levels: conceptual, logical and operational, and infrastructural.

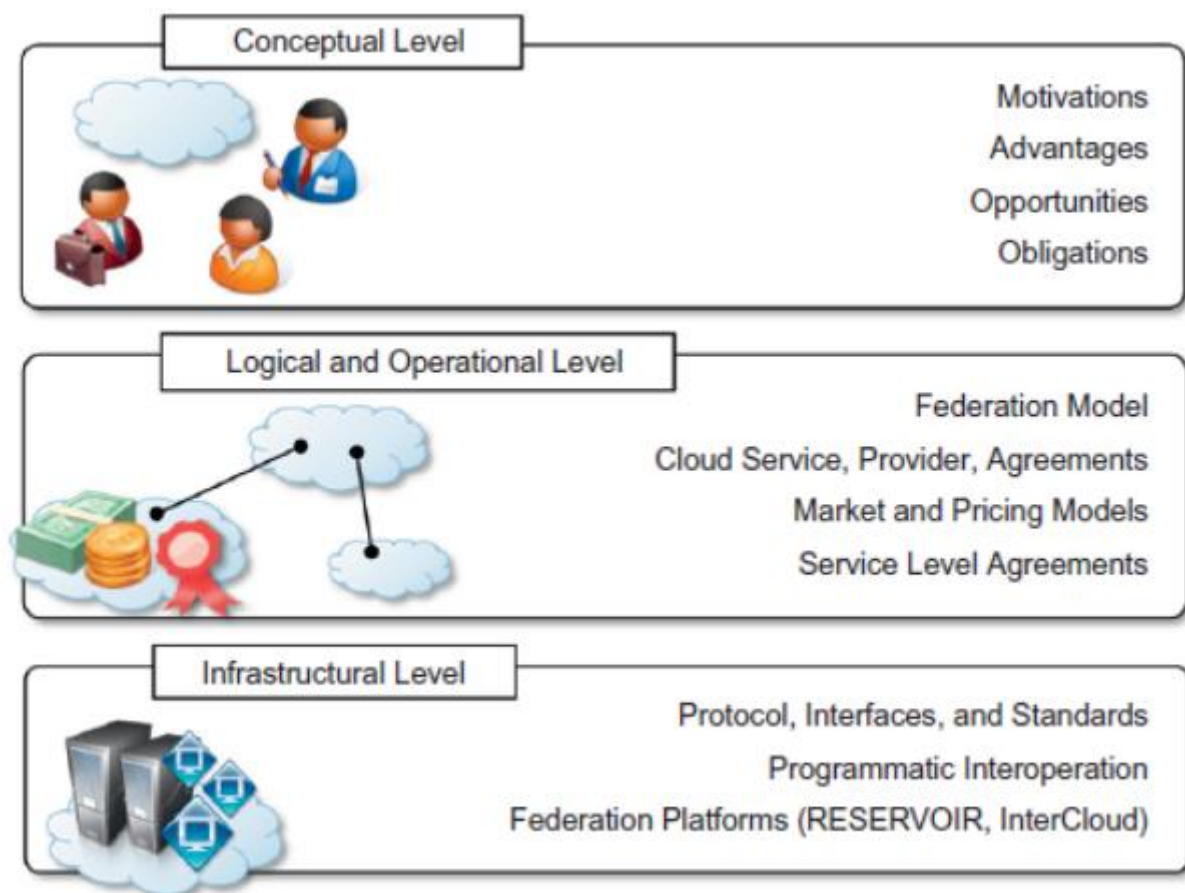


Figure provides a comprehensive view of the challenges faced in designing and implementing an organizational structure that coordinates together cloud services that belong to different administrative domains and makes them operate within a context of a single unified service middleware.

Each cloud federation level presents different challenges and operates at a different layer of the IT stack. It then requires the use of different approaches and technologies. Taken together, the solutions to the challenges faced at each of these levels constitute a reference model for a cloud federation.

CONCEPTUAL LEVEL

The conceptual level addresses the challenges in presenting a cloud federation as a favourable solution with respect to the use of services leased by single cloud providers. In this level it is important to clearly identify the advantages for either service providers or service consumers in joining a federation and to delineate the new opportunities that a federated environment creates with respect to the single-provider solution.

Elements of concern at this level are:

- Motivations for cloud providers to join a federation.
- Motivations for service consumers to leverage a federation.
- Advantages for providers in leasing their services to other providers.
- Obligations of providers once they have joined the federation.
- Trust agreements between providers.
- Transparency versus consumers.

Among these aspects, the most relevant are the motivations of both service providers and consumers in joining a federation.

LOGICAL & OPERATIONAL LEVEL

- The logical and operational level of a federated cloud identifies and addresses the challenges in devising a framework that enables the aggregation of providers that belong to different administrative domains within a context of a single overlay infrastructure, which is the cloud federation.
- At this level, policies and rules for interoperation are defined. Moreover, this is the layer at which decisions are made as to how and when to lease a service to—or to leverage a service from—another provider.
- The logical component defines a context in which agreements among providers are settled and services are negotiated, whereas the operational component characterizes and shapes the dynamic behaviour of the federation as a result of the single providers' choices.

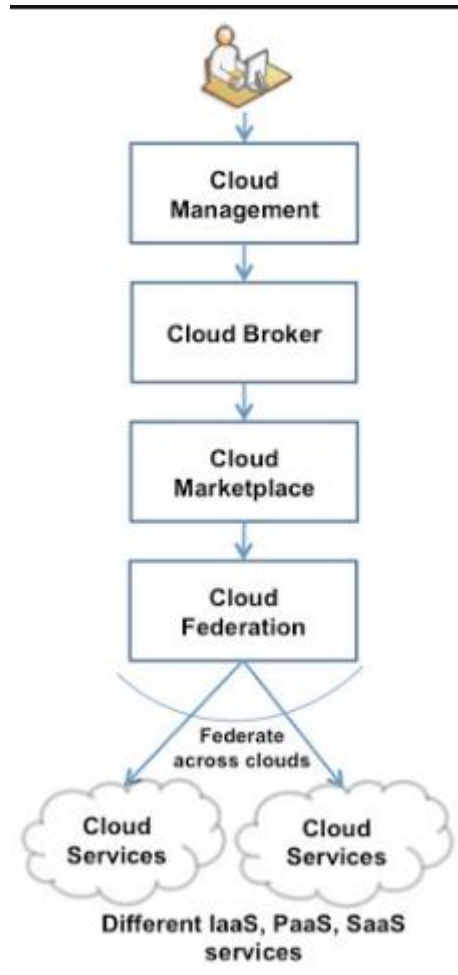
This is the level where MOCC is implemented and realized. It is important at this level to address the following challenges:

- How should a federation be represented?
- How should we model and represent a cloud service, a cloud provider, or an agreement?
- How should we define the rules and policies that allow providers to join a federation?
- What are the mechanisms in place for settling agreements among providers?
- What are provider's responsibilities with respect to each other?
- When should providers and consumers take advantage of the federation?
- Which kinds of services are more likely to be leased or bought?
- How should we price resources that are leased, and which fraction of resources should we lease? **The logical and operational level provides opportunities for both academia and industry.**

INFRASTRUCTURE LEVEL

The infrastructural level addresses the technical challenges involved in enabling heterogeneous cloud computing systems to interoperate seamlessly.

It deals with the technology barriers that keep separate cloud computing systems belonging to different administrative domains. By having standardized protocols and interfaces, these barriers can be overcome.

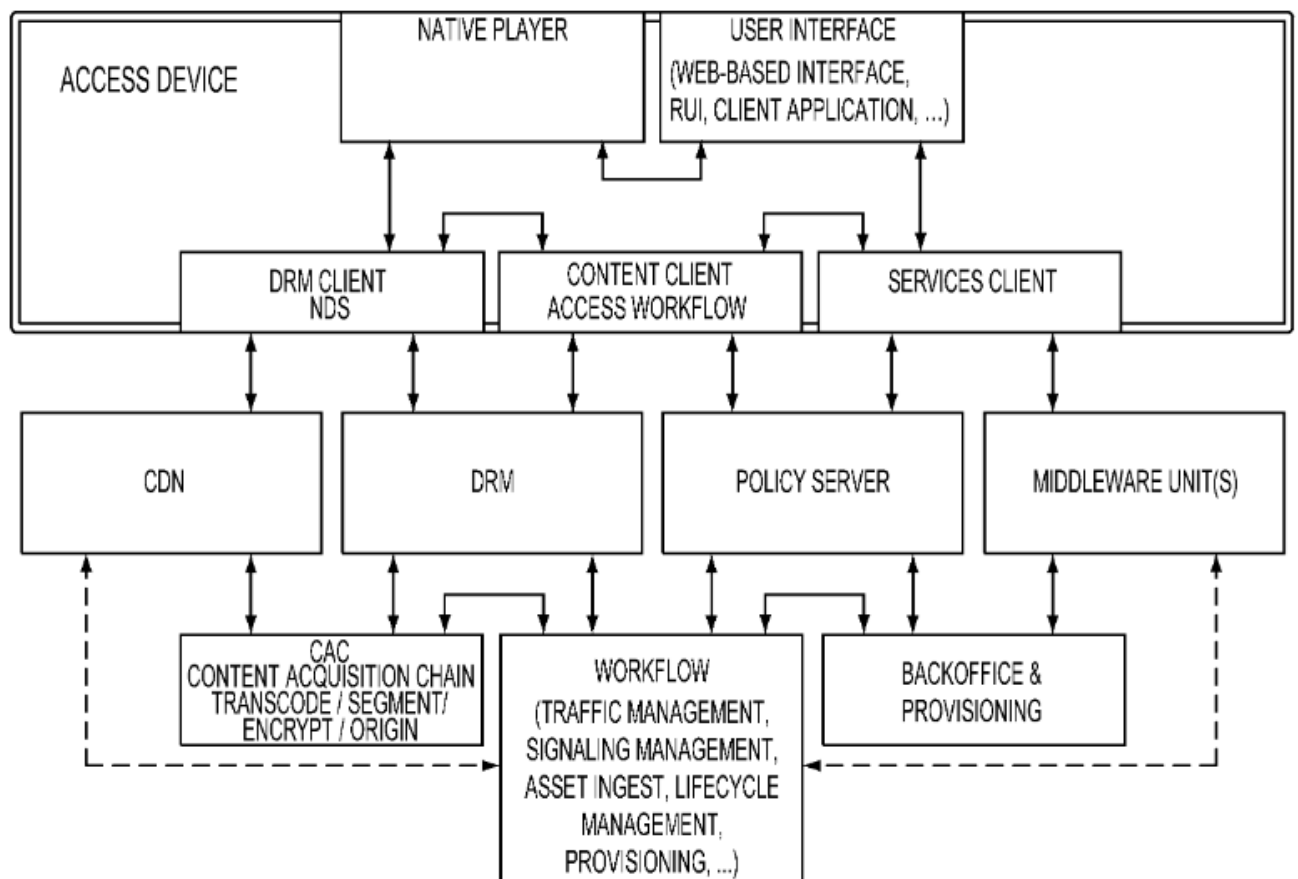


At this level it is important to address the following issues:

- What kind of standards should be used?
- How should design interfaces and protocols be designed for interoperation?
- Which are the technologies to use for interoperation?
- How can we realize a software system, design platform components, and services enabling interoperability?

Interoperation and composition among different cloud computing vendors is possible only by means of open standards and interfaces. Moreover, interfaces and protocols change considerably at each layer of the Cloud Computing Reference Model.

5.10 Federated Services and Applications



5.11 Future of Federation.

The federated cloud model is a force for real democratization in the cloud market. It's how businesses will be able to use local cloud providers to connect with customers, partners and employees anywhere in the world. It's how end users will finally get to realize the promise of the cloud. And, it's how data center operators and other service providers will finally be able to compete with, and beat, today's so-called global cloud providers.

The future of cloud computing as one big public cloud. Others believe that enterprises will ultimately build a single large cloud to host all their corporate services. This is, of course, because the benefit of cloud computing is dependent on large – very large – scale infrastructure, which provides administrators and service administrators and consumers the ability for ease of deployment, self service, elasticity, resource pooling and economies of scale. However, as cloud continues to evolve – so do the services being offered.

Cloud Services & Hybrid Clouds

Services are now able to reach a wider range of consumers, partners, competitors and public audiences. It is also clear that storage, compute power, streaming, analytics and other advanced services are best served when they are in an environment tailored for the proficiency of that service.

One method of addressing the need of these service environments is through the advent of hybrid clouds. Hybrid clouds, by definition, are composed of multiple distinct cloud infrastructures connected in a manner that enables services and data access across the combined infrastructure. The intent is to leverage the additional benefits that hybrid cloud offers without disrupting the traditional cloud benefits. While hybrid cloud benefits come through the ability to distribute the work stream, the goal is to continue to realize the ability for managing peaks in demand, to quickly make services available and capitalize on new business opportunities.

The Solution: Federation

Federation creates a hybrid cloud environment with an increased focus on maintaining the integrity of corporate policies and data integrity. Think of federation as a pool of clouds connected

through a channel of gateways; gateways which can be used to optimize a cloud for a service or set of specific services. Such gateways can be used to segment service audiences or to limit access to specific data sets. In essence, federation has the ability for enterprises to service their audiences with economy of scale without exposing critical applications or vital data through weak policies or vulnerabilities.

- Many would raise the question: if Federation creates multiples of clouds, doesn't that mean cloud benefits are diminished? I believe the answer is no, due to the fact that a fundamental change has transformed enterprises through the original adoption of cloud computing, namely the creation of a flexible environment able to adapt rapidly to changing needs based on policy and automation.
 - Cloud end-users are often tied to a unique cloud provider, because of the different APIs, image formats, and access methods exposed by different providers that make very difficult for an average user to move its applications from one cloud to another, so leading to a vendor lock-in problem.
 - Many SMEs have their own on-premise private cloud infrastructures to support the internal computing necessities and workloads. These infrastructures are often over-sized to satisfy peak demand periods, and avoid performance slow-down. Hybrid cloud (or cloud bursting) model is a solution to reduce the on-premise infrastructure size, so that it can be dimensioned for an average load, and it is complemented with external resources from a public cloud provider to satisfy peak demands.
 - Many big companies (e.g. banks, hosting companies, etc.) and also many large institutions maintain several distributed data-centers or server-farms, for example to serve to multiple geographically distributed offices, to implement HA, or to guarantee server proximity to the end user. Resources and networks in these distributed data-centers are usually configured as non-cooperative separate elements, so that usually every single service or workload is deployed in a unique site or replicated in multiple sites.
 - Many educational and research centers often deploy their own computing infrastructures, that usually do not cooperate with other institutions, except in some punctual situations (e.g. in joint projects or initiatives). Many times, even different departments within the same institution maintain their own non-cooperative infrastructures.
- This Study Group will evaluate the main challenges to enable the provision of federated cloud infrastructures, with special emphasis on inter-cloud networking and security issues:
- Security and Privacy
 - Interoperability and Portability
 - Performance and Networking Cost

It is important to bring perspectives from Europe and USA in order to define the basis for an open cloud market, addressing barriers to adoption and meeting regulatory, legal, geographic, trust and performance constraints.

This group will directly contribute to the first two key actions of the European Cloud Strategy "Unleashing the Potential of Cloud Computing in Europe".

The **first key action** aims at "Cutting through the Jungle of Standards" to help the adoption of cloud computing by encouraging compliance of cloud services with respect to standards and thus providing evidence of compliance to legal and audit obligations. These standards aim to avoid customer lock in by promoting interoperability, data portability and reversibility.

The **second key action** "Safe and Fair Contract Terms and Conditions" aims to protect the cloud consumer from insufficiently specific and balanced contracts with cloud providers that do not "provide for liability for data integrity, confidentiality or service continuity". The cloud consumer is often presented with "take-it-or-leave-it standard contracts that might be cost-saving for the provider but is

often undesirable for the user”. The commission aims to develop with “stakeholders model terms for cloud computing service level agreements for contracts”.

Server to Server Sharing

The first version of this ideas was implemented in ownCloud 7.0 as “Server to Server Sharing”. ownCloud already knew the concept of sharing anonymous links with people outside of the server. And, as ownCloud offered both a WebDAV interface and could mount external WebDAV shares, it was possible to manually hook a ownCloud into another ownCloud server. Therefore the first obvious step was to add a “Add to your ownCloud” button to this link shares, allowing people to connect such public links with their cloud by mounting it as a external WebDAV resource.

Interface: Various cloud service providers have different APIs, pricing models and cloud infrastructure. Open cloud computing interface is necessary to be initiated to provide a common application programming interface for multiple cloud environments. The simplest solution is to use a software component that allows the federated system to connect with a given cloud environment. Another solution can be to perform the federation at Infrastructure level and not at application or service level.

Networking: Virtual machines in the cloud may be located in different network architectures using different addressing schemes. To interconnect these VMs a virtual network can be formed on the underlying physical network with uniform IP addressing scheme. When services are running on remote clouds, main concern is security of the sensitive strategic information running on remote cloud. C. Heterogeneity of resource: Each cloud service providers offers different VMs with varying processing memory and storage capacity resulting in unbalanced processing load and system instability. It is likely that the cloud owner will purchase latest models of hardware available at the time of purchase while it is unlikely to retire the older model nodes until their useful life is over. This creates heterogeneity.

Federated Cloud Sharing

Server to server sharing already helped a lot to establish some bridges between many small islands created by the ability to self-host your cloud solution. But it was still not the kind of integration people where used to from the large centralized services and it only worked for ownCloud, not across various open source file sync and share solutions.

Trusted Servers

In order to make it easier to find people on other servers we introduced the concept of “trusted servers” as one of our last steps. This allows administrator to define other servers they trust. If two servers trust each other they will sync their user lists. This way the share dialogue can auto-complete not only local users but also users on other trusted servers. The administrator can decide to define the lists of trusted servers manually or allow the server to auto add every other server to which at least one federated share was successfully created. This way it is possible to let your cloud server learn about more and more other servers over time, connect with them and increase the network of trusted servers.

Open Challenges: where we’re taking Federated Cloud Sharing

Of course there are still many areas to improve. For example the way you can discover users on different server to share with them, for which we’re working on a global, shared address book solution. Another point is that at the moment this is limited to sharing files. A logical next step would be to extend this to many other areas like address books, calendars and to real-time text, voice and video communication and we are, of course, planning for that.